# LIGHT SUMMARIZER

A CAPSTONE AND RESEARCH PROJECT

Robert Martin Baltisoto

Shielo Mariell Leodones

Gieselle Sabbaluca

Sofiah Heaven Galman

Nueva Ecija University of Science and Technology

Peñaranda Off-Campus

[REQUIRES RESEARCHER CONFIRMATION: degree program and academic unit]

[REQUIRES RESEARCHER CONFIRMATION: month and year of submission]

**Title Page**

LIGHT summarizer

A Capstone and Research Project

Presented to the Faculty of

[REQUIRES RESEARCHER CONFIRMATION: official college, department and program]

Nueva Ecija University of Science and Technology

Peñaranda Off-Campus

In Partial Fulfillment of the Requirements for

[REQUIRES RESEARCHER CONFIRMATION: exact degree and submission wording]

by

Robert Martin Baltisoto

Shielo Mariell Leodones

Gieselle Sabbaluca

Sofiah Heaven Galman

[REQUIRES RESEARCHER CONFIRMATION: submission date]

The institution and intended campus are identified in the researcher-supplied draft. Their inclusion does not establish institutional endorsement or completed submission. This editable manuscript contains verified technical evidence and clearly identified research components awaiting confirmation.

## Approval Sheet

UNSIGNED TEMPLATE — REQUIRES INSTITUTIONAL COMPLETION

The capstone project entitled **LIGHT summarizer**, prepared by Robert Martin Baltisoto, Shielo Mariell Leodones, Gieselle Sabbaluca and Sofiah Heaven Galman, is submitted for review under the requirements of [official program]. Approval, examination outcomes and acceptance dates shall be entered only by the authorized institution after the corresponding action.

Research Adviser: [name, credentials and signature]

Date: [actual date]

Examination Committee Chair: [name, credentials and signature]

Committee Member: [name, credentials and signature]

Committee Member: [name, credentials and signature]

Examination date and decision: [actual record]

Accepted by: [authorized official, designation and signature]

Date of acceptance: [actual date]

No signature, grade, approval or acceptance is represented by this template. Replace the wording with the institution's prescribed form before submission.

## Joint Undertaking

PENDING INSTITUTION-APPROVED FORM

Insert the official undertaking applicable to this capstone, including the institution's required declarations concerning authorship, responsibilities, intellectual property and any notarization. The reference manuscript's legal statements and personal details have not been copied. The four researchers must review and execute the actual institutional form; no attestation is made in this draft.

Signatories: Robert Martin Baltisoto; Shielo Mariell Leodones; Gieselle Sabbaluca; Sofiah Heaven Galman.

Required form version, signing date, witness and notarization details: [REQUIRES RESEARCHER CONFIRMATION].

## English Critic Certification

PENDING AUTHENTIC CERTIFICATION

Insert a certification issued by the person who actually reviewed the manuscript's language, using the institution's approved wording. Critic name, qualifications, review date, document version and signature: [REQUIRES RESEARCHER CONFIRMATION]. This page is a document slot and does not certify that language review has occurred.

## Acknowledgement

RESEARCHER-COMPLETED PERSONAL STATEMENT

[The researchers should identify the people and organizations that actually contributed to the project and describe each contribution accurately. Confirm the adviser, technical reviewers, campus personnel and any participants before naming them. Do not acknowledge a consultation, evaluation or institutional approval that has not occurred.]

## Dedication

RESEARCHER-COMPLETED PERSONAL STATEMENT

[Insert the dedication selected and approved by the four researchers. Personal experiences, relationships and sentiments have not been invented for this manuscript.]

## Table of Contents

LIGHT SUMMARIZER.....i
Title Page.....ii
Approval Sheet.....iii
Joint Undertaking.....iv
English Critic Certification.....v
Acknowledgement.....vi
Dedication.....vii
Table of Contents.....viii
List of Tables.....xiv
List of Figures.....xvii
Abstract.....xxi
CHAPTER I THE PROBLEM AND ITS BACKGROUND.....1
Introduction.....1
Review of Related Literature and Studies.....3
Extractive ranking and information selection.....3
Automatic comparison with reference summaries.....4
Faithfulness and factuality.....5
Evaluation through complementary evidence.....6
Product-quality framework and edition control.....7
Synthesis and project contribution.....8
Conceptual Framework.....9
Statement of the Problem.....11
Objectives of the Study.....12
Scope and Delimitation.....13
Significance of the Study.....15
Definition of Terms.....16
CHAPTER II METHODS AND PROCEDURES.....19
Research Design.....19
Development Approach Described in the Researcher Draft
.....22
Planning and analysis.....22
Design.....22

Implementation.....23
Testing, integration and maintenance.....23
Supplied development schedule.....24
Technical Examination Procedure.....29
Research Locale.....31
Research Respondents.....32
Sampling Procedure.....33
Research Instruments.....34
Proposed Data-Gathering Procedure.....37
Data Analysis Technique.....39
Technical findings.....39
Questionnaire responses.....39
Summary quality and comments.....41
Ethical and Methodological Boundaries.....42
CHAPTER III RESULTS AND DISCUSSION.....43
1. Problems Encountered in the Existing System.....43
2. How the Developed System Addresses the Identified Problems.....44
3. Essential Features and Functionalities.....46
Entry.....46
Navigation.....47
Appearance.....48
Accounts.....49
Terms.....54
Summary input.....55
Summary controls.....57
Summarization.....61
Recovery.....62
Processing.....63
Nutshell.....64
Results.....65
Feedback.....71
Results.....72
Dashboard.....72

History.....74
Analytics.....75
Profile.....76
Accounts.....78
Administration.....79
Evaluation.....85
API-only.....87
Health.....89
Backend services.....90
Evaluation.....93
Operations.....97
Results.....100
Processing.....100
Security Features.....106
User Experience Features.....107
4. Design and Development of the System.....109
4.1 Planning Stage.....109
4.2 Design Stage.....110
4.3 Development Stage.....229
4.4 Testing Stage.....249
4.5 Deployment Stage.....253
4.6 Operational Stage.....254
5. IT Consultants' System Evaluation.....255
Functional Suitability.....255
Performance Efficiency.....256
Compatibility.....257
Usability.....259
Reliability.....260
Security.....261
Portability.....262
Overall IT Consultants' Evaluation Summary.....263
6. End-Users' System Evaluation.....264
Functional Suitability.....264
Performance Efficiency.....265

Usability.....266
Overall End-Users' Evaluation Summary.....267
CHAPTER IV SUMMARY, CONCLUSIONS AND RECOMMENDATIONS. 269
1. Summary.....269
2. Conclusions.....272
3. Recommendations.....273
REFERENCES.....277
Project and Documentary Sources.....278
APPENDIX A. RESEARCH ADVISER REQUEST LETTER TEMPLATE 281
APPENDIX B. DATA-GATHERING PERMISSION REQUEST TEMPLATE .....283
APPENDIX C. ENGLISH CRITIC REVIEW REQUEST TEMPLATE. .285
APPENDIX D. CONSULTATION REQUEST AND MONITORING FORM 287
APPENDIX E. RESEARCH ADVISER DECISION LETTER TEMPLATE .....289
APPENDIX F — USER MANUAL.....291
A. Landing Page and Shared Navigation.....293
B. Sign In and Administrator Verification.....295
C. Create Account.....299
D. Recover Password.....301
E. Set a New Password.....303
F. Terms and Conditions.....305
G. Summarizer Workspace.....308
G.1. Select the source.....309
G.2. Choose settings and submit.....311
G.3. Errors, drafts, and Nutshell.....312
H. Processing and Recovery.....313
I. Summary Result, Source Review, and Export.....316
J. Summary Feedback.....319
K. Unavailable Result.....321
L. Personal Dashboard.....323
M. Summary History.....325
N. Analytics.....326
O. Profile and Account Deactivation.....328

P. Logout Confirmation.....330
Q. Administrative Console.....332
Q.1. Review system information.....333
Q.2. Manage users.....333
Q.3. Review feedback and landing telemetry.....334
R. Administrator Audit Logs.....336
S. Assigned Independent Reviews.....338
T. Evaluation Reports, Compatibility Routes, and Diagnostics.....339
U. Common Recovery Guidance and Evidence Trace.....341
APPENDIX G. LIGHT SUMMARIZER MAINTENANCE MANUAL.....343
1. Introduction.....343
2. System Overview.....343
3. Major Features.....346
4. System Requirements.....347
5. Preventive Maintenance Procedures.....349
Database and schema.....349
File storage and retention.....350
Security and dependency maintenance.....351
Worker and performance monitoring.....352
Proposed backup and recovery procedure.....353
6. Troubleshooting Guide.....355
7. Roles and Responsibilities.....358
8. Data Security and Privacy.....359
9. Maintenance Schedule.....361
10. Conclusion.....363
APPENDIX H. PROPOSED TECHNICAL EVALUATION INSTRUMENT 364
Instrument Status and Purpose.....364
Directions and Response Scale.....365
Functional Suitability.....367
Performance Efficiency.....368
Compatibility.....370
Usability.....371
Reliability.....372

Security.....374
Portability.....375
Reviewer Comments and Researcher Coding.....377
APPENDIX I. PROPOSED END-USER EVALUATION INSTRUMENT. 378
Instrument Status and Directions.....378
Functional Suitability.....379
Perceived Performance Efficiency.....380
Usability.....381
APPENDIX J. DATA-GATHERING AND SUPPORTING DOCUMENTS. 384
Status of Research-Event Evidence.....384
Photograph and Activity-Record Template.....386
APPENDIX K. RESEARCHER CURRICULUM VITAE.....387
Robert Martin Baltisoto.....387
Shielo Mariell Leodones.....389
Gieselle Sabbaluca.....390
Sofiah Heaven Galman.....391
APPENDIX L. TECHNICAL EVIDENCE AND REPRODUCTION GUIDE
.....392
Purpose and Boundaries.....392
Controlled Reproduction Principles.....395

# List of Tables

Table 1. Research questions, evidence sources and limits of inference.....20

Table 2. Researcher-supplied provisional Gantt schedule, April–October 2026.....24

Table 3. Proposed respondent groups and outstanding sampling information.....33

Table 4. Proposed questionnaire response coding.....35

Table 5. Proposed descriptive interpretation intervals for questionnaire means.....40

Table 6. Relationship between supplied project needs and implemented capabilities.....45

Table 7. Data dictionary for admin_audit_logs.....155

Table 8. Data dictionary for admin_security_challenges. .....158

Table 9. Data dictionary for documents.....160

Table 10. Data dictionary for document_summaries.. .161

Table 11. Data dictionary for evaluation_assignments. .....164

Table 12. Data dictionary for evaluation_audit_log.. 167

Table 13. Data dictionary for evaluation_challenge_cases.....169

Table 14. Data dictionary for evaluation_content_units. .....171

Table 15. Data dictionary for evaluation_content_unit_reviews.....173

Table 16. Data dictionary for evaluation_datasets.. .176

Table 17. Data dictionary for evaluation_documents.. 178

Table 18. Data dictionary for evaluation_errors.....181

Table 19. Data dictionary for evaluation_metrics.. .183

Table 20. Data dictionary for evaluation_outputs.. .186

Table 21. Data dictionary for evaluation_ratings.. .189

Table 22. Data dictionary for evaluation_references. 192

Table 23. Data dictionary for evaluation_runs.....195

Table 24. Data dictionary for feedback.....199
Table 25. Data dictionary for feedback_comments.....202
Table 26. Data dictionary for guest_sessions.....204
Table 27. Data dictionary for landing_page_events.. .205
Table 28. Data dictionary for landing_page_sessions. 207
Table 29. Data dictionary for nutshell_generations.. 209
Table 30. Data dictionary for password_resets.....212
Table 31. Data dictionary for rate_limits.....214
Table 32. Data dictionary for schema_migrations.....216
Table 33. Data dictionary for summaries.....217
Table 34. Data dictionary for summary_artifacts.....221
Table 35. Data dictionary for summary_jobs.....222
Table 36. Data dictionary for users.....226
Table 37. Data dictionary for user_tokens.....228
Table 38. Technologies and observed development environment.....230
Table 39. Audit verification procedures and outcomes. .....250
Table 40. Functional Suitability — pending human evaluation.....256
Table 41. Performance Efficiency — pending human evaluation.....257
Table 42. Compatibility — pending human evaluation.. 258
Table 43. Usability — pending human evaluation.....259
Table 44. Reliability — pending human evaluation.. . .260
Table 45. Security — pending human evaluation.....261
Table 46. Portability — pending human evaluation.. . .262
Table 47. Overall IT consultants' summary — pending human evaluation.....263
Table 48. Functional Suitability for end users — pending human evaluation.....265
Table 49. Performance Efficiency for end users — pending human evaluation.....266
Table 50. Usability for end users — pending human evaluation.....267

Table 51. Overall end-users’ summary — pending human evaluation.....268
Table 52. Blank consultation and monitoring record.. 288
Table 53. Available access contexts and interface boundaries.....292
Table 54. Source methods and user actions.....310
Table 55. Summary settings in the main interface.. . .311
Table 56. Result toolbar actions and verification limits.....318
Table 57. Common recovery situations and user guidance. .....341
Table 58. Maintainer's path and component reference. 344
Table 59. Evidence-based troubleshooting guide.....355
Table 60. Proposed maintenance schedule; adoption and execution are not verified.....361
Table 61. Response scale for the proposed technical and end-user questionnaires.....365
Table 62. Proposed technical functional-suitability items.....367
Table 63. Proposed technical performance-efficiency items.....368
Table 64. Proposed technical compatibility items.. . .370
Table 65. Proposed technical usability items.....371
Table 66. Proposed technical reliability items.....372
Table 67. Proposed technical security items.....374
Table 68. Proposed technical portability items.....375
Table 69. Proposed end-user functional-suitability items.....379
Table 70. Proposed end-user performance items.....380
Table 71. Proposed end-user usability items.....381
Table 72. Supporting-document register and current evidence status.....384
Table 73. Technical evidence supplied with the manuscript.....392

## List of Figures

Figure 1. Proposed research paradigm for LIGHT summarizer.....10
Figure 2. Researcher-supplied provisional schedule, activities 1 to 7.....27
Figure 3. Researcher-supplied provisional schedule, activities 8 to 14.....28
Figure 4. Researcher-supplied provisional schedule, activities 15 to 21.....29
Figure 5. Implemented actors and major use cases. . .111
Figure 6. Guest visitor use cases.....113
Figure 7. Registered account use cases.....114
Figure 8. Administrator use cases.....115
Figure 9. Assigned evaluator use cases.....116
Figure 10. Context diagram of LIGHT summarizer.....118
Figure 11. Level 1 data flows and logical stores. . .119
Figure 12. Level 2 summary-generation data flow.....120
Figure 13. Level 2 account and authentication data flow.....122
Figure 14. Level 1 account and session detail.....124
Figure 15. Level 1 source and summary detail.....125
Figure 16. Level 1 result and reading-helper detail. 126
Figure 17. Level 1 feedback and usage detail.....128
Figure 18. Level 1 administration and evaluation detail.....129
Figure 19. Level 2 summary intake detail.....130
Figure 20. Level 2 summary output detail.....131
Figure 21. Level 2 account verification detail.....133
Figure 22. Level 2 session and identity-change detail. .....135
Figure 23. Accounts and authentication ERD panel. . .138
Figure 24. Summary ownership, jobs and artifacts ERD panel.....139
Figure 25. Ratings and comments ERD panel.....140

Figure 26. Document extraction and cached combinations ERD panel.....142
Figure 27. Landing sessions and events ERD panel.. . .144
Figure 28. Stores without declared foreign keys ERD panel.....146
Figure 29. Evaluation datasets, documents, runs and outputs ERD panel.....148
Figure 30. References, content units and challenge cases ERD panel.....149
Figure 31. Assignments, ratings, errors and metrics ERD panel.....150
Figure 32. Content-unit reviews ERD panel.....151
Figure 33. Evaluator and reviewer account relationships ERD panel.....152
Figure 34. Dataset, run and reference authorship ERD panel.....153
Figure 35. Landing information and navigation — isolated template capture.....232
Figure 36. Source intake and summary settings — isolated template capture.....233
Figure 37. Sign-in form — isolated template capture. 234
Figure 38. Registration form — isolated template capture.....235
Figure 39. Recovery request form — isolated template capture.....236
Figure 40. Password reset form — isolated template capture.....237
Figure 41. Administrator code verification state — isolated template capture.....238
Figure 42. Terms acceptance state — isolated template capture.....239
Figure 43. Personal dashboard empty state — isolated template capture.....240
Figure 44. History empty state — isolated template capture.....241
Figure 45. Analytics empty state — isolated template capture.....242

Figure 46. Account profile display — isolated template capture.....243
Figure 47. Background processing state — isolated template capture.....244
Figure 48. Structured result and output controls — isolated template capture.....245
Figure 49. Unavailable-result recovery state — isolated template capture.....246
Figure 50. Administrator overview — isolated template capture.....247
Figure 51. Administrator audit-log filters — isolated template capture.....248
Figure 52. Assigned evaluation empty state — isolated template capture.....249
Figure 53. LIGHT landing page rendered from source with an anonymous fixture.....293
Figure 54. Sign-in form rendered with an anonymous fixture.....295
Figure 55. Conditional administrator email-code screen using a synthetic pending-login fixture.....297
Figure 56. Registration form rendered without submitting account data.....299
Figure 57. Password-recovery form rendered without sending an email.....301
Figure 58. Password-reset form rendered with no recovery request in progress.....303
Figure 59. Public terms page rendered with an anonymous fixture.....305
Figure 60. Pending-acceptance page using a synthetic registered-account fixture.....306
Figure 61. Summary workspace showing source selection and generation settings.....308
Figure 62. Processing layout using a synthetic pending request; execution was blocked.....313
Figure 63. Result interface with explicitly synthetic summary text, not model output.....316

Figure 64. Unavailable-result state produced with a missing synthetic record.....321
Figure 65. Personal dashboard with an empty synthetic registered-account fixture.....323
Figure 66. History page with no synthetic saved summaries.....325
Figure 67. Analytics page displaying the synthetic empty-data state.....326
Figure 68. Profile page displaying a reserved synthetic account identity.....328
Figure 69. Logout confirmation opened from a synthetic signed-in profile.....330
Figure 70. Administrative console rendered with synthetic administrator access and empty records.....332
Figure 71. Audit-log viewer with a synthetic existing table and no log rows.....336
Figure 72. Assigned-review page with an active synthetic account and no assignments.....338

**Abstract**

LIGHT summarizer is a web application intended to assist users in reviewing lengthy written material through configurable summaries and related reading functions. The researcher-supplied draft identifies students of Nueva Ecija University of Science and Technology, Peñaranda Off-Campus, as intended users and describes an Agile development approach. These statements provide project context; completed institutional research activities and participant evaluation records were not supplied. This manuscript therefore reports the inspected implementation and reproducible technical evidence while presenting the human-evaluation design as proposed.

The technical examination covered PHP application logic, the Python processing engine, public interfaces, database definitions, configuration and operational scripts. Direct metadata queries verified 31 tables, 312 columns and 35 foreign keys in the configured MariaDB database without reading application records. Source inspection identified text, URL, PDF and DOCX input paths; configurable summary modes, depth and presentation; history, feedback, translation, audio, administration and a separate evaluation subsystem. Selected behavior was examined through isolated synthetic fixtures and source-preserving interface rendering.

The evidence establishes implemented workflows and specific limitations. Isolated tests reproduced missing ownership checks in the

document API and incorrect suffix-byte audio delivery. Additional findings include password-reset schema drift, incomplete persistent-token revocation, an alternate administrator status path that bypasses safeguards, and text cleaning that can reject ordinary prose. These findings prevent an unqualified claim of deployment readiness. The manuscript provides corrected architecture documentation, a complete data dictionary, user and maintenance manuals, and proposed technical and end-user instruments. No respondent counts, questionnaire means, acceptance ratings, learning gains or time savings are asserted. Human assessment and institutional completion remain necessary before conclusions about educational usefulness or user acceptance can be drawn.

Keywords: article summarization; natural language processing; software audit; source fidelity; evaluation design; LIGHT summarizer.

# CHAPTER I

# THE PROBLEM AND ITS BACKGROUND

## Introduction

Academic reading requires more than reducing the number of words on a page. A useful summary must identify the central subject, retain important supporting information and preserve the source's qualifications. A shorter passage that changes a numerical result, omits a condition or attributes an assertion to the wrong entity may be easier to scan while being less useful for study. A summarization tool must consequently be examined both as a software application and as a system that transforms information.

The researcher-supplied project draft identifies lengthy academic articles, journals and other readings as the intended materials for LIGHT summarizer. It describes students of Nueva Ecija University of Science and Technology, Peñaranda Off-Campus, as the intended primary users and presents manual identification of important ideas, note-taking and repeated review as motivating concerns. These concerns are the researchers' stated rationale. No interview transcripts, needs-assessment dataset or observed time-and-motion

study was supplied to establish their prevalence at the campus. The present manuscript treats them as a project motivation to be investigated, rather than as measured findings about the student population.

LIGHT summarizer brings article input, summary generation and supporting reading actions into a browser-based workflow. The inspected implementation accepts pasted text, article URLs and supported PDF/DOCX files. It provides analysis profiles, summary-depth choices and output formats, together with features for nutshell explanations, history, feedback, translation and audio. Registered users and guests have different ownership and retention contexts, while administrators have monitoring and account-management functions. These statements describe implemented pathways; availability and correctness still depend on configuration, source material and the limitations documented in Chapter III.

The project addresses two related questions. First, can the implemented software accept, process, store and present information with appropriate functional and access boundaries? Second, do the resulting summaries and interface support the intended reading tasks? Source inspection and isolated tests can

answer parts of the first question. The second requires approved source material, a defined evaluation protocol and actual judgments or task observations. Treating these evidence types separately allows the manuscript to document existing work without inventing student experiences or beneficial outcomes.

## Review of Related Literature and Studies

### Extractive ranking and information selection

Mihalcea and Tarau (2004) introduced TextRank as an unsupervised graph-ranking approach to keyword and sentence extraction. Text units are represented through relationships, allowing their importance to depend on the surrounding graph. This provides a useful conceptual basis for a comparative extractive baseline: sentence selection can be studied without assuming that a generated paraphrase is necessary for every reading task. The original study does not establish the quality of LIGHT's implementation or its suitability for the intended campus. TextRank study.

The inspected software contains a broader set of processing choices than a single named ranking method. Its evaluation subsystem

includes lead-N, TF-IDF and TextRank comparison paths as well as the proposed pipeline and optional hybrid generation. A fair comparison would hold the source documents and relevant length conditions constant and record the specific implementation and configuration. Merely listing multiple algorithms does not show that one produces better summaries. In this project, those comparisons remain an evaluation capability until a controlled run and its evidence are supplied.

**Automatic comparison with reference summaries**

Lin (2004) described ROUGE as a family of automatic summary-comparison measures using overlap with reference summaries, including n-grams and longest common subsequences. Such measures provide reproducible observations about correspondence to chosen references. They require explicit reference provenance and scoring settings; overlap should not be treated as a complete reading-comprehension measure. ROUGE study.

For LIGHT, this literature supports storing reference versions and metric status separately from generated outputs. A missing reference or unavailable model should remain

visible as missing evidence. A score of zero represents a computed result and is therefore different from a computation that could not be performed. The current evaluation design makes that distinction. The manuscript does not provide ROUGE scores because no approved research corpus and completed benchmark results were supplied for this study.

**Faithfulness and factuality**

Maynez, Narayan, Bohnet and McDonald (2020) examined faithfulness in neural abstractive summarization and reported unsupported content in the systems they evaluated. Their analysis also considered the relationship between automatic measures and human judgments. The study supports examining whether a summary is grounded in its source instead of relying on fluency alone. Its reported model outcomes cannot be transferred to LIGHT or used to estimate this application's error rate. Faithfulness and factuality study.

The corresponding concern in LIGHT is preservation of source meaning throughout extraction, cleaning, selection and presentation. A faithful selection cannot be made from information removed incorrectly during preprocessing. Likewise, evidence

labels are useful only when their text and locations remain aligned with the source. The technical audit therefore examines the pipeline before considering aggregate quality claims. Optional synthesis and factuality checks are described according to the configuration and evidence available; their presence in code is not a guarantee that every output is correct.

**Evaluation through complementary evidence**

Fabbri and colleagues (2021) presented SummEval, combining automatic evaluation with expert and crowd-sourced judgments of summarization outputs. Their work highlights the need for explicit, comparable evaluation protocols and human evidence alongside automatic measures. For LIGHT, the relevant implication is methodological: system usability, source fidelity and automatic similarity should be reported as distinct observations. SummEval's datasets and results do not constitute an evaluation of this project. SummEval study.

A user may find an interface convenient while overlooking an omitted qualification. A technical reviewer may identify an authorization defect without judging whether a

summary is suitable for a particular lesson. The proposed protocol consequently separates application-quality questionnaires, source-based summary review and technical tests. The application's ordinary star-rating feedback is also distinct from a controlled study, since its exposure conditions, respondent eligibility and duplicate-response controls may differ from the research protocol.

**Product-quality framework and edition control**

ISO/IEC 25010:2011 described an eight-characteristic product-quality model. ISO records that edition as withdrawn on 4 March 2024; the 2023 product-quality edition has nine characteristics. These edition differences matter when naming an evaluation framework. ISO/IEC 25010:2011 record; ISO/IEC 25010:2023 record.

To preserve the reference manuscript's organization, this draft proposes seven selected domains using the older template's vocabulary: functional suitability, performance efficiency, compatibility, usability, reliability, security and portability. Maintainability is omitted from that selected questionnaire. The items are original project-specific questions, not

reproduced ISO clauses, and this selection is not a complete conformity assessment of either edition. The adviser must approve retention of the historical template or require revision to another framework before administration. Source-code maintainability can still be discussed through the technical audit without pretending that an omitted survey domain was measured.

**Synthesis and project contribution**

The reviewed literature suggests that summary evaluation should consider information selection, reference correspondence and source faithfulness, while software assessment should address the conditions under which users obtain and interpret an output. These perspectives inform the proposed study rather than establishing its results. The bounded contribution documented here is the integration and examination of an existing application: its workflows, processing controls, storage model, interface, operational requirements and reproducible defects. A claim of algorithmic superiority, improved academic performance or campus-wide adoption would require additional evidence.

**Conceptual Framework**

The proposed framework organizes the development and assessment of the intended reading tool. Inputs comprise supported articles and documents, researcher-supplied requirements, implemented source and schema, and the approved criteria needed for future evaluation. The process comprises system refinement, source extraction and NLP summarization, safe technical testing, and human evaluation once its prerequisites are satisfied. Expected outputs comprise summaries and supporting reading features, evidence-grounded system documentation, user and maintenance manuals, and future evaluation findings only when actual records are available. Technical findings and later participant feedback inform further refinement. These are proposed relationships and implemented processing stages, not proof of completed research activities, learning gains or automatic model retraining.

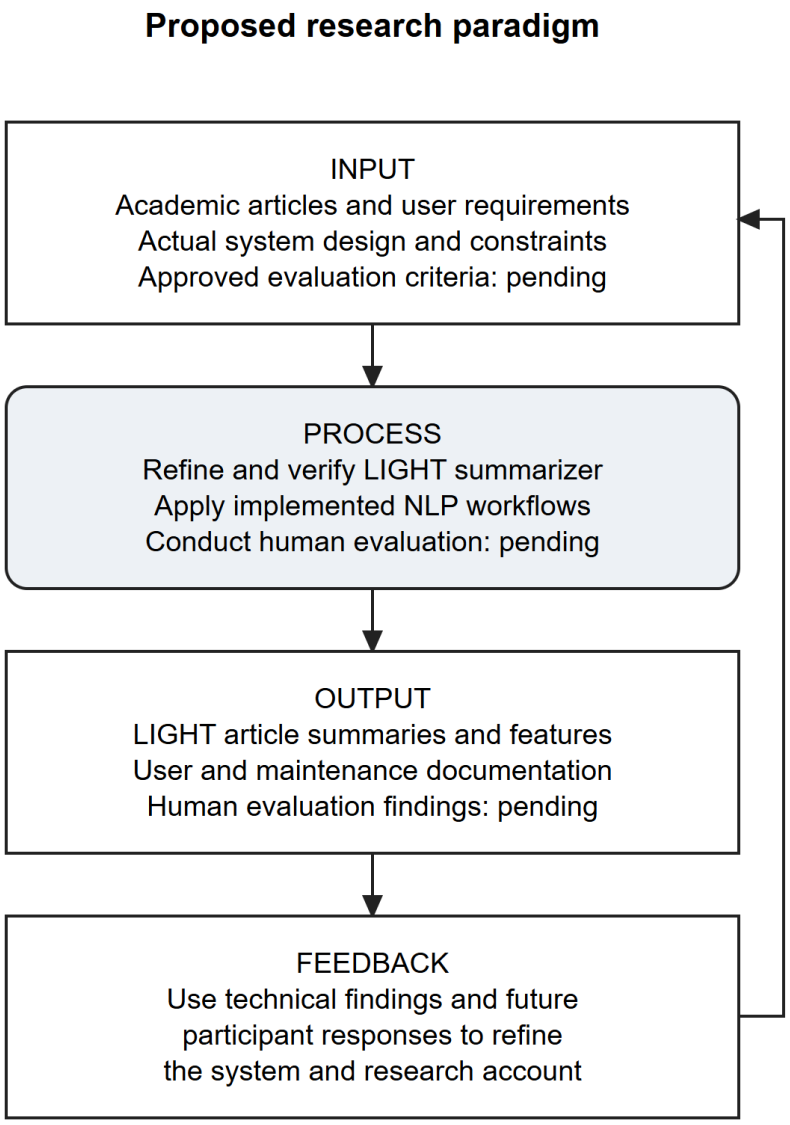



**Figure 1. Proposed research paradigm for LIGHT summarizer.**

The framework in Figure 1 organizes the study's intended inquiry. It does not assert a proven causal relationship between application

use and learning. Software availability, source genre, extraction quality, chosen depth, browser/device conditions and model availability may influence the observed experience. A future effectiveness study would need to control or document these conditions and select an outcome measure that matches its claim.

**Statement of the Problem**

This project concerns the implementation and assessment of LIGHT summarizer as a tool for handling lengthy written materials. The following questions are proposed for researcher and adviser approval; they have not been represented as a previously approved research protocol.

1. What reading and summarization needs should guide the use of LIGHT among the intended users, and which of those needs are supported by documented stakeholder evidence?
2. How does the implemented system acquire articles, generate and present summaries, manage user access and retain supporting records?

3. What functional, processing, security and reliability limitations can be established through source inspection, schema verification and isolated testing?
4. How would qualified technical evaluators assess the system under the seven selected application-quality domains after the required safeguards and evaluation conditions are established?
5. How would intended end users assess functional suitability, perceived performance and usability after completing approved representative tasks?
6. What improvements are warranted by the combined technical findings and, when available, independently collected human evaluation evidence?

The fourth and fifth questions remain unanswered in the absence of actual respondents and response records. They are included to complete the proposed study design, not to imply that a questionnaire has been administered.

**Objectives of the Study**

The general objective is to document and assess LIGHT summarizer as an implemented

article-processing application while establishing a defensible plan for evaluating its intended use.

The specific objectives are to identify source-supported requirements and outstanding stakeholder information; map actual roles, functions, data flows and storage; verify selected behaviors using controlled evidence; document known defects and operational constraints; prepare project-specific technical and end-user instruments for review; and derive recommendations whose strength is limited by the available evidence. Measuring learning improvement, comparative reading time or adoption is outside these objectives unless the researchers approve an additional study design with appropriate records.

**Scope and Delimitation**

The technical scope covers the supplied PHP/Python application, its public interface files, configuration, checked-in database scripts, live schema metadata and operational utilities as examined on 17 October 2026. It includes guest and account contexts, main summarization, result actions, history and feedback, administrator features, document-combination caching and the separate

evaluation subsystem. The database description uses the actual summaries, summary_artifacts, feedback and related tables; older draft labels such as summary_records and feedback_records are not actual schema entities.

The main pipeline supports eight analysis modes, six depth values and five output formats. The document-combination/evaluation path uses five lengths. These are interface or configuration options, not promises of exact sentence/word counts for every source. Extraction quality may depend on the document structure and available libraries. Scanned-document recognition, complex layout interpretation, translation quality and voice-model availability must not be assumed from file acceptance alone.

The audit preserved the application and production data. It used direct metadata queries and synthetic isolated checks; it did not conduct live destructive testing, read private database records or execute real account, email or cleanup workflows. A source-preserving interface rendering is not proof of authenticated database integration. Optional models were not treated as active merely because related code exists. Known access-

control, installation, lifecycle and preprocessing defects delimit the conclusions.

The intended locale comes from the researcher draft. The population, sample size, recruitment method, actual study dates and institutional approvals remain to be confirmed. No real human response dataset, controlled educational experiment or verified production deployment was supplied. The manuscript therefore contains proposed research instruments and blank institutional forms, not fabricated evaluation tables or completion certificates.

**Significance of the Study**

For intended student users, the project offers a means of organizing article input and accessing shorter views alongside supporting reading functions. Its potential value is assistance with review; improved comprehension, reduced workload and learning gains remain outcomes to be investigated. Users must still compare important assertions with the original material.

For instructors and researchers, a documented processing and evaluation workflow can help distinguish an output's convenience

from its source fidelity. Versioned references and clearly stated review criteria could support future assessment if an approved corpus and protocol are established. They do not replace academic judgment or source citation.

For administrators and maintainers, the audit supplies an actual schema inventory, workflow descriptions, access-control limitations and operational procedures. This evidence can guide prioritization of corrective work and a safer deployment decision. For future developers, the preserved distinction between existing code, tested behavior and proposed activities provides a traceable starting point for implementation and research replication.

**Definition of Terms**

**Analysis profile.** A selectable emphasis, such as general, academic or technical, passed to the summary-processing logic. A profile is not a separately trained model established by this audit.

**Article source.** The permitted text, URL content or supported uploaded document supplied for processing.

**Extractive summary.** A summary formed primarily by selecting information units from the source. In this manuscript, use of the term does not guarantee perfect preservation of context.

**Source faithfulness.** Agreement between an output's assertions and the information supported by its source, including conditions, entities and numerical details.

**Summary depth.** A requested level of detail. The main application's allowed values differ from those of the standalone combination/evaluation path.

**Nutshell.** The application's concise explanatory output feature, distinct from the persisted main-summary workflow.

**Guest context.** Access associated with a session-held guest token instead of a registered account; it is not a third database user role.

**Recoverable job.** A stored processing request with state, ownership and bounded recovery information that supports progress tracking and certain retry/cancel actions.

**Artifact.** Stored source or generated content associated with a summary record.

**Evaluator.** An eligible account assigned to review specified outputs. Assignment does not create a separate role value in users.role.

**Technical audit.** Examination of implementation evidence and selected controlled behavior. It is distinct from respondent opinion and from a completed penetration-test certification.

**Source-supported.** A statement present in supplied project material or inspected code whose broader historical or operational claim may still require verification.

**Pending evaluation.** A planned assessment for which actual response or measurement evidence has not been supplied.

# CHAPTER II

# METHODS AND PROCEDURES

## Research Design

This manuscript combines a documented examination of the existing software with a proposed descriptive evaluation of its intended use. The completed technical work concerns source structure, database metadata, controlled behavior and interface presentation. The proposed human component concerns technical reviewers' and intended users' assessments under stated tasks and conditions. These components answer different questions and require separate evidence. An automated test is not a respondent, a rendered page is not proof of a completed user session, and a service method is not proof that a research procedure occurred.

The researcher-supplied draft describes development through an Agile Development Life Cycle and includes a schedule spanning April to October 2026. This is a supplied account of the intended development approach. Dated sprint records, backlog revisions, review minutes, stakeholder consultations and acceptance records were not supplied. The manuscript can therefore describe the draft's

stage organization and the implementation available for inspection, but cannot verify historical adherence to Agile practices or mark scheduled activities as completed.

The proposed assessment is descriptive rather than a controlled causal experiment. It is designed to report observed software behavior and judgments following representative tasks. It cannot establish that LIGHT improves examination scores, causes better comprehension or reduces reading time without an additional approved design, appropriate comparison conditions and actual measurements. The questions and instruments remain subject to adviser and institutional review.

**Table 1. Research questions, evidence sources and limits of inference.**

| Inquiry | Evidence or proposed method | Permitted interpretation |
|---|---|---|
| Intended needs | Researcher draft; future approved stakeholder inquiry | Draft motivation is known; prevalence and priority of needs remain unmeasured. |
| Implemented functions and storage | Source inspection, configuration, route mapping and read-only schema metadata | Describes observed implementation and constraints. |

| Inquiry | Evidence or proposed method | Permitted interpretation |
|---|---|---|
| Functional and technical limitations | Isolated synthetic fixtures, automated tests and source-preserving interface examination | Establishes specific tested behavior under recorded conditions. |
| Technical reviewer assessment | Proposed Appendix H after approved staged tasks | Describes eligible reviewers' judgments; does not certify security or standard conformance. |
| End-user assessment | Proposed Appendix I after representative tasks | Describes participants' experience within the sample and conditions. |
| Improvements | Traceable synthesis of defects and future response evidence | Supports prioritized recommendations without inventing unobserved benefits. |

Table 1 links each inquiry to the evidence needed to answer it. The distinction prevents source inspection and synthetic tests from being presented as student or consultant responses. It also identifies the questions that remain open until an approved evaluation is completed.

## Development Approach Described in the Researcher Draft

### Planning and analysis

The supplied draft associates planning with identifying academic-reading concerns, defining scope and objectives, and selecting features such as adjustable summaries, nutshell output, Filipino translation, audio, history and feedback. Analysis is presented as the clarification of requirements and feature relationships. In the present documentation, these statements are reconciled with the available application. Features are described from their actual entry points, processing paths and storage rather than presumed from a planned diagram.

### Design

The draft includes use cases and data-flow diagrams, but some flows are incomplete or reversed and certain store names do not match the database. This manuscript uses diagrams derived from the audited source and actual schema. Guest, registered-user, administrator and assigned-evaluator contexts are distinguished where their permissions differ. The ERD shows actual foreign keys, while

explanatory notes identify important logical links that are not enforced constraints. Reconstructing current design does not establish the chronological sequence in which the researchers originally normalized or implemented it.

**Implementation**

Implementation is evidenced by the PHP application, Python engine, SQL definitions, assets and operational scripts supplied in the workspace. The main PHP workflow validates a request, records a processing job, invokes local processing, stores an output and serves result/history actions. The database and service layers also support document-combination caching, feedback and evaluation records. Optional processing branches are identified separately from configured defaults. Code and runtime configuration form the basis of the technical description; the older project's narrative does not override them.

**Testing, integration and maintenance**

The supplied schedule includes researcher tests, end-user evaluation, consultant evaluation and maintenance activities. The

technical audit provides its own dated isolated test evidence, which must not be retroactively represented as the researchers' historical tests. Likewise, source code for an evaluation system does not establish that an evaluation was conducted. Maintenance scripts and a worker monitor are implemented, while an automatic backup program and completed restore drill were not verified. Appendix G separates available operations from proposed practice.

**Supplied development schedule**

The following table transcribes the 21 activities and colored intervals in the supplied Gantt image. Its four numbered slots per month do not specify calendar-day boundaries. The intervals are retained as provisional month/week labels, without invented dates or completion percentages. Confirmation is required whether this is an original plan, revised plan or as-completed record.

**Table 2. Researcher-supplied provisional Gantt schedule, April–October 2026.**

| No. | Stage | Activity | Supplied interval |
|---|---|---|---|
| 1 | Planning | Brainstorming | April weeks 1–2 |

| No. | Stage | Activity | Supplied interval |
|---|---|---|---|
| 2 | Planning | Data Gathering | April weeks 3 –4 |
| 3 | Planning | Determining Data Requirements | May weeks 1–2 |
| 4 | Planning | Defining System Scope and Objectives | May weeks 3–4 |
| 5 | Analysis | Gathering Requirements | May week 1– June week 1 |
| 6 | Analysis | Defining Features | May week 4– June week 2 |
| 7 | Analysis | Defining the System Objectives | June weeks 2–4 |
| 8 | Design | Creating System Models | May week 4– June week 4 |
| 9 | Design | Structuring the Front-End | June week 2– July week 4 |
| 10 | Design | Planning the Back-End Interface | July week 2– August week 2 |
| 11 | Implementation | Creating System Flow | June week 4– July week 4 |
| 12 | Implementation | Choosing Database | July week 2– August week 3 |
| 13 | Implementation | Structuring Tables and Variables | August week 1 –September week 2 |
| 14 | Implementation | Coding of the System | August week 2 –September week 2 |
| 15 | Testing and integration | Finalizing the System Functionalities | August week 4 –September week 3 |

| No. | Stage | Activity | Supplied interval |
|---|---|---|---|
| 16 | Testing and integration | System Testing by Researchers | September week 3–October week 1 |
| 17 | Testing and integration | Evaluation by End-Users | October weeks 1–2 |
| 18 | Testing and integration | Evaluation of IT Consultants | October weeks 2–3 |
| 19 | Maintenance | Improving the System Performance | October weeks 1–2 |
| 20 | Maintenance | Fixing Identified Issues | October weeks 2–3 |
| 21 | Maintenance | Continuous System Backup | October weeks 3–4 |

Table 2 is evidence of the researchers' supplied planning artifact. It is not evidence of respondent participation, actual consultant ratings, completed fixes or backup execution. Supporting records are listed in the research-information register.

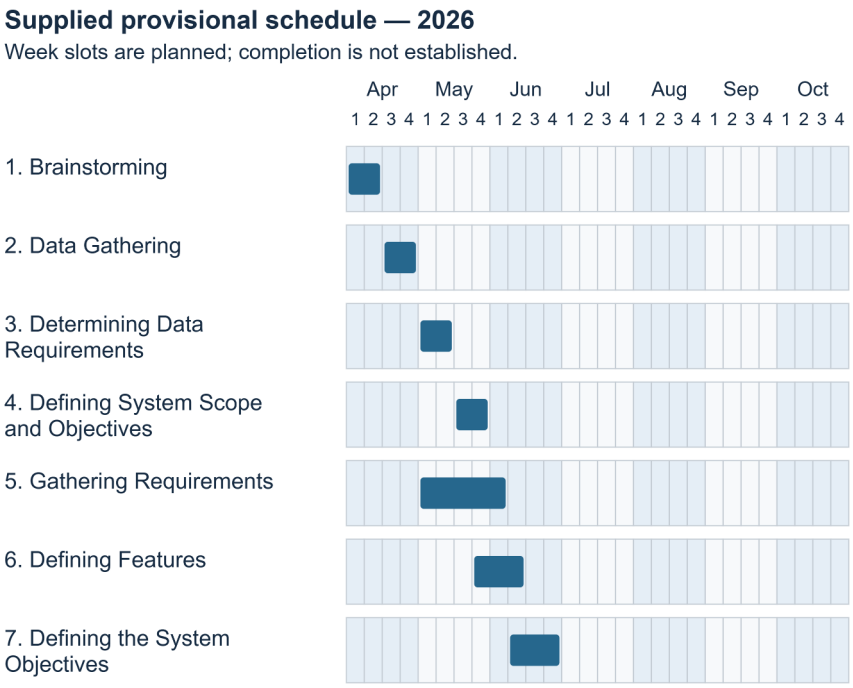



**Figure 2. Researcher-supplied provisional schedule, activities 1 to 7.**

Figure 2 illustrates the supplied month and week slots for these activities. The chart is an original redraw of the researcher draft, not a record of completed work. No exact calendar dates, participant events or evaluation results are inferred from its shaded intervals.

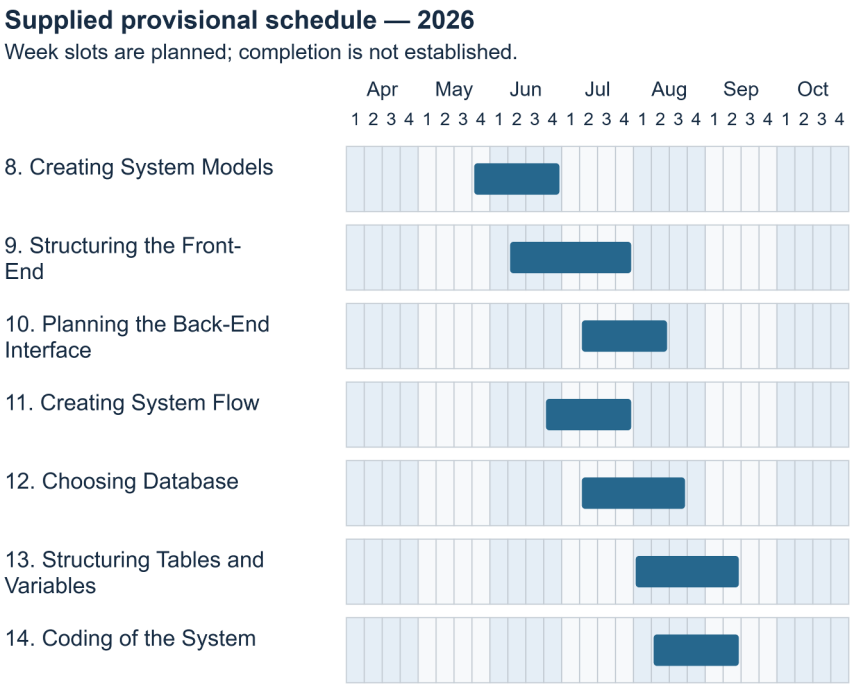



**Figure 3. Researcher-supplied provisional schedule, activities 8 to 14.**

Figure 3 illustrates the supplied month and week slots for these activities. The chart is an original redraw of the researcher draft, not a record of completed work. No exact calendar dates, participant events or evaluation results are inferred from its shaded intervals.

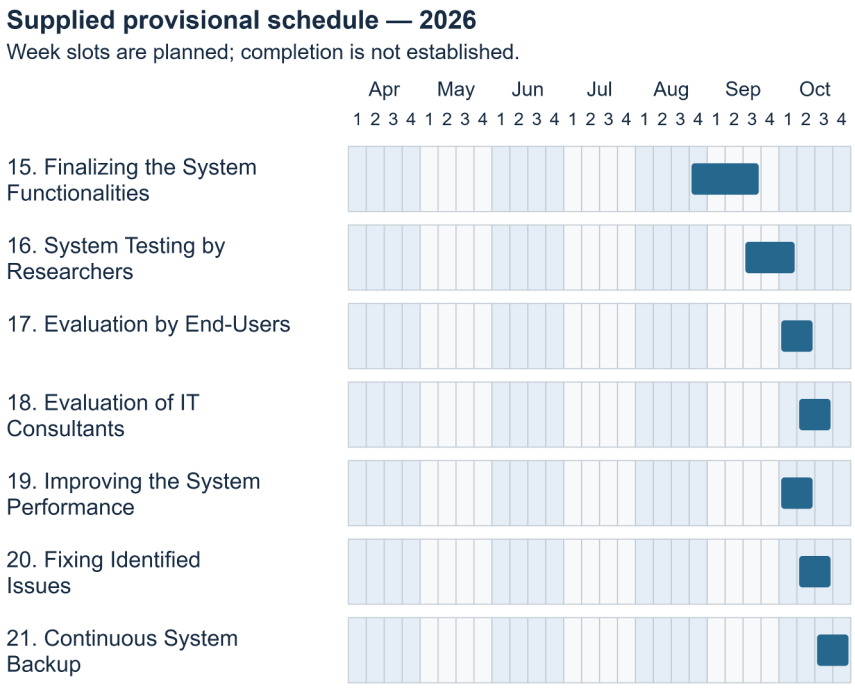



**Figure 4. Researcher-supplied provisional schedule, activities 15 to 21.**

Figure 4 illustrates the supplied month and week slots for these activities. The chart is an original redraw of the researcher draft, not a record of completed work. No exact calendar dates, participant events or evaluation results are inferred from its shaded intervals.

## Technical Examination Procedure

The technical examination began with study of the full reference manuscript's structure and formatting, followed by inspection of the

supplied application. Source files, routes, service responsibilities, dependencies and configuration were mapped before manuscript claims were drafted. The reference's institutional records, survey outcomes and project-specific statements were excluded from the LIGHT evidence base.

Database inspection used direct PDO queries for table, column, index, check and foreign-key metadata. It did not load the web bootstrap or retrieve user records. This distinction was necessary because ordinary application startup can touch guest records and occasionally purge expired data. The resulting snapshot verified 31 tables, 312 columns and 35 foreign keys. Comparison with the base schema and migrations identified differences between the configured database and a fresh-install definition.

Executable checks used isolated copies, synthetic text or files, and in-memory test storage where appropriate. The actual document controller/service was exercised against a synthetic SQLite database to test owner boundaries. Audio range handling used a synthetic file. The Python suite and selected smoke checks ran in an isolated environment with the recorded dependency and optional-

model limitations. Interface capture used an isolated rendering arrangement whose scope is described in the UI audit. Production account actions, email transmission, cleanup, migrations and live content-processing requests were not performed.

Each reported technical result distinguishes expected behavior, observed behavior, status and evidence location. Source-supported capability and untested integration are labelled separately. The audit's failures are retained rather than excluded to create a favorable success rate. Chapter III reports the actual results; the method does not convert them into human usability ratings.

**Research Locale**

The researcher draft identifies Nueva Ecija University of Science and Technology, Peñaranda Off-Campus, as the intended setting and identifies students as intended primary users. This is the supplied locale statement. The exact academic unit, program, study population, venue, contact official and permission reference remain [REQUIRES RESEARCHER CONFIRMATION]. No campus event, installation, demonstration or data-collection session is asserted.

The technical audit occurred in the supplied local Windows/XAMPP workspace. That computing environment is an implementation context, not evidence that the campus hosted the application. A campus photograph or location map should be inserted only from an authentic authorized source and labelled according to its actual purpose; none is fabricated here.

**Research Respondents**

Two proposed groups address application quality from different perspectives. Technical reviewers should have relevant software-development, testing, deployment or information-systems experience sufficient for the tasks they are asked to judge. Intended end users should belong to the confirmed target population and have a legitimate need to read the selected materials. The institution must approve eligibility, recruitment and any age-related consent requirements before participation.

**Table 3. Proposed respondent groups and outstanding sampling information.**

| Group | Proposed role in the study | Target number | Actual valid responses |
|---|---|---|---|
| IT consultants or qualified technical reviewers | Review seven selected application-quality domains using staged evidence and tasks | REQUIRES APPROVAL | NOT PROVIDED |
| Intended student/end-user group | Review functional suitability, perceived performance and usability after tasks | REQUIRES APPROVAL | NOT PROVIDED |

Table 3 keeps the two proposed groups separate and leaves their numbers unresolved. The groups must not be combined into an invented total. The number of researchers is not the sample size. Application accounts, feedback rows, test cases and browser screenshots are not substitute respondent counts. No demographic distribution or response rate is computed in this draft.

## Sampling Procedure

Purposive selection of technically qualified reviewers is proposed because the instrument requires relevant competence. For end users, the researchers should first obtain the eligible population frame and choose an

institution-approved recruitment method that matches the desired scope of inference. If practical recruitment uses a nonprobability volunteer sample, the resulting limitations must be stated; its findings must not be presented as statistically representative of all campus students.

No numerical sample-size formula is applied without a population, target precision, design assumptions and actual recruitment plan. The researchers must document invitations, eligibility checks, refusals/withdrawals and usable responses without publishing unnecessary identifiers. Participation should be voluntary and should not affect grades or access to ordinary services. The final sampling procedure must describe what actually happened, including deviations from the plan.

**Research Instruments**

Appendix H contains 28 proposed technical-review items: four each for functional suitability, performance efficiency, compatibility, usability, reliability, security and portability. Appendix I contains 12 proposed end-user items: four each for functional suitability, perceived performance and usability. These are original project-

specific items organized using selected domains from the older reference template. Maintainability is excluded, so the seven-domain questionnaire is not the complete ISO/IEC 25010:2011 model. The withdrawn 2011 edition and revised 2023 edition must not be conflated. Official 2011 record; official 2023 record.

Both proposed questionnaires use the same four-point agreement scale, with a separate not-observed/not-applicable response. The application itself has ordinary 1–5 feedback and a separate 1–5 blind-summary rubric. Those built-in records are different instruments and must not be merged numerically with the proposed four-point questionnaires. Source-fidelity review, if undertaken through the evaluation module, requires its own corpus, rubric, evaluator training and analysis specification.

**Table 4. Proposed questionnaire response coding.**

| Response | Numeric value | Meaning |
|---|---|---|
| Strongly Disagree | 1 | The statement clearly does not describe the observed experience/evidence. |

| Response | Numeric value | Meaning |
|---|---|---|
| Disagree | 2 | The statement generally does not describe the observed experience/evidence. |
| Agree | 3 | The statement generally describes the observed experience/evidence. |
| Strongly Agree | 4 | The statement clearly describes the observed experience/evidence. |
| Not observed / Not applicable | No numeric value | The respondent lacks the required exposure or the item does not apply; reason requested. |

Table 4 assigns numeric values only to observed agreement responses. An observed failure should receive an appropriate disagreement response and an explanatory note, not be removed as “not applicable.” Unobserved security or recovery properties should not be guessed. Questionnaire review and pilot testing are proposed before use; no content-validity score, reliability coefficient or pilot outcome is supplied. Item revisions should be versioned and completed before final collection.

**Proposed Data-Gathering Procedure**

First, the researchers should finalize the study questions, eligibility criteria, source-material permissions, privacy arrangements and instruments with the adviser. They should obtain the appropriate institutional approval and participant information/consent materials before recruitment. Copies or references belong in the supporting appendices. This draft's blank letters are not evidence of approval.

Second, the researchers should address the known access-control and processing defects or establish a reviewed restricted evaluation environment. The environment must use staging-controlled accounts and permitted nonconfidential documents. Record the code version, engine setting, model availability, browser, device and network conditions. A successful demonstration must not be inferred from a screenshot or planned agenda.

Third, participants should receive consistent task instructions without suggestions about desired ratings. Representative tasks may include submitting a prepared text, changing depth/format, comparing the output with the source, locating

history, trying a supported file, and using available reading actions. Failed or unavailable actions must be recorded as such. Technical reviewers may additionally inspect staged ownership denials, errors, installation and recovery evidence; real private records must not be exposed for evaluation.

Fourth, each respondent should complete the approved questionnaire independently, using a study code instead of unnecessary personal identifiers. Record which tasks were attempted and whether help was required. A separate optional comment field can capture specific difficulties. Store consent/identity linkage apart from coded responses, with an approved access and retention arrangement. Record actual collection dates and deviations.

Finally, the researchers should validate the response file for duplicate codes, out-of-range values, omissions and documented withdrawals. Keep the raw responses protected and maintain a reproducible analysis copy with a codebook. Present unresolved missing data and negative findings. Institutional closure, participant communication and final retention/disposal should follow the approved protocol. None of these human-collection steps

is represented as completed in this manuscript.

**Data Analysis Technique**

**Technical findings**

Technical cases are classified as passed, failed, skipped, blocked or not tested according to recorded execution. The denominator for a pass rate must state whether skipped or unexecuted cases are excluded. Source-only observations are not added to executed-case totals. Findings are grouped by workflow and consequence, with reproduction/evidence references. Timings, if reported, must identify hardware, input, operation and repetition; configured timeout values are not observed latency.

**Questionnaire responses**

For item j, let f1j, f2j, f3j and f4j denote the valid response frequencies at values 1 through 4. Let n_j = f1j + f2j + f3j + f4j. The proposed descriptive weighted mean is:

WM_j = (1f1j + 2f2j + 3f3j + 4f4j) / n_j.

If n_j is zero, report “not estimable,” not zero. Blank, withdrawn and not-observed responses are excluded from the numeric

denominator and reported separately. The weighted mean is a descriptive convention for the ordered response scale, not proof that the underlying intervals are equal or that a quality standard has been satisfied. Frequencies and valid counts should accompany it; medians may be included to make the ordinal distribution visible.

For a domain with four items, the proposed respondent-level domain score is the arithmetic mean of answered items only when at least three have valid numeric responses. Report the number of qualifying respondents. A group domain mean averages those respondent-level scores, giving each qualifying respondent equal weight. If an overall summary is approved, average available domain means with equal domain weights only when every required domain has an estimable mean, and disclose differing respondent coverage. Keep technical and end-user groups separate.

**Table 5. Proposed descriptive interpretation intervals for questionnaire means.**

| Unrounded mean interval | Interpretation |
|---|---|
| 1.00 to 1.75 inclusive | Strongly Disagree |
| Greater than 1.75 to 2.50 inclusive | Disagree |
| Greater than 2.50 to 3.25 inclusive | Agree |

| Unrounded mean interval | Interpretation |
|---|---|
| Greater than 3.25 to 4.00 inclusive | Strongly Agree |

Table 5 provides nonoverlapping descriptive intervals. Apply the interval before rounding for display so boundary values are not reassigned by formatting. These labels are proposed analysis conventions, not ISO-prescribed acceptance thresholds. An “Agree” result cannot override a reproduced authorization failure or establish actual confidentiality. No means or interpretations are calculated until genuine response data exist.

**Summary quality and comments**

Any later source-based evaluation should preserve reference provenance, document split, profile/depth, system version and metric implementation. Report automatic scores separately from human criteria and include failures/missing measures. The reviewed evaluation literature motivates this separation but does not supply LIGHT's outcomes. Comments may be organized into transparent descriptive themes with supporting de-identified examples; report how coding was performed and avoid identifying participants through quotations. No inferential

significance test or effectiveness claim is proposed without a suitable design and dataset.

**Ethical and Methodological Boundaries**

The study must use permitted source materials, voluntary participation and approved handling of personal information. Translation or optional remote processing can create additional data-transfer considerations that should be disclosed before use. The technical audit's synthetic fixtures avoid substituting real private records for test content. The manuscript preserves failures, unsupported historical claims and missing evidence explicitly. Final conclusions must reflect the actual approved protocol and collected data, including any limitations that remain after corrective work.

# CHAPTER III

# RESULTS AND DISCUSSION

This chapter presents the implemented LIGHT summarizer and the evidence obtained during the technical audit on 17 October 2026. It distinguishes three kinds of information: needs and development intentions described in the researchers' supplied draft; implementation supported by source code and database metadata; and behavior observed in bounded tests. No completed consultant or end-user questionnaire responses were supplied. Consequently, the human-evaluation sections contain pending tables and cannot establish acceptance, learning benefits or compliance with a quality standard.

## 1. Problems Encountered in the Existing System

The researchers' draft identifies a practical reading problem: students face extensive academic material and may have difficulty selecting main ideas, preserving important details and allocating reading time. Its intended setting is the NEUST Peñaranda Off-Campus academic community. These statements provide the project's motivation; the available records do not establish how

often the problems occurred, how many people experienced them, or what interviews or observations produced them. The term existing system therefore refers to the described reading and summarization practice, rather than to a verified earlier software product.

The audit also found implementation defects. Those defects concern the developed software and are discussed under testing and limitations. They are not substituted for the original research problem. A missing ownership check, for example, is an engineering finding; it is not evidence that students previously experienced a reading difficulty.

## 2. How the Developed System Addresses the Identified Problems

The application brings several reading-support activities into one interface. A user can supply supported material, choose a summary organization and depth, and inspect a result with source-related detail. Registered users have additional history and account functions. Administrators have account, feedback, storage and activity views. These functions provide mechanisms through which the proposed needs may be addressed, but their presence does not establish that users read

faster, understand more, or prefer the application to another method.

**Table 6. Relationship between supplied project needs and implemented capabilities.**

| Supplied need | Implemented response | Evidence and boundary |
|---|---|---|
| Difficulty identifying main ideas | Source extraction, sentence selection, modes, depths and result organization | Source traced; one legacy fixture returned a summary; over-filtering and omission defects remain |
| Extensive material in different forms | PDF/DOCX, pasted text and article URL intake | Interface/ source supported; representative live uploads and external URLs not submitted |
| Need to revisit useful material | Summary history, result access, text export and clipboard copy | Copy and TXT export passed with synthetic content; saved history was not exercised |
| Need for alternative ways to read output | Filipino translation, audio and concise Nutshell paths | Implemented; external translation, speech quality and successful Nutshell web generation unverified |
| Need to manage use and collect feedback | Role-restricted administration, ratings, comments and analytics | Source supported; synthetic interfaces only; telemetry is not a research survey |

Table 6 relates the draft’s needs to the actual implementation without treating an expected benefit as a measured outcome. The evidence column limits each claim to the work performed. Evaluation by the intended participants is still needed to determine whether the functions address their needs in practice.

## 3. Essential Features and Functionalities

The following inventory describes each distinct capability in language intended for readers who are not software developers. Each identifier corresponds to the full evidence, file, function, input and database record in FEATURE_INVENTORY.csv. “Implemented but not verified” means implementation was inspected without completing its operational workflow. “Partially working” identifies an implemented facility with a supported limitation. “Verified working” applies only to the narrow action expressly tested. Benefits describe intended use rather than measured learning or productivity gains.

### Entry

**F001. Landing information and navigation.** The feature allows visitor to learn the

purpose and open the summarizer or account forms. The intended output is Landing sections; target page. It connects with navigation and result presentation.

The interaction proceeds as follows: open the landing page; then read about light and use start/summarize; then use footer terms or navigation account links. Inputs are Navigation links; scroll. Behind the interface, the implementation performs Render project information and navigation links.

The audit classification is implemented but not verified. Admin landing requests redirect; footer social links lead to generic social-platform homepages, not verified project accounts. Evidence is recorded under F001 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**Navigation**

**F002. Responsive mobile menu.** The feature allows visitor to open and close site navigation on small screens. The intended output is Expanded navigation with aria-expanded state. It connects with navigation and result presentation.

The interaction proceeds as follows: press the menu button; then choose a link or press escape. Inputs are Menu button; Escape. Behind the interface, the implementation performs Toggle menu visibility and aria-expanded; close on Escape.

The audit classification is verified working. Opening and Escape verified at 390px; complete keyboard/screen-reader audit not performed. Evidence is recorded under F002 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**Appearance**

**F003. Light/dark theme preference.** The feature allows visitor to switch the reading theme. The intended output is Light or dark document data-theme. It connects with navigation and result presentation.

The interaction proceeds as follows: use the theme control; then revisit a supported page to reuse preference. Inputs are Theme button; saved localStorage preference. Behind the interface, the implementation performs Toggle supported page theme and retain browser preference.

The audit classification is verified working. Source inspection; theme persists locally, not as a saved account preference. Evidence is recorded under F003 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**Accounts**

**F004. Create account.** The feature allows visitor to register with an email and password. The intended output is Validation feedback or account/session redirect. It connects with account registration.

The interaction proceeds as follows: open create account; then enter the required fields; then review and accept terms; then submit create account. Inputs are Username; email; password; confirmation; terms acceptance; CSRF. Behind the interface, the implementation performs Validate account inputs and password policy; check uniqueness; create the account.

The audit classification is implemented but not verified. No account was created. Registration form copy mentions custom profiles; dedicated editable saved-profile UI was not found. Evidence is recorded under F004

in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F005. Sign in and remember session.** The feature allows registered user to sign in to account features. The intended output is Session or validation error. It connects with login and logout; remembered login.

The interaction proceeds as follows: enter your username/email and password; then choose remember-me only on a trusted device if desired; then submit sign-in. Inputs are Username or email; password; optional remember-me; CSRF. Behind the interface, the implementation performs Check credentials and status; regenerate session; optionally issue or rotate a remembered user token.

The audit classification is implemented but not verified. Credential and remember-cookie flows not exercised. Evidence is recorded under F005 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F006. Administrator email code verification.** The feature allows administrator to complete the additional administrator login step. The intended output is Verified login or code

error. It connects with administrator email otp.

The interaction proceeds as follows: sign in with administrator credentials; then enter the emailed one-time code; then submit verification or cancel to sign in again. Inputs are Pending admin session; emailed code; CSRF. Behind the interface, the implementation performs Create pending administrator-code session state; send and verify time-limited code; rate-limit attempts; record security activity.

The audit classification is implemented but not verified. Source-supported conditional UI only; no email sent; retired challenge routes are redirects, not an active challenge feature. Evidence is recorded under F006 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F007. Password recovery.** The feature allows account holder to request a password-reset code. The intended output is Generic recovery outcome and reset flow. It connects with password reset.

The interaction proceeds as follows: open recover password; then enter email; then

choose send otp and follow the response. Inputs are Email; CSRF. Behind the interface, the implementation performs Locate eligible account; issue a time-limited reset code; request email delivery.

The audit classification is partially working. Recovery form rendered; sending, delivery and account existence outcomes not tested. Reset-token storage differs between the live database and clean-install SQL; email delivery was not exercised. Evidence is recorded under F007 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F008. Reset password using OTP.** The feature allows account holder to replace a forgotten password after verification. The intended output is Reset success or validation/code error. It connects with password reset.

The interaction proceeds as follows: open reset form after requesting a code; then enter email and code; then enter matching new passwords; then submit reset. Inputs are Email; OTP; new password; confirmation; CSRF. Behind the interface, the implementation performs Validate reset code and new password; replace password hash and consume reset token.

The audit classification is partially working. No password changed. A password change does not revoke existing remembered tokens or active sessions; the fresh-install reset schema also differs. Evidence is recorded under F008 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F009. Password visibility and strength hints.** The feature allows visitor to review password entry before submitting. The intended output is Plain/masked password; client strength/match hints. It connects with registration and password reset.

The interaction proceeds as follows: enter a password; then use eye button to toggle visibility; then read available strength/mismatch hint. Inputs are Password input and visibility button. Behind the interface, the implementation performs Toggle the password input masking and display local password-strength hints.

The audit classification is partially working. Click toggle verified on registration; eye buttons tabindex=-1 excludes keyboard Tab navigation. The visibility control responds to clicks but uses

tabindex=-1, excluding normal keyboard tab access. Evidence is recorded under F009 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**Terms**

**F010. Read and accept terms.** The feature allows visitor or registered user to review usage conditions and continue after agreement. The intended output is Session/account acceptance or exit/login redirect. It connects with terms acceptance.

The interaction proceeds as follows: open terms and conditions; then read the sections; then check agreement and confirm; choose decline if not agreeing. Inputs are Acceptance checkbox; accept/decline action; CSRF. Behind the interface, the implementation performs Display the current terms and record an authenticated acceptance.

The audit classification is partially working. Reading rendered; acceptance/decline not executed; two terms pages have different guest actions. The decline link does not submit the server-side decline action; terms enforcement also depends on the route

allowlist. Evidence is recorded under F010 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**Summary input**

**F011. Upload PDF or DOCX.** The feature allows guest or registered user to supply a supported local document. The intended output is Selected filename or input error. It connects with input and preference normalization; upload acceptance.

The interaction proceeds as follows: choose upload file; then browse or drag a file; then use replace/remove to correct the selection. Inputs are One PDF/DOCX; optional title. Behind the interface, the implementation performs Select PDF/DOCX; on submission validate multipart origin, extension, MIME, signature and package structure; store a randomized upload file.

The audit classification is implemented but not verified. No file uploaded. Effective limit depends on PHP/config; renderer displayed 2.0 MiB for its PHP environment. Evidence is recorded under F011 in the feature

inventory; a template capture or function test does not verify the full saved workflow.

**F012. Paste text source.** The feature allows guest or registered user to summarize readable pasted text. The intended output is Text selected as source. It connects with input and preference normalization.

The interaction proceeds as follows: choose paste text; then paste permitted document text; then review source and settings. Inputs are Text up to server-provided byte cap. Behind the interface, the implementation performs Accept pasted text; on submission normalize and validate source length.

The audit classification is implemented but not verified. Source-selector interaction verified; extraction/model processing not run. Evidence is recorded under F012 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F013. Article URL source.** The feature allows guest or registered user to request summarization of an accessible article URL. The intended output is URL source or validation error. It connects with input and preference normalization; url acquisition.

The interaction proceeds as follows: choose url; then paste the complete public address; then submit when source/settings are ready. Inputs are HTTP(S) URL without embedded credentials. Behind the interface, the implementation performs Accept an HTTP(S) article address; validate destination and redirects; acquire bounded source content.

The audit classification is implemented but not verified. Selection and invalid-empty-URL rejection verified locally; fetching and safety policy not exercised. Evidence is recorded under F013 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**Summary controls**

**F014. Optional document title and focus.** The feature allows guest or registered user to name a result and prioritize a topic/question. The intended output is Selected title/focus in request. It connects with input and preference normalization.

The interaction proceeds as follows: optionally enter document title; then optionally enter focus / topic / question. Inputs are Optional title; optional focus text

(UI maxlength 300). Behind the interface, the implementation performs Accept optional title and focus and validate their length for the processing request.

The audit classification is implemented but not verified. Focus changes requested prioritization; cannot promise answer accuracy. Evidence is recorded under F014 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F015. Five output organizations.** The feature allows guest or registered user to choose paragraph, bullets, hybrid, structured sections or key takeaways. The intended output is Updated description and request setting. It connects with input and preference normalization.

The interaction proceeds as follows: open output format; then choose one format and read its description. Inputs are output_format select. Behind the interface, the implementation performs Select one of five output organizations and pass its validated value to generation.

The audit classification is implemented but not verified. Structured selection verified;

output quality not evaluated. Evidence is recorded under F015 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F016. Eight analysis modes.** The feature allows guest or registered user to choose general, academic, executive, technical, study, news, legal or scientific emphasis. The intended output is Updated mode description and request setting. It connects with input and preference normalization.

The interaction proceeds as follows: open analysis mode; then choose the intended emphasis. Inputs are analysis_mode select. Behind the interface, the implementation performs Select one of eight analysis modes and pass its validated value to generation.

The audit classification is implemented but not verified. Academic selection verified; modes do not establish domain correctness. Evidence is recorded under F016 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F017. Six summary depths.** The feature allows guest or registered user to choose very brief, brief, short, balanced, detailed or comprehensive depth. The intended output is

aria-pressed and hidden depth fields update. It connects with input and preference normalization.

The interaction proceeds as follows: select a summary depth button; then read the depth explanation. Inputs are Depth button. Behind the interface, the implementation performs Select one of six main-interface depths and pass its validated value to generation.

The audit classification is implemented but not verified. Comprehensive selection verified; exact word length is not guaranteed. Evidence is recorded under F017 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F018. Optional evidence-controlled synthesis.** The feature allows guest or registered user when configured to opt in to configured language-model synthesis. The intended output is Requested synthesis path or extractive fallback behavior. It connects with input and preference normalization; python integration.

The interaction proceeds as follows: when generation method is displayed, read the data-transfer notice; then opt in only when authorized to process the source there. Inputs

are Conditional use_llm_synthesis checkbox. Behind the interface, the implementation performs Request optional synthesis only when both request flag and server configuration permit the configured provider.

The audit classification is implemented but not verified. Conditional source only; flag disabled in synthetic renderer; external synthesis not invoked. Evidence is recorded under F018 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**Summarization**

**F019. Submit summary request.** The feature allows guest or registered user to start processing a selected source. The intended output is Processing redirect or recoverable error. It connects with input and preference normalization; recoverable summary jobs; python integration; summary persistence.

The interaction proceeds as follows: review source and settings; then choose summarize document; then follow processing/status feedback. Inputs are Source; title; focus; format; mode; depth; CSRF; request token. Behind the interface, the implementation

performs Validate request; resolve source; create or reuse a job; dispatch processing and persist successful output.

The audit classification is partially working. Submission not executed; all harness POSTs blocked. The legacy cleaner can discard legitimate prose containing register. Upload word-count and processing-time persistence are incomplete. Evidence is recorded under F019 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**Recovery**

**F020. Source validation and one-hour draft recovery.** The feature allows guest or registered user to correct invalid input and recover tab-local source/settings. The intended output is Friendly explanation, recovery action and restored settings. It connects with input and preference normalization.

The interaction proceeds as follows: correct the named field; then use retry only where offered; then for file recovery, select the file again. Inputs are Input errors; saved sessionStorage fields. Behind the interface,

the implementation performs Validate browser inputs; display catalogued errors; retain and recover an eligible one-hour browser draft.

The audit classification is implemented but not verified. Empty URL validation verified; file bytes cannot be restored; source text is retained in browser tab storage for up to one hour. Evidence is recorded under F020 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**Processing**

**F021. Job progress, retry and cancel.** The feature allows visitor with pending job to follow an asynchronous summary and recover from failure. The intended output is Status, completion redirect or recovery message. It connects with recoverable summary jobs.

The interaction proceeds as follows: remain on the processing page; then if offered, check connection/status or retry; then use cancel when available to return to settings. Inputs are Existing job/session; retry or cancel controls. Behind the interface, the implementation performs Resolve owned job;

inspect or update state; execute, retry or cancel through guarded actions.

The audit classification is implemented but not verified. Only synthetic pending page rendered; automatic execute/poll requests deliberately blocked. Evidence is recorded under F021 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**Nutshell**

**F022. Generate and copy concise nutshell.** The feature allows registered active account to produce a short standalone rendition from an eligible source. The intended output is Nutshell text, not-needed notice or recovery error. It connects with python integration; usage analytics.

The interaction proceeds as follows: prepare a source; then choose generate nutshell; then read the panel and use copy after success. Inputs are Selected source/mode; CSRF; idempotency key. Behind the interface, the implementation performs Require a registered account; request concise generation; record successful Nutshell activity and expose copy controls.

The audit classification is implemented but not verified. Guest redirect verified. Generation not run. Backend supports six mode labels; legal/scientific selections are normalized to general by this endpoint. The synthetic direct call returned insufficient_content with an empty nutshell; it did not demonstrate a successful concise summary. Web saving was not run. Evidence is recorded under F022 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**Results**

**F023. Read structured summary and details.** The feature allows authorized owner or shared-link viewer to read the summary and available metadata. The intended output is Summary, key points, keywords, section analysis and details when stored. It connects with history and sharing; structured display.

The interaction proceeds as follows: open an accessible result; then read the primary summary; then expand available source/details sections. Inputs are Authorized summary id; optional share token. Behind the interface, the implementation performs Resolve owner,

guest or bearer access; load summary/artifact and render supported result sections.

The audit classification is implemented but not verified. Synthetic prose rendered; optional rich sections depend on saved output and were not all populated. Evidence is recorded under F023 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F024. Inspect source and evidence passages.** The feature allows authorized result viewer to compare output with retained source passages. The intended output is Source text, evidence/coverage blocks where stored. It connects with history and sharing; structured display.

The interaction proceeds as follows: choose check source; then expand original text; then follow numbered evidence references if present. Inputs are Source disclosure and evidence anchor clicks. Behind the interface, the implementation performs Display available source text and evidence metadata from an accessible result.

The audit classification is implemented but not verified. Source-backed evidence links do not prove factual accuracy; populated metadata

not end-to-end tested. Evidence is recorded under F024 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F025. Copy summary.** The feature allows authorized result viewer to copy the displayed summary. The intended output is Clipboard text and status. It connects with navigation and result presentation.

The interaction proceeds as follows: open a completed result; then choose copy. Inputs are Copy button. Behind the interface, the implementation performs Copy the rendered summary text to the browser clipboard.

The audit classification is verified working. Clipboard success verified on loopback with explicit browser permission and synthetic content; other browser permission contexts untested. Evidence is recorded under F025 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F026. Print or save PDF.** The feature allows authorized result viewer to open browser print output for a completed summary. The intended output is Printed output or browser Save as

PDF. It connects with navigation and result presentation.

The interaction proceeds as follows: choose pdf; then review the browser print preview; then select a printer or save as pdf. Inputs are PDF button; browser print dialog. Behind the interface, the implementation performs Invoke browser printing with the result print stylesheet; PDF saving depends on browser options.

The audit classification is implemented but not verified. Uses window.print(), not a server-generated PDF endpoint; print dialog not exercised. Evidence is recorded under F026 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F027. Download text.** The feature allows authorized result viewer to save summary text locally. The intended output is Client-generated .txt file. It connects with navigation and result presentation.

The interaction proceeds as follows: choose text export; then save the generated download. Inputs are Text export button. Behind the interface, the implementation performs Create

a local text download containing the rendered summary.

The audit classification is verified working. Synthetic result exported to evidence/synthetic-summary-download.txt and exact text checked; no real summary exported. Evidence is recorded under F027 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F028. Listen to summary audio.** The feature allows authorized result viewer with session to request and listen to generated speech. The intended output is Audio player/status or service error. It connects with audio generation and delivery.

The interaction proceeds as follows: choose listen on a completed summary; then use audio controls after successful generation. Inputs are Summary id; share token if applicable; CSRF. Behind the interface, the implementation performs Validate permitted audio request; normalize text; generate or reuse Piper output; serve token-controlled file with byte-range handling.

The audit classification is partially working. Local worker availability required; no audio generated or fetched. The isolated

suffix-range request returned the whole file; audio delivery declares public cacheability. Real Piper synthesis and listening were not performed. Evidence is recorded under F028 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F029. Filipino translation.** The feature allows session-bearing result viewer to request a Filipino rendition of displayed summary text. The intended output is Translation panel or error. It connects with python integration.

The interaction proceeds as follows: choose translate on a completed result; then read returned filipino translation with original as reference. Inputs are Text; target language tl; CSRF. Behind the interface, the implementation performs Validate translation request; call configured translation provider through Python; return translated text.

The audit classification is implemented but not verified. Worker not invoked. Endpoint accepts en/tl; UI requests Filipino. Evidence is recorded under F029 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**Feedback**

**F030. Rate, comment and update feedback.** The feature allows guest/user with summary access to submit a usefulness rating with optional reasons/comment. The intended output is Saved/updated feedback confirmation or error. It connects with rating and comments.

The interaction proceeds as follows: choose a star rating; then optionally add reasons and a short comment; then submit feedback; use update feedback for an existing response. Inputs are 1–5 stars; optional reasons; comment up to 500 UI characters; CSRF. Behind the interface, the implementation performs Validate one-to-five rating and permitted summary context; update latest rating and append comment events.

The audit classification is implemented but not verified. No rating submitted; anonymous presentation does not mean no user/guest identifier is stored. Evidence is recorded under F030 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**Results**

**F031. Unavailable result recovery.** The feature allows visitor to receive a safe unavailable-result message. The intended output is 404 recovery view or missing-id redirect. It connects with history and sharing.

The interaction proceeds as follows: review the unavailable message; then use back to summary settings or obtain an authorized link. Inputs are Missing, unauthorized or inaccessible result id. Behind the interface, the implementation performs Resolve requested result; render unavailable or recovery view when it cannot be returned.

The audit classification is implemented but not verified. 404 template verified with synthetic absent id; live authorization not tested. Evidence is recorded under F031 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**Dashboard**

**F032. Personal dashboard and recent summaries.** The feature allows registered non-admin to return to recent work and inspect

personal activity. The intended output is Recent five summaries and aggregate/empty dashboard. It connects with personal summary/nutshell history and activity; administrator scope where stated.

The interaction proceeds as follows: open dashboard after signing in; then choose an available recent result or start a summary. Inputs are Account session. Behind the interface, the implementation performs Read recent summaries and Nutshell activity; calculate account dashboard counters and display recent work.

The audit classification is implemented but not verified. Empty state rendered; no live account statistics read. Evidence is recorded under F032 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F033. Reuse inferred preferred settings.** The feature allows registered non-admin to reuse settings inferred from previous completed summaries. The intended output is Validated mode/format/depth preselection. It connects with personal summary/nutshell history and activity; administrator scope where stated.

The interaction proceeds as follows: if preferred settings are shown, choose the reuse link; then review the resulting controls before submitting. Inputs are Preferred-settings link. Behind the interface, the implementation performs Infer frequently used settings from the account's saved summary/artifact history and expose reuse controls.

The audit classification is implemented but not verified. No separate editable custom-profile storage interface found; history-derived preference is conditional. Evidence is recorded under F033 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**History**

**F034. Browse summary and nutshell history.** The feature allows registered user or admin to reopen retained results and read prior nutshells. The intended output is History previews and full-report links. It connects with personal summary/nutshell history and activity; administrator scope where stated.

The interaction proceeds as follows: open history; then choose a title or access full

report; then read nutshell history when available. Inputs are Account; optional page query. Behind the interface, the implementation performs Read the requested history page and list summary and Nutshell records for the current context.

The audit classification is partially working. The history page displays 10 summaries per page by default (service cap 50) and 20 Nutshell records; no visible history pagination controls are supplied. Saved history was not exercised against production records. Evidence is recorded under F034 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**Analytics**

**F035. Personal/system activity and date filters.** The feature allows registered user or admin to inspect activity over a chosen period. The intended output is Counts, reduction, charts, performance and recent activity. It connects with personal summary/nutshell history and activity; administrator scope where stated.

The interaction proceeds as follows: open analytics through an available dashboard link; then choose a preset or from/to dates; then apply filter and review available charts. Inputs are 7d/30d/3m/year/custom dates; role. Behind the interface, the implementation performs Apply date bounds and role context to summary/Nutshell activity counts, distributions and trends.

The audit classification is partially working. Empty state rendered; populated charts not validated; admin scope differs from personal scope. Stored upload source counts and elapsed times can be incomplete, affecting derived activity measures. Evidence is recorded under F035 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**Profile**

**F036. View account details.** The feature allows registered account to view username, email and join date. The intended output is Read-only identity card. It connects with account display, self-deactivation and logout.

The interaction proceeds as follows: open profile; then review account details; then use

sidebar to return to dashboard or history. Inputs are Authenticated account. Behind the interface, the implementation performs Find the current account profile and render stored account details.

The audit classification is implemented but not verified. Synthetic identity rendered; no edit username/email/photo form exists. Evidence is recorded under F036 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F037. Deactivate own account.** The feature allows registered account subject to backend protections to deactivate account after confirming password. The intended output is Inactive account and sign-out; summaries retained. It connects with login and logout; account administration.

The interaction proceeds as follows: read the deactivation description; then enter current password; then submit only when deactivation is intended. Inputs are Current password; CSRF. Behind the interface, the implementation performs Validate authenticated account action; deactivate the account and end applicable access.

The audit classification is implemented but not verified. Not executed; backend has administrator protections; source retention is not hard deletion. Evidence is recorded under F037 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**Accounts**

**F038. Logout confirmation.** The feature allows signed-in account to confirm leaving the active session. The intended output is Cancelled modal or logout redirect. It connects with login and logout; remembered login.

The interaction proceeds as follows: choose logout; then choose cancel to remain or log out to leave. Inputs are Logout link; Cancel/Log out. Behind the interface, the implementation performs Confirm logout in the browser; clear session and applicable remembered-login state.

The audit classification is implemented but not verified. Escape closed modal on desktop/mobile; actual logout not executed; GET logout target remains source-supported only. Evidence is recorded under F038 in the

feature inventory; a template capture or function test does not verify the full saved workflow.

**Administration**

**F039. System overview and storage counters.** The feature allows administrator to inspect aggregate system and upload counters. The intended output is User/summary/guest/rating counts; storage size/count. It connects with usage analytics; storage administration.

The interaction proceeds as follows: open admin; then choose a statistics period; then review overview and storage cards. Inputs are Stats-period select. Behind the interface, the implementation performs Read aggregate system counters and inspect upload storage totals for the administrator.

The audit classification is implemented but not verified. All captured zeros are synthetic; cleanup/delete-file dispatcher actions exist but visible maintenance buttons were not found. Evidence is recorded under F039 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F040. Search and filter users.** The feature allows administrator to filter the displayed account table. The intended output is Visible rows and count. It connects with account administration.

The interaction proceeds as follows: enter username/email/id text; then choose role and status; then use clear filters to reset. Inputs are Search text; role; active-status filters. Behind the interface, the implementation performs Retrieve and filter the administrator’s account list by supplied search/status criteria.

The audit classification is implemented but not verified. Client-side filter over loaded rows; fixture had no users, so nonempty match behavior not browser-tested. Evidence is recorded under F040 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F041. Deactivate managed user.** The feature allows administrator to disable an eligible user account. The intended output is Account deactivation or error. It connects with account administration.

The interaction proceeds as follows: find the intended account; then read the

confirmation on deactivate; then confirm only when intended. Inputs are User id; CSRF; confirmation. Behind the interface, the implementation performs Validate administrator status-changing request and update target account status; dedicated and alternate paths have different safeguards.

The audit classification is partially working. Destructive/stateful action source-inspected only; not executed. The alternate toggle-status action bypasses protections present in the dedicated deactivation method. Evidence is recorded under F041 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F042. Permanently delete managed user.** The feature allows administrator to remove an eligible account and associated data. The intended output is Deletion or protected-account/error result. It connects with account administration.

The interaction proceeds as follows: find and verify the intended account; then read the explicit permanent-deletion warning; then confirm only under approved administration procedures. Inputs are User id; CSRF; confirmation. Behind the interface, the

implementation performs Validate authorized hard-delete action; remove target account and process related cleanup according to source and constraints.

The audit classification is implemented but not verified. Not executed; deletion is distinct from deactivation and may remove linked data. Evidence is recorded under F042 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F043. Filter feedback and quality telemetry.** The feature allows administrator to review anonymous-presented feedback and summary setting patterns. The intended output is Feedback/comment tables; mode/depth/reason aggregates. It connects with rating and comments.

The interaction proceeds as follows: review quality telemetry; then filter rating/comment state and sort order; then open a result link where authorized. Inputs are Rating filter; has-comment filter; sort. Behind the interface, the implementation performs Filter and aggregate feedback and retrieve related summary information for administrator display.

The audit classification is implemented but not verified. Empty fixture only; underlying records are linkable internally despite anonymous display. Evidence is recorded under F043 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F044. Landing telemetry dashboard.** The feature allows administrator to inspect landing engagement metrics. The intended output is Session, CTA, dwell, LCP, device, scroll, trend and referrer panels. It connects with usage analytics.

The interaction proceeds as follows: scroll to landing page analytics; then read labels and applicable date periods; then interpret missing values as unavailable. Inputs are Stored landing sessions/events. Behind the interface, the implementation performs Collect permitted landing session/event telemetry and aggregate visit/conversion/device/referrer indicators.

The audit classification is implemented but not verified. No telemetry emitted; the landing view itself normally sends beacons; counters are not human survey evaluation. Evidence is recorded under F044 in the feature

inventory; a template capture or function test does not verify the full saved workflow.

**F045. Audit log viewer.** The feature allows administrator to review privileged action logs. The intended output is Filtered log rows and pagination. It connects with administrative action accountability.

The interaction proceeds as follows: open audit logs; then choose filters and submit; then use pagination when present. Inputs are Action/admin/date filters; page. Behind the interface, the implementation performs Require administrator access; filter audit records and join actor identity for display.

The audit classification is implemented but not verified. Synthetic missing-table empty state captured; real audit log contents not read. Evidence is recorded under F045 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F046. Database health JSON.** The feature allows administrator to check required-table/column presence. The intended output is Status JSON and development-dependent detail. It connects with navigation and result presentation.

The interaction proceeds as follows: open the diagnostic endpoint under administrator access and interpret status fields. Inputs are Authenticated administrator session. Behind the interface, the implementation performs Require administrator context; query database metadata and return health information.

The audit classification is implemented but not verified. Anonymous isolated 403 verified; authenticated database health not checked against live schema. Evidence is recorded under F046 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**Evaluation**

**F047. Assigned independent summary review.** The feature allows active account with assignment to read an assigned source/summary and score it independently. The intended output is Submitted-and-locked assessment or error. It connects with blind evaluation assignments.

The interaction proceeds as follows: open the assigned review link while signed in; then read source and blinded summary; then score each criterion separately; then submit once;

contact administrator for corrections. Inputs are Assignment token; seven 1–5 criteria; optional comment; CSRF. Behind the interface, the implementation performs Check current assignment access; display blinded output and accept the required review criteria when submitted.

The audit classification is implemented but not verified. Empty/denied UI captured; no assignment or rating created; this 1–5 content rubric is not the capstone ISO usability instrument. Evidence is recorded under F047 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F048. Evaluation export and report rendering**. The feature allows administrator to export an existing evaluation run. The intended output is Download or HTML report. It connects with evaluation reports and export.

The interaction proceeds as follows: use an existing approved run identifier; then request the selected format; then protect exported records. Inputs are Positive run id; json/csv/html format. Behind the interface, the implementation performs Require administrator access; build available

evaluation report/export with supported identifier removal and spreadsheet safeguards.

The audit classification is partially working. No real run exported; only anonymous 403 tested. evaluation.php now redirects, so legacy management tabs are not operational entry points. Export code exists, but links to the retired evaluation-management page redirect; no real evaluation report was generated. Evidence is recorded under F048 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**API-only**

**F049. Document Q&A endpoint.** The feature allows authorized summary viewer to request source-grounded answers through the API. The intended output is Answer/passage/confidence JSON or error. It connects with python integration; history and sharing.

The interaction proceeds as follows: use an authorized api client; then provide an accessible source summary and question; then inspect source evidence. Inputs are POST JSON summary_id, question, optional share_token; no CSRF check found in this endpoint. Behind the

interface, the implementation performs Accept a source-related question; obtain permitted source context; rank passages and return a grounded answer or unavailable result.

The audit classification is implemented but not verified. No Q&A interaction widget found in current result.js/result.php. Do not advertise a visible chat feature. The API-only endpoint has no main-interface chat widget; a source-grounded Python answer passed in isolation. The route lacks a CSRF check and may disclose raw exception detail. Evidence is recorded under F049 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F050. Document and multi-document REST operations.** The feature allows Intended document caller; dispatcher does not enforce authentication or ownership to access document operations through routed API. The intended output is Document/status/summary response. It connects with document combination api.

The interaction proceeds as follows: consult controller dispatch contract before using an integration. Inputs are Method/path and controller-specific payload. Behind the interface, the implementation performs

Dispatch document creation, retrieval, cached-summary combinations, listing, batch or streaming operations; ownership enforcement is missing.

The audit classification is partially working. No multi-file selector in main form; result can render multi_document metadata when present. Actual controller/service fixtures returned another owner's document and an anonymous document with HTTP 200. The dispatcher also lacks a CSRF check. Evidence is recorded under F050 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**Health**

**F051. Static service health.** The feature allows visitor or monitor to confirm the PHP health endpoint responds. The intended output is status ok / service ai-summarizer JSON. It connects with navigation and result presentation.

The interaction proceeds as follows: request health.php; then check http status and json service label. Inputs are GET only in this audit. Behind the interface, the implementation performs Return a static JSON

health response without loading the application bootstrap.

The audit classification is verified working. This static check does not establish database, NLP worker or full application readiness. Evidence is recorded under F051 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**Backend services**

**F052. Guest lifecycle.** Guest access keeps a temporary session so eligible results can be retrieved before an account is used. The intended output is Guest access token; eligible records transferred or expired. It connects with main processing, administration and reproducible operation.

The interaction proceeds as follows: use the named authorized entry point in an isolated deployment; then provide the specified inputs; then inspect the response and related records. Inputs are Temporary token and existing guest-owned records. Behind the interface, the implementation performs create token; touch expiry; claim summaries/feedback; purge expired guest data.

The audit classification is partially working. Source-confirmed; GET bootstrap mutates state; lifecycle incomplete for other stores. Evidence is recorded under F052 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F053. Python integration.** The PHP application passes processing work to Python and interprets its response. The intended output is Structured Python response or bounded error. It connects with main processing, administration and reproducible operation.

The interaction proceeds as follows: use the named authorized entry point in an isolated deployment; then provide the specified inputs; then inspect the response and related records. Inputs are Action, normalized payload and worker configuration. Behind the interface, the implementation performs daemon dispatch; CLI dispatch; structured response; timeout; terminate child tree.

The audit classification is implemented but not verified. Source-confirmed; authenticated loopback protocol3; Python behavior separately audited. Evidence is recorded under F053 in

the feature inventory; a template capture or function test does not verify the full saved workflow.

**F054. Summary persistence.** Successful processing is intended to save a summary and supporting source for later display. The intended output is Saved summary identifier and artifact. It connects with main processing, administration and reproducible operation.

The interaction proceeds as follows: use the named authorized entry point in an isolated deployment; then provide the specified inputs; then inspect the response and related records. Inputs are Normalized output, source and metadata. Behind the interface, the implementation performs normalize worker contract; store summary; store source/artifact; attach metadata.

The audit classification is partially working. Source-confirmed; uploaded source word counts and processing time incomplete. Evidence is recorded under F054 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F055. Storage administration.** Administrators can inspect upload storage and invoke cleanup

controls. The intended output is File counts or deletion outcome. It connects with main processing, administration and reproducible operation.

The interaction proceeds as follows: use the named authorized entry point in an isolated deployment; then provide the specified inputs; then inspect the response and related records. Inputs are Authorized cleanup action and upload directory. Behind the interface, the implementation performs count uploads; delete aged uploads; delete all uploads.

The audit classification is partially working. Source-confirmed destructive capability; age-only cleanup is not reference-aware orphan detection; not executed. Evidence is recorded under F055 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**Evaluation**

**F056. Evaluation dataset preparation.** A maintainer can prepare material for controlled algorithm evaluation through implemented services and import tools. The intended output is Dataset, reference and content-unit

records. It connects with main processing, administration and reproducible operation.

The interaction proceeds as follows: use the named authorized entry point in an isolated deployment; then provide the specified inputs; then inspect the response and related records. Inputs are Dataset metadata, source documents and reference annotations. Behind the interface, the implementation performs create dataset; add document/provenance/split; add references; add content units; add challenge cases; activate/deactivate.

The audit classification is implemented but not verified. Source-confirmed service/CLI; not evidence of reachable browser controls or completed research corpus. Evidence is recorded under F056 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F057. Evaluation run control.** Evaluation run control records the settings used for a comparison so results can be traced to their inputs. The intended output is Queued immutable run snapshot and output work items. It connects with main processing, administration and reproducible operation.

The interaction proceeds as follows: use the named authorized entry point in an isolated deployment; then provide the specified inputs; then inspect the response and related records. Inputs are Dataset, profile/length/system combinations and configuration. Behind the interface, the implementation performs queue immutable snapshot; cancel; resume; code fingerprint; bounded combinations.

The audit classification is implemented but not verified. Source-confirmed; five lengths/eight profiles/five systems/ten ablations; optionalLLM may unavailable. Evidence is recorded under F057 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F058. Offline evaluation worker.** An offline worker processes queued comparison items and records available measurements. The intended output is Generated outputs, metric records and error states. It connects with main processing, administration and reproducible operation.

The interaction proceeds as follows: use the named authorized entry point in an isolated deployment; then provide the specified inputs; then inspect the response and related records.

Inputs are Queued run and installed evaluation dependencies. Behind the interface, the implementation performs single-database lock; generate outputs; persist metrics; record failures; refresh report.

The audit classification is implemented but not verified. Source-confirmed; production worker not run; missing metric statuses retained. Evidence is recorded under F058 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F059. Evaluation review and correction.** Authorized review services record corrections and detailed analysis without treating automatic measurements as human ratings. The intended output is Recorded correction/review with audit history. It connects with main processing, administration and reproducible operation.

The interaction proceeds as follows: use the named authorized entry point in an isolated deployment; then provide the specified inputs; then inspect the response and related records. Inputs are Output, rating correction reason, error label or content-unit review. Behind the interface, the implementation performs correct

rating with reason; classify error; verify content unit representation.

The audit classification is implemented but not verified. Source-confirmed; source method existence does not establish current UI reachability. Evidence is recorded under F059 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**Operations**

**F060. Migration and installation.** Installation tools establish or update database structures; they must be rehearsed against a disposable database. The intended output is Schema changes and migration ledger. It connects with main processing, administration and reproducible operation.

The interaction proceeds as follows: use the named authorized entry point in an isolated deployment; then provide the specified inputs; then inspect the response and related records. Inputs are Database configuration and checked-in SQL/migrations. Behind the interface, the implementation performs import base schemas; apply migrations; migration ledger/status.

The audit classification is partially working. Source-confirmed; status maywrite; passwordresetfreshinstalldrift; not executed. Evidence is recorded under F060 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F061. Retention cleanup.** Retention helpers limit selected old telemetry, comments and temporary recovery material. The intended output is Pruned eligible records/payloads. It connects with main processing, administration and reproducible operation.

The interaction proceeds as follows: use the named authorized entry point in an isolated deployment; then provide the specified inputs; then inspect the response and related records. Inputs are Retention interval or expiry timestamps. Behind the interface, the implementation performs landing90day default; audit90day default; feedbackcomments90day default; expired job payloads; guest expiry.

The audit classification is implemented but not verified. Source-confirmed; notproofscheduled; dryrunbootstrapcanwrite; no automaticbackupfound. Evidence is recorded under F061 in the feature inventory; a

template capture or function test does not verify the full saved workflow.

**F062. Windows worker supervision.** Maintainers can start, monitor and stop the local Python worker using supplied Windows scripts. The intended output is Worker lifecycle, health checks and scheduled-task configuration. It connects with main processing, administration and reproducible operation.

The interaction proceeds as follows: use the named authorized entry point in an isolated deployment; then provide the specified inputs; then inspect the response and related records. Inputs are Python executable, worker paths and task settings. Behind the interface, the implementation performs install logon task; healthcheck30seconds; hidden launch; selftest; verify process before stop; remove task.

The audit classification is implemented but not verified. Source-confirmed; installed/runningtaskstate not inspected. Evidence is recorded under F062 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**Results**

**F063. Bearer-token result access.** A person holding an authorized share link can retrieve its associated result without becoming its owner. The intended output is Result display for token holder. It connects with source intake, processing and result retrieval.

The interaction proceeds as follows: open the relevant result or call the documented internal action through its intended boundary; then provide the required source or access token; then inspect the resulting output and any warnings. Inputs are Summary identifier and bearer share token. Behind the interface, the implementation performs Look up matching token and summary; load result artifact.

The audit classification is implemented but not verified. Source-supported; live share-link lifecycle and revocation were not exercised. Evidence is recorded under F063 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**Processing**

**F064. PDF and DOCX source extraction.** The engine converts supported document content

into text and structural hints before summarization. The intended output is Extracted source and document context. It connects with source intake, processing and result retrieval.

The interaction proceeds as follows: open the relevant result or call the documented internal action through its intended boundary; then provide the required source or access token; then inspect the resulting output and any warnings. Inputs are Supported source document. Behind the interface, the implementation performs Read pages/paragraphs/tables; reconstruct text; apply readability checks.

The audit classification is implemented but not verified. Parsing fixtures run in isolation; representative real uploads were not submitted. Blank-first-page alignment probe was inconclusive. Evidence is recorded under F064 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F065. Bounded OCR fallback.** Image-only PDF content can be passed to an OCR fallback when suitable local tools are installed. The intended output is Recovered text or a

controlled unreadable-content error. It connects with source intake, processing and result retrieval.

The interaction proceeds as follows: open the relevant result or call the documented internal action through its intended boundary; then provide the required source or access token; then inspect the resulting output and any warnings. Inputs are Unreadable selected PDF pages. Behind the interface, the implementation performs Select at most 12 pages; render at 150 DPI; use a 15-second OCR-call timeout and check a 55-second elapsed budget between pages.

The audit classification is implemented but not verified. Source-supported only. The 55-second value is a between-page elapsed-budget check, not a hard total deadline: rasterization has no local timeout. Real OCR tools and scanned-document accuracy were not demonstrated. Evidence is recorded under F065 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F066. Source cleaning and reconstruction.** The engine attempts to remove navigation and extraction noise while preserving useful

paragraphs. The intended output is Cleaned paragraphs or an error when content is unavailable. It connects with source intake, processing and result retrieval.

The interaction proceeds as follows: open the relevant result or call the documented internal action through its intended boundary; then provide the required source or access token; then inspect the resulting output and any warnings. Inputs are Extracted or pasted text. Behind the interface, the implementation performs Normalize and classify blocks; remove detected noise; retain readable content.

The audit classification is partially working. A synthetic valid paragraph containing register was discarded. Rewording or paragraph separation retained content; this is a confirmed over-filtering defect. Evidence is recorded under F066 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F067. Default legacy summary selection.** The configured engine ranks and organizes source passages using text statistics, document hints and length constraints. The intended output is Structured summary payload and evidence

metadata. It connects with source intake, processing and result retrieval.

The interaction proceeds as follows: open the relevant result or call the documented internal action through its intended boundary; then provide the required source or access token; then inspect the resulting output and any warnings. Inputs are Source, mode, format, depth, title and focus. Behind the interface, the implementation performs Extract; clean; rank sentence candidates; select; simplify; check heuristic facts; format.

The audit classification is partially working. The checked-in NIST fixture returned a nonempty summary. General quality, omission rates and saved web workflows remain unverified; the preceding cleaning stage has a reproduced defect. Evidence is recorded under F067 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F068. Optional v2 summary engine.** An alternative engine implementation provides a configurable experimental processing path. The intended output is Alternative summary result. It connects with source intake, processing and result retrieval.

The interaction proceeds as follows: open the relevant result or call the documented internal action through its intended boundary; then provide the required source or access token; then inspect the resulting output and any warnings. Inputs are Document and supported engine configuration. Behind the interface, the implementation performs Build evidence representation and compose output under the alternative engine contract.

The audit classification is partially working. Not enabled by the audited default configuration. One regression failed because an inline-heading fixture omitted the introductory outlier definition. Evidence is recorded under F068 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**F069. Automated comparison metrics and statistics.** Evaluation helpers compare generated text with supplied references and summarize available measurements. The intended output is Metric records and statistical report payload. It connects with source intake, processing and result retrieval.

The interaction proceeds as follows: open the relevant result or call the documented

internal action through its intended boundary; then provide the required source or access token; then inspect the resulting output and any warnings. Inputs are Sources, outputs, references and paired document-level observations. Behind the interface, the implementation performs Compute supported overlap/fact proxies; report unavailable metrics; group comparisons; apply statistical helpers.

The audit classification is implemented but not verified. Tests establish selected contracts in isolation. No human survey responses or approved research dataset were collected; optional metrics require their resources. Evidence is recorded under F069 in the feature inventory; a template capture or function test does not verify the full saved workflow.

**Security Features**

Authentication uses password hashing and session handling, with an emailed code in the administrator sign-in path. Remembered user sessions use stored token material, while administrator auto-login is refused. Password recovery has a time-limited code flow. Most principal mutation routes use request

validation, and database repositories use prepared statements. The source includes role checks, result ownership or bearer-token checks, escaping in result rendering, rate-limit facilities and upload validation. URL acquisition validates schemes and destination addresses, pins the selected address and checks redirects to reduce server-side request risks.

These controls do not provide uniform protection across all interfaces. Isolated execution of the document API’s actual controller and service returned source content for another user and for an anonymous context. The separate Q&A and document dispatchers lack a CSRF check. Password replacement does not revoke existing remembered tokens or all active sessions, and an alternate administrator status action bypasses protections used by the dedicated deactivation action. These findings must remain visible alongside the control descriptions. No comprehensive penetration test or claim of security certification is made.

**User Experience Features**

The interface offers a mobile navigation menu, theme selection, source tabs,

explanatory labels, validation messages, progress and recovery views, and output controls. Eighteen local interaction assertions passed in the isolated browser fixtures. Forty-six screenshots cover twenty-three interface states at desktop and mobile widths; the tested fixtures did not show document-level horizontal overflow. These observations establish limited layout and interaction behavior, rather than accessibility conformance across browsers and assistive technologies.

Remaining usability issues include password-visibility controls excluded from the normal tab sequence, audit-log filters without programmatic labels, a missing favicon, history without visible paging beyond its default 10-summary page, and links to a retired evaluation-management route. Some pages represent empty or unavailable states. Such states are documented because users require understandable recovery paths as well as successful-result screens.

## 4. Design and Development of the System

### 4.1 Planning Stage

The supplied researcher draft describes an Agile approach and an April–October 2026 activity schedule. Chapter II transcribes that schedule as supplied planning material. It does not establish completed sprints, meetings, testing sessions or deployment milestones. The confirmed project title is LIGHT summarizer; the institution’s required title wording, program, adviser and approval records remain subject to researcher confirmation.

The implemented requirements can be reconstructed from source: multiple source types, selectable output settings, background jobs, result reuse, accounts, administrative facilities and controlled evaluation services. Constraints include document and request limits, local Python dependencies, database consistency, external translation/mail services, and unavailable human-evaluation records. This reconstruction documents the current design; it is not a retroactive assertion that a historical requirements meeting occurred.

## 4.2 Design Stage

### A. Use Case Diagram

The only persisted account roles are user and admin. A guest is a temporary access context, and an evaluator is a user with an assignment, not a third account role. The use-case model distinguishes browser access contexts. Maintenance is an operational responsibility described in Appendix G, rather than an additional stored account role. Restrictions stated in the diagram apply to intended authorized paths; the document API's reproduced authorization defect remains an exception requiring repair.

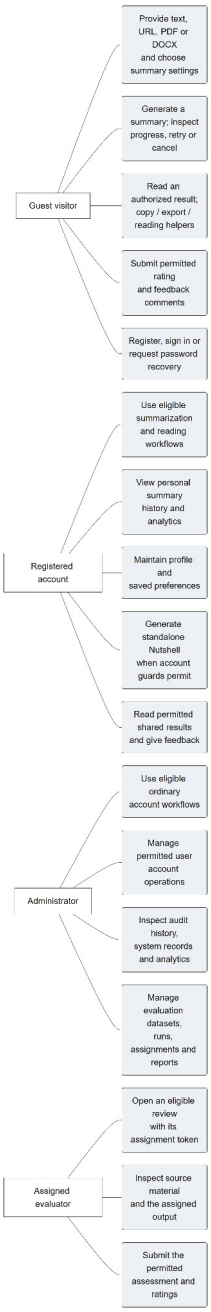



**Figure 5. Implemented actors and major use cases.**

Figure 5 illustrates guest, registered-user, administrator and assigned-evaluator

activities. Guest access does not grant the authenticated Nutshell workflow. A maintenance operator is a deployment responsibility rather than a database account role. Source inspection supports the relationships; only the named test cases establish runtime observations.

The following enlarged role panels provide the readable detail of the overview. Their associations do not introduce additional database roles.

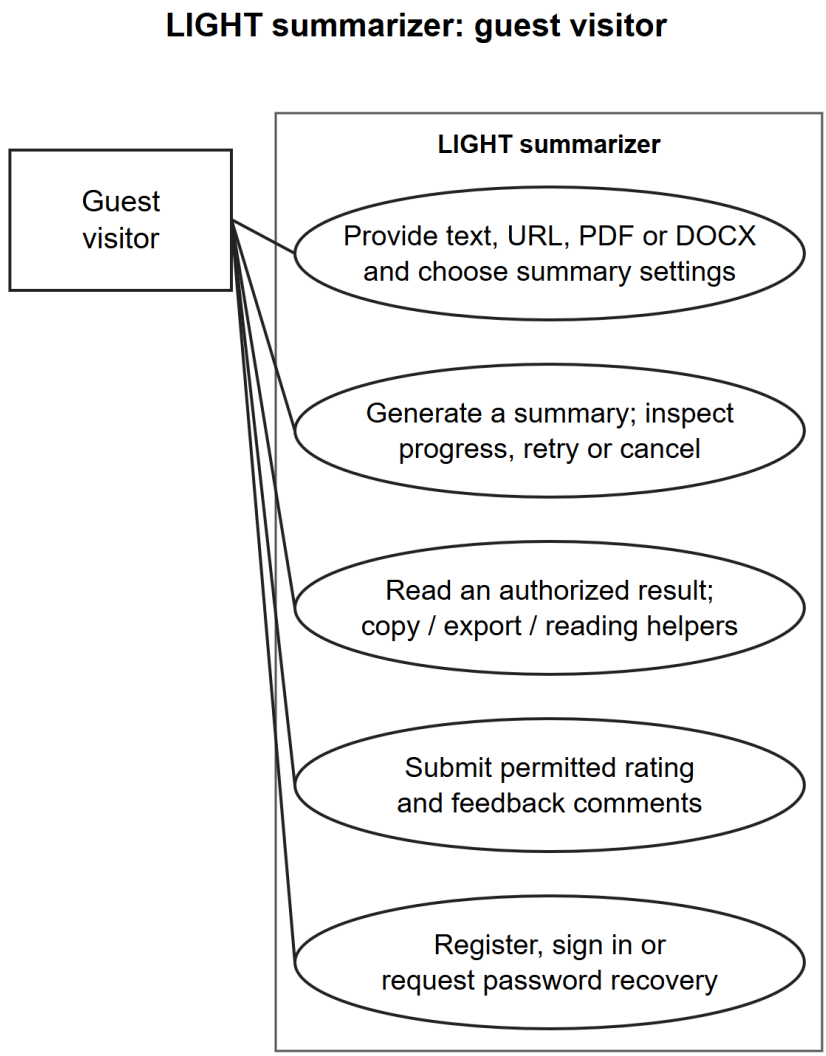



**Figure 6. Guest visitor use cases.**

Figure 6 illustrates the authorized activities for the named access context. The panel is derived from the same role model as

the overview; existence of a use case is source-supported and does not override the verification limits in the feature inventory.

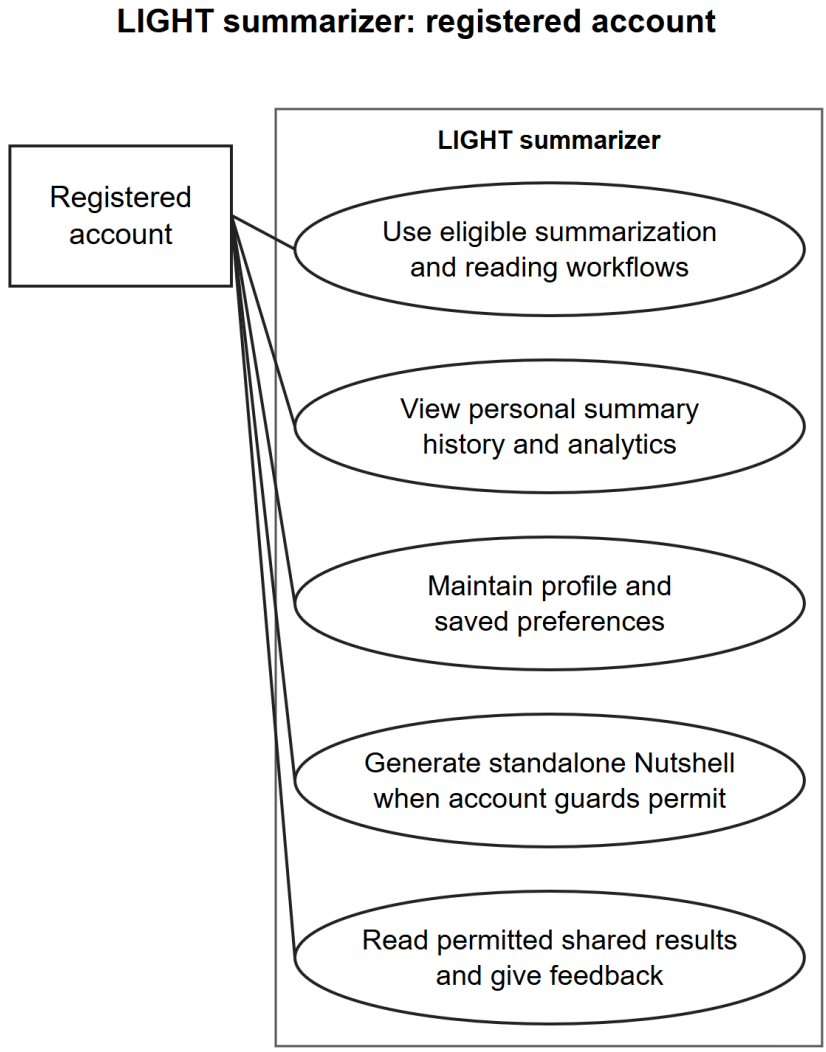



**Figure 7. Registered account use cases.**

Figure 7 illustrates the authorized activities for the named access context. The panel is derived from the same role model as the overview; existence of a use case is source-supported and does not override the verification limits in the feature inventory.

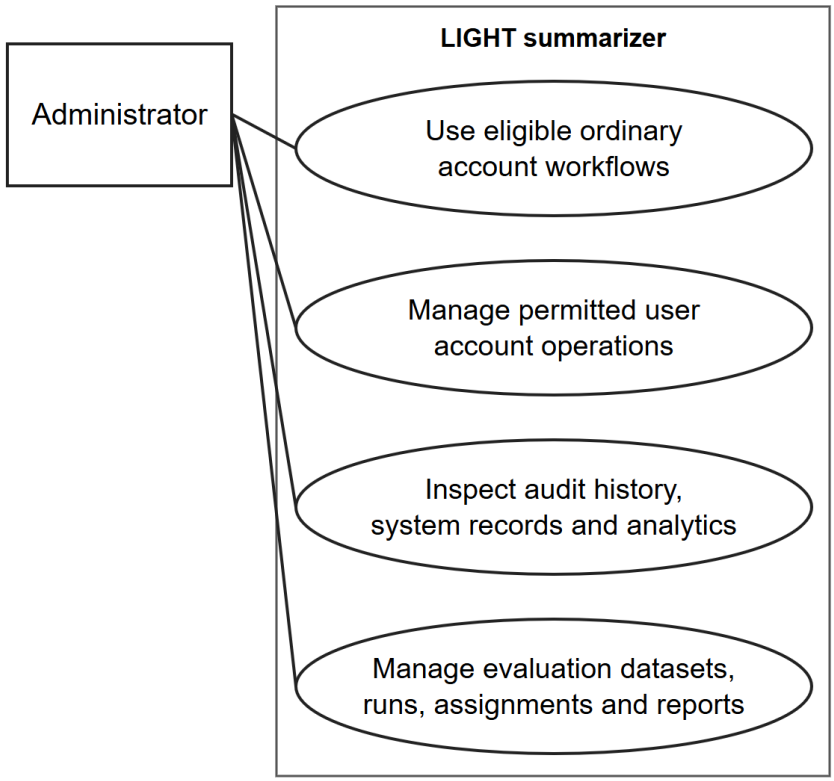



**Figure 8. Administrator use cases.**

Figure 8 illustrates the authorized activities for the named access context. The panel is derived from the same role model as the overview; existence of a use case is source-supported and does not override the verification limits in the feature inventory.

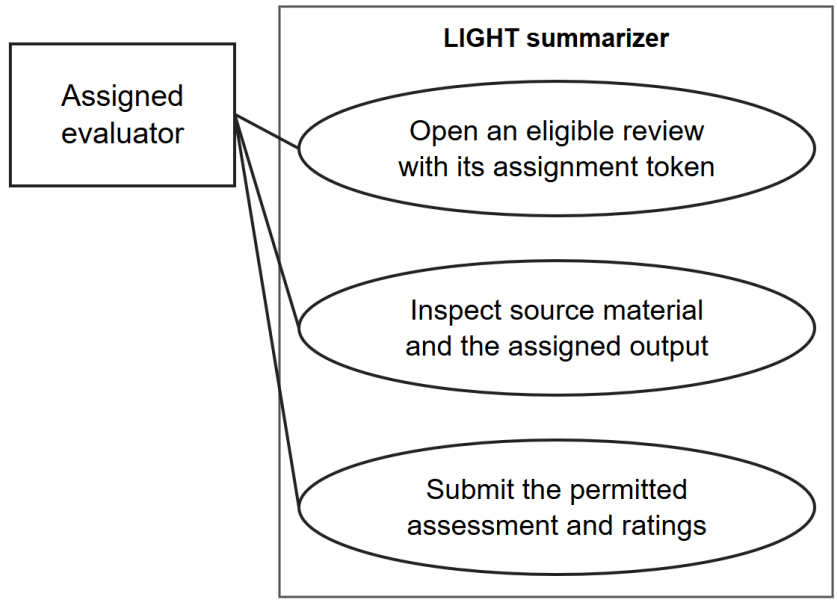



**Figure 9. Assigned evaluator use cases.**

Figure 9 illustrates the authorized activities for the named access context. The panel is derived from the same role model as

the overview; existence of a use case is source-supported and does not override the verification limits in the feature inventory.

**B. Data Flow Diagram**

The context model treats LIGHT summarizer as one process. Level 1 expands that process into account handling, source processing, result access, supporting output services, administration and evaluation. Detailed models then expand summary generation and authentication. Logical store identifiers retain their meaning across the levels. File storage and PHP session state are shown distinctly from declared database entities. External providers are included where the inspected code can contact them; their presence does not mean those integrations were exercised.

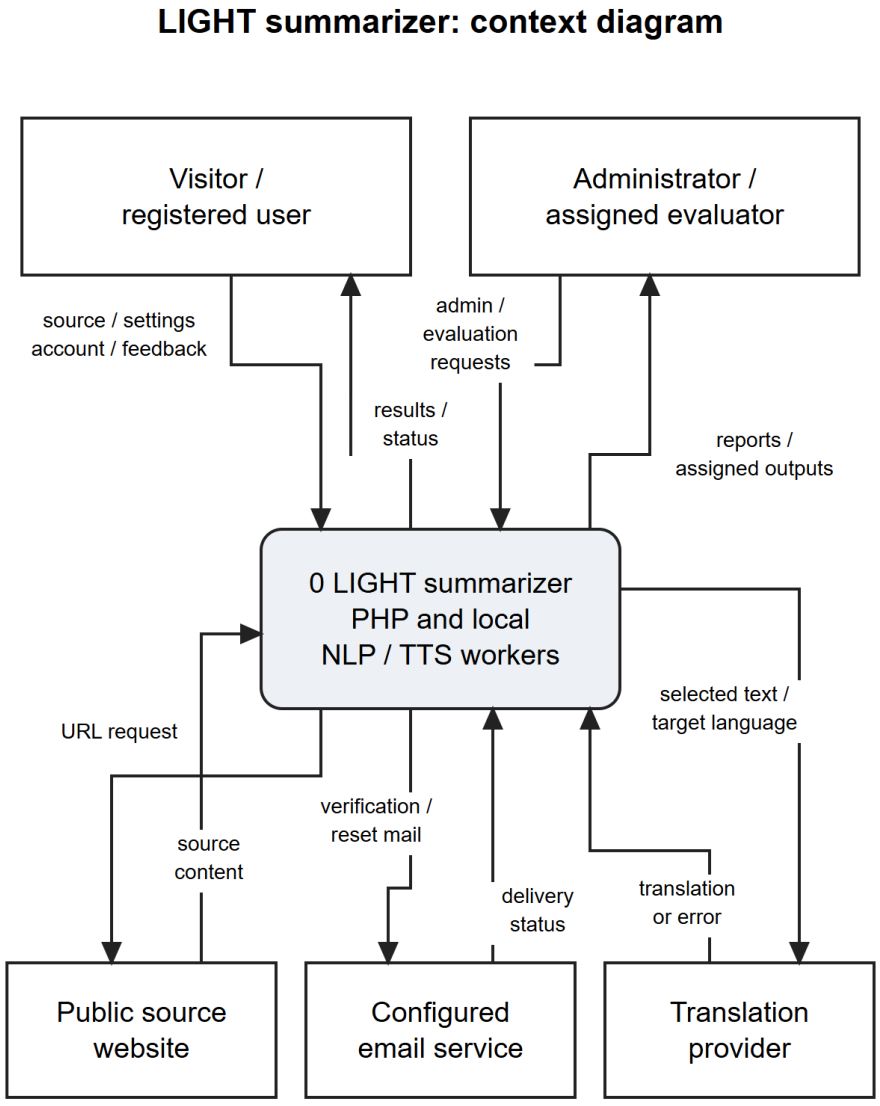



**Figure 10. Context diagram of LIGHT summarizer.**

Figure 10 illustrates the boundary between people, the application, and external information or delivery services. Submitted

sources and requests enter the application, while summaries, access decisions and administrative information return through application processes.

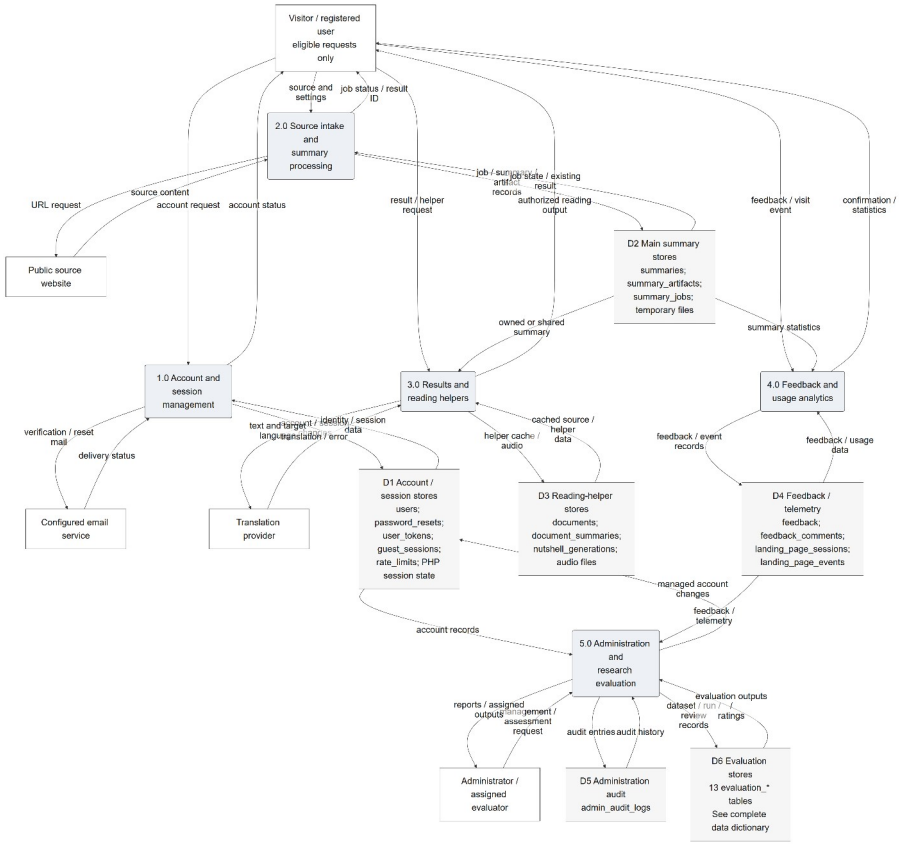



**Figure 11. Level 1 data flows and logical stores.**

Figure 11 illustrates the division of work among the major application processes. Accounts, generated records, feedback, telemetry, audit information and evaluation material have distinct storage responsibilities. Logical grouping is for

explanation; the data dictionary provides the exact tables.

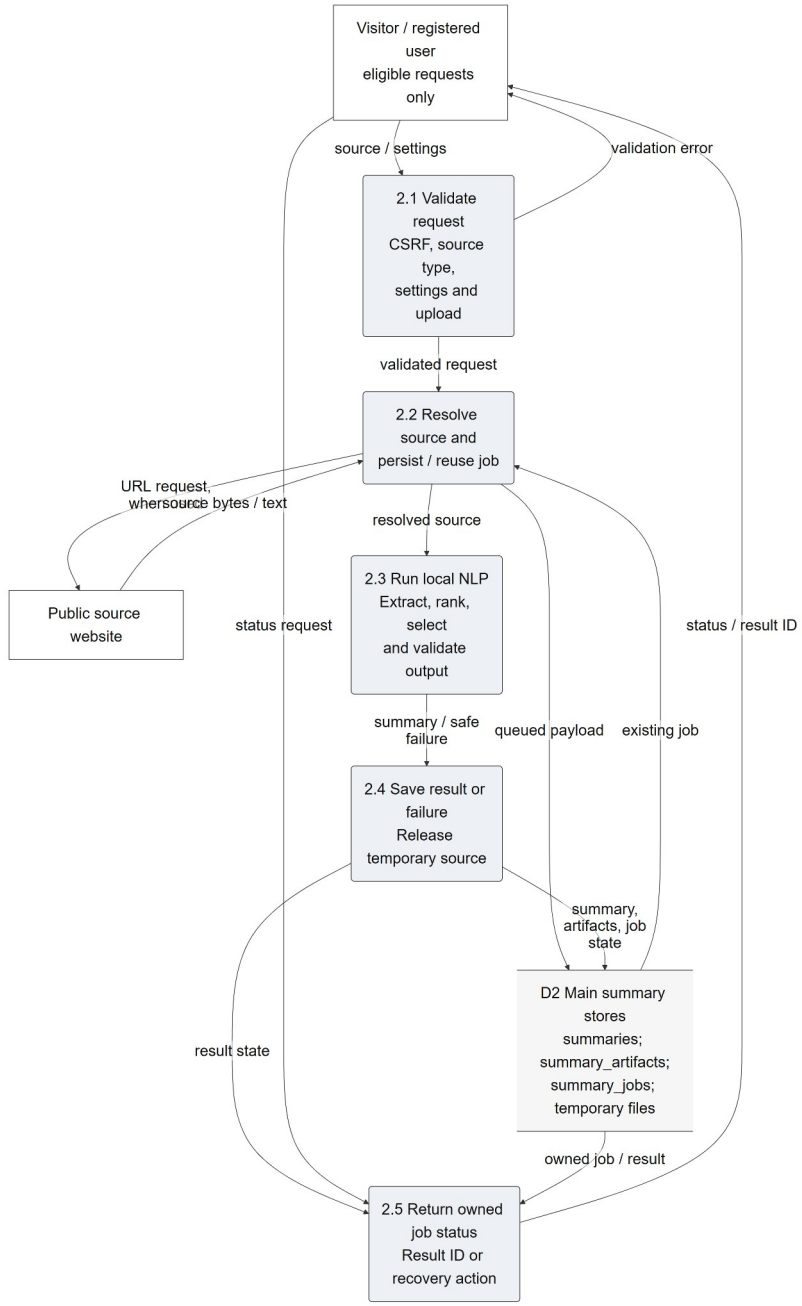



**Figure 12. Level 2 summary-generation data flow.**

Figure 12 illustrates source validation and extraction, job creation and execution, Python processing, persistence and status/result response. The output passes through an application process before returning to a user. A worker response alone is not evidence that the PHP save step succeeded.

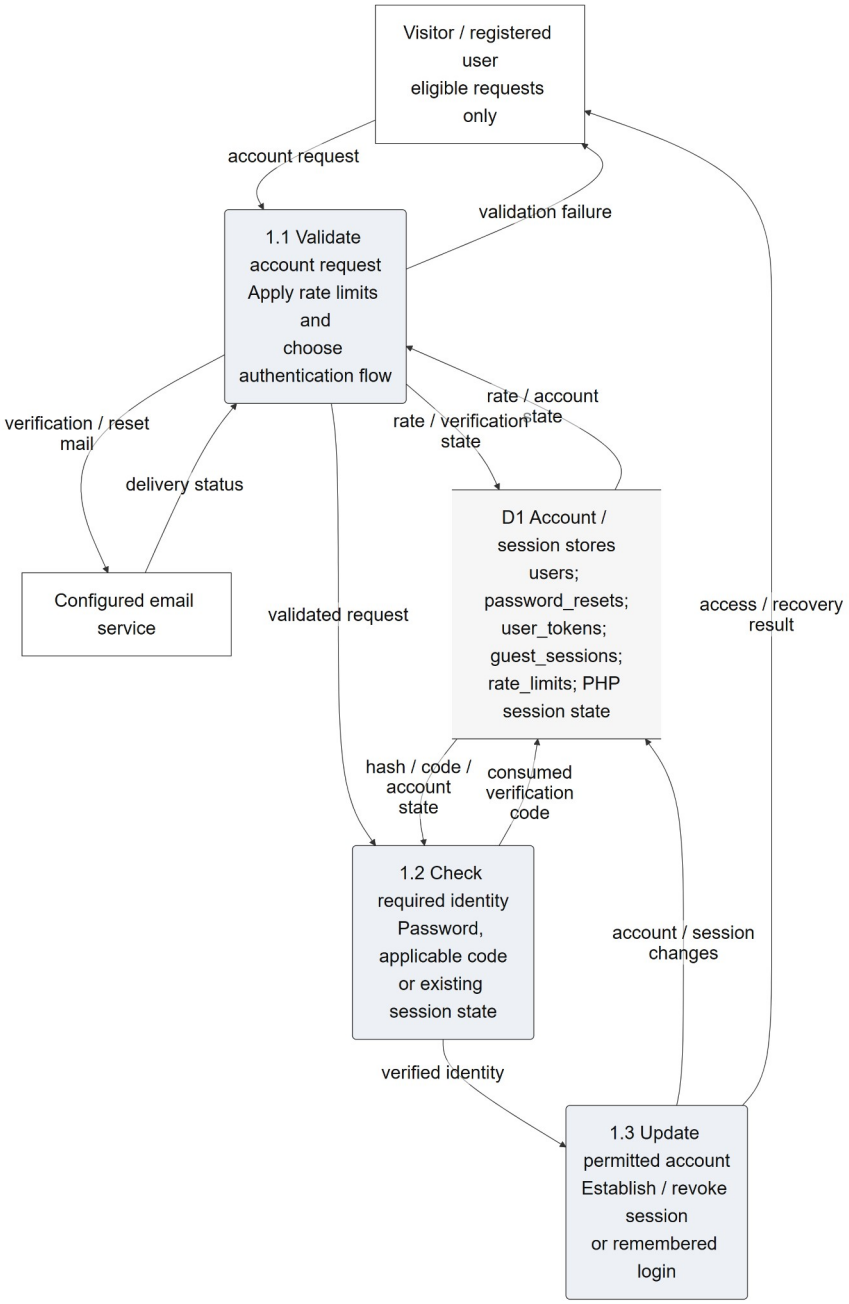



**Figure 13. Level 2 account and authentication data flow.**

Figure 13 illustrates registration, credentials, administrator email-code

handling, password recovery and session/token establishment. Session verification state belongs to the account store group; it is not represented as a nonexistent administrator-code table.

The editable diagram sources and focused print panels accompany the manuscript. The models describe the inspected application contract, including optional services, while the issue register identifies places where implementation does not enforce the intended boundary. They do not add a student database, course-record system, messaging service, or research-participant role that the application does not contain.

The following enlarged panels retain the process and store identifiers used in the overview. They divide a dense drawing for print legibility rather than define additional processes.

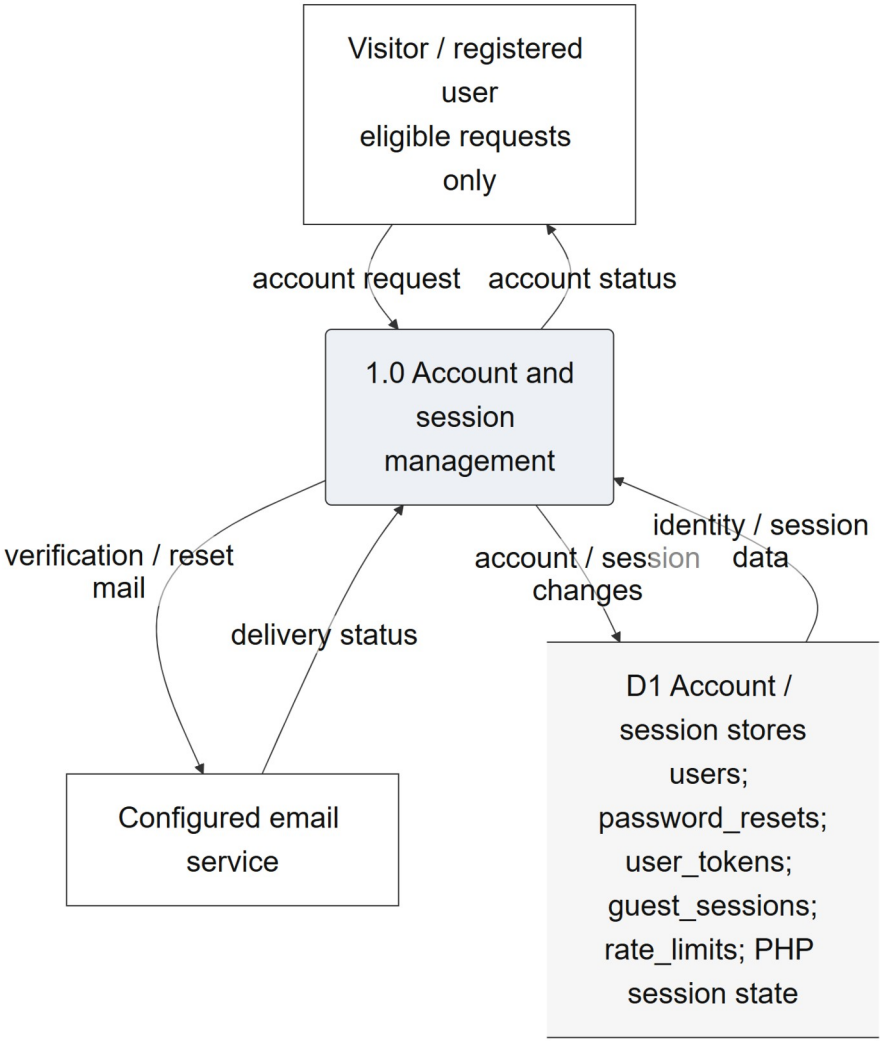



**Figure 14. Level 1 account and session detail.**

Figure 14 illustrates the named portion of the inspected data flow. Inputs and outputs pass through an application process; persistent tables, file storage and PHP session state retain their documented distinctions. The complete overview and

editable sources retain the cross-panel context.

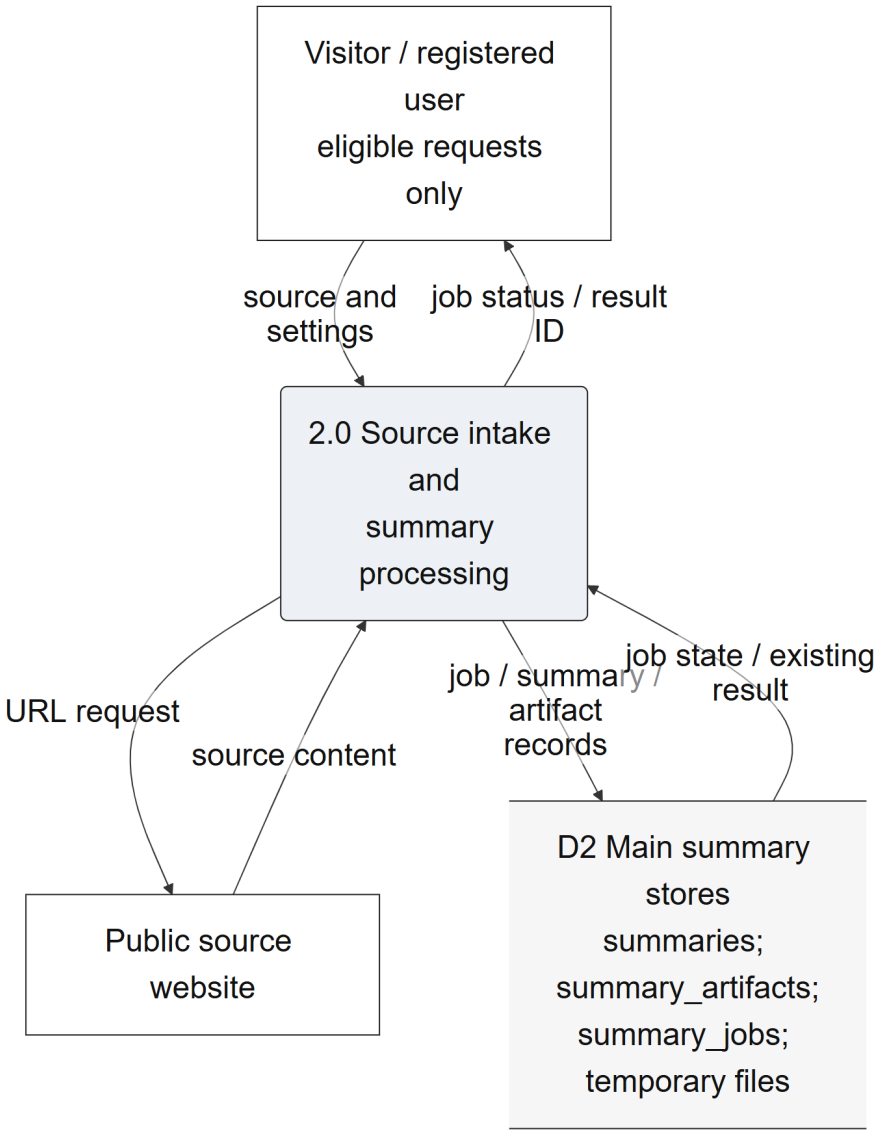



**Figure 15. Level 1 source and summary detail.**

Figure 15 illustrates the named portion of the inspected data flow. Inputs and outputs pass through an application process;

persistent tables, file storage and PHP session state retain their documented distinctions. The complete overview and editable sources retain the cross-panel context.

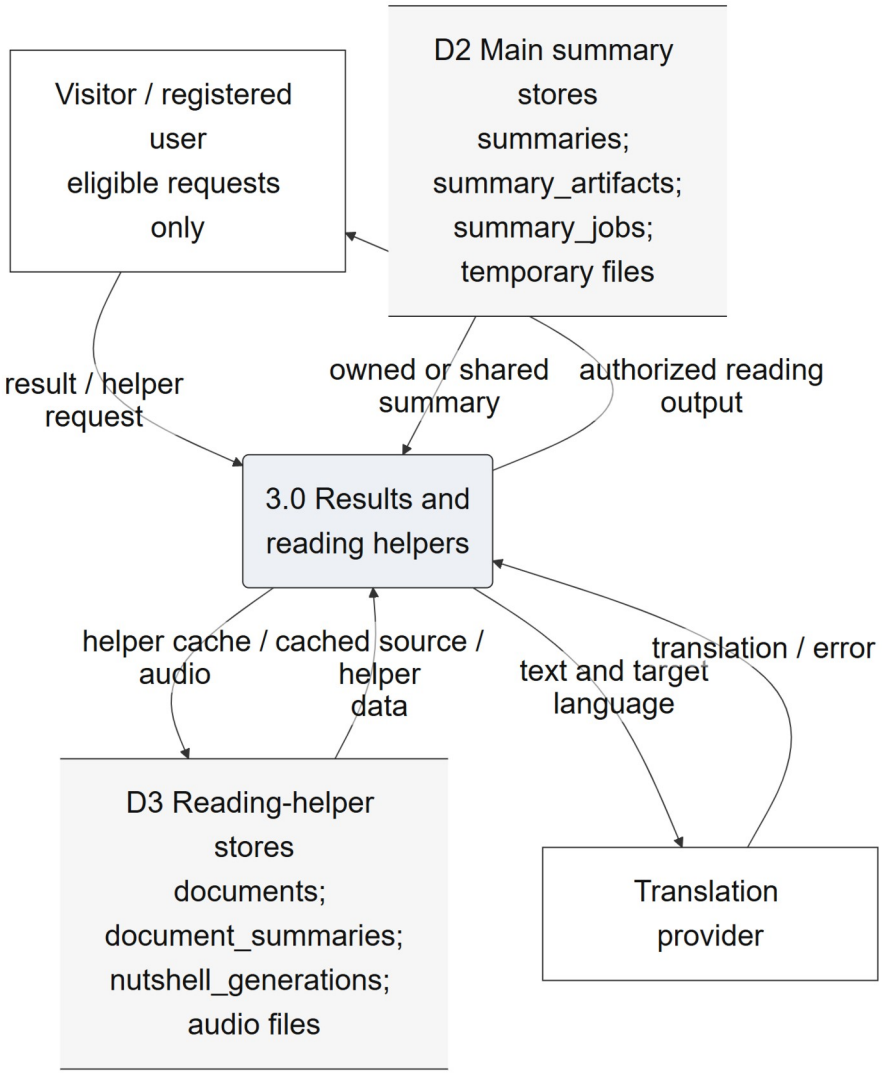



**Figure 16. Level 1 result and reading-helper detail.**

Figure 16 illustrates the named portion of the inspected data flow. Inputs and outputs pass through an application process; persistent tables, file storage and PHP session state retain their documented distinctions. The complete overview and editable sources retain the cross-panel context.

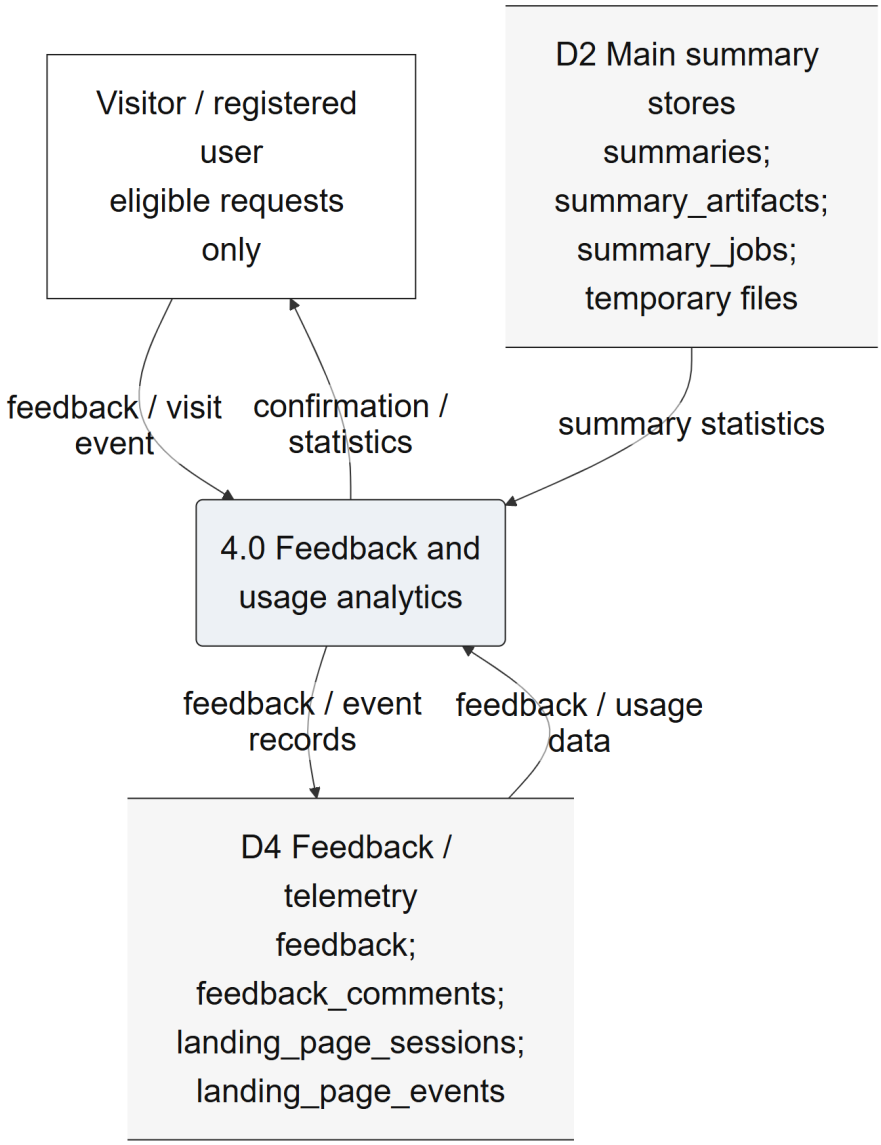



**Figure 17. Level 1 feedback and usage detail.**

Figure 17 illustrates the named portion of the inspected data flow. Inputs and outputs pass through an application process; persistent tables, file storage and PHP session state retain their documented distinctions. The complete overview and

editable sources retain the cross-panel context.

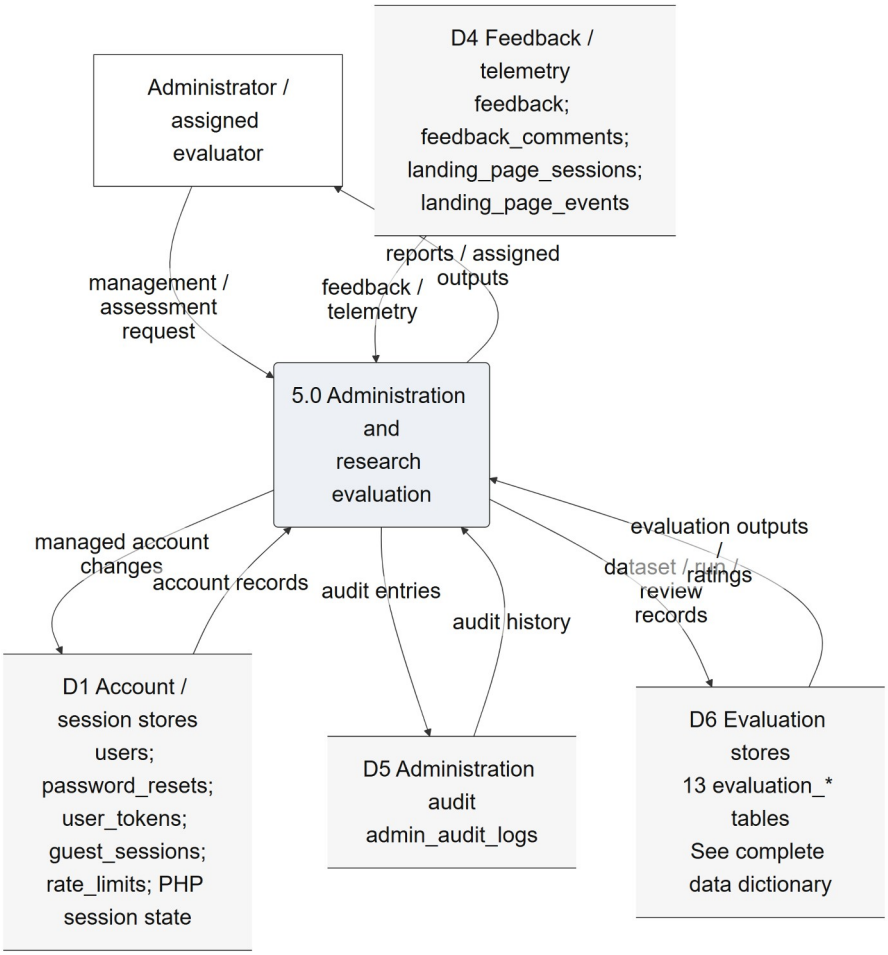



**Figure 18. Level 1 administration and evaluation detail.**

Figure 18 illustrates the named portion of the inspected data flow. Inputs and outputs pass through an application process; persistent tables, file storage and PHP session state retain their documented distinctions. The complete overview and

editable sources retain the cross-panel context.

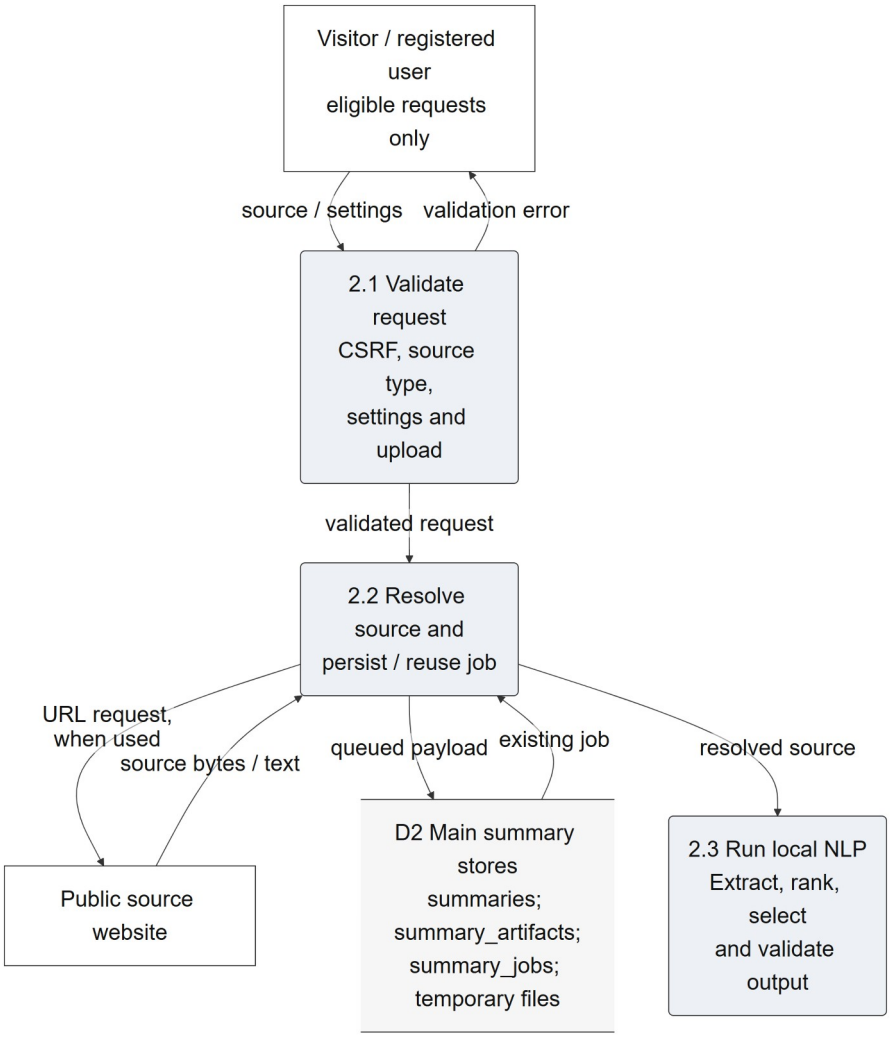



**Figure 19. Level 2 summary intake detail.**

Figure 19 illustrates the named portion of the inspected data flow. Inputs and outputs pass through an application process; persistent tables, file storage and PHP session state retain their documented

distinctions. The complete overview and editable sources retain the cross-panel context.

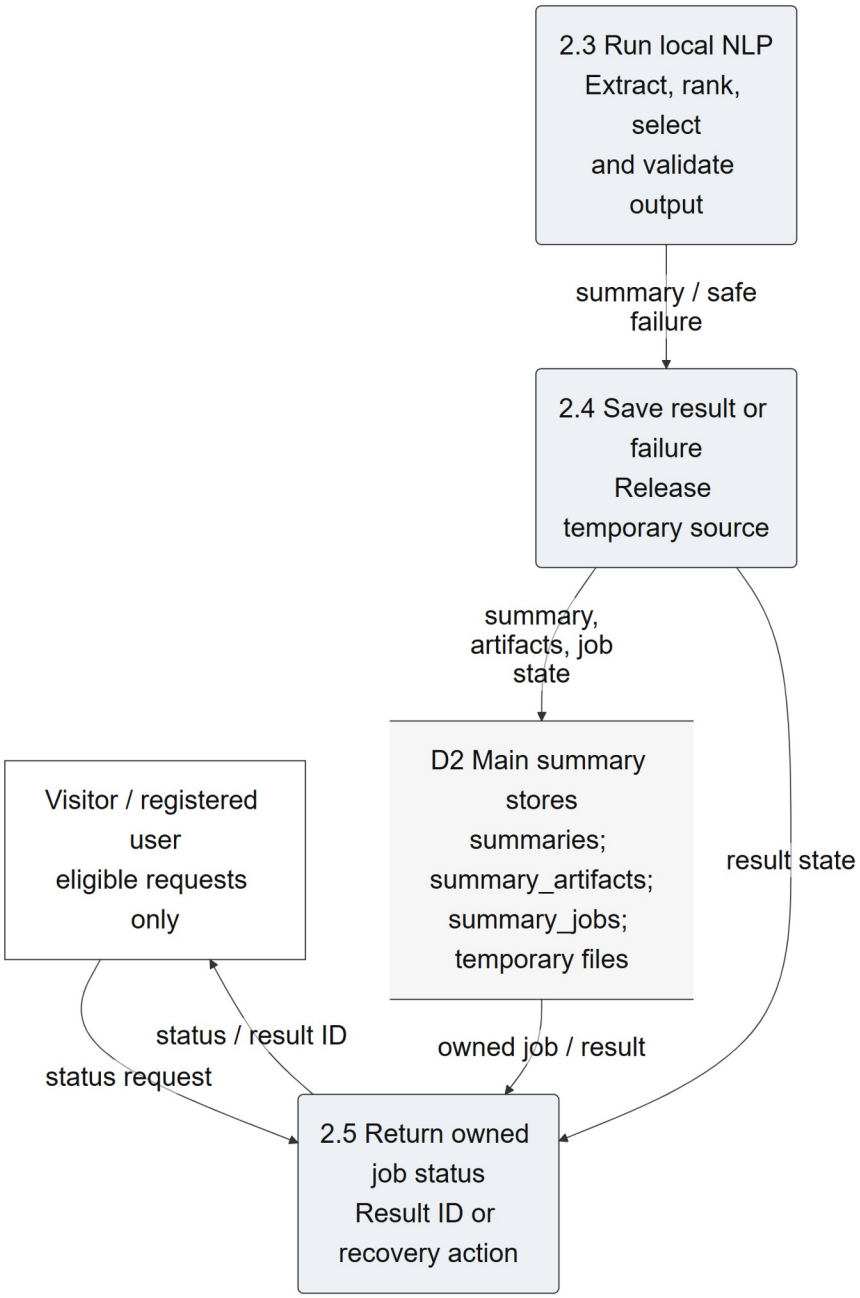



**Figure 20. Level 2 summary output detail.**

Figure 20 illustrates the named portion of the inspected data flow. Inputs and outputs pass through an application process; persistent tables, file storage and PHP session state retain their documented distinctions. The complete overview and editable sources retain the cross-panel context.

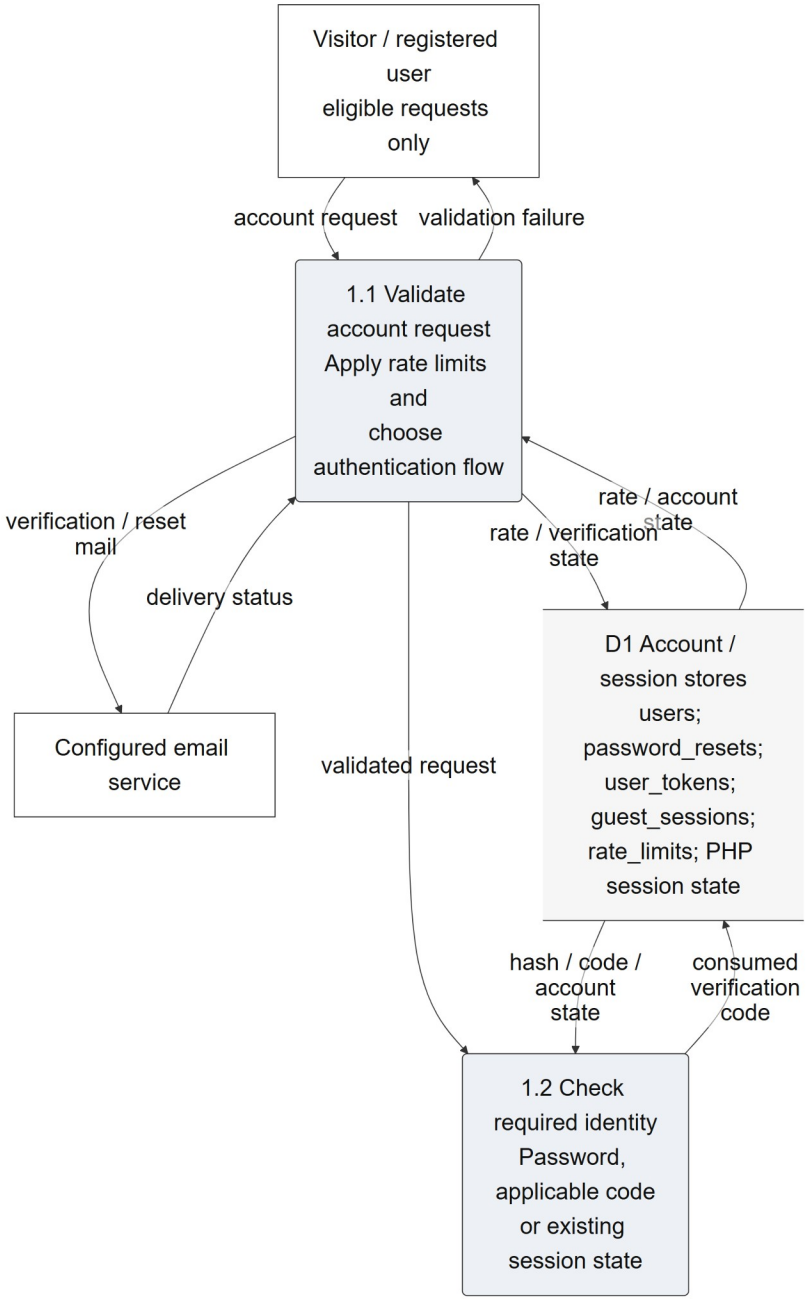



**Figure 21. Level 2 account verification detail.**

Figure 21 illustrates the named portion of the inspected data flow. Inputs and outputs

pass through an application process; persistent tables, file storage and PHP session state retain their documented distinctions. The complete overview and editable sources retain the cross-panel context.

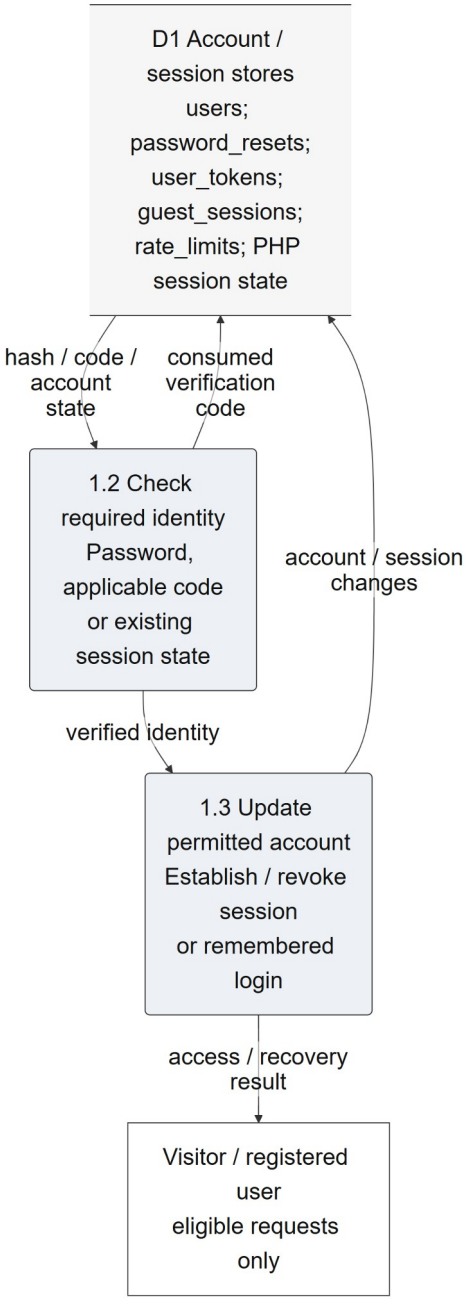



**Figure 22. Level 2 session and identity-change detail.**

Figure 22 illustrates the named portion of the inspected data flow. Inputs and outputs

pass through an application process; persistent tables, file storage and PHP session state retain their documented distinctions. The complete overview and editable sources retain the cross-panel context.

**C. Database Normalization**

The available evidence is a current schema snapshot, not a documented historical sequence from unnormalized forms to third normal form. Accordingly, this section describes present design decisions and constraints. User accounts are separated from generated summaries. Source artifacts, jobs, ratings and comment events have separate stores. Evaluation datasets, documents, references, generated outputs and ratings are represented independently, allowing repeated observations without placing every detail in one wide record.

An illustrative dependency is that a user identifier determines that account’s stored attributes, while a summary identifier determines its generated-summary record. Separate job and artifact identifiers support asynchronous processing and supporting material. This explanation is based on

declared keys and current code use; it does not prove that every semantic dependency has been exhaustively normalized. JSON/text payloads, caches, counters and snapshots deliberately retain compound or derived information, so a blanket claim that all tables are in third normal form would be unsupported.

The database declares thirty-five foreign keys. Several identifier-looking fields have no declared relationship: examples include documents.user_id, nutshell_generations.user_id and landing_page_sessions.user_id. Guest-token associations also depend on application logic. The summary_artifacts.summary_id column is not unique, so the physical schema permits more than one artifact per summary. These distinctions are preserved in the ERDs and dictionary rather than replaced with assumed one-to-one relationships.

**D. Entity Relationship Diagram**

The complete ERD was generated from the read-only MariaDB metadata snapshot. To keep the printed relationships legible, twelve focused panels repeat shared entities where necessary. Their combined coverage was

mechanically checked against all thirty-one entities and thirty-five declared foreign keys. Each relationship uses the actual key and nullability information. A missing line is not evidence that two concepts are unrelated; it means the live schema does not enforce that relationship as a foreign key.

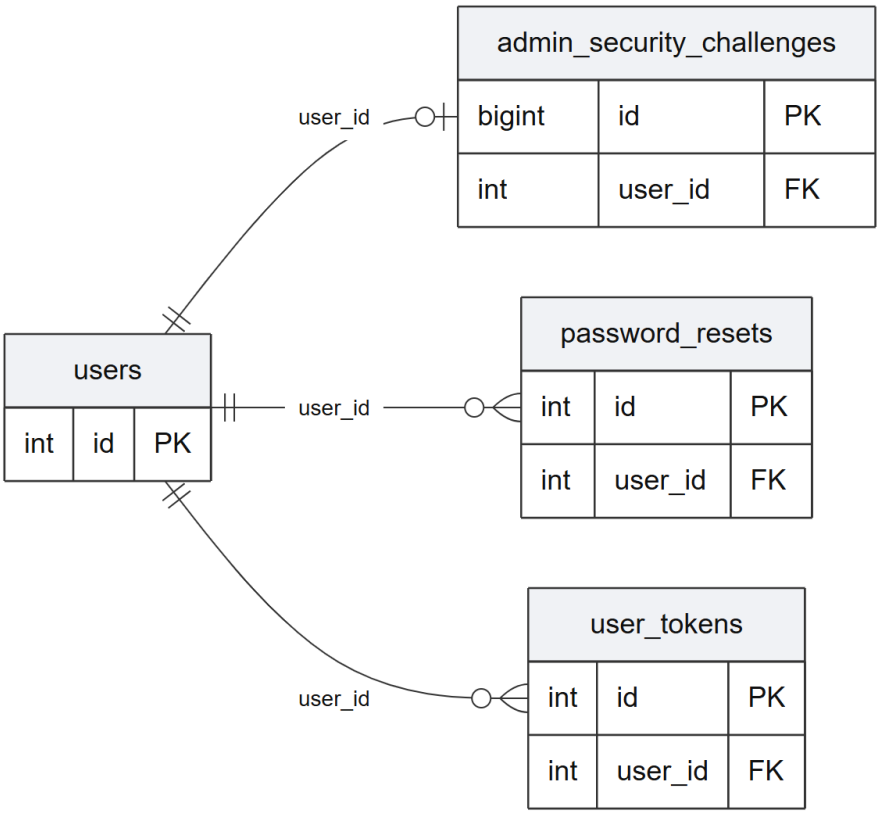



**Figure 23. Accounts and authentication ERD panel.**

Figure 23 illustrates the declared relationships in the accounts and authentication portion of the current database. Shared entities repeat across panels

to preserve context. Primary and foreign keys are shown here; all columns and exact constraints appear in the data dictionary.

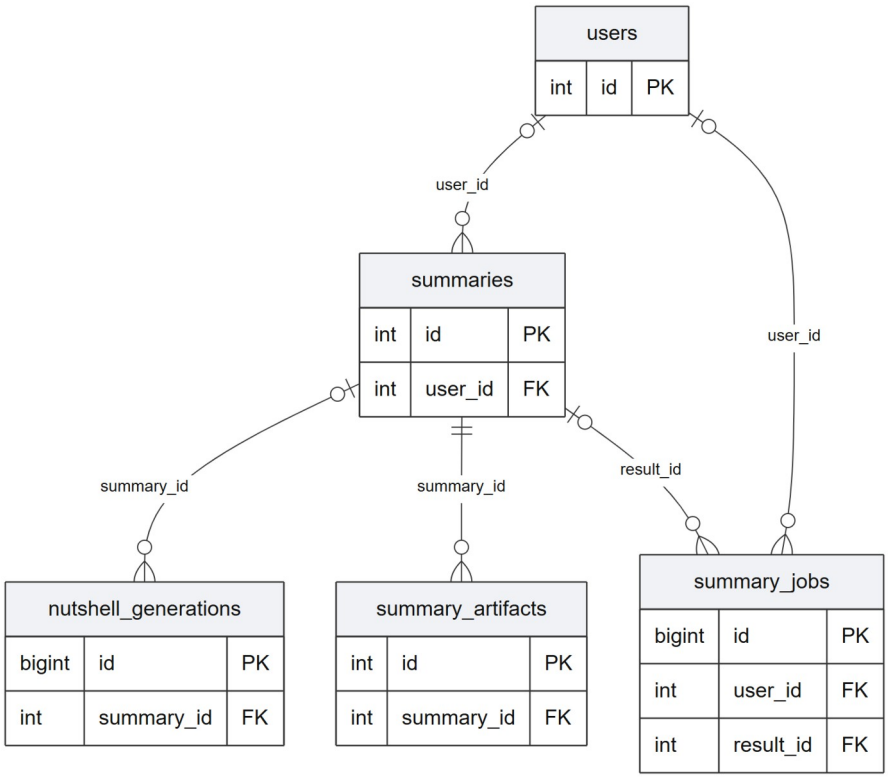



**Figure 24. Summary ownership, jobs and artifacts ERD panel.**

Figure 24 illustrates the declared relationships in the summary ownership, jobs and artifacts portion of the current database. Shared entities repeat across panels to preserve context. Primary and foreign keys are shown here; all columns and exact constraints appear in the data dictionary.

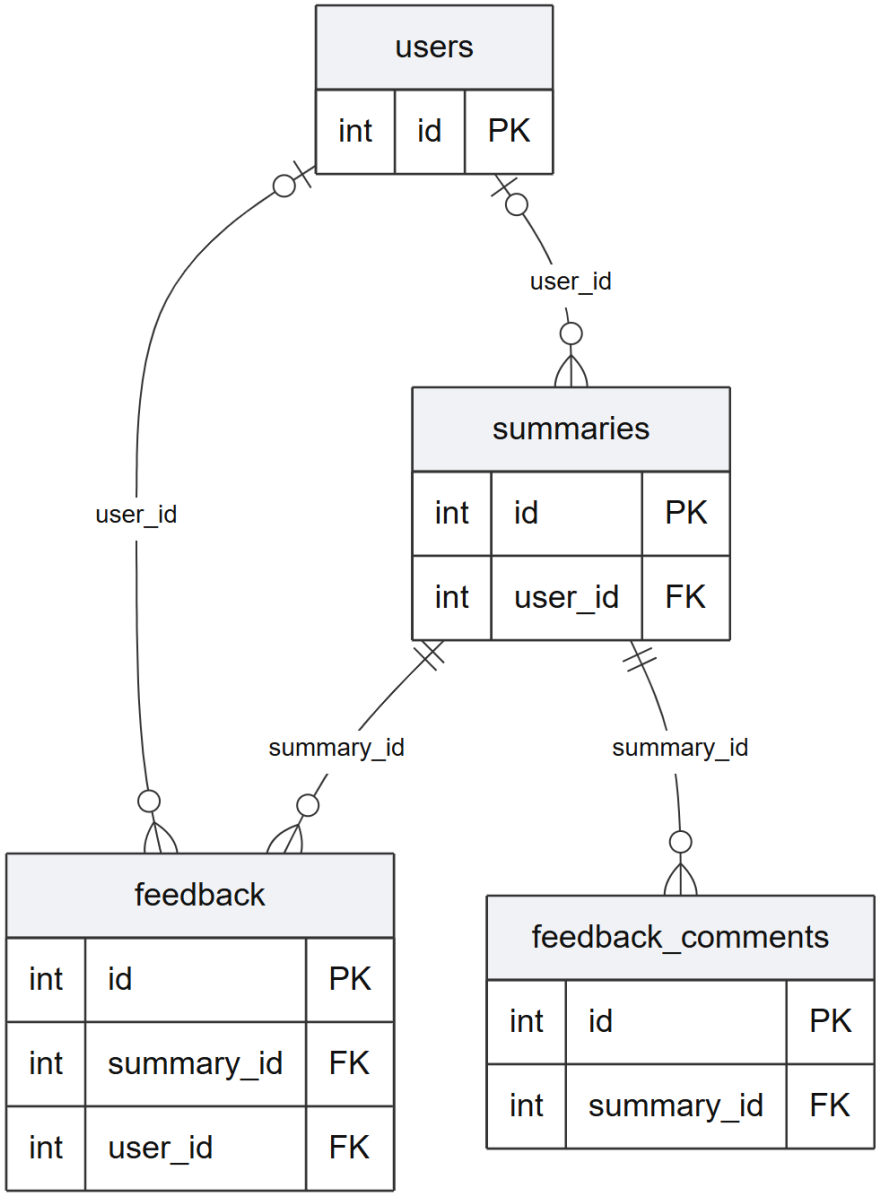



**Figure 25. Ratings and comments ERD panel.**

Figure 25 illustrates the declared relationships in the ratings and comments portion of the current database. Shared entities repeat across panels to preserve

context. Primary and foreign keys are shown here; all columns and exact constraints appear in the data dictionary.

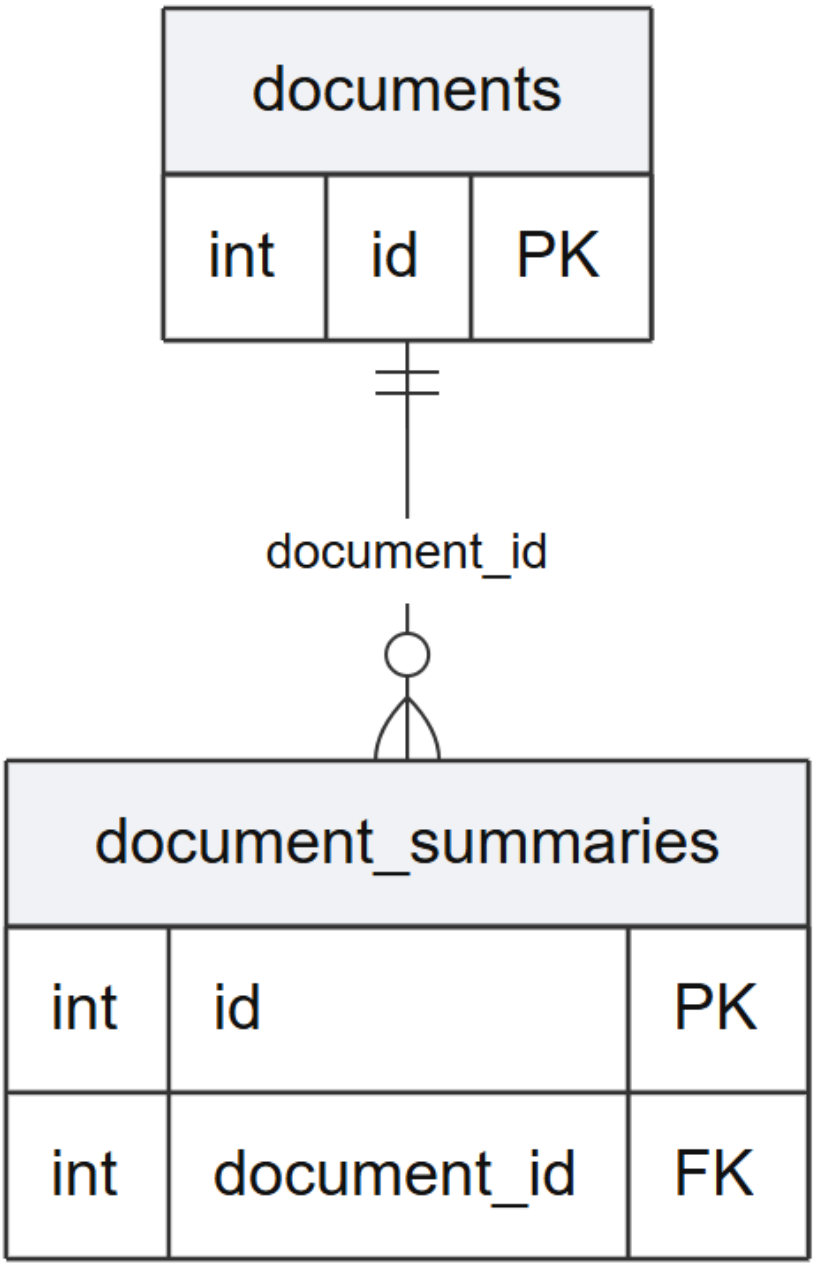



**Figure 26. Document extraction and cached combinations ERD panel.**

Figure 26 illustrates the declared relationships in the document extraction and

cached combinations portion of the current database. Shared entities repeat across panels to preserve context. Primary and foreign keys are shown here; all columns and exact constraints appear in the data dictionary.

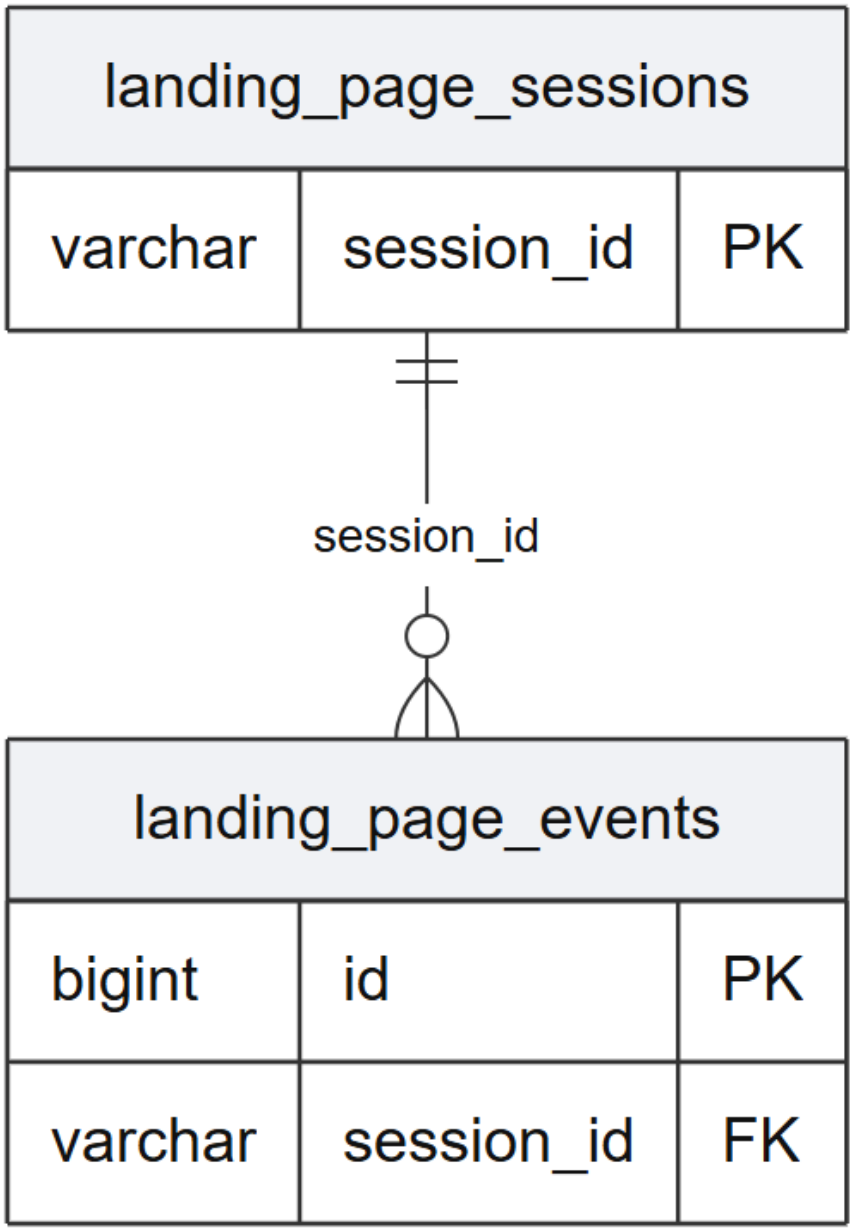



**Figure 27. Landing sessions and events ERD panel.**

Figure 27 illustrates the declared relationships in the landing sessions and events portion of the current database. Shared

entities repeat across panels to preserve context. Primary and foreign keys are shown here; all columns and exact constraints appear in the data dictionary.

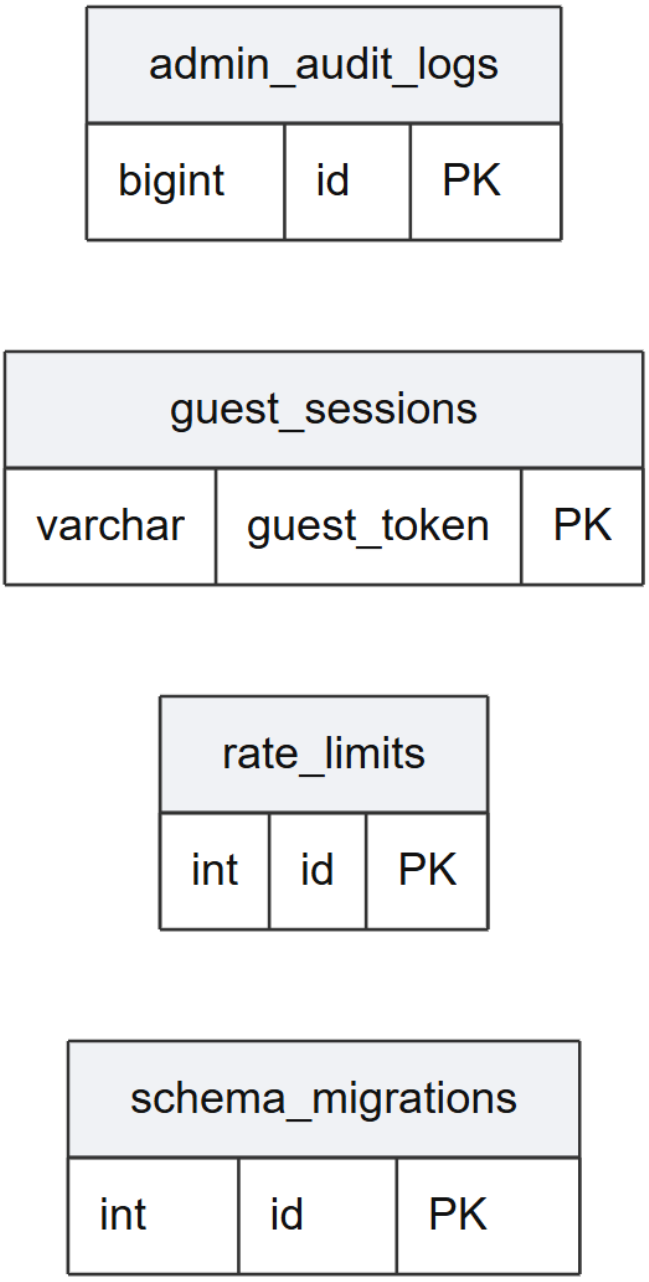



**Figure 28. Stores without declared foreign keys ERD panel.**

Figure 28 illustrates the current tables that have no declared foreign-key edges. Their

identifiers and application-level associations must not be mistaken for enforced relational constraints. The panel intentionally contains no inferred relationship lines.

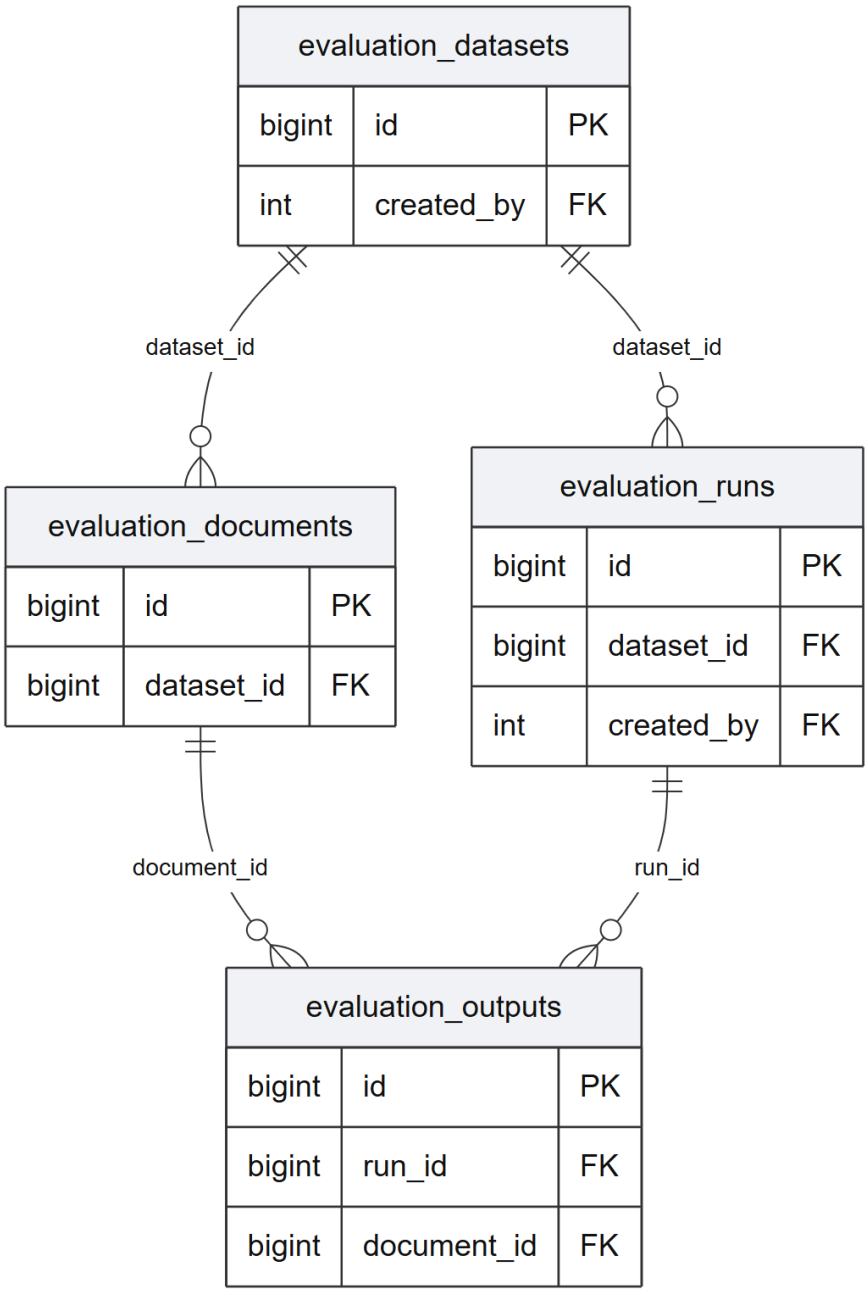



**Figure 29. Evaluation datasets, documents, runs and outputs ERD panel.**

Figure 29 illustrates the declared relationships in the evaluation datasets,

documents, runs and outputs portion of the current database. Shared entities repeat across panels to preserve context. Primary and foreign keys are shown here; all columns and exact constraints appear in the data dictionary.

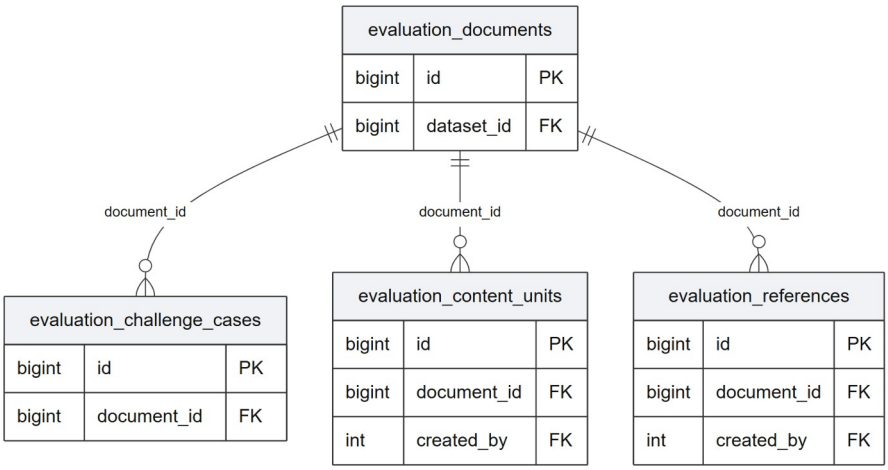



**Figure 30. References, content units and challenge cases ERD panel.**

Figure 30 illustrates the declared relationships in the references, content units and challenge cases portion of the current database. Shared entities repeat across panels to preserve context. Primary and foreign keys are shown here; all columns and exact constraints appear in the data dictionary.

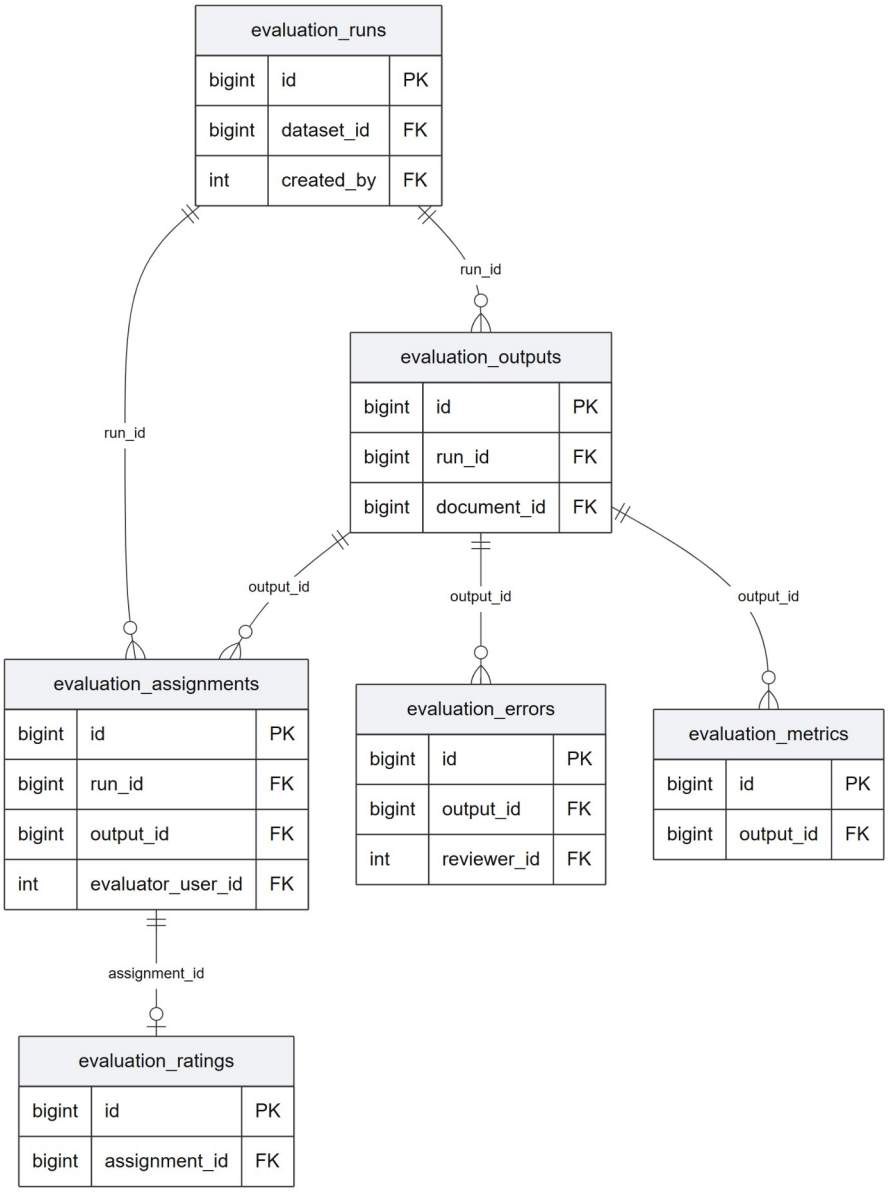



**Figure 31. Assignments, ratings, errors and metrics ERD panel.**

Figure 31 illustrates the declared relationships in the assignments, ratings, errors and metrics portion of the current database. Shared entities repeat across panels

to preserve context. Primary and foreign keys are shown here; all columns and exact constraints appear in the data dictionary.

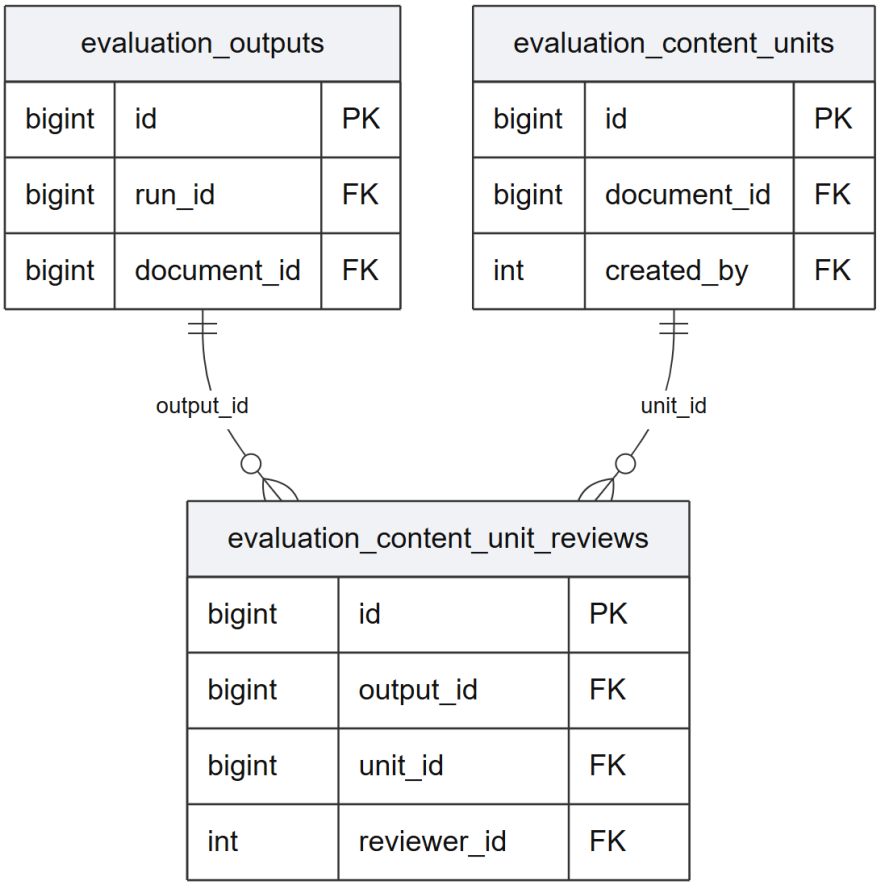



**Figure 32. Content-unit reviews ERD panel.**

Figure 32 illustrates the declared relationships in the content-unit reviews portion of the current database. Shared entities repeat across panels to preserve context. Primary and foreign keys are shown here; all columns and exact constraints appear in the data dictionary.

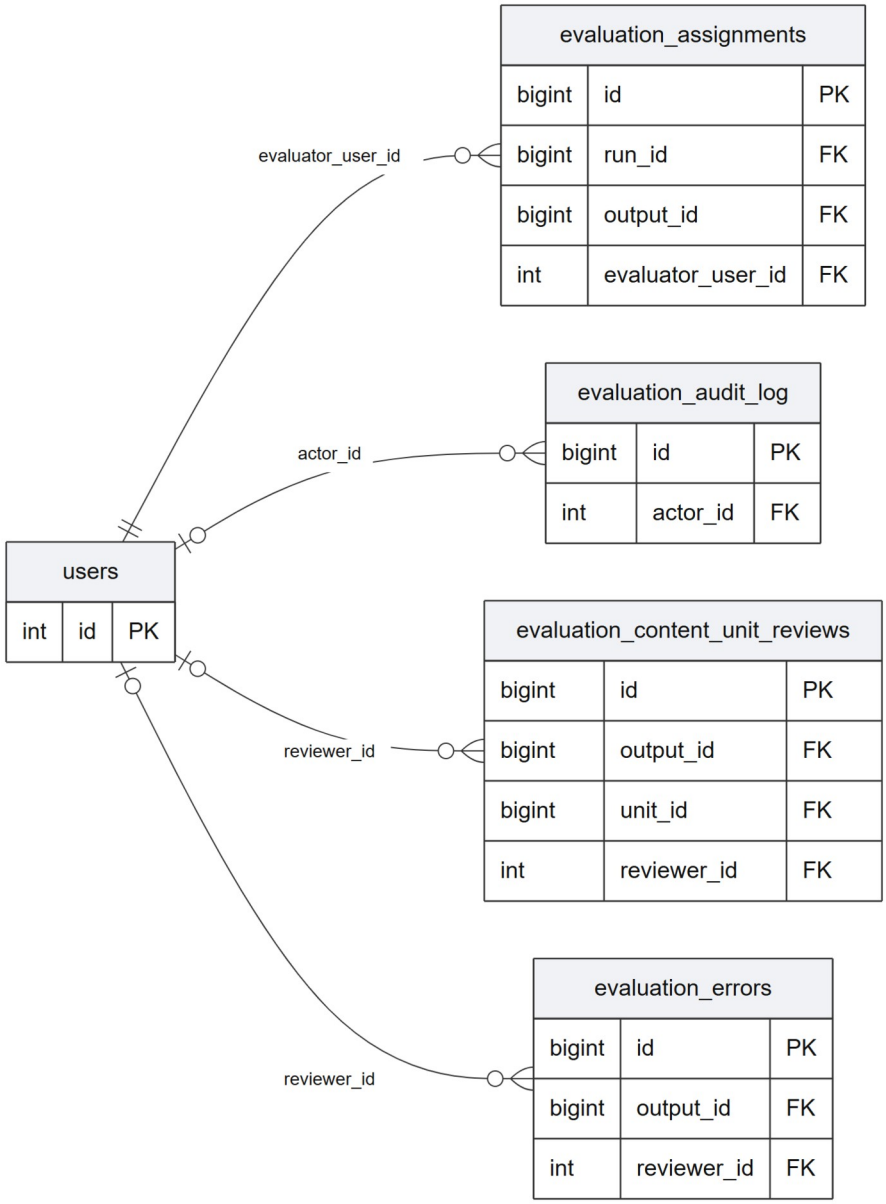



**Figure 33. Evaluator and reviewer account relationships ERD panel.**

Figure 33 illustrates the declared relationships in the evaluator and reviewer account relationships portion of the current database. Shared entities repeat across panels

to preserve context. Primary and foreign keys are shown here; all columns and exact constraints appear in the data dictionary.

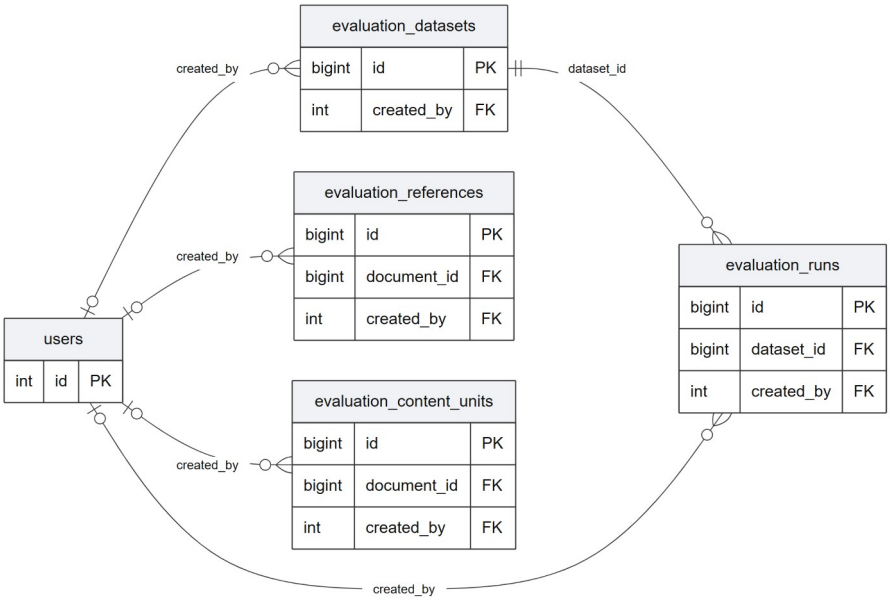



**Figure 34. Dataset, run and reference authorship ERD panel.**

Figure 34 illustrates the declared relationships in the dataset, run and reference authorship portion of the current database. Shared entities repeat across panels to preserve context. Primary and foreign keys are shown here; all columns and exact constraints appear in the data dictionary.

## E. Data Dictionary

The following tables document every column in the observed database: thirty-one tables

and three hundred twelve fields. Types, lengths, nullability, defaults, keys and declared referential actions are metadata facts. Descriptions explain likely field purposes based on names and inspected service use; they do not imply that legacy fields are actively used. No application rows, passwords, tokens or personal records were extracted for this dictionary.

The live password_resets structure uses used_at, whereas the base installation SQL defines used and created_at. No inspected migration resolves that difference. Consequently, the dictionary represents the observed database, while a new installation requires a separate schema-reconciliation and rehearsal task. A successful connection to the current database would not establish that fresh installation is reliable.

######### Data dictionary — verified live schema

Metadata was read directly from information_schema without loading the application bootstrap or reading application rows. The snapshot contains 31 tables, 312 columns and 35 foreign-key constraints. SQL types, defaults, nullability, keys and

relationships below are observed metadata. Field-purpose prose is an interpretation of names and inspected service use, not a claim that every field is actively used.

The current live schema differs from the fresh-install scripts for password_resets: live uses used_at and has no used/created_at columns, while database/schema.sql defines used and created_at and no checked-in migration adds used_at. Do not treat the base SQL alone as the deployed schema.

Tables are presented alphabetically. Each table has a manuscript caption and a stable dictionary identifier.

### DD-01. admin_audit_logs

Administrative action evidence and target references.

**Table 7. Data dictionary for admin_audit_logs.**

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| id | Unique row identifier. | bigint(20) unsigned | NOT NULL; PRIMARY KEY; auto_increment; default SQL NULL / no explicit non-null default |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| admin_id | Admin id. | int(11) | NULL allowed; default NULL |
| action | Action. | varchar(100) | NOT NULL; default SQL NULL / no explicit non-null default |
| entity_type | Entity type. | varchar(100) | NULL allowed; default NULL |
| entity_id | Entity id. | bigint(20) unsigned | NULL allowed; default NULL |
| ip_address | Ip address. | varchar(45) | NULL allowed; default NULL |
| user_agent | User agent. | varchar(255) | NULL allowed; default NULL |
| details | Details. | text | NULL allowed; default NULL |
| created_at | Creation timestamp. | datetime | NOT NULL; default current_timestamp() |

Indexes and constraints:

- PRIMARY: UNIQUE (id).
- idx_admin_audit_entity: nonunique (entity_type, entity_id).

- idx_admin_audit_logs_action: nonunique (action).
- idx_admin_audit_logs_admin_id: nonunique (admin_id).
- idx_admin_audit_logs_created_at: nonunique (created_at).

The table supports administrative action evidence and target references. Its constraints describe enforced storage behavior; runtime verification is reported separately.

Table 7 records the observed columns and constraints for admin_audit_logs. The descriptions explain their storage purpose; runtime behavior remains subject to the separately stated verification limits.

**DD-02. admin_security_challenges**

Legacy administrator challenge storage; current login uses emailed codes.

**Table 8. Data dictionary for admin_security_challenges.**

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| id | Unique row identifier. | bigint(20) unsigned | NOT NULL; PRIMARY KEY; auto_increment; default SQL NULL / no explicit non-null default |
| user_id | Registered account identifier; an enforced relation only where listed below. | int(11) | NOT NULL; default SQL NULL / no explicit non-null default; FK → users.id; DELETE CASCADE; UPDATE CASCADE |
| challenge_hash | Challenge hash. | varchar(255) | NOT NULL; default SQL NULL / no explicit non-null default |
| failed_attempts | Failed attempts. | int(10) unsigned | NOT NULL; default 0 |
| locked_until | Locked until. | datetime | NULL allowed; default NULL |
| created_at | Creation timestamp. | datetime | NOT NULL; default current_timestamp() |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| updated_at | Most recent update timestamp. | datetime | NULL allowed; default NULL |

Indexes and constraints:

- PRIMARY: UNIQUE (id).
- idx_admin_security_locked_until: nonunique (locked_until).
- uq_admin_security_user: UNIQUE (user_id).

The table supports legacy administrator challenge storage; current login uses emailed codes. Its constraints describe enforced storage behavior; runtime verification is reported separately.

Table 8 records the observed columns and constraints for admin_security_challenges. The descriptions explain their storage purpose; runtime behavior remains subject to the separately stated verification limits.

**DD-03. documents**

Independent document API source content and nominal owner fields.

**Table 9. Data dictionary for documents.**

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| id | Unique row identifier. | int(11) | NOT NULL; PRIMARY KEY; auto_increment; default SQL NULL / no explicit non-null default |
| content | Stored source document text. | longtext | NOT NULL; default SQL NULL / no explicit non-null default |
| user_id | Registered account identifier; an enforced relation only where listed below. | int(11) | NULL allowed; default NULL |
| guest_token | Anonymous owner/session token; not a foreign key unless explicitly listed. | varchar(64) | NULL allowed; default NULL |
| created_at | Creation timestamp. | timestamp | NOT NULL; default current_timestamp() |

Indexes and constraints:

- PRIMARY: UNIQUE (id).
- idx_documents_created_at: nonunique (created_at).
- idx_documents_guest: nonunique (guest_token).
- idx_documents_user: nonunique (user_id).

The table supports independent document api source content and nominal owner fields. Its constraints describe enforced storage behavior; runtime verification is reported separately.

Table 9 records the observed columns and constraints for documents. The descriptions explain their storage purpose; runtime behavior remains subject to the separately stated verification limits.

### DD-04. document_summaries

Cached summary for each document/profile/length combination.

**Table 10. Data dictionary for document_summaries.**

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| id | Unique row identifier. | int(11) | NOT NULL; PRIMARY KEY; auto_increment; default SQL NULL / no explicit non-null default |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| document_id | Related document identifier in this table's domain. | int(11) | NOT NULL; default SQL NULL / no explicit non-null default; FK → documents.id; DELETE CASCADE; UPDATE RESTRICT |
| profile | Content emphasis/profile selection. | varchar(32) | NOT NULL; default SQL NULL / no explicit non-null default |
| length | Summary depth selection. | varchar(32) | NOT NULL; default SQL NULL / no explicit non-null default |
| summary | Summary. | longtext | NOT NULL; default SQL NULL / no explicit non-null default |
| word_count | Word count. | int(11) | NOT NULL; default SQL NULL / no explicit non-null default |
| source_word_count | Source word count. | int(11) | NOT NULL; default SQL NULL / no explicit non-null default |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| compression_ratio | Module-specific compression metric; check service semantics before comparisons. | decimal(5, 2) | NOT NULL; default SQL NULL / no explicit non-null default |
| estimated_reading_time_seconds | Estimated reading time seconds. | int(11) | NOT NULL; default SQL NULL / no explicit non-null default |
| created_at | Creation timestamp. | timestamp | NOT NULL; default current_timestamp() |

Indexes and constraints:

- PRIMARY: UNIQUE (id).
- idx_doc_summaries_created_at: nonunique (created_at).
- idx_document_id: nonunique (document_id).
- uq_doc_profile_length: UNIQUE (document_id, profile, length).

The table supports cached summary for each document/profile/length combination. Its constraints describe enforced storage behavior; runtime verification is reported separately.

Table 10 records the observed columns and constraints for document_summaries. The descriptions explain their storage purpose; runtime behavior remains subject to the separately stated verification limits.

### DD-05. evaluation_assignments

Blinded output assignments to registered evaluator accounts.

**Table 11. Data dictionary for evaluation_assignments.**

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| id | Unique row identifier. | bigint(20) unsigned | NOT NULL; PRIMARY KEY; auto_increment; default SQL NULL / no explicit non-null default |
| assignment_token | Assignment token. | char(48) | NOT NULL; default SQL NULL / no explicit non-null default |
| run_id | Related evaluation run identifier. | bigint(20) unsigned | NOT NULL; default SQL NULL / no explicit non-null default; FK → evaluation_runs.id; DELETE RESTRICT; UPDATE RESTRICT |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| output_id | Related generated evaluation output identifier. | bigint(20) unsigned | NOT NULL; default SQL NULL / no explicit non-null default; FK → evaluation_outputs.id; DELETE RESTRICT; UPDATE RESTRICT |
| evaluator_user_id | Evaluator user id. | int(11) | NOT NULL; default SQL NULL / no explicit non-null default; FK → users.id; DELETE RESTRICT; UPDATE RESTRICT |
| evaluator_code | Evaluator code. | varchar(20) | NOT NULL; default SQL NULL / no explicit non-null default |
| blind_label | Evaluator-visible label that conceals system identity. | varchar(50) | NOT NULL; default SQL NULL / no explicit non-null default |
| presentation_order | Presentation order. | int(10) unsigned | NOT NULL; default SQL NULL / no explicit non-null default |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| randomization_seed | Randomization seed. | char(64) | NOT NULL; default SQL NULL / no explicit non-null default |
| status | Domain status marker. | varchar(20) | NOT NULL; default 'assigned' |
| created_at | Creation timestamp. | datetime | NOT NULL; default current_timestamp() |
| submitted_at | Submitted at. | datetime | NULL allowed; default NULL |

Indexes and constraints:

- PRIMARY: UNIQUE (id).
- assignment_token: UNIQUE (assignment_token).
- fk_eval_assignment_run: nonunique (run_id).
- idx_eval_assignment_user: nonunique (evaluator_user_id, status, presentation_order).
- uq_eval_assignment: UNIQUE (output_id, evaluator_user_id).

The table supports blinded output assignments to registered evaluator accounts. Its constraints describe enforced storage behavior; runtime verification is reported separately.

Table 11 records the observed columns and constraints for evaluation_assignments. The descriptions explain their storage purpose; runtime behavior remains subject to the separately stated verification limits.

### DD-06. evaluation_audit_log

Changes to evaluation records with old/new values and reasons.

**Table 12. Data dictionary for evaluation_audit_log.**

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| id | Unique row identifier. | bigint(20) unsigned | NOT NULL; PRIMARY KEY; auto_increment; default SQL NULL / no explicit non-null default |
| actor_id | Actor id. | int(11) | NULL allowed; default NULL; FK → users.id; DELETE SET NULL; UPDATE RESTRICT |
| action | Action. | varchar(100) | NOT NULL; default SQL NULL / no explicit non-null default |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| entity_type | Entity type. | varchar(50) | NOT NULL; default SQL NULL / no explicit non-null default |
| entity_id | Entity id. | bigint(20) unsigned | NOT NULL; default SQL NULL / no explicit non-null default |
| previous_json | Previous json. | longtext | NULL allowed; default NULL |
| replacement_json | Replacement json. | longtext | NULL allowed; default NULL |
| reason | Reason. | text | NULL allowed; default NULL |
| created_at | Creation timestamp. | datetime | NOT NULL; default current_timestamp() |

Indexes and constraints:

- PRIMARY: UNIQUE (id).
- fk_eval_audit_user: nonunique (actor_id).
- idx_eval_audit_entity: nonunique (entity_type, entity_id).

The table supports changes to evaluation records with old/new values and reasons. Its constraints describe enforced storage

behavior; runtime verification is reported separately.

Table 12 records the observed columns and constraints for evaluation_audit_log. The descriptions explain their storage purpose; runtime behavior remains subject to the separately stated verification limits.

### DD-07. evaluation_challenge_cases

Expected facts and prohibited distortions for challenge cases.

**Table 13. Data dictionary for evaluation_challenge_cases.**

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| id | Unique row identifier. | bigint(20) unsigned | NOT NULL; PRIMARY KEY; auto_increment; default SQL NULL / no explicit non-null default |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| document_id | Related document identifier in this table's domain. | bigint(20) unsigned | NOT NULL; default SQL NULL / no explicit non-null default; FK → evaluation_documents.id; DELETE RESTRICT; UPDATE RESTRICT |
| category | Category. | varchar(100) | NOT NULL; default SQL NULL / no explicit non-null default |
| expected_facts_json | Expected facts json. | longtext | NOT NULL; default SQL NULL / no explicit non-null default |
| prohibited_distortions_json | Prohibited distortions json. | longtext | NOT NULL; default SQL NULL / no explicit non-null default |
| notes | Notes. | text | NULL allowed; default NULL |
| created_at | Creation timestamp. | datetime | NOT NULL; default current_timestamp() |

Indexes and constraints:

- PRIMARY: UNIQUE (id).

- fk_eval_challenge_doc: nonunique (document_id).

The table supports expected facts and prohibited distortions for challenge cases. Its constraints describe enforced storage behavior; runtime verification is reported separately.

Table 13 records the observed columns and constraints for evaluation_challenge_cases. The descriptions explain their storage purpose; runtime behavior remains subject to the separately stated verification limits.

### DD-08. evaluation_content_units

Essential source facts to assess coverage.

**Table 14. Data dictionary for evaluation_content_units.**

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| id | Unique row identifier. | bigint(20) unsigned | NOT NULL; PRIMARY KEY; auto_increment; default SQL NULL / no explicit non-null default |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| document_id | Related document identifier in this table’s domain. | bigint(20) unsigned | NOT NULL; default SQL NULL / no explicit non-null default; FK → evaluation_documents.id; DELETE RESTRICT; UPDATE RESTRICT |
| unit_text | Unit text. | text | NOT NULL; default SQL NULL / no explicit non-null default |
| unit_type | Unit type. | varchar(100) | NOT NULL; default 'essential_information' |
| created_by | Created by. | int(11) | NULL allowed; default NULL; FK → users.id; DELETE SET NULL; UPDATE RESTRICT |
| created_at | Creation timestamp. | datetime | NOT NULL; default current_timestamp() |

Indexes and constraints:

- PRIMARY: UNIQUE (id).
- fk_eval_unit_doc: nonunique (document_id).

- fk_eval_unit_user: nonunique (created_by).

The table supports essential source facts to assess coverage. Its constraints describe enforced storage behavior; runtime verification is reported separately.

Table 14 records the observed columns and constraints for evaluation_content_units. The descriptions explain their storage purpose; runtime behavior remains subject to the separately stated verification limits.

### DD-09. evaluation_content_unit_reviews

Human judgments of whether output represents a content unit.

**Table 15. Data dictionary for evaluation_content_unit_reviews.**

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| id | Unique row identifier. | bigint(20) unsigned | NOT NULL; PRIMARY KEY; auto_increment; default SQL NULL / no explicit non-null default |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| output_id | Related generated evaluation output identifier. | bigint(20) unsigned | NOT NULL; default SQL NULL / no explicit non-null default; FK → evaluation_outputs.id; DELETE RESTRICT; UPDATE RESTRICT |
| unit_id | Unit id. | bigint(20) unsigned | NOT NULL; default SQL NULL / no explicit non-null default; FK → evaluation_content_units.id; DELETE RESTRICT; UPDATE RESTRICT |
| represented | Represented. | tinyint(1) | NOT NULL; default SQL NULL / no explicit non-null default |
| notes | Notes. | text | NULL allowed; default NULL |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| reviewer_id | Reviewer id. | int(11) | NULL allowed; default NULL; FK → users.id; DELETE SET NULL; UPDATE RESTRICT |
| created_at | Creation timestamp. | datetime | NOT NULL; default current_timestamp() |

Indexes and constraints:

- PRIMARY: UNIQUE (id).
- fk_eval_review_unit: nonunique (unit_id).
- fk_eval_review_user: nonunique (reviewer_id).
- uq_eval_unit_review: UNIQUE (output_id, unit_id).

The table supports human judgments of whether output represents a content unit. Its constraints describe enforced storage behavior; runtime verification is reported separately.

Table 15 records the observed columns and constraints for evaluation_content_unit_reviews. The descriptions explain their storage purpose; runtime behavior remains subject to the separately stated verification limits.

### DD-10. evaluation_datasets

Versioned evaluation collections and sealing state.

**Table 16. Data dictionary for evaluation_datasets.**

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| id | Unique row identifier. | bigint(20) unsigned | NOT NULL; PRIMARY KEY; auto_increment; default SQL NULL / no explicit non-null default |
| version | Version. | varchar(100) | NOT NULL; default SQL NULL / no explicit non-null default |
| name | Name. | varchar(255) | NOT NULL; default SQL NULL / no explicit non-null default |
| notes | Notes. | text | NULL allowed; default NULL |
| target_count | Target count. | int(10) unsigned | NOT NULL; default 48 |
| active | Enabled/active state. | tinyint(1) | NOT NULL; default 1 |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| sealed_at | Sealed at. | datetime | NULL allowed; default NULL |
| created_by | Created by. | int(11) | NULL allowed; default NULL; FK → users.id; DELETE SET NULL; UPDATE RESTRICT |
| created_at | Creation timestamp. | datetime | NOT NULL; default current_timestamp() |

Indexes and constraints:

- PRIMARY: UNIQUE (id).
- fk_eval_dataset_user: nonunique (created_by).
- version: UNIQUE (version).

The table supports versioned evaluation collections and sealing state. Its constraints describe enforced storage behavior; runtime verification is reported separately.

Table 16 records the observed columns and constraints for evaluation_datasets. The descriptions explain their storage purpose; runtime behavior remains subject to the separately stated verification limits.

## DD-11. evaluation_documents

Dataset source documents, split, profile and provenance.

**Table 17. Data dictionary for evaluation_documents.**

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| id | Unique row identifier. | bigint(20) unsigned | NOT NULL; PRIMARY KEY; auto_increment; default SQL NULL / no explicit non-null default |
| dataset_id | Dataset id. | bigint(20) unsigned | NOT NULL; default SQL NULL / no explicit non-null default; FK → evaluation_datasets.id; DELETE RESTRICT; UPDATE RESTRICT |
| title | Title. | varchar(255) | NOT NULL; default SQL NULL / no explicit non-null default |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| category | Category. | varchar(100) | NOT NULL; default SQL NULL / no explicit non-null default |
| source_text | Source text. | longtext | NOT NULL; default SQL NULL / no explicit non-null default |
| source_hash | Hash used to identify source content. | char(64) | NOT NULL; default SQL NULL / no explicit non-null default |
| source_reference | Source reference. | varchar(500) | NULL allowed; default NULL |
| original_word_count | Source word count; main file path currently counts the empty input string. | int(10) unsigned | NOT NULL; default SQL NULL / no explicit non-null default |
| profile | Content emphasis/profile selection. | varchar(32) | NOT NULL; default SQL NULL / no explicit non-null default |
| dataset_split | Dataset split. | varchar(20) | NOT NULL; default SQL NULL / no explicit non-null default |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| provenance | Provenance. | text | NOT NULL; default SQL NULL / no explicit non-null default |
| notes | Notes. | text | NULL allowed; default NULL |
| active | Enabled/active state. | tinyint(1) | NOT NULL; default 1 |
| created_at | Creation timestamp. | datetime | NOT NULL; default current_timestamp() |

Indexes and constraints:

- PRIMARY: UNIQUE (id).
- idx_eval_doc_dataset: nonunique (dataset_id, dataset_split, active).

The table supports dataset source documents, split, profile and provenance. Its constraints describe enforced storage behavior; runtime verification is reported separately.

Table 17 records the observed columns and constraints for evaluation_documents. The descriptions explain their storage purpose; runtime behavior remains subject to the separately stated verification limits.

## DD-12. evaluation_errors

Reviewer-classified error observations.

**Table 18. Data dictionary for evaluation_errors.**

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| id | Unique row identifier. | bigint(20) unsigned | NOT NULL; PRIMARY KEY; auto_increment; default SQL NULL / no explicit non-null default |
| output_id | Related generated evaluation output identifier. | bigint(20) unsigned | NOT NULL; default SQL NULL / no explicit non-null default; FK → evaluation_outputs.id; DELETE RESTRICT; UPDATE RESTRICT |
| category | Category. | varchar(100) | NOT NULL; default SQL NULL / no explicit non-null default |
| notes | Notes. | text | NULL allowed; default NULL |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| reviewer_id | Reviewer id. | int(11) | NULL allowed; default NULL; FK → users.id; DELETE SET NULL; UPDATE RESTRICT |
| created_at | Creation timestamp. | datetime | NOT NULL; default current_timestamp() |

Indexes and constraints:

- PRIMARY: UNIQUE (id).
- fk_eval_error_output: nonunique (output_id).
- fk_eval_error_user: nonunique (reviewer_id).
- idx_eval_error_category: nonunique (category).

The table supports reviewer-classified error observations. Its constraints describe enforced storage behavior; runtime verification is reported separately.

Table 18 records the observed columns and constraints for evaluation_errors. The descriptions explain their storage purpose; runtime behavior remains subject to the separately stated verification limits.

### DD-13. evaluation_metrics

Metric values, metric versions and availability/error status.

**Table 19. Data dictionary for evaluation_metrics.**

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| id | Unique row identifier. | bigint(20) unsigned | NOT NULL; PRIMARY KEY; auto_increment; default SQL NULL / no explicit non-null default |
| output_id | Related generated evaluation output identifier. | bigint(20) unsigned | NOT NULL; default SQL NULL / no explicit non-null default; FK → evaluation_outputs.id; DELETE RESTRICT; UPDATE RESTRICT |
| metric_name | Metric name. | varchar(100) | NOT NULL; default SQL NULL / no explicit non-null default |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| metric_version | Metric version. | varchar(100) | NOT NULL; default SQL NULL / no explicit non-null default |
| status | Domain status marker. | varchar(30) | NOT NULL; default SQL NULL / no explicit non-null default |
| metric_value | Metric value, nullable when unavailable. | double | NULL allowed; default NULL |
| precision_value | Precision value. | double | NULL allowed; default NULL |
| recall_value | Recall value. | double | NULL allowed; default NULL |
| f1_value | F1 value. | double | NULL allowed; default NULL |
| details_json | Details json. | longtext | NULL allowed; default NULL |
| error_message | Error message. | text | NULL allowed; default NULL |
| created_at | Creation timestamp. | datetime | NOT NULL; default current_timestamp() |

Indexes and constraints:

- PRIMARY: UNIQUE (id).
- idx_eval_metric_name: nonunique (metric_name, status).
- uq_eval_metric: UNIQUE (output_id, metric_name).

The table supports metric values, metric versions and availability/error status. Its constraints describe enforced storage behavior; runtime verification is reported separately.

Table 19 records the observed columns and constraints for evaluation_metrics. The descriptions explain their storage purpose; runtime behavior remains subject to the separately stated verification limits.

**DD-14. evaluation_outputs**

Generated output for a run/document/mode/system/ablation combination.

**Table 20. Data dictionary for evaluation_outputs.**

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| id | Unique row identifier. | bigint(20) unsigned | NOT NULL; PRIMARY KEY; auto_increment; default SQL NULL / no explicit non-null default |
| run_id | Related evaluation run identifier. | bigint(20) unsigned | NOT NULL; default SQL NULL / no explicit non-null default; FK → evaluation_runs.id; DELETE RESTRICT; UPDATE RESTRICT |
| document_id | Related document identifier in this table’s domain. | bigint(20) unsigned | NOT NULL; default SQL NULL / no explicit non-null default; FK → evaluation_documents.id; DELETE RESTRICT; UPDATE RESTRICT |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| profile | Content emphasis/profile selection. | varchar(32) | NOT NULL; default SQL NULL / no explicit non-null default |
| mode | Mode. | varchar(32) | NOT NULL; default SQL NULL / no explicit non-null default |
| system_name | System name. | varchar(50) | NOT NULL; default SQL NULL / no explicit non-null default |
| ablation | Ablation. | varchar(100) | NOT NULL; default '' |
| status | Domain status marker. | varchar(30) | NOT NULL; default 'queued' |
| summary_text | Summary text. | longtext | NULL allowed; default NULL |
| word_count | Word count. | int(10) unsigned | NULL allowed; default NULL |
| source_word_count | Source word count. | int(10) unsigned | NULL allowed; default NULL |
| target_words | Target words. | int(10) unsigned | NULL allowed; default NULL |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| processing_seconds | Processing seconds. | double | NULL allowed; default NULL |
| timings_json | Timings json. | text | NULL allowed; default NULL |
| configuration_hash | Hash of canonical run configuration. | char(64) | NULL allowed; default NULL |
| configuration_json | Configuration json. | longtext | NULL allowed; default NULL |
| provenance_json | Serialized execution/source provenance. | longtext | NULL allowed; default NULL |
| warnings_json | Warnings json. | longtext | NULL allowed; default NULL |
| error_message | Error message. | text | NULL allowed; default NULL |
| generated_at | Generated at. | datetime | NULL allowed; default NULL |

Indexes and constraints:

- PRIMARY: UNIQUE (id).
- fk_eval_output_doc: nonunique (document_id).
- idx_eval_output_run_status: nonunique (run_id, status).

- uq_eval_output: UNIQUE (run_id, document_id, mode, system_name, ablation).

The table supports generated output for a run/document/mode/system/ablation combination. Its constraints describe enforced storage behavior; runtime verification is reported separately.

Table 20 records the observed columns and constraints for evaluation_outputs. The descriptions explain their storage purpose; runtime behavior remains subject to the separately stated verification limits.

### DD-15. evaluation_ratings

Human criterion ratings for an assignment.

**Table 21. Data dictionary for evaluation_ratings.**

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| id | Unique row identifier. | bigint(20) unsigned | NOT NULL; PRIMARY KEY; auto_increment; default SQL NULL / no explicit non-null default |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| assignment_id | Assignment id. | bigint(20) unsigned | NOT NULL; default SQL NULL / no explicit non-null default; FK → evaluation_assignments.id; DELETE RESTRICT; UPDATE RESTRICT |
| relevance | Relevance. | tinyint(3) unsigned | NOT NULL; default SQL NULL / no explicit non-null default |
| factual_consistency | Factual consistency. | tinyint(3) unsigned | NOT NULL; default SQL NULL / no explicit non-null default |
| coverage | Coverage. | tinyint(3) unsigned | NOT NULL; default SQL NULL / no explicit non-null default |
| coherence | Coherence. | tinyint(3) unsigned | NOT NULL; default SQL NULL / no explicit non-null default |
| readability | Readability. | tinyint(3) unsigned | NOT NULL; default SQL NULL / no explicit non-null default |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| conciseness | Conciseness. | tinyint(3) unsigned | NOT NULL; default SQL NULL / no explicit non-null default |
| non_redundancy | Non redundancy. | tinyint(3) unsigned | NOT NULL; default SQL NULL / no explicit non-null default |
| overall_usefulness | Overall usefulness. | tinyint(3) unsigned | NULL allowed; default NULL |
| comments | Comments. | text | NULL allowed; default NULL |
| content_unit_verification_json | Content unit verification json. | longtext | NULL allowed; default NULL |
| created_at | Creation timestamp. | datetime | NOT NULL; default current_timestamp() |
| corrected_at | Corrected at. | datetime | NULL allowed; default NULL |

Indexes and constraints:

- PRIMARY: UNIQUE (id).
- assignment_id: UNIQUE (assignment_id).
- CHECK ck_eval_rating_range: `relevance between 1 and 5 and factual_consistency between 1 and 5 and coverage between 1 and 5 and coherence between 1 and 5 and

readability between 1 and 5 and conciseness between 1 and 5 and non_redundancy between 1 and 5 and (overall_usefulness is null or overall_usefulness between 1 and 5)`.

The table supports human criterion ratings for an assignment. Its constraints describe enforced storage behavior; runtime verification is reported separately.

Table 21 records the observed columns and constraints for evaluation_ratings. The descriptions explain their storage purpose; runtime behavior remains subject to the separately stated verification limits.

### DD-16. evaluation_references

Versioned reference summaries and review status.

**Table 22. Data dictionary for evaluation_references.**

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| id | Unique row identifier. | bigint(20) unsigned | NOT NULL; PRIMARY KEY; auto_increment; default SQL NULL / no explicit non-null default |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| document_id | Related document identifier in this table’s domain. | bigint(20) unsigned | NOT NULL; default SQL NULL / no explicit non-null default; FK → evaluation_documents.id; DELETE RESTRICT; UPDATE RESTRICT |
| reference_text | Reference text. | longtext | NOT NULL; default SQL NULL / no explicit non-null default |
| reference_version | Reference version. | varchar(100) | NOT NULL; default SQL NULL / no explicit non-null default |
| author_code | Author code. | varchar(100) | NULL allowed; default NULL |
| review_status | Reference-review lifecycle status. | varchar(20) | NOT NULL; default 'draft' |
| created_by | Created by. | int(11) | NULL allowed; default NULL; FK → users.id; DELETE SET NULL; UPDATE RESTRICT |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| created_at | Creation timestamp. | datetime | NOT NULL; default current_timestamp() |

Indexes and constraints:

- PRIMARY: UNIQUE (id).
- fk_eval_reference_user: nonunique (created_by).
- uq_eval_reference: UNIQUE (document_id, reference_version).

The table supports versioned reference summaries and review status. Its constraints describe enforced storage behavior; runtime verification is reported separately.

Table 22 records the observed columns and constraints for evaluation_references. The descriptions explain their storage purpose; runtime behavior remains subject to the separately stated verification limits.

**DD-17. evaluation_runs**

Frozen configuration and dataset snapshots for comparison runs.

**Table 23. Data dictionary for evaluation_runs.**

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| id | Unique row identifier. | bigint(20) unsigned | NOT NULL; PRIMARY KEY; auto_increment; default SQL NULL / no explicit non-null default |
| run_uuid | Run uuid. | char(32) | NOT NULL; default SQL NULL / no explicit non-null default |
| name | Name. | varchar(255) | NOT NULL; default SQL NULL / no explicit non-null default |
| dataset_id | Dataset id. | bigint(20) unsigned | NOT NULL; default SQL NULL / no explicit non-null default; FK → evaluation_datasets.id; DELETE RESTRICT; UPDATE RESTRICT |
| dataset_version | Dataset version. | varchar(100) | NOT NULL; default SQL NULL / no explicit non-null default |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| dataset_split | Dataset split. | varchar(20) | NOT NULL; default SQL NULL / no explicit non-null default |
| summarizer_version | Summarizer version. | varchar(100) | NOT NULL; default 'pending-worker' |
| evaluation_version | Evaluation version. | varchar(30) | NOT NULL; default '1.0.0' |
| configuration_hash | Hash of canonical run configuration. | char(64) | NOT NULL; default SQL NULL / no explicit non-null default |
| configuration_json | Configuration json. | longtext | NOT NULL; default SQL NULL / no explicit non-null default |
| dataset_snapshot_json | Frozen dataset content and reference snapshot. | longtext | NOT NULL; default SQL NULL / no explicit non-null default |
| snapshot_hash | Snapshot hash. | char(64) | NOT NULL; default SQL NULL / no explicit non-null default |
| provenance_json | Serialized execution/source provenance. | longtext | NULL allowed; default NULL |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| report_json | Report json. | longtext | NULL allowed; default NULL |
| report_pending | Report pending. | tinyint(1) | NOT NULL; default 1 |
| report_error | Report error. | text | NULL allowed; default NULL |
| status | Domain status marker. | varchar(30) | NOT NULL; default 'queued' |
| documents_total | Documents total. | int(10) unsigned | NOT NULL; default SQL NULL / no explicit non-null default |
| documents_processed | Documents processed. | int(10) unsigned | NOT NULL; default 0 |
| outputs_total | Outputs total. | int(10) unsigned | NOT NULL; default SQL NULL / no explicit non-null default |
| generated_summaries | Generated summaries. | int(10) unsigned | NOT NULL; default 0 |
| failed_summaries | Failed summaries. | int(10) unsigned | NOT NULL; default 0 |
| created_by | Created by. | int(11) | NULL allowed; default NULL; FK → users.id; DELETE SET NULL; UPDATE RESTRICT |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| notes | Notes. | text | NULL allowed; default NULL |
| error_message | Error message. | text | NULL allowed; default NULL |
| started_at | Started at. | datetime | NULL allowed; default NULL |
| completed_at | Completed at. | datetime | NULL allowed; default NULL |
| heartbeat_at | Heartbeat at. | datetime | NULL allowed; default NULL |
| created_at | Creation timestamp. | datetime | NOT NULL; default current_timestamp() |

Indexes and constraints:

- PRIMARY: UNIQUE (id).
- fk_eval_run_dataset: nonunique (dataset_id).
- fk_eval_run_user: nonunique (created_by).
- idx_eval_run_status: nonunique (status, created_at).
- run_uuid: UNIQUE (run_uuid).

The table supports frozen configuration and dataset snapshots for comparison runs. Its constraints describe enforced storage

behavior; runtime verification is reported separately.

Table 23 records the observed columns and constraints for evaluation_runs. The descriptions explain their storage purpose; runtime behavior remains subject to the separately stated verification limits.

### DD-18. feedback

Latest owner-associated rating, reasons and optional legacy identifying fields.

**Table 24. Data dictionary for feedback.**

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| id | Unique row identifier. | int(11) | NOT NULL; PRIMARY KEY; auto_increment; default SQL NULL / no explicit non-null default |
| summary_id | Related summary identifier. | int(11) | NOT NULL; default SQL NULL / no explicit non-null default; FK → summaries.id; DELETE CASCADE; UPDATE RESTRICT |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| user_id | Registered account identifier; an enforced relation only where listed below. | int(11) | NULL allowed; default NULL; FK → users.id; DELETE CASCADE; UPDATE RESTRICT |
| guest_token | Anonymous owner/session token; not a foreign key unless explicitly listed. | varchar(64) | NULL allowed; default NULL |
| rating | Numeric feedback rating. | tinyint(4) | NOT NULL; default SQL NULL / no explicit non-null default |
| name | Name. | varchar(100) | NULL allowed; default NULL |
| email | Email. | varchar(150) | NULL allowed; default NULL |
| impression | Impression. | varchar(50) | NULL allowed; default NULL |
| reasons | Serialized selected feedback reason codes. | longtext | NULL allowed; default NULL |
| comment | Free-text feedback comment. | text | NULL allowed; default NULL |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| created_at | Creation timestamp. | timestamp | NOT NULL; default current_timestamp() |
| updated_at | Most recent update timestamp. | timestamp | NOT NULL; on update current_timestamp(); default current_timestamp() |

Indexes and constraints:

- PRIMARY: UNIQUE (id).
- idx_guest_owner: nonunique (guest_token, user_id).
- idx_summary: nonunique (summary_id).
- uq_guest_summary: UNIQUE (summary_id, guest_token).
- uq_user_summary: UNIQUE (summary_id, user_id).
- user_id: nonunique (user_id).

The table supports latest owner-associated rating, reasons and optional legacy identifying fields. Its constraints describe enforced storage behavior; runtime verification is reported separately.

Table 24 records the observed columns and constraints for feedback. The descriptions explain their storage purpose; runtime behavior remains subject to the separately stated verification limits.

### DD-19. feedback_comments

Feedback comment/reason event ledger without direct user/guest columns.

**Table 25. Data dictionary for feedback_comments.**

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| id | Unique row identifier. | int(11) | NOT NULL; PRIMARY KEY; auto_increment; default SQL NULL / no explicit non-null default |
| summary_id | Related summary identifier. | int(11) | NOT NULL; default SQL NULL / no explicit non-null default; FK → summaries.id; DELETE CASCADE; UPDATE RESTRICT |
| rating | Numeric feedback rating. | tinyint(4) | NULL allowed; default NULL |
| reasons | Serialized selected feedback reason codes. | longtext | NULL allowed; default NULL |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| comment | Free-text feedback comment. | text | NOT NULL; default SQL NULL / no explicit non-null default |
| created_at | Creation timestamp. | timestamp | NOT NULL; default current_timestamp() |

Indexes and constraints:

- PRIMARY: UNIQUE (id).
- idx_fc_created: nonunique (created_at).
- idx_fc_summary: nonunique (summary_id).

The table supports feedback comment/reason event ledger without direct user/guest columns. Its constraints describe enforced storage behavior; runtime verification is reported separately.

Table 25 records the observed columns and constraints for feedback_comments. The descriptions explain their storage purpose; runtime behavior remains subject to the separately stated verification limits.

### DD-20. guest_sessions

Anonymous session identity and inactivity expiry.

**Table 26. Data dictionary for guest_sessions.**

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| guest_token | Anonymous owner/session token; not a foreign key unless explicitly listed. | varchar(64) | NOT NULL; PRIMARY KEY; default SQL NULL / no explicit non-null default |
| created_at | Creation timestamp. | datetime | NOT NULL; default current_timestamp() |
| last_activity | Last activity. | datetime | NOT NULL; default current_timestamp() |
| expires_at | Expires at. | datetime | NOT NULL; default SQL NULL / no explicit non-null default |

Indexes and constraints:

- PRIMARY: UNIQUE (guest_token).
- idx_guest_sessions_activity: nonunique (last_activity).
- idx_guest_sessions_expires: nonunique (expires_at).

The table supports anonymous session identity and inactivity expiry. Its constraints describe enforced storage behavior; runtime verification is reported separately.

Table 26 records the observed columns and constraints for guest_sessions. The descriptions explain their storage purpose; runtime behavior remains subject to the separately stated verification limits.

### DD-21. landing_page_events

Individual landing-page events associated with a session.

**Table 27. Data dictionary for landing_page_events.**

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| id | Unique row identifier. | bigint(20) unsigned | NOT NULL; PRIMARY KEY; auto_increment; default SQL NULL / no explicit non-null default |
| session_id | Session id. | varchar(64) | NOT NULL; default SQL NULL / no explicit non-null default; FK → landing_page_sessions.session_id; DELETE CASCADE; UPDATE RESTRICT |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| event_type | Event type. | varchar(50) | NOT NULL; default SQL NULL / no explicit non-null default |
| event_label | Event label. | varchar(100) | NULL allowed; default NULL |
| event_value | Event value. | int(11) | NULL allowed; default NULL |
| created_at | Creation timestamp. | timestamp | NOT NULL; default current_timestamp() |

Indexes and constraints:

- PRIMARY: UNIQUE (id).
- idx_lpe_created: nonunique (created_at).
- idx_lpe_session: nonunique (session_id, event_type).

The table supports individual landing-page events associated with a session. Its constraints describe enforced storage behavior; runtime verification is reported separately.

Table 27 records the observed columns and constraints for landing_page_events. The descriptions explain their storage purpose; runtime behavior remains subject to the separately stated verification limits.

## DD-22. landing_page_sessions

Aggregated visitor engagement, device and web-vital measurements.

**Table 28. Data dictionary for landing_page_sessions.**

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| session_id | Session id. | varchar(64) | NOT NULL; PRIMARY KEY; default SQL NULL / no explicit non-null default |
| user_id | Registered account identifier; an enforced relation only where listed below. | int(11) | NULL allowed; default NULL |
| guest_token | Anonymous owner/session token; not a foreign key unless explicitly listed. | varchar(64) | NULL allowed; default NULL |
| ip_hash | Ip hash. | char(64) | NOT NULL; default SQL NULL / no explicit non-null default |
| user_agent | User agent. | varchar(255) | NULL allowed; default NULL |
| device_category | Device category. | enum('mobile','tablet','desktop') | NOT NULL; default 'desktop' |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| referrer | Referrer. | varchar(512) | NULL allowed; default NULL |
| lcp_ms | Lcp ms. | int(11) | NULL allowed; default NULL |
| fid_ms | Fid ms. | int(11) | NULL allowed; default NULL |
| inp_ms | Inp ms. | int(11) | NULL allowed; default NULL |
| cls_score | Cls score. | decimal(6,4) | NULL allowed; default NULL |
| max_scroll_percent | Max scroll percent. | tinyint(3) unsigned | NOT NULL; default 0 |
| dwell_time_seconds | Dwell time seconds. | smallint(5) unsigned | NOT NULL; default 0 |
| converted | Converted. | tinyint(1) | NOT NULL; default 0 |
| created_at | Creation timestamp. | timestamp | NOT NULL; default current_timestamp() |
| updated_at | Most recent update timestamp. | timestamp | NOT NULL; on update current_timestamp(); default current_timestamp() |

Indexes and constraints:

- PRIMARY: UNIQUE (session_id).

- idx_lps_converted: nonunique (converted).
- idx_lps_created: nonunique (created_at).
- idx_lps_device: nonunique (device_category).

The table supports aggregated visitor engagement, device and web-vital measurements. Its constraints describe enforced storage behavior; runtime verification is reported separately.

Table 28 records the observed columns and constraints for landing_page_sessions. The descriptions explain their storage purpose; runtime behavior remains subject to the separately stated verification limits.

### DD-23. nutshell_generations

Completed/failed short-summary generation ledger and generation metrics.

**Table 29. Data dictionary for nutshell_generations.**

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| id | Unique row identifier. | bigint(20) unsigned | NOT NULL; PRIMARY KEY; auto_increment; default SQL NULL / no explicit non-null default |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| summary_id | Related summary identifier. | int(11) | NULL allowed; default NULL; FK → summaries.id; DELETE SET NULL; UPDATE RESTRICT |
| user_id | Registered account identifier; an enforced relation only where listed below. | int(11) | NULL allowed; default NULL |
| guest_token | Anonymous owner/session token; not a foreign key unless explicitly listed. | varchar(64) | NULL allowed; default NULL |
| input_type | Input type. | varchar(20) | NOT NULL; default 'text' |
| nutshell_text | Nutshell text. | text | NULL allowed; default NULL |
| status | Domain status marker. | enum('completed','failed') | NOT NULL; default 'completed' |
| failure_reason | Failure reason. | varchar(80) | NULL allowed; default NULL |
| analysis_mode | Analysis mode. | varchar(24) | NOT NULL; default 'general' |
| output_format | Output format. | varchar(24) | NOT NULL; default 'paragraph' |
| word_count | Word count. | int(10) unsigned | NOT NULL; default 0 |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| source_word_count | Source word count. | int(10) unsigned | NOT NULL; default 0 |
| primary_summary_word_count | Primary summary word count. | int(10) unsigned | NOT NULL; default 0 |
| compression_ratio | Module-specific compression metric; check service semantics before comparisons. | decimal(8, 4) | NOT NULL; default 0.0000 |
| processing_time_ms | Processing time ms. | int(10) unsigned | NOT NULL; default 0 |
| algorithm_version | Algorithm version. | varchar(32) | NOT NULL; default 'nutshell-extractive-v2' |
| idempotency_key | Idempotency key. | char(64) | NULL allowed; default NULL |
| created_at | Creation timestamp. | timestamp | NOT NULL; default current_timestamp() |

Indexes and constraints:

- PRIMARY: UNIQUE (id).
- idx_ng_created: nonunique (created_at).
- idx_ng_status_created: nonunique (status, created_at).
- idx_ng_summary: nonunique (summary_id).
- idx_ng_user_created: nonunique (user_id, created_at).
- uq_ng_idempotency_key: UNIQUE (idempotency_key).

The table supports completed/failed short-summary generation ledger and generation metrics. Its constraints describe enforced storage behavior; runtime verification is reported separately.

Table 29 records the observed columns and constraints for nutshell_generations. The descriptions explain their storage purpose; runtime behavior remains subject to the separately stated verification limits.

### DD-24. password_resets

Hashed recovery codes and expiry/consumption state.

**Table 30. Data dictionary for password_resets.**

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| id | Unique row identifier. | int(11) | NOT NULL; PRIMARY KEY; auto_increment; default SQL NULL / no explicit non-null default |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| user_id | Registered account identifier; an enforced relation only where listed below. | int(11) | NOT NULL; default SQL NULL / no explicit non-null default; FK → users.id; DELETE CASCADE; UPDATE RESTRICT |
| token_hash | One-way recovery-code verifier. | varchar(64) | NOT NULL; default SQL NULL / no explicit non-null default |
| expires_at | Expires at. | datetime | NOT NULL; default SQL NULL / no explicit non-null default |
| used_at | Used at. | datetime | NULL allowed; default NULL |

Indexes and constraints:

- PRIMARY: UNIQUE (id).
- idx_token: nonunique (token_hash).
- user_id: nonunique (user_id).

The table supports hashed recovery codes and expiry/consumption state. Its constraints describe enforced storage behavior; runtime verification is reported separately.

Table 30 records the observed columns and constraints for password_resets. The descriptions explain their storage purpose; runtime behavior remains subject to the separately stated verification limits.

### DD-25. rate_limits

Per-action identity counters and time windows.

**Table 31. Data dictionary for rate_limits.**

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| id | Unique row identifier. | int(11) | NOT NULL; PRIMARY KEY; auto_increment; default SQL NULL / no explicit non-null default |
| action_type | Action type. | varchar(30) | NOT NULL; default SQL NULL / no explicit non-null default |
| identifier | Identifier. | varchar(255) | NOT NULL; default SQL NULL / no explicit non-null default |
| attempts | Attempts. | int(11) | NULL allowed; default 0 |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| window_start | Window start. | timestamp | NOT NULL; default current_timestamp() |

Indexes and constraints:

- PRIMARY: UNIQUE (id).
- idx_window: nonunique (window_start).
- uq_action_id: UNIQUE (action_type, identifier).

The table supports per-action identity counters and time windows. Its constraints describe enforced storage behavior; runtime verification is reported separately.

Table 31 records the observed columns and constraints for rate_limits. The descriptions explain their storage purpose; runtime behavior remains subject to the separately stated verification limits.

### DD-26. schema_migrations

Migration application bookkeeping.

**Table 32. Data dictionary for schema_migrations.**

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| id | Unique row identifier. | int(11) | NOT NULL; PRIMARY KEY; auto_increment; default SQL NULL / no explicit non-null default |
| migration | Migration. | varchar(255) | NOT NULL; default SQL NULL / no explicit non-null default |
| applied_at | Applied at. | timestamp | NOT NULL; default current_timestamp() |

Indexes and constraints:

- PRIMARY: UNIQUE (id).
- migration: UNIQUE (migration).

The table supports migration application bookkeeping. Its constraints describe enforced storage behavior; runtime verification is reported separately.

Table 32 records the observed columns and constraints for schema_migrations. The descriptions explain their storage purpose; runtime behavior remains subject to the separately stated verification limits.

### DD-27. summaries

Summary ownership, sharing capability, metadata, aggregate metrics and cached Nutshell.

**Table 33. Data dictionary for summaries.**

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| id | Unique row identifier. | int(11) | NOT NULL; PRIMARY KEY; auto_increment; default SQL NULL / no explicit non-null default |
| user_id | Registered account identifier; an enforced relation only where listed below. | int(11) | NULL allowed; default NULL; FK → users.id; DELETE CASCADE; UPDATE RESTRICT |
| guest_token | Anonymous owner/session token; not a foreign key unless explicitly listed. | varchar(64) | NULL allowed; default NULL |
| share_token | Random capability token used by shared-summary links. | varchar(64) | NOT NULL; default SQL NULL / no explicit non-null default |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| article_title | Article title. | varchar(255) | NOT NULL; default SQL NULL / no explicit non-null default |
| author_name | Author name. | varchar(255) | NULL allowed; default NULL |
| input_type | Input type. | enum('text','pdf','docx','url') | NOT NULL; default SQL NULL / no explicit non-null default |
| summary_style | Summary style. | enum('standard_paragraph','bullet_points','hybrid','academic_summary','simple_summary','executive_summary') | NULL allowed; default 'standard_paragraph' |
| summary_length | Summary length. | varchar(32) | NULL allowed; default NULL |
| original_word_count | Source word count; main file path currently counts the empty input string. | int(10) unsigned | NULL allowed; default NULL |
| summary_word_count | Generated summary word count. | int(10) unsigned | NULL allowed; default NULL |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| processing_time | Elapsed summary processing seconds; current main caller does not supply this value. | decimal(10,3) | NULL allowed; default NULL |
| article_category | Article category. | varchar(100) | NULL allowed; default NULL |
| status | Domain status marker. | enum('pending','processing','completed','failed') | NOT NULL; default 'completed' |
| error_message | Error message. | text | NULL allowed; default NULL |
| created_at | Creation timestamp. | timestamp | NOT NULL; default current_timestamp() |
| nutshell_text | Nutshell text. | text | NULL allowed; default NULL |
| nutshell_word_count | Nutshell word count. | int(10) unsigned | NULL allowed; default NULL |
| nutshell_generated_at | Nutshell generated at. | timestamp | NULL allowed; default NULL |

Indexes and constraints:

- PRIMARY: UNIQUE (id).
- idx_created_owner: nonunique (created_at, user_id).
- idx_guest: nonunique (guest_token).

- idx_guest_owner: nonunique (guest_token, user_id).
- idx_status: nonunique (status).
- idx_summaries_analytics_user_created: nonunique (user_id, created_at).
- idx_summaries_created_at: nonunique (created_at).
- idx_summaries_guest_token: nonunique (guest_token).
- idx_summaries_nutshell_generated: nonunique (nutshell_generated_at).
- idx_summaries_status_created: nonunique (status, created_at).
- idx_summaries_user_id: nonunique (user_id).
- idx_user: nonunique (user_id).
- uq_share_token: UNIQUE (share_token).

The table supports summary ownership, sharing capability, metadata, aggregate metrics and cached nutshell. Its constraints describe enforced storage behavior; runtime verification is reported separately.

Table 33 records the observed columns and constraints for summaries. The descriptions explain their storage purpose; runtime behavior remains subject to the separately stated verification limits.

**DD-28. summary_artifacts**

Source text and structured generated output stored apart from summary metadata.

**Table 34. Data dictionary for summary_artifacts.**

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| id | Unique row identifier. | int(11) | NOT NULL; PRIMARY KEY; auto_increment; default SQL NULL / no explicit non-null default |
| summary_id | Related summary identifier. | int(11) | NOT NULL; default SQL NULL / no explicit non-null default; FK → summaries.id; DELETE CASCADE; UPDATE RESTRICT |
| original_text | Persisted original/source text (main summary service applies pattern masking). | longtext | NOT NULL; default SQL NULL / no explicit non-null default |
| generated_summary | Structured summary JSON/text payload. | longtext | NOT NULL; default SQL NULL / no explicit non-null default |

Indexes and constraints:

- PRIMARY: UNIQUE (id).
- fk_summary_artifacts_summary_id: nonunique (summary_id).

The table supports source text and structured generated output stored apart from summary metadata. Its constraints describe enforced storage behavior; runtime verification is reported separately.

Table 34 records the observed columns and constraints for summary_artifacts. The descriptions explain their storage purpose; runtime behavior remains subject to the separately stated verification limits.

### DD-29. summary_jobs

Durable background work state, owner, payload, retry and result references.

**Table 35. Data dictionary for summary_jobs.**

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| id | Unique row identifier. | bigint(20) unsigned | NOT NULL; PRIMARY KEY; auto_increment; default SQL NULL / no explicit non-null default |
| job_id | Job id. | varchar(64) | NOT NULL; default SQL NULL / no explicit non-null default |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| user_id | Registered account identifier; an enforced relation only where listed below. | int(11) | NULL allowed; default NULL; FK → users.id; DELETE SET NULL; UPDATE RESTRICT |
| guest_token | Anonymous owner/session token; not a foreign key unless explicitly listed. | varchar(64) | NULL allowed; default NULL |
| state | Current durable work state. | enum('queued','processing','completed','failed','cancelled','timed_out') | NOT NULL; default 'queued' |
| payload | Job input JSON retained according to processing/retry lifecycle. | longtext | NOT NULL; default SQL NULL / no explicit non-null default |
| idempotency_key | Idempotency key. | varchar(128) | NOT NULL; default SQL NULL / no explicit non-null default |
| result_id | Result id. | int(11) | NULL allowed; default NULL; FK → summaries.id; DELETE SET NULL; UPDATE RESTRICT |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| error_message | Error message. | text | NULL allowed; default NULL |
| error_details | Error details. | text | NULL allowed; default NULL |
| worker_pid | Worker pid. | int(11) | NULL allowed; default NULL |
| retry_count | Retry count. | tinyint(3) unsigned | NOT NULL; default 0 |
| started_at | Started at. | datetime | NULL allowed; default NULL |
| completed_at | Completed at. | datetime | NULL allowed; default NULL |
| created_at | Creation timestamp. | timestamp | NOT NULL; default current_timestamp() |
| updated_at | Most recent update timestamp. | timestamp | NOT NULL; on update current_timestamp(); default current_timestamp() |

Indexes and constraints:

- PRIMARY: UNIQUE (id).
- idx_summary_jobs_created: nonunique (created_at).
- idx_summary_jobs_guest: nonunique (guest_token).

- idx_summary_jobs_result: nonunique (result_id).
- idx_summary_jobs_state: nonunique (state).
- idx_summary_jobs_user: nonunique (user_id).
- uq_summary_jobs_idempotency: UNIQUE (idempotency_key).
- uq_summary_jobs_job_id: UNIQUE (job_id).

The table supports durable background work state, owner, payload, retry and result references. Its constraints describe enforced storage behavior; runtime verification is reported separately.

Table 35 records the observed columns and constraints for summary_jobs. The descriptions explain their storage purpose; runtime behavior remains subject to the separately stated verification limits.

**DD-30. users**

Registered accounts, role, active status, password hash and acceptance state.

**Table 36. Data dictionary for users.**

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| id | Unique row identifier. | int(11) | NOT NULL; PRIMARY KEY; auto_increment; default SQL NULL / no explicit non-null default |
| username | Username. | varchar(50) | NOT NULL; default SQL NULL / no explicit non-null default |
| email | Email. | varchar(100) | NOT NULL; default SQL NULL / no explicit non-null default |
| password_hash | One-way password verifier, not a readable password. | varchar(255) | NOT NULL; default SQL NULL / no explicit non-null default |
| role | Account access category. | enum('user','admin') | NULL allowed; default 'user' |
| active | Enabled/active state. | tinyint(1) | NULL allowed; default 1 |
| terms_accepted | Whether the account accepted terms. | tinyint(1) | NOT NULL; default 0 |

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| terms_accepted_at | Recorded acceptance time. | datetime | NULL allowed; default NULL |
| created_at | Creation timestamp. | timestamp | NOT NULL; default current_timestamp() |

Indexes and constraints:

- PRIMARY: UNIQUE (id).
- email: UNIQUE (email).
- idx_role: nonunique (role).
- username: UNIQUE (username).

The table supports registered accounts, role, active status, password hash and acceptance state. Its constraints describe enforced storage behavior; runtime verification is reported separately.

Table 36 records the observed columns and constraints for users. The descriptions explain their storage purpose; runtime behavior remains subject to the separately stated verification limits.

### DD-31. user_tokens

Persistent sign-in selectors and hashed validators.

**Table 37. Data dictionary for user_tokens.**

| Field Name | Description | Data Type/Length | Constraints |
|---|---|---|---|
| id | Unique row identifier. | int(11) | NOT NULL; PRIMARY KEY; auto_increment; default SQL NULL / no explicit non-null default |
| user_id | Registered account identifier; an enforced relation only where listed below. | int(11) | NOT NULL; default SQL NULL / no explicit non-null default; FK → users.id; DELETE CASCADE; UPDATE RESTRICT |
| selector | Public lookup part of a persistent sign-in token. | varchar(255) | NOT NULL; default SQL NULL / no explicit non-null default |
| hashed_validator | Hash of persistent sign-in secret. | varchar(255) | NOT NULL; default SQL NULL / no explicit non-null default |
| expires | Expires. | datetime | NOT NULL; default SQL NULL / no explicit non-null default |

Indexes and constraints:

- PRIMARY: UNIQUE (id).
- selector_idx: UNIQUE (selector).
- user_id: nonunique (user_id).

The table supports persistent sign-in selectors and hashed validators. Its constraints describe enforced storage behavior; runtime verification is reported separately.

Table 37 records the observed columns and constraints for user_tokens. The descriptions explain their storage purpose; runtime behavior remains subject to the separately stated verification limits.

### 4.3 Development Stage

The application uses PHP-rendered pages with JavaScript and CSS, a MariaDB database, and Python language-processing services. It is organized around controllers, services and repositories rather than a verified external PHP application framework. The inspected configuration defaults to the legacy summary engine; the independent v2 engine and optional LLM synthesis must not be described as the enabled main algorithm. Translation may use external providers, and email verification requires a working delivery configuration.

**Table 38. Technologies and observed development environment.**

| Component | Observed implementation | Verification boundary |
|---|---|---|
| Web application | PHP templates, controllers, services, repositories; JavaScript and CSS | Original source inspected; PHP/JS syntax checked |
| PHP runtime | XAMPP PHP 8.1.25 for syntax/metadata; PHP 8.3.29 for isolated templates | Other configured installations may differ |
| Database | MariaDB 10.4.32; 31 tables, 312 columns, 35 foreign keys | Metadata read; application rows not read |
| Processing | Python legacy pipeline and optional v2 path | Existing tests ran using available Python 3.14 packages in isolation |
| Document/NLP facilities | pypdf, python-docx, text statistics, optional models and OCR | Resource availability varies; no model downloads performed |
| Audio/translation | Piper voice path; deep-translator/provider paths | Actual speech and live translation not validated |
| Local environment | Windows workspace under XAMPP htdocs | Public hosting, TLS and deployment readiness not verified |

Table 38 records observed technology separately from installation assumptions. In

particular, the test interpreter is not proof that every optional dependency supports the same environment. Older setup claims of Python 3.8 conflict with current language/API usage; a tested installation matrix is required.

The screenshots below are original captures of this application’s templates. They were rendered with empty or synthetic records in a read-only harness that bypassed production startup. Each carries an audit label. They illustrate interface organization, controls and error states; they are not screenshots of genuine users, real study results, completed logins or saved production summaries. Appendix F supplies step-by-step use instructions and additional states.

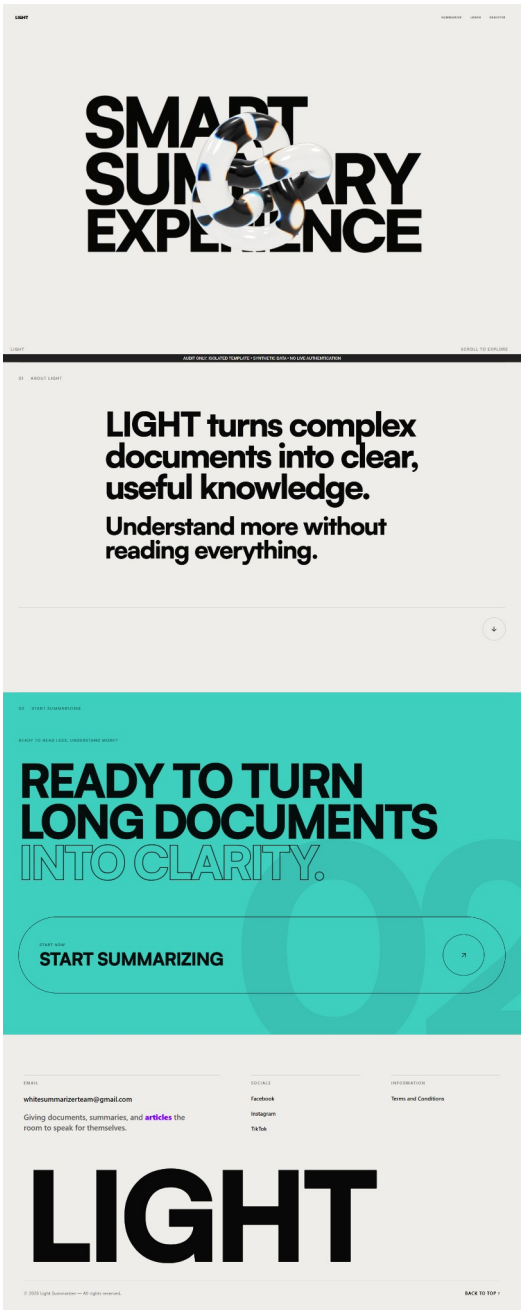



**Figure 35. Landing information and navigation — isolated template capture.**

Figure 35 illustrates the entry page and links to the summarizer and account pages. The

text establishes the interface's intended purpose, not a measured benefit.

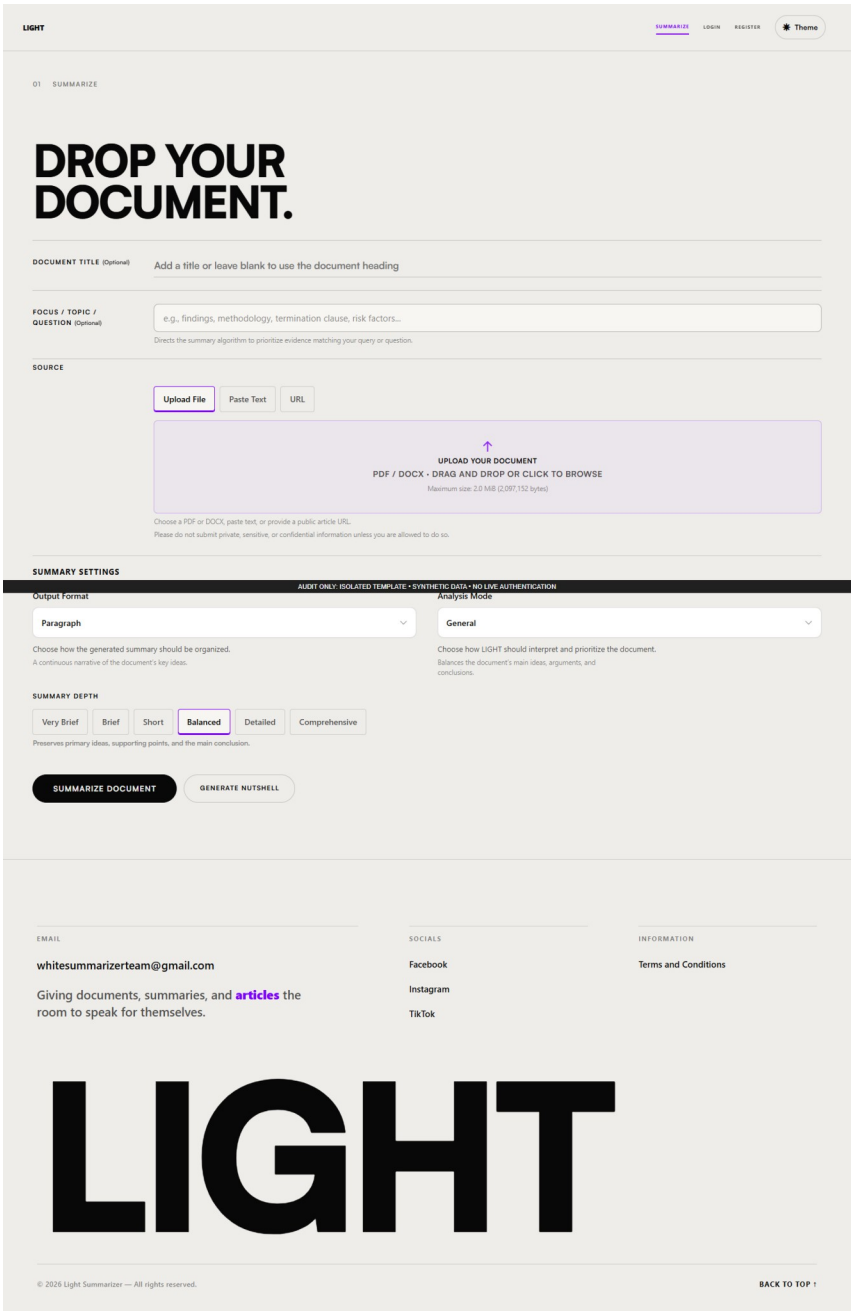



**Figure 36. Source intake and summary settings — isolated template capture.**

Figure 36 illustrates the source choices, title/focus inputs, eight analysis modes, six depths and five output organizations. Local selection tests do not verify all generation combinations.

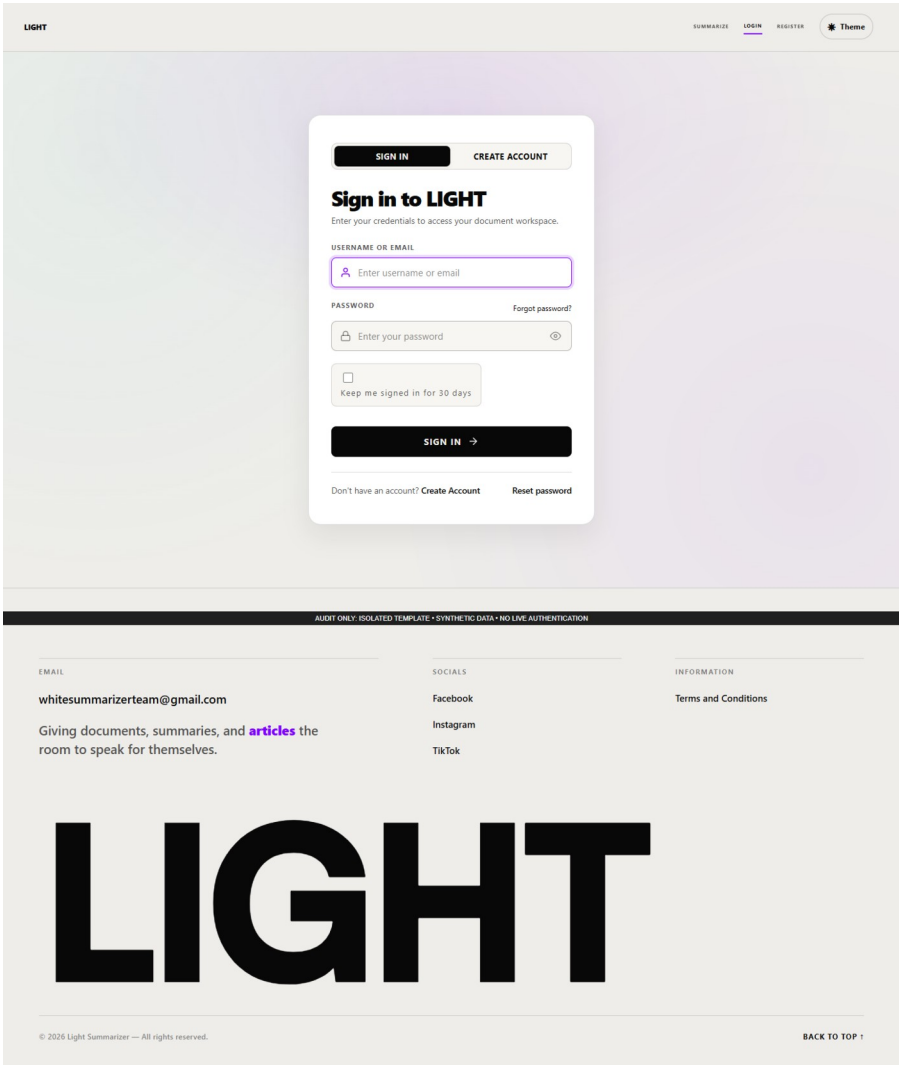



**Figure 37. Sign-in form — isolated template capture.**

Figure 37 illustrates credential and remembered-session controls. The synthetic render does not authenticate an account.

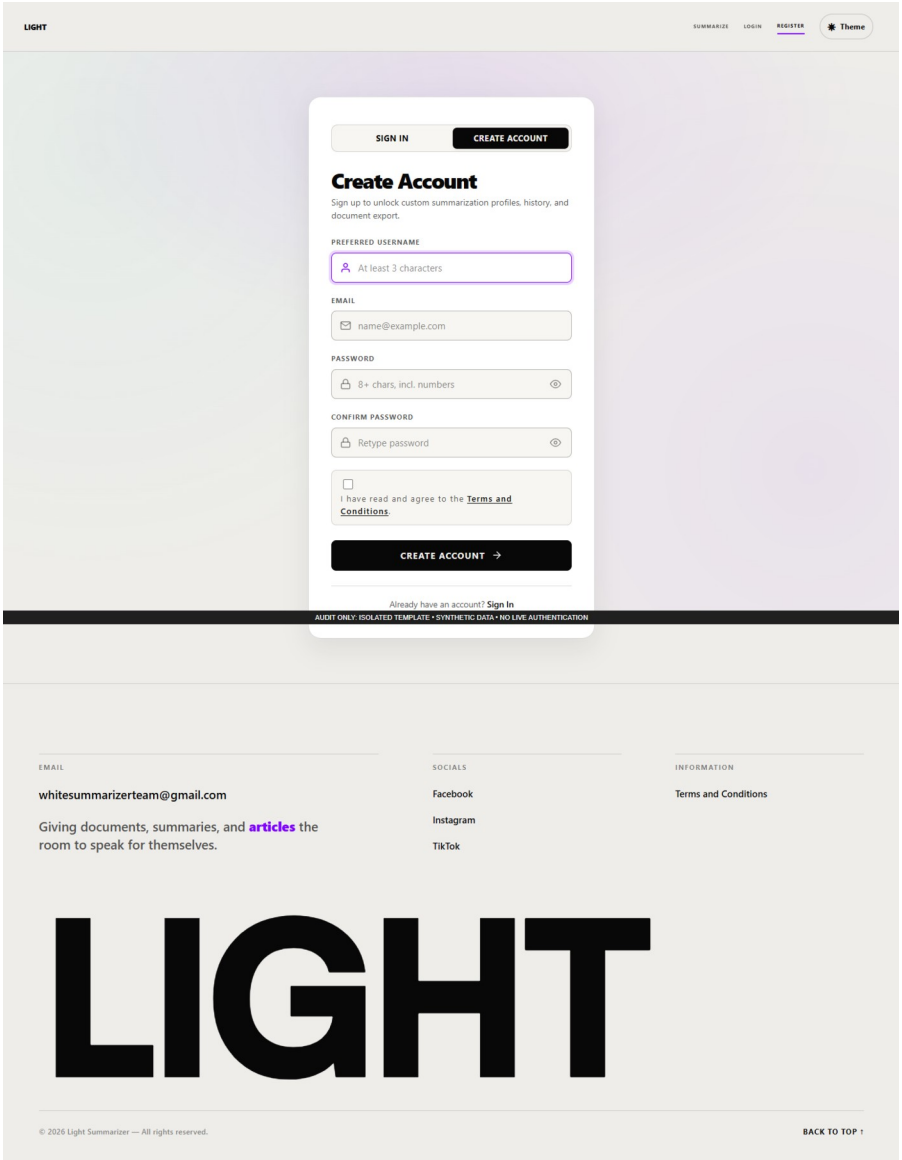



**Figure 38. Registration form — isolated template capture.**

Figure 38 illustrates the information requested for account creation. No account was created during this audit.

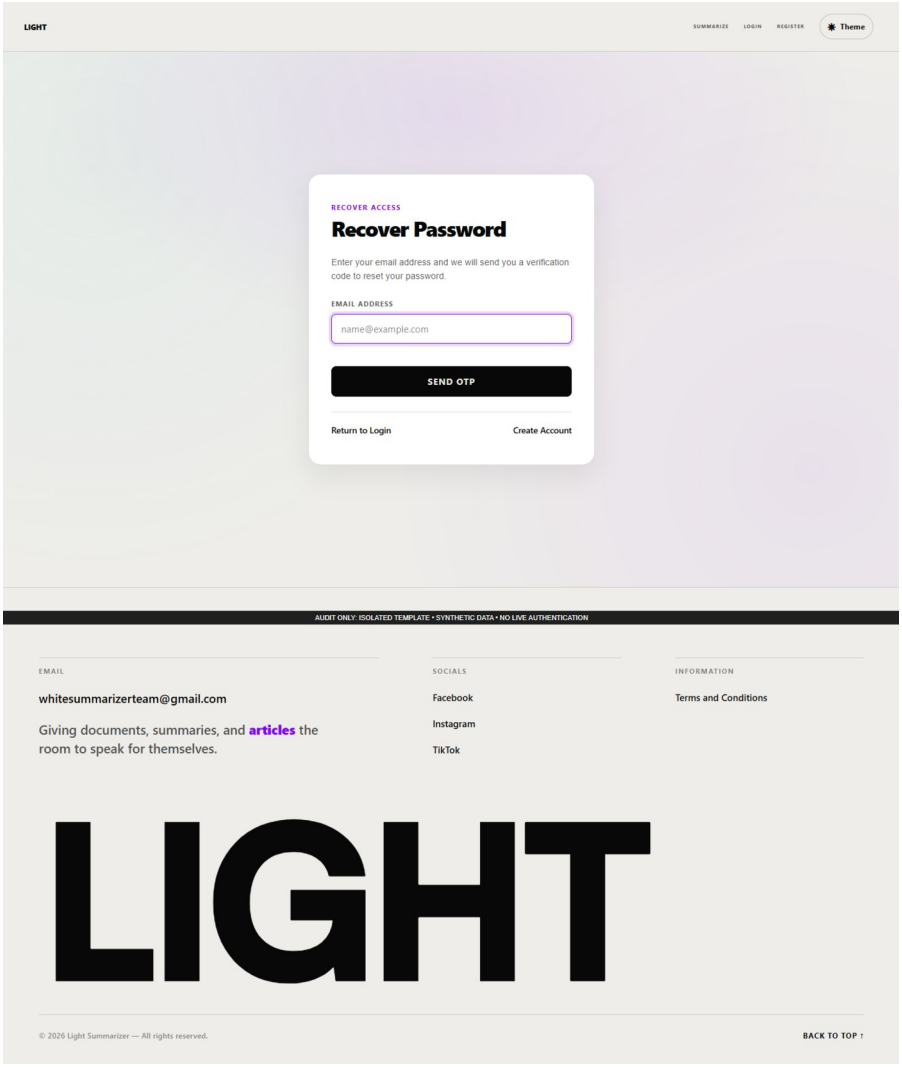



**Figure 39. Recovery request form — isolated template capture.**

Figure 39 illustrates the entry point for requesting a password-reset code. Email

delivery and token consumption were not exercised.

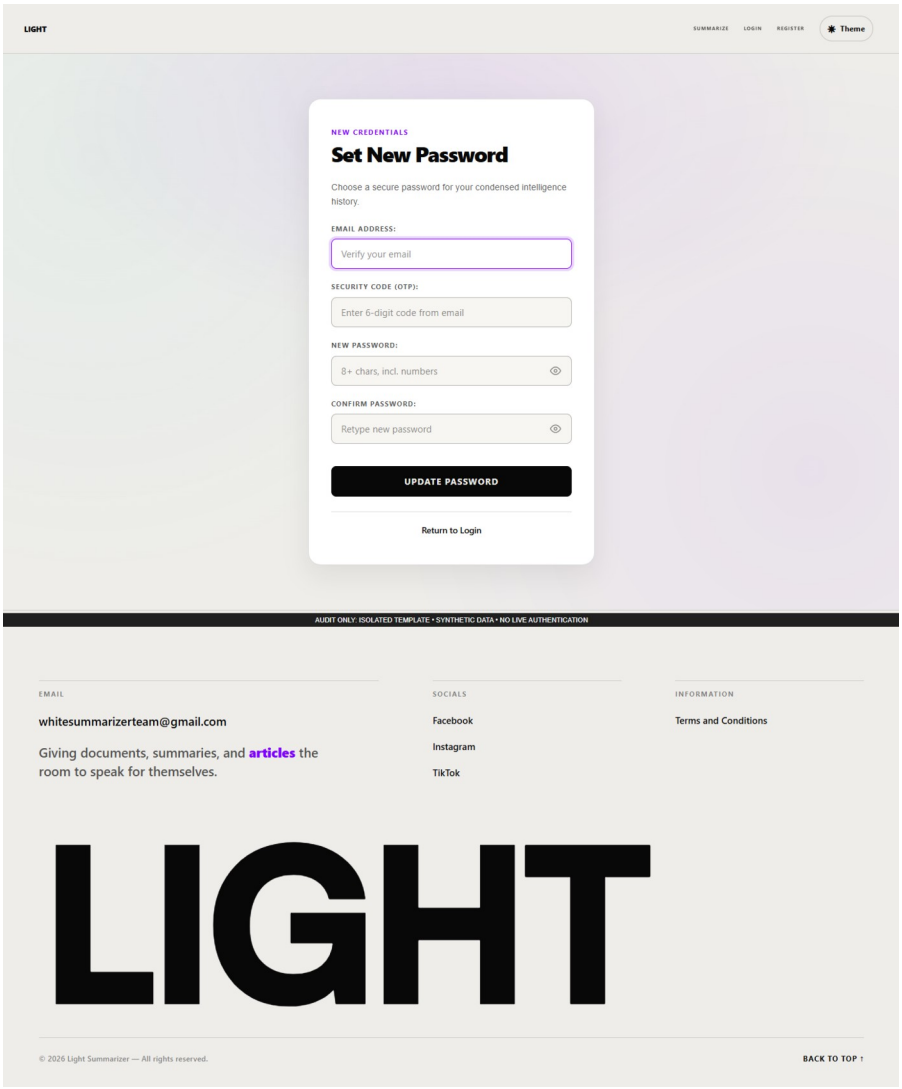



**Figure 40. Password reset form — isolated template capture.**

Figure 40 illustrates code and new-password inputs. The known schema and token-revocation limitations apply to this workflow.

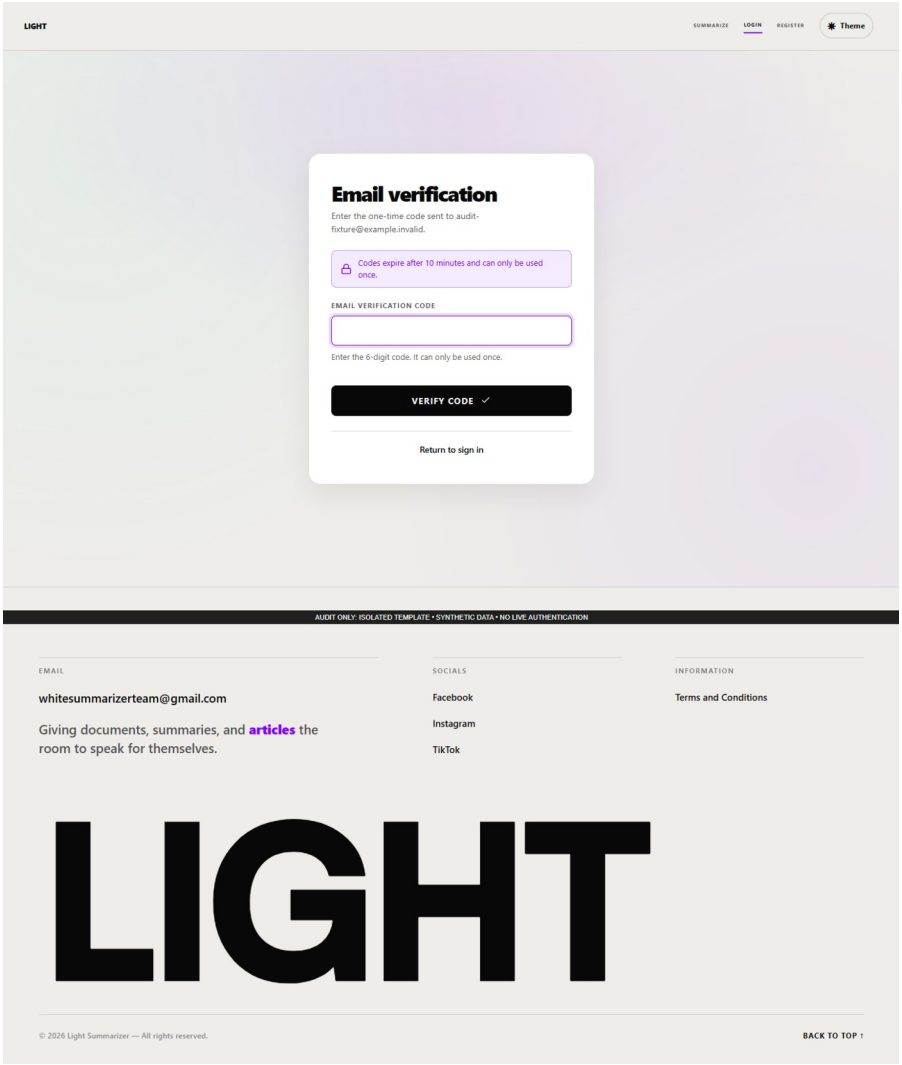



**Figure 41. Administrator code verification state — isolated template capture.**

Figure 41 illustrates the additional emailed-code step for administrator access. The displayed state was injected in isolation and proves no email delivery.

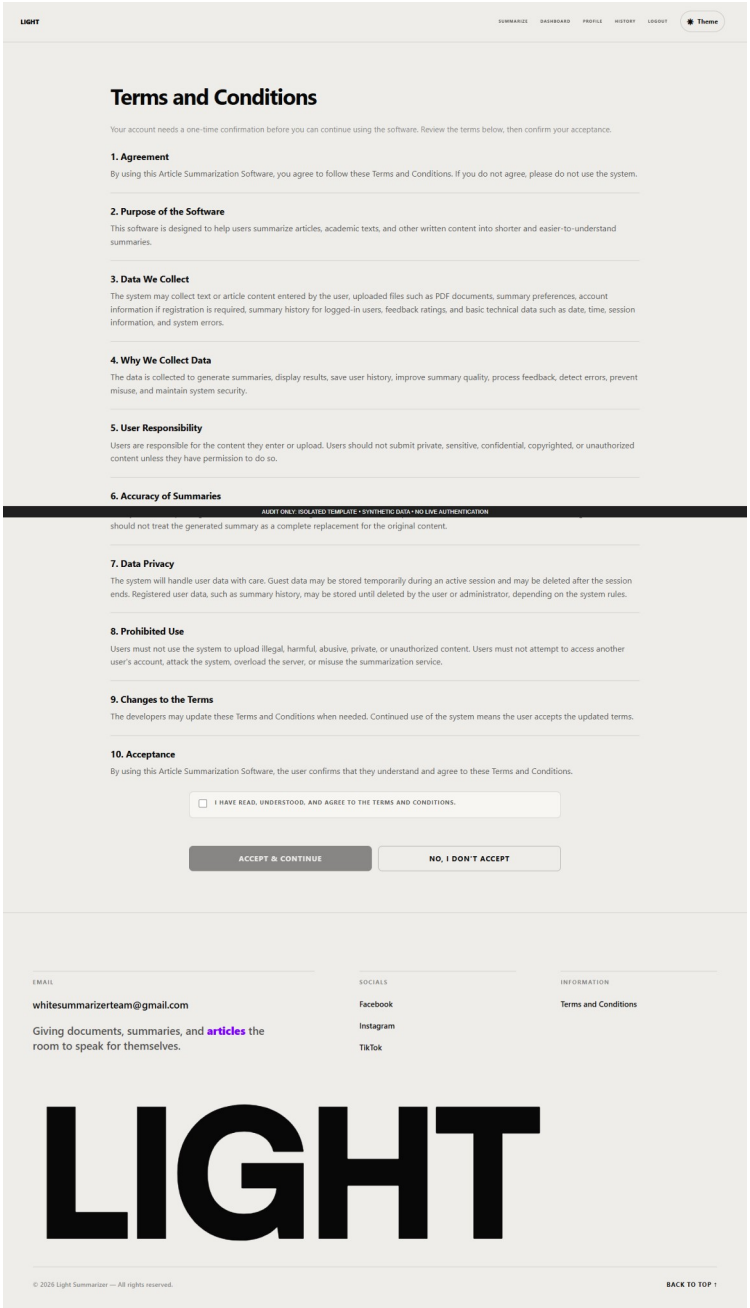



**Figure 42. Terms acceptance state — isolated template capture.**

Figure 42 illustrates the checkbox and acceptance control for an account with pending

terms. Local enabling behavior passed; persistence was not submitted.

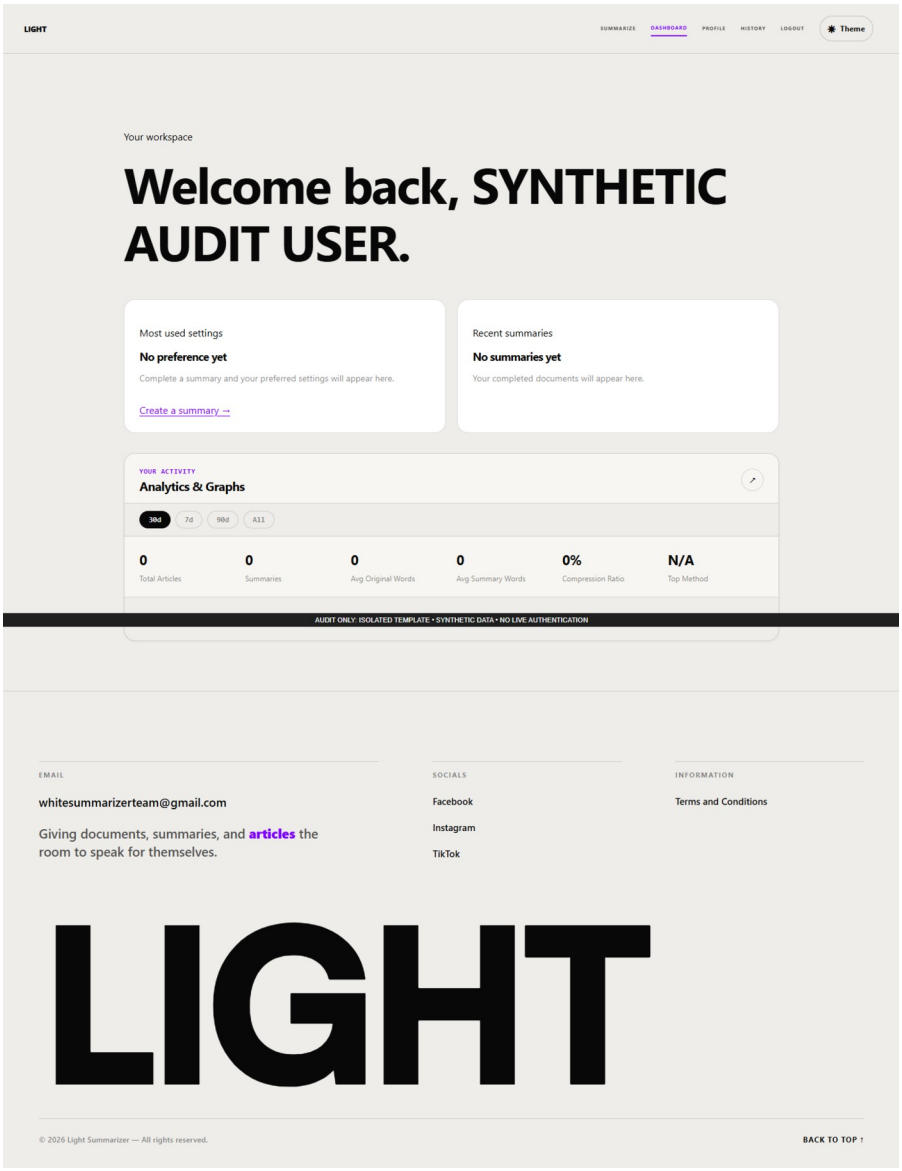



**Figure 43. Personal dashboard empty state — isolated template capture.**

Figure 43 illustrates the organization of recent work and account-related activity when

the fixture contains no saved records. Its counters are not actual usage statistics.

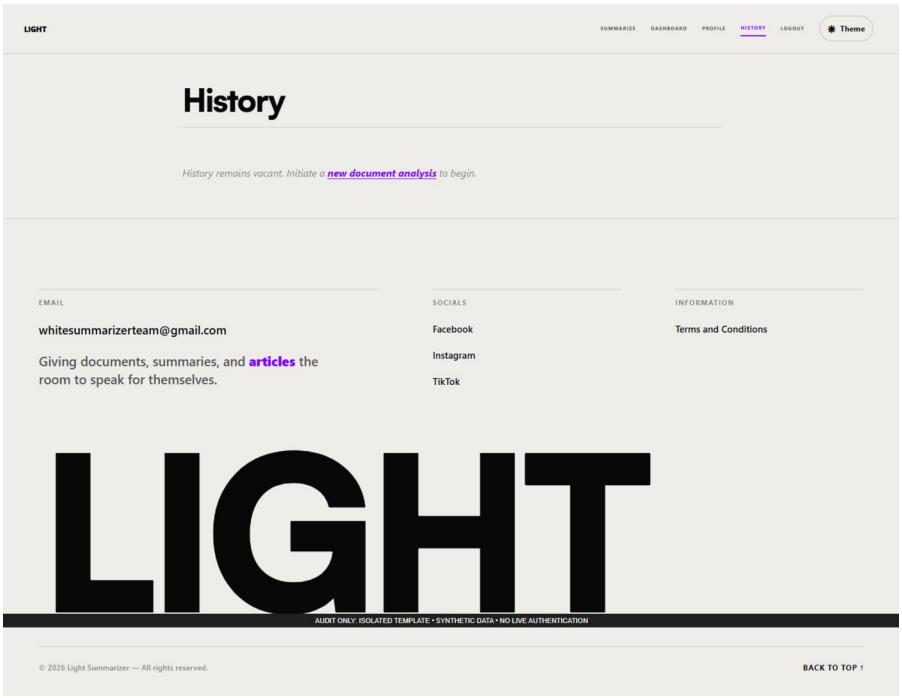



**Figure 44. History empty state — isolated template capture.**

Figure 44 illustrates the saved-work area and navigation. The backend history limit and absent paging controls remain known limitations.

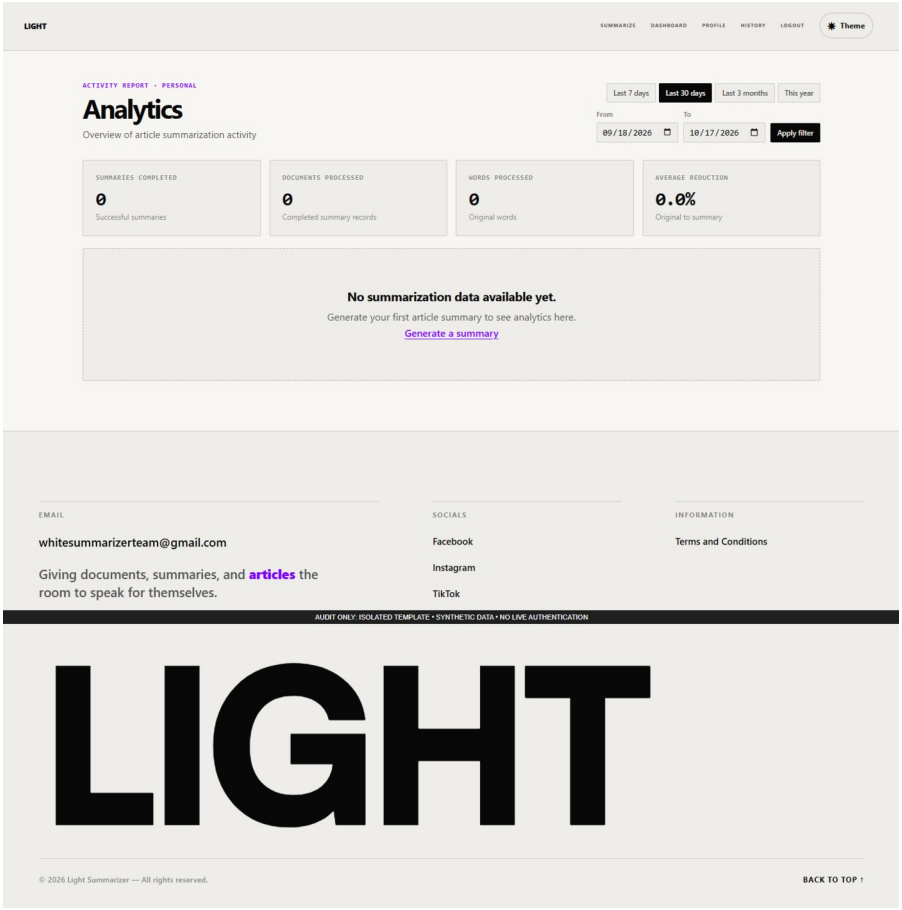



**Figure 45. Analytics empty state — isolated template capture.**

Figure 45 illustrates the presentation of activity filters and summaries. Synthetic empty values cannot be used as research data.

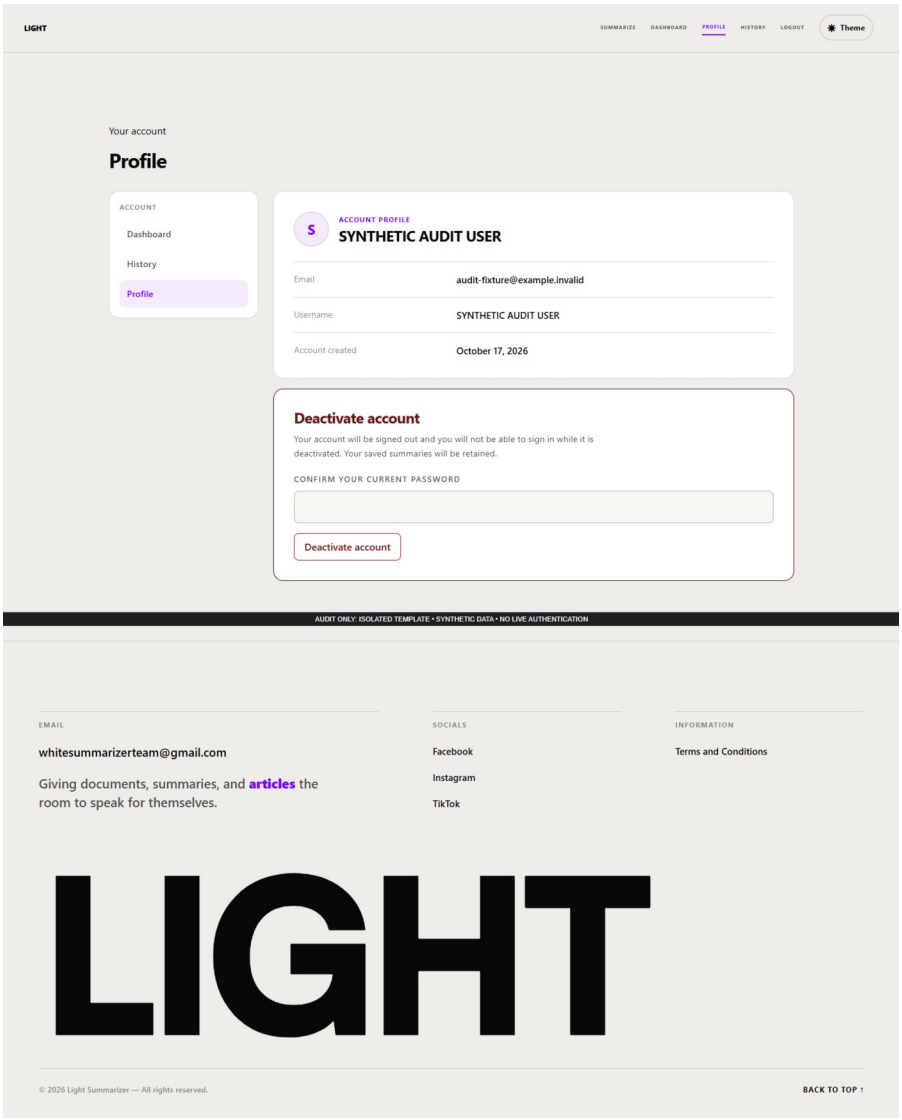



**Figure 46. Account profile display — isolated template capture.**

Figure 46 illustrates the account-information page and available account actions. The sample identity is synthetic, and profile editing is not an implemented page feature.

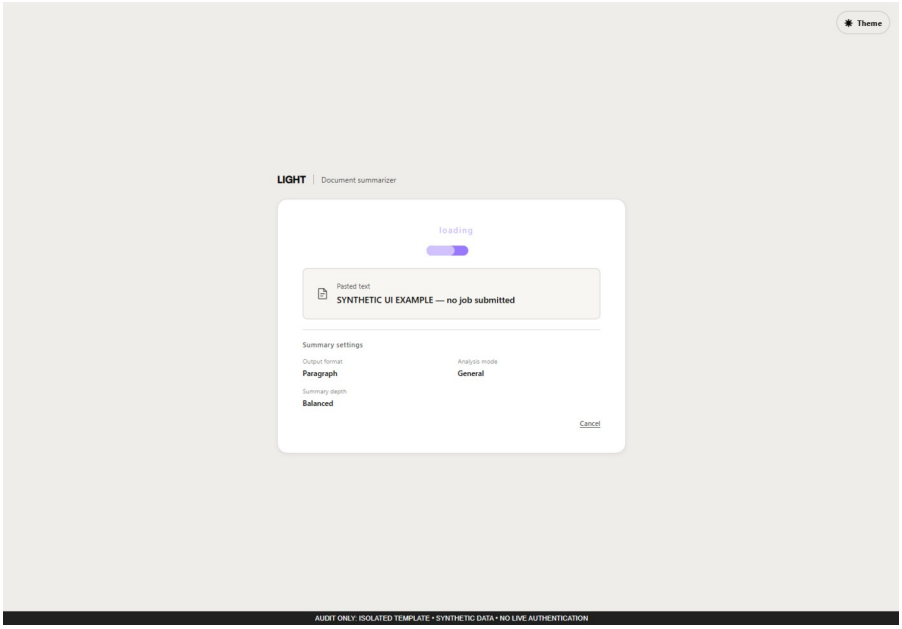



**Figure 47. Background processing state — isolated template capture.**

Figure 47 illustrates progress and recovery controls around a summary job. This fixture did not run a production job or establish successful persistence.

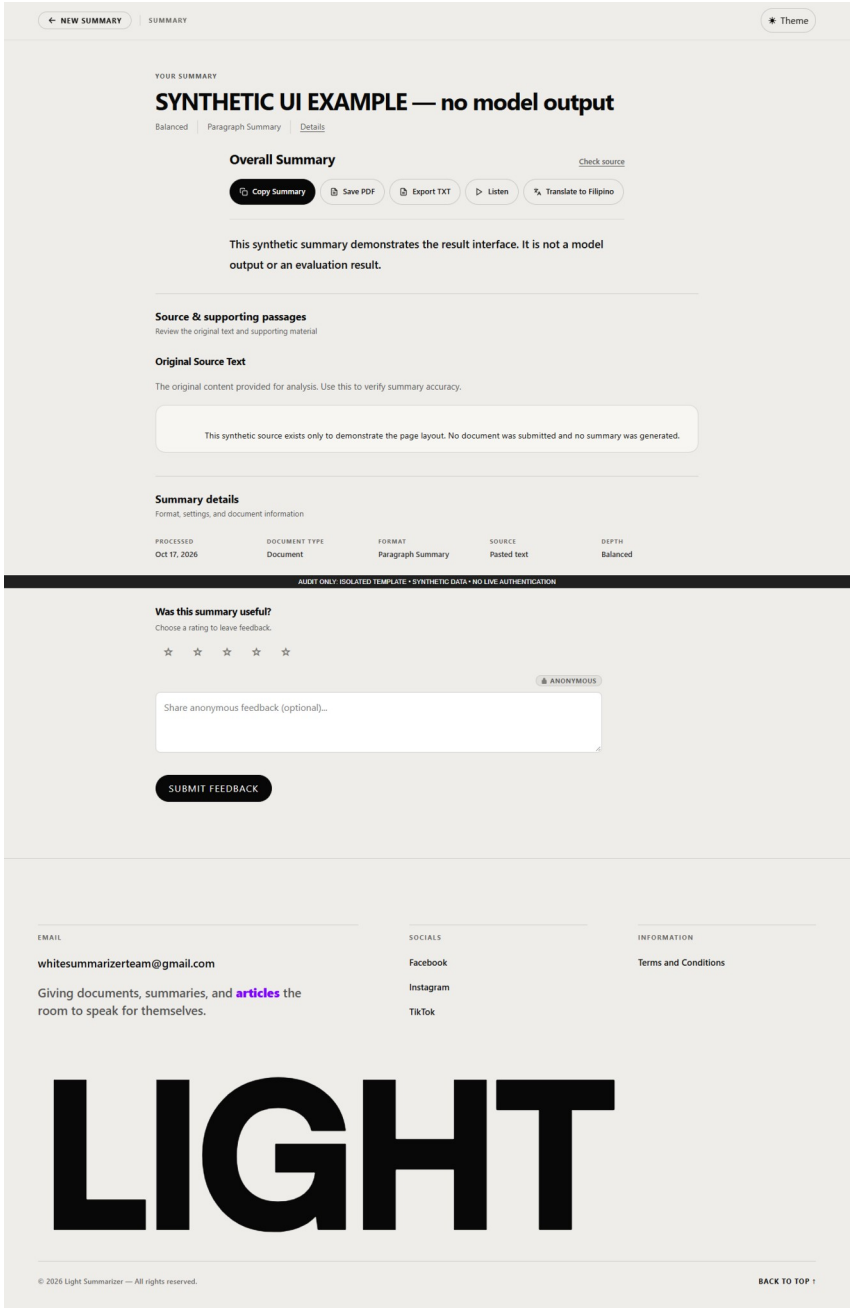



**Figure 48. Structured result and output controls — isolated template capture.**

Figure 48 illustrates the result layout, evidence-related detail and copy, text export,

print, audio, translation and feedback controls. The displayed text is synthetic rather than a generated evaluation output.

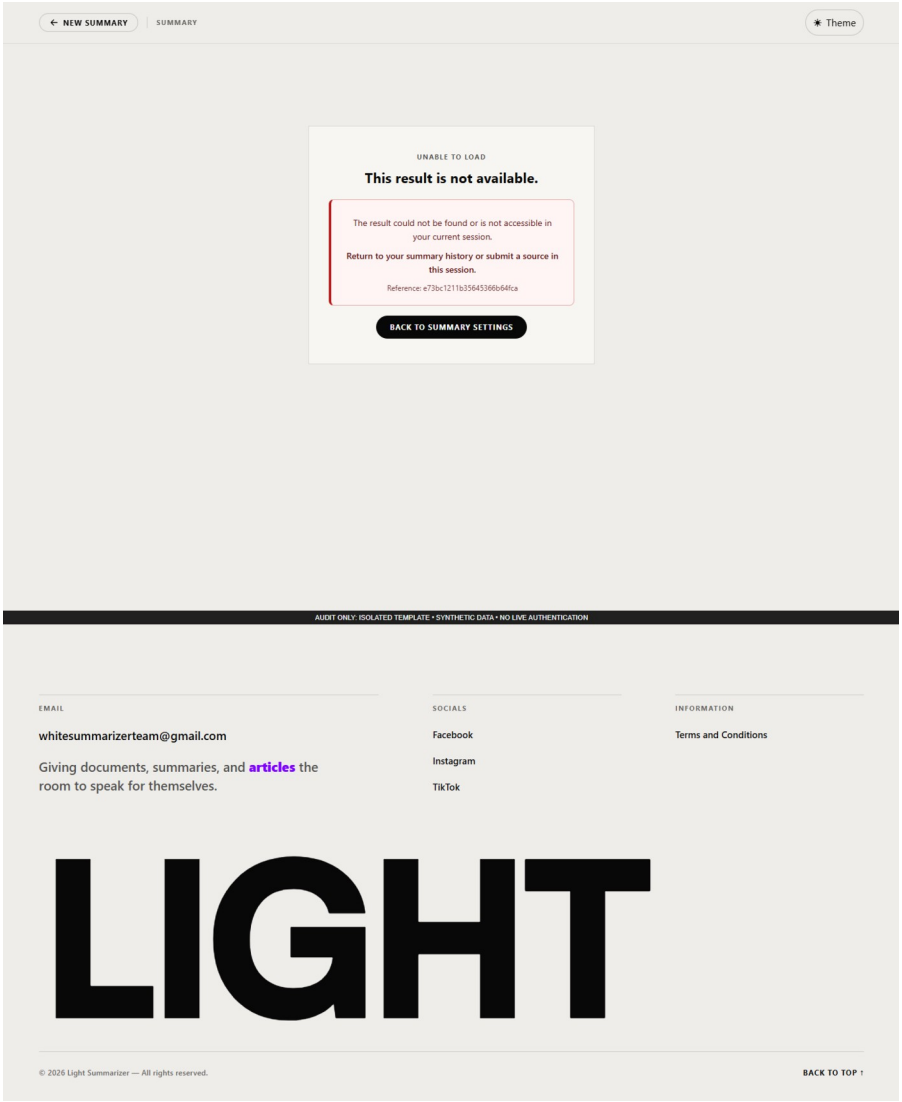



**Figure 49. Unavailable-result recovery state — isolated template capture.**

Figure 49 illustrates the message shown when an accessible result cannot be supplied. This state is part of the user’s recovery path.

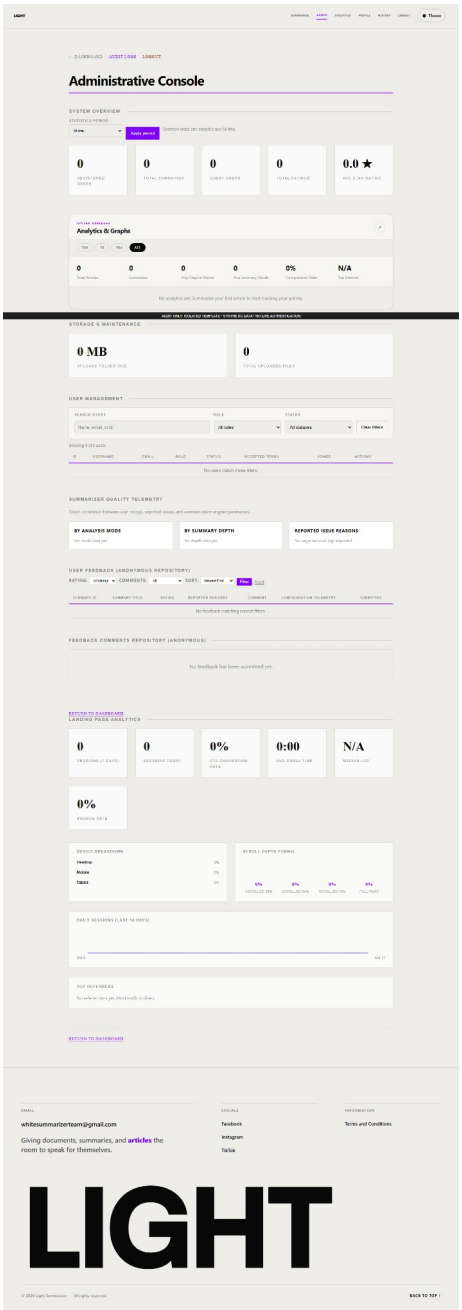



**Figure 50. Administrator overview — isolated template capture.**

Figure 50 illustrates the arrangement of account, storage, feedback and activity

administration. Empty synthetic records prevent disclosure of production data.

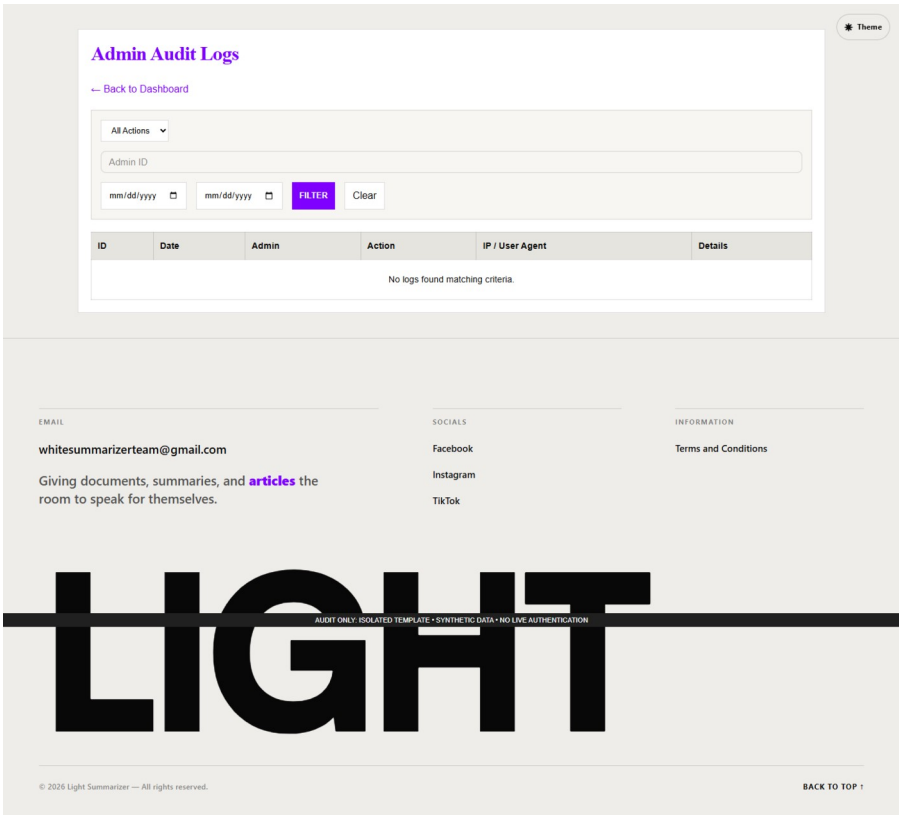



**Figure 51. Administrator audit-log filters — isolated template capture.**

Figure 51 illustrates the filtering controls and log presentation. Missing programmatic labels were observed and require correction.

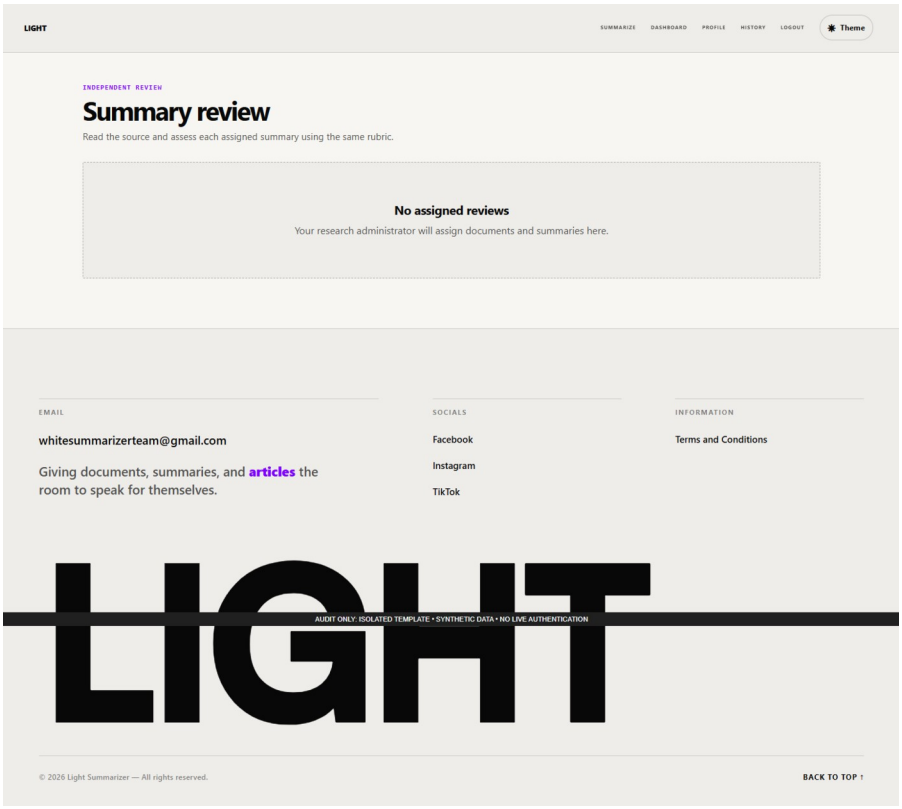



**Figure 52. Assigned evaluation empty state — isolated template capture.**

Figure 52 illustrates the reviewer view when no eligible assignment is supplied. No respondent rating or completed evaluation is represented.

## 4.4 Testing Stage

The audit used source inspection, syntax checks, read-only database metadata, isolated Python tests, synthetic controller fixtures and local browser rendering. Normal anonymous requests can create or update guest state and

can trigger expiry cleanup. To preserve production data, live authenticated workflows, uploads, mutations, email delivery and background generation were therefore not performed. Three live requests were restricted to static health or redirects that do not load the application bootstrap, without following redirects.

**Table 39. Audit verification procedures and outcomes.**

| Test group | Expected behavior | Actual result | Scope/status |
|---|---|---|---|
| PHP and JavaScript syntax | All selected files parse | 150 passed | Syntax only |
| Existing Python tests | Asserted contracts hold | 595 passed, 1 failed, 4 skipped; 62 subtests passed | Isolated confirmation run, 600 test cases |
| Optional v2 definition fixture | Retain introductory outlier definition | Expected sentence absent | Failed regression |
| Legacy cleaning probe | Preserve valid library announcement | Paragraph containing register removed | Reproduced defect |
| Legacy NIST fixture | Return nonempty summary | Nonempty response | Direct function pass |
| Source Q&A fixture | Return source passage containing 120 books | Expected passage returned | Direct function pass |

| Test group | Expected behavior | Actual result | Scope/status |
|---|---|---|---|
| Nutshell synthetic fixture | Inspect generation response | Empty insufficient_ content response | Successful generation not demonstrated |
| Document access boundary | Deny other owner and anonymous caller | Both returned HTTP 200 with source | Two failed authorization assertions |
| Audio suffix range | Return 206 and final 100 of 1,000 bytes | Returned 200 and all 1,000 bytes | Failed delivery assertion |
| Browser local interactions | Named controls behave as expected | 18 assertions matched | Isolated interface only |
| Anonymous guards | Source-specified denial/redirect | 19 responses matched | Bootstrap bypassed; not complete authorization proof |
| Responsive fixtures | Fit tested viewports | 46 captures; no document-level overflow | 23 synthetic/empty states |
| Live no-bootstrap routes | Health/ redirect response | 3 expected responses | No redirects followed |

Table 39 reports individual evidence classes rather than combining unrelated observations into a misleading overall pass percentage. The full Python case register contains test identifiers, procedures/assertion names and outcomes; logs retain the initial permission-limited attempt and the confirmation run. The four skipped cases require a live translation result URL, which was not supplied. The test

environment did not include every optional model resource.

The principal confirmed defects concern document access, whole-paragraph cleaning and audio range handling. Source inspection additionally identified incomplete upload metrics, reset-token persistence, a conflicting fresh-install reset schema, age-based cleanup described as orphan cleanup, and incomplete guest lifecycle coverage. The screenshot review identified interaction and navigation limitations. All are recorded with paths, evidence and remediation direction in KNOWN_ISSUES.md. The application was preserved rather than repaired during the audit.

An initial blank-first-page PDF probe was rejected by readability validation before it could test page-reference alignment. Therefore no measured page-number defect is claimed from that probe. Likewise, the raw Nutshell harness used PASS to mean that a response returned; its payload was insufficient_content with empty output. The manuscript interprets the payload, not the misleadingly broad harness label.

### 4.5 Deployment Stage

The verified environment is a local Windows/XAMPP workspace with an accessible MariaDB server and PHP runtime. No production hostname, public hosting account, TLS certificate, deployment approval or successful external launch was established. A claim that the application has been deployed to its intended research locale would require separate records.

Deployment requires a compatible PHP runtime and extensions, MariaDB credentials and schema, a functioning Python environment, writable application storage and appropriate process permissions. Optional OCR, language models, voices, translation and mail each require their own configuration and verification. The source supplies worker supervision and migration tools, but their existence is not proof that scheduled tasks were installed or that fresh installation succeeds. Appendix G gives staging-first procedures and identifies commands that can mutate state even when described as status or dry-run operations.

### 4.6 Operational Stage

Operational responsibility includes checking worker availability, processing failures, storage growth, mail and translation failures, database health and access-control behavior. Administrators can view activity and feedback, while maintainers have CLI and filesystem responsibilities. These responsibilities describe the implemented operational surface and recommended practice, not a verified staffing arrangement.

Cleanup helpers exist for selected telemetry, audit events, comments, temporary job payloads and guest records. Scheduled execution was not verified. Upload cleanup uses age rather than a complete database-reference check, so it cannot safely be characterized as proven orphan detection. Automated backup and a successful restore drill were not established. A responsible deployment should rehearse a consistent database-and-storage backup, verify restoration in isolation and retain a documented rollback procedure.

## 5. IT Consultants' System Evaluation

PENDING EVALUATION. No genuine consultant responses, respondent count, approved sampling procedure or final rating scale were supplied. The following seven domains preserve the requested selected ISO/IEC 25010:2011-style organization. They omit maintainability and therefore do not constitute the full eight-characteristic 2011 product-quality model. The 2011 edition has been withdrawn and replaced; the institution must approve the chosen edition and adaptation before administration. Appendix H contains proposed items. The four-point coding in Chapter II is a draft, not an approved research instrument or the application's separate five-star feedback scale.

### Functional Suitability

Whether the specified tasks produce appropriate results. Relevant sub-characteristics for the proposed technical instrument are functional completeness, correctness and appropriateness. The available audit can guide test tasks and item review, but it cannot provide a consultant's rating.

**Table 40. Functional Suitability — pending human evaluation.**

| Item/domain | Valid responses | Weighted mean | Verbal interpretation |
|---|---|---|---|
| Approved items for functional suitability | Pending | Pending | Pending approved scale and responses |

Table 40 is an unpopulated research template. No score, number of respondents or interpretation can be inferred from it. After approved administration, valid response frequencies and the stated analysis rule must support each value; audit findings remain separate evidence.

The result discussion remains pending because no completed responses are available. Before evaluation, the researcher should resolve or disclose the relevant known limitations, preserve the tested software version and document the conditions under which the task is performed.

**Performance Efficiency**

Whether representative tasks meet approved time and resource expectations. Relevant sub-characteristics for the proposed technical instrument are time behaviour, resource utilisation and capacity. The available audit

can guide test tasks and item review, but it cannot provide a consultant’s rating.

**Table 41. Performance Efficiency — pending human evaluation.**

| Item/domain | Valid responses | Weighted mean | Verbal interpretation |
|---|---|---|---|
| Approved items for performance efficiency | Pending | Pending | Pending approved scale and responses |

Table 41 is an unpopulated research template. No score, number of respondents or interpretation can be inferred from it. After approved administration, valid response frequencies and the stated analysis rule must support each value; audit findings remain separate evidence.

The result discussion remains pending because no completed responses are available. Before evaluation, the researcher should resolve or disclose the relevant known limitations, preserve the tested software version and document the conditions under which the task is performed.

**Compatibility**

Whether the application works alongside relevant software and exchanges supported data. Relevant sub-characteristics for the

proposed technical instrument are co-existence and interoperability. The available audit can guide test tasks and item review, but it cannot provide a consultant’s rating.

**Table 42. Compatibility — pending human evaluation.**

| Item/domain | Valid responses | Weighted mean | Verbal interpretation |
|---|---|---|---|
| Approved items for compatibility | Pending | Pending | Pending approved scale and responses |

Table 42 is an unpopulated research template. No score, number of respondents or interpretation can be inferred from it. After approved administration, valid response frequencies and the stated analysis rule must support each value; audit findings remain separate evidence.

The result discussion remains pending because no completed responses are available. Before evaluation, the researcher should resolve or disclose the relevant known limitations, preserve the tested software version and document the conditions under which the task is performed.

**Usability**

Whether intended users can understand, operate and recover from the interface. Relevant sub-characteristics for the proposed technical instrument are appropriateness recognisability, learnability, operability, user error protection, interface aesthetics and accessibility. The available audit can guide test tasks and item review, but it cannot provide a consultant's rating.

**Table 43. Usability — pending human evaluation.**

| Item/domain | Valid responses | Weighted mean | Verbal interpretation |
|---|---|---|---|
| Approved items for usability | Pending | Pending | Pending approved scale and responses |

Table 43 is an unpopulated research template. No score, number of respondents or interpretation can be inferred from it. After approved administration, valid response frequencies and the stated analysis rule must support each value; audit findings remain separate evidence.

The result discussion remains pending because no completed responses are available. Before evaluation, the researcher should resolve or disclose the relevant known

limitations, preserve the tested software version and document the conditions under which the task is performed.

**Reliability**

Whether the system behaves consistently and recovers under defined conditions. Relevant sub-characteristics for the proposed technical instrument are maturity, availability, fault tolerance and recoverability. The available audit can guide test tasks and item review, but it cannot provide a consultant’s rating.

**Table 44. Reliability — pending human evaluation.**

| Item/domain | Valid responses | Weighted mean | Verbal interpretation |
|---|---|---|---|
| Approved items for reliability | Pending | Pending | Pending approved scale and responses |

Table 44 is an unpopulated research template. No score, number of respondents or interpretation can be inferred from it. After approved administration, valid response frequencies and the stated analysis rule must support each value; audit findings remain separate evidence.

The result discussion remains pending because no completed responses are available.

Before evaluation, the researcher should resolve or disclose the relevant known limitations, preserve the tested software version and document the conditions under which the task is performed.

**Security**

Whether authorized use and protection of information are adequately supported. Relevant sub-characteristics for the proposed technical instrument are confidentiality, integrity, non-repudiation, accountability and authenticity. The available audit can guide test tasks and item review, but it cannot provide a consultant’s rating.

**Table 45. Security — pending human evaluation.**

| Item/domain | Valid responses | Weighted mean | Verbal interpretation |
|---|---|---|---|
| Approved items for security | Pending | Pending | Pending approved scale and responses |

Table 45 is an unpopulated research template. No score, number of respondents or interpretation can be inferred from it. After approved administration, valid response frequencies and the stated analysis rule must support each value; audit findings remain separate evidence.

The result discussion remains pending because no completed responses are available. Before evaluation, the researcher should resolve or disclose the relevant known limitations, preserve the tested software version and document the conditions under which the task is performed.

**Portability**

Whether the application can be installed or adapted within a specified environment. Relevant sub-characteristics for the proposed technical instrument are adaptability, installability and replaceability. The available audit can guide test tasks and item review, but it cannot provide a consultant's rating.

**Table 46. Portability — pending human evaluation.**

| Item/domain | Valid responses | Weighted mean | Verbal interpretation |
|---|---|---|---|
| Approved items for portability | Pending | Pending | Pending approved scale and responses |

Table 46 is an unpopulated research template. No score, number of respondents or interpretation can be inferred from it. After approved administration, valid response

frequencies and the stated analysis rule must support each value; audit findings remain separate evidence.

The result discussion remains pending because no completed responses are available. Before evaluation, the researcher should resolve or disclose the relevant known limitations, preserve the tested software version and document the conditions under which the task is performed.

**Overall IT Consultants' Evaluation Summary**

An overall result requires approved domain aggregation rules and genuine domain observations. Empty domains must not be treated as zero scores or silently excluded to create a favorable mean.

**Table 47. Overall IT consultants' summary — pending human evaluation.**

| Item/domain | Valid responses | Weighted mean | Verbal interpretation |
|---|---|---|---|
| Seven approved consultant domains | Pending | Pending | Pending approved scale and responses |

Table 47 is an unpopulated research template. No score, number of respondents or interpretation can be inferred from it. After approved administration, valid response

frequencies and the stated analysis rule must support each value; audit findings remain separate evidence.

**6. End-Users' System Evaluation**

PENDING EVALUATION. The supplied material identifies intended academic users but does not provide completed end-user questionnaires, consent records, sample counts or task results. The proposed instrument uses ordinary language and keeps task suitability, perceived performance and usability separate. It cannot establish classroom learning gains without an appropriate additional research design.

**Functional Suitability**

This domain asks whether the available input, summary and result functions meet the participant's specified reading task. Responses must come from actual participants using the approved version and protocol.

**Table 48. Functional Suitability for end users — pending human evaluation.**

| Item/domain | Valid responses | Weighted mean | Verbal interpretation |
|---|---|---|---|
| Approved end-user items for functional suitability | Pending | Pending | Pending approved scale and responses |

Table 48 is an unpopulated research template. No score, number of respondents or interpretation can be inferred from it. After approved administration, valid response frequencies and the stated analysis rule must support each value; audit findings remain separate evidence.

The narrative result is pending. The interface captures and engineering test outcomes cannot be relabeled as satisfaction responses, and no comparison with an earlier manual process can be made from the available evidence.

**Performance Efficiency**

This domain asks whether waiting and response behavior are acceptable for the agreed task and device. Responses must come from actual participants using the approved version and protocol.

**Table 49. Performance Efficiency for end users — pending human evaluation.**

| Item/domain | Valid responses | Weighted mean | Verbal interpretation |
|---|---|---|---|
| Approved end-user items for performance efficiency | Pending | Pending | Pending approved scale and responses |

Table 49 is an unpopulated research template. No score, number of respondents or interpretation can be inferred from it. After approved administration, valid response frequencies and the stated analysis rule must support each value; audit findings remain separate evidence.

The narrative result is pending. The interface captures and engineering test outcomes cannot be relabeled as satisfaction responses, and no comparison with an earlier manual process can be made from the available evidence.

**Usability**

This domain asks whether the participant can understand the steps, controls, messages and result presentation. Responses must come from actual participants using the approved version and protocol.

**Table 50. Usability for end users — pending human evaluation.**

| Item/domain | Valid responses | Weighted mean | Verbal interpretation |
|---|---|---|---|
| Approved end-user items for usability | Pending | Pending | Pending approved scale and responses |

Table 50 is an unpopulated research template. No score, number of respondents or interpretation can be inferred from it. After approved administration, valid response frequencies and the stated analysis rule must support each value; audit findings remain separate evidence.

The narrative result is pending. The interface captures and engineering test outcomes cannot be relabeled as satisfaction responses, and no comparison with an earlier manual process can be made from the available evidence.

**Overall End-Users' Evaluation Summary**

The proposed summary combines only the domains and item weights approved before data collection. Participants with missing or not-applicable answers must be handled transparently under that rule.

**Table 51. Overall end-users' summary — pending human evaluation.**

| Item/domain | Valid responses | Weighted mean | Verbal interpretation |
|---|---|---|---|
| Three approved end-user domains | Pending | Pending | Pending approved scale and responses |

Table 51 is an unpopulated research template. No score, number of respondents or interpretation can be inferred from it. After approved administration, valid response frequencies and the stated analysis rule must support each value; audit findings remain separate evidence.

# CHAPTER IV

# SUMMARY, CONCLUSIONS AND RECOMMENDATIONS

## 1. Summary

LIGHT summarizer is an implemented PHP and Python web application intended to support article and document summarization. The researchers' draft places its intended academic use at NEUST Peñaranda Off-Campus and describes an Agile approach with an April–October 2026 schedule. The audit does not establish that the planned activities, locale-based evaluation or institutional approvals were completed. This distinction limits the conclusions that can be drawn from the documentation.

In relation to the proposed questions about users' difficulties, the supplied draft identifies extensive reading, selection of main ideas and preservation of important details as motivations. No interview dataset, baseline observation or respondent frequency distribution was available. The needs therefore remain researcher-supplied problem statements rather than independently measured research findings.

In relation to system development and required functionality, the audit documented sixty-nine distinct capabilities. These include supported source intake, configurable summarization, job processing, result controls, history, accounts, administration, evaluation services and maintenance facilities. Five narrowly scoped capabilities were verified working; eighteen have supported limitations and are classified partially working; forty-six are implemented but remain unverified end-to-end. These counts use a documented capability grouping and do not imply that every source method or output combination is a separate feature.

The observed database contains thirty-one tables, three hundred twelve columns and thirty-five declared foreign keys. The diagrams and data dictionary reflect those actual constraints. Source inspection also identified differences between the current reset-token table and the clean-install SQL, so the observed database should not be confused with a successfully rehearsed installation process.

In relation to technical verification, one hundred fifty PHP/JavaScript files passed syntax checks. The confirmed isolated Python

run produced 595 passed, one failed and four skipped test cases, with sixty-two passed subtests. Eighteen local browser assertions, nineteen isolated anonymous-guard expectations and three live no-bootstrap response checks matched expectations. Forty-six original template captures document twenty-three desktop/mobile interface states. Those captures contain synthetic or empty data, and no completed login or production generation workflow is inferred from them.

Important limitations were also demonstrated. The document API returned source content to an unauthorized owner context and to an anonymous context in actual controller/service fixtures. A legacy cleaning rule removed a meaningful paragraph containing the ordinary word register. An audio suffix-range request returned the entire file, and an optional v2 regression omitted an expected definition. Additional source-supported concerns involve reset-token persistence, administrator-status safeguards, metrics persistence, guest cleanup coverage and deployment configuration.

In relation to consultant and end-user evaluation, no genuine completed questionnaires or approved sample records were

supplied. The evaluation instruments and result tables are proposed or pending. The audit therefore provides no weighted means, acceptance ratings, measured time savings, learning improvements or certification claim.

## 2. Conclusions

The available implementation demonstrates a substantial software foundation for source-based summarization and supporting account and administrative activities. Its capability and schema documentation can be traced to actual files, method inventories and read-only metadata. The narrow tests establish specific behavior within their recorded boundaries; they do not establish complete operational reliability or quality for the intended student population.

The current evidence does not support describing the application as uniformly secure, fully verified, production-ready or validated by consultants and end users. Reproduced authorization and processing defects require attention before evaluation or wider deployment. The optional v2 path should also remain distinct from the configured legacy engine when reporting results.

The research conclusions concerning user needs, effectiveness and acceptance remain pending. A supplied methodology narrative and schedule can support planning documentation, but they cannot substitute for actual development records, approved participant procedures or collected responses. The manuscript is consequently an evidence-grounded documentation foundation with explicit research-completion requirements.

**3. Recommendations**

The first implementation priority is to enforce authentication and ownership across every document API action, including retrieval and cached combinations, and to apply suitable CSRF and request controls. Regression checks should demonstrate denial for anonymous and cross-owner contexts while preserving authorized access. Existing reset sessions and remembered tokens should be addressed when passwords change, and all administrator status-changing routes should apply consistent self/last-administrator safeguards and audit recording.

The processing team should replace whole-paragraph navigation-word filtering with a context-sensitive rule that preserves ordinary

prose, then retain the reproduced library fixture as a regression case. The optional v2 inline-heading failure should be corrected and reviewed against representative documents. Audio range handling and cache visibility should be corrected, and real audio generation should be tested with the intended voice and language configuration.

Installation and data integrity should be addressed in a disposable staging environment. The reset-token schema should be reconciled with checked-in migrations, the supported runtime/dependency matrix should be established, and a clean install should be rehearsed. Upload source counts and elapsed-time persistence should be verified before relying on analytics. Guest lifecycle coverage and cleanup should be reviewed across all stores, and upload deletion should check references rather than infer orphan status solely from age.

A complete authenticated test deployment should then cover registration, code delivery, password reset, terms, source submission, jobs, retries, cancellation, saved result access, feedback, account administration, exports and evaluator assignments. Tests should include actual supported files,

realistic documents and unavailable-service cases without using production personal data. Accessibility review should address keyboard access and labels and should include assistive-technology testing. Representative performance and recovery drills should be designed around explicit acceptance criteria.

The researchers should confirm the institution, program, title wording, adviser, locale, development records, participant groups, sampling procedure and approved evaluation scale. The proposed research questions and instruments should be reviewed before administration. Any genuine data should retain its provenance, consent and anonymization arrangements. Weighted means and conclusions should be calculated only after validated responses are supplied, with missing and not-applicable observations handled under a declared rule.

Before final submission, institutional reviewers should approve the adapted reference formatting and research sections, replace all explicit placeholders, verify current literature and citations, and review the regenerated Word and PDF files. Before deployment, maintainers should resolve the priority defects, verify production

configuration and rehearse backup and restoration. These are future actions grounded in observed gaps, not claims that the activities have already occurred.

## REFERENCES

Fabbri, A. R., Kryściński, W., McCann, B., Xiong, C., Socher, R., & Radev, D. (2021). SummEval: Re-evaluating summarization evaluation. *Transactions of the Association for Computational Linguistics, 9*, 391–409. Publisher record and paper. DOI: 10.1162/tacl_a_00373.

International Organization for Standardization & International Electrotechnical Commission. (2011). *ISO/IEC 25010:2011: Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. Official standard record. Edition status checked 17 October 2026: withdrawn 4 March 2024. Used only to identify the historical template's framework; no claim of complete conformity is made.

International Organization for Standardization & International Electrotechnical Commission. (2023). *ISO/IEC 25010:2023: Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*. Official standard record. Official record checked 17 October 2026.

Lin, C.-Y. (2004). ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out* (pp. 74–81). Association for Computational Linguistics. Paper and bibliographic record.

Maynez, J., Narayan, S., Bohnet, B., & McDonald, R. (2020). On faithfulness and factuality in abstractive summarization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics* (pp. 1906–1919). Association for Computational Linguistics. Paper and bibliographic record. DOI: 10.18653/v1/2020.acl-main.173.

Mihalcea, R., & Tarau, P. (2004). TextRank: Bringing order into text. In *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing* (pp. 404–411). Association for Computational Linguistics. Paper and bibliographic record. The citation uses the ACL bibliographic title; the PDF title displays “Texts.”

**Project and Documentary Sources**

LIGHT summarizer project materials. (n.d.). *Application source, database definitions, configuration and operational scripts* [Unpublished software workspace supplied for

audit]. Inspected 17 October 2026. Relevant evidence is identified in the technical audit reports, source maps and schema snapshot. Repository authorship chronology and release history are not inferred from file presence.

Researcher-supplied draft. (n.d.). *Development of an NLP-Based Article Summarization Tool for NEUST–Penaranda Off Campus* [Unpublished draft and embedded diagrams supplied during this task]. The document supplies the intended locale, a reported Agile approach and an April–October 2026 schedule. The date of authoring, institutional approval and activity completion are not established. The explicit current title and researcher-name spellings supplied by the user govern this manuscript.

*AccredX-Manuscript-Final.pdf*. (n.d.). [User-supplied reference manuscript, 184 PDF pages]. Consulted solely for organization, page conventions and appendix structure. Its project-specific claims, bibliography, institutional approvals, participant results and personal records are not evidence for LIGHT summarizer.

The external research references above were verified through their primary publication or

official standard records. This is a focused verified literature base, not a claim to an exhaustive systematic review. Any additional institution-required literature should be independently verified and integrated before final submission; unverified citations from the format reference have not been transferred.

## APPENDIX A. RESEARCH ADVISER REQUEST LETTER TEMPLATE

UNSIGNED TEMPLATE — NOT A SENT OR APPROVED LETTER

[Actual date]

[Prospective adviser's name and designation]

[Official academic unit]

Nueva Ecija University of Science and Technology

[Confirmed campus/address]

Subject: Request for research advisement for LIGHT summarizer

Dear [appropriate salutation]:

We, Robert Martin Baltisoto, Shielo Mariell Leodones, Gieselle Sabbaluca and Sofiah Heaven Galman, respectfully request your consideration as adviser for our capstone project, LIGHT summarizer, under [confirmed program and course]. The project concerns an article-summarization application intended to support the review of written materials. The accompanying draft distinguishes implemented software, technical audit findings and proposed human evaluation.

We seek guidance on the research questions, scope, intended campus population, evaluation design and institutional requirements. The current technical documentation identifies unresolved defects and does not claim completed participant assessment. We propose consultation arrangements of [schedule to be agreed], subject to your availability and the institution's procedures.

Thank you for considering this request.

Respectfully,

Robert Martin Baltisoto — [signature]

Shielo Mariell Leodones — [signature]

Gieselle Sabbaluca — [signature]

Sofiah Heaven Galman — [signature]

Adviser's response/official appointment reference: [to be completed by the authorized person].

## APPENDIX B. DATA-GATHERING PERMISSION REQUEST TEMPLATE

UNSIGNED TEMPLATE — NO PERMISSION OR COLLECTION IS IMPLIED

[Actual date]

[Authorized campus/department official]

[Designation and office]

[Confirmed institutional address]

Subject: Request to conduct approved evaluation activities for LIGHT summarizer

Dear [appropriate salutation]:

We respectfully request permission to conduct the proposed activities described in protocol [version/date] for LIGHT summarizer at [confirmed locale]. The proposed participants are [approved eligibility group], with [approved recruitment method and target number]. Requested dates, venue and duration are [to be specified]. No recruitment or collection is represented as already completed by this request.

The activity would involve [approved tasks] using permitted nonconfidential source materials and a reviewed evaluation environment. Participants would receive the

approved information and consent materials, may decline without academic penalty, and would answer a coded questionnaire independently. The protocol should specify [responsible contact], [access controls], [retention/deletion period] and [withdrawal procedure]. Any required ethics or institutional clearance must be obtained before the corresponding activity.

We attach [actual protocol], [approved instruments], [participant information/consent form] and [technical readiness record]. We request your written decision and any conditions required by the institution.

Respectfully,

Robert Martin Baltisoto; Shielo Mariell Leodones; Gieselle Sabbaluca; Sofiah Heaven Galman — [signatures].

Noted by: [actual adviser and signature, if required].

Official decision/reference/date: [authorized official to complete].

## APPENDIX C. ENGLISH CRITIC REVIEW REQUEST TEMPLATE

UNSIGNED TEMPLATE — REVIEW HAS NOT BEEN CERTIFIED

[Actual date]

[English critic's name, credentials and designation]

[Office/contact details approved for inclusion]

Subject: Request for language review of LIGHT summarizer manuscript

Dear [appropriate salutation]:

We respectfully request a language and presentation review of manuscript version [identifier/date] entitled LIGHT summarizer. The review is requested for clarity, grammar, consistency and academic presentation. Technical findings, research approvals and evaluation results should retain their evidence qualifications; language revision should not convert proposed activities into completed claims.

Please advise us of the agreed review scope, schedule and requirements. If a certification is issued after the review, we request that it identify the reviewed version and actual

completion date. This request does not serve as the certification itself.

Respectfully,

Robert Martin Baltisoto; Shielo Mariell Leodones; Gieselle Sabbaluca; Sofiah Heaven Galman — [signatures].

Review agreement and subsequent certification reference: [to be supplied].

## APPENDIX D. CONSULTATION REQUEST AND MONITORING FORM

UNSIGNED REQUEST TEMPLATE AND BLANK LOG — NO CONSULTATION HISTORY IS IMPLIED

[Actual date]

[Adviser/consultant name and designation]

Subject: Request for project consultation and documented follow-up

Dear [appropriate salutation]:

We request a consultation on LIGHT summarizer concerning [specific issue or manuscript section]. The materials proposed for review are [actual version/evidence references]. Our requested agenda is [questions], with a proposed meeting arrangement of [date, time and medium]. Please confirm an available arrangement and any preparatory requirements.

We will record the actual advice, agreed actions and subsequent evidence of completion. The monitoring form below is blank and should be completed only after an authentic consultation. A scheduled meeting alone does not establish that advice was received or an action completed.

Respectfully,

Robert Martin Baltisoto; Shielo Mariell Leodones; Gieselle Sabbaluca; Sofiah Heaven Galman — [signatures].

**Table 52. Blank consultation and monitoring record.**

| Actual date / medium | Topic and reviewed version | Advice / agreed action | Responsible person / follow-up evidence |
|---|---|---|---|
| [actual entry] | [actual entry] | [actual entry] | [actual entry] |

Table 52 provides a blank structure for an authentic consultation record. Each completed row should identify the reviewed material, actual advice and follow-up evidence; a scheduled activity is not a completed consultation.

Adviser/consultant confirmation: [name, signature or authentic reference]. Researcher acknowledgement: [actual record]. Do not prefill the supplied April–October schedule as completed consultation entries.

## APPENDIX E. RESEARCH ADVISER DECISION LETTER TEMPLATE

FOR COMPLETION BY THE AUTHORIZED ADVISER — NOT AN APPROVAL

[Actual date]

[Authorized recipient and designation]

[Academic unit and institution]

Subject: Adviser review decision concerning LIGHT summarizer

Dear [appropriate salutation]:

The manuscript/project version presented for review is [version and date], prepared by Robert Martin Baltisoto, Shielo Mariell Leodones, Gieselle Sabbaluca and Sofiah Heaven Galman. The scope of the review performed is [actual review scope and supporting record].

Decision: [the adviser must enter the actual decision, such as revisions required or endorsed for the specified next step]. Conditions and outstanding requirements: [actual conditions]. The decision applies to [specific submission or activity] and should not be interpreted beyond that scope.

Adviser: [full name and credentials]

Designation: [official designation]

Signature: [authentic signature]

Date/reference: [actual record]

Replace this template with the institution's prescribed decision/approval document when available. An adviser decision does not automatically constitute participant-research permission, ethics clearance, examination approval or deployment authorization.

## APPENDIX F — USER MANUAL

This manual describes LIGHT, the document summarization application found in the audited source. It distinguishes guest, registered-account, administrator, and assigned-reviewer access. The instructions describe implemented controls; successful submission, authentication, email delivery, worker operation, and data persistence still require an approved end-to-end demonstration.

**Screenshot evidence limitation.** Every screenshot in this appendix was captured from the original application templates in an isolated browser renderer on 17 October 2026. Anonymous or synthetic account sessions and empty/synthetic records were supplied to avoid altering the existing application database. A black provenance banner identifies that condition. The displayed account, summary, zero counters, and empty lists are examples, not actual usage or research results. External fonts and service requests were blocked. These screenshots explain the interface; they do not prove a working deployment or successful authentication. Capture URLs, desktop/mobile versions, and exact checks appear in

screenshots/README.md and ../system-audit/browser-readonly-results.json.

**Table 53. Available access contexts and interface boundaries.**

| Access context | Available interface |
|---|---|
| Guest | Public information and account forms, summary workspace, accessible results, and permitted result actions. Nutshell, history, and analytics direct guests toward registration. |
| Registered account | Summary workspace, Nutshell, dashboard, own history/analytics, profile, and accessible results. An active account is required for assigned reviews. |
| Administrator | Administrative console, system-wide analytics, user-management actions, audit logs, and protected diagnostics/exports. Administrators can also use summary-related interfaces. |
| Assigned reviewer | An active account receives access to its assigned blinded summary reviews. Assignment access is separate from merely having an account. |

Table 53 presents available access contexts and interface boundaries. The stated status and qualifications govern its interpretation.

## A. Landing Page and Shared Navigation

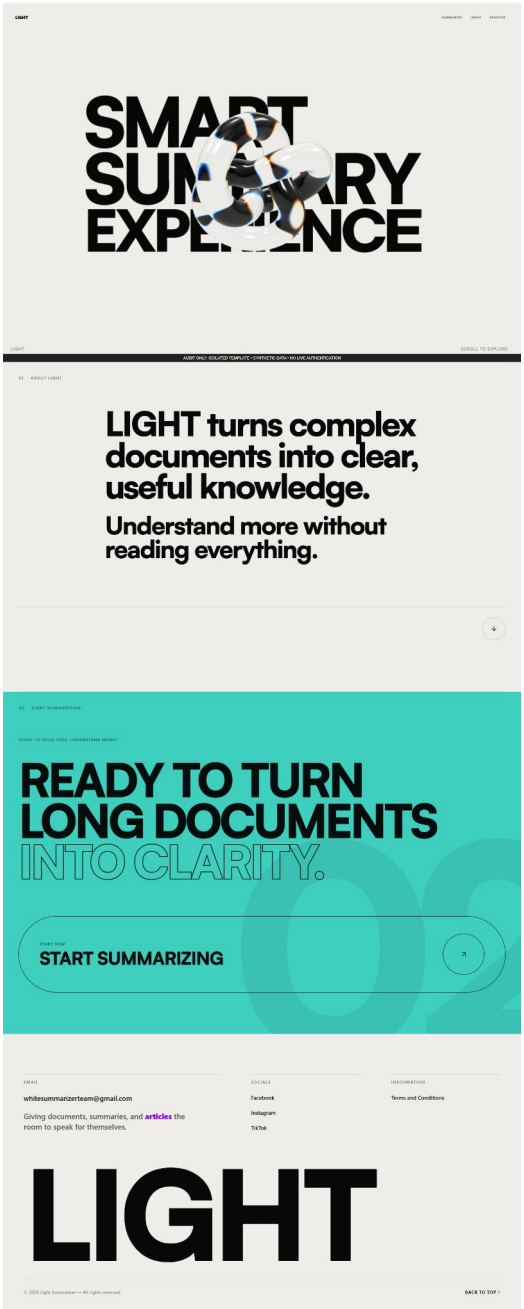



**Figure 53. LIGHT landing page rendered from source with an anonymous fixture.**

The landing page introduces the application's purpose and leads to the document workspace. Figure 53 shows its shared navigation and information sections. Select **Summarize** to open the workspace. Select **Login** or **Register** to access account forms. The About LIGHT section explains the intended purpose; the older About address redirects to that section. Signed-in administrator visits to the landing route lead to the summarizer.

1. Open the application address supplied by the operator.
2. Read the landing information and select **Summarize** or the start link.
3. On a narrow screen, select the menu button to reveal navigation. Select a link or press **Escape** to close it.
4. Use the footer **Terms and Conditions** link to read the conditions. **Back to top** returns to the beginning of the current page.

The footer email link opens the reader's email application. It does not itself submit a support request. The social links currently open general Facebook, Instagram, and TikTok homepages; the audit did not verify project-specific social accounts. The shared theme control appears on supported nonlanding pages.

Theme preferences are stored in that browser. Mobile menu opening and Escape closure were verified in isolation; external links were not opened.

## B. Sign In and Administrator Verification

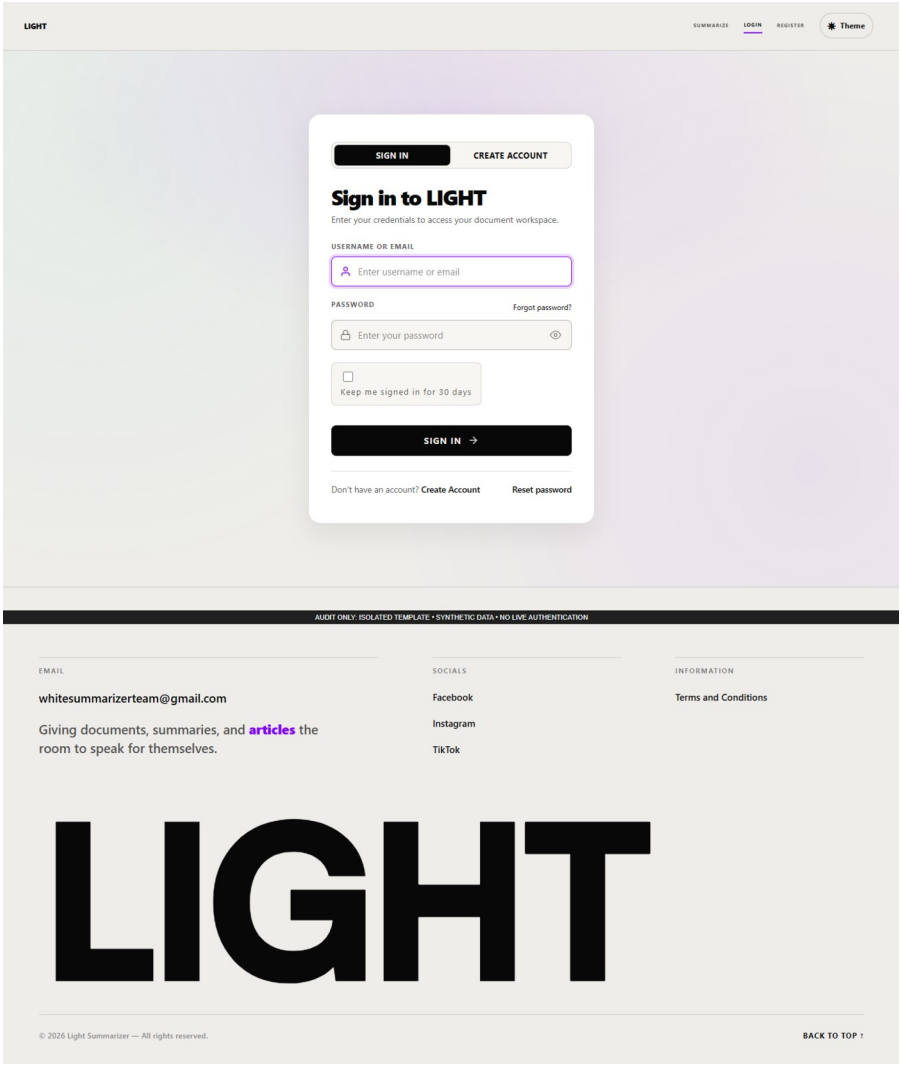



**Figure 54. Sign-in form rendered with an anonymous fixture.**

Figure 54 shows the ordinary account form. Enter a username or email and the account password, then select **Sign In**. The password visibility button changes masking on screen. The regular sign-in form offers **Keep me signed in for 30 days**; use that option only where retaining the account session is appropriate. **Forgot password?** and **Reset password** lead to recovery. **Create Account** opens registration.

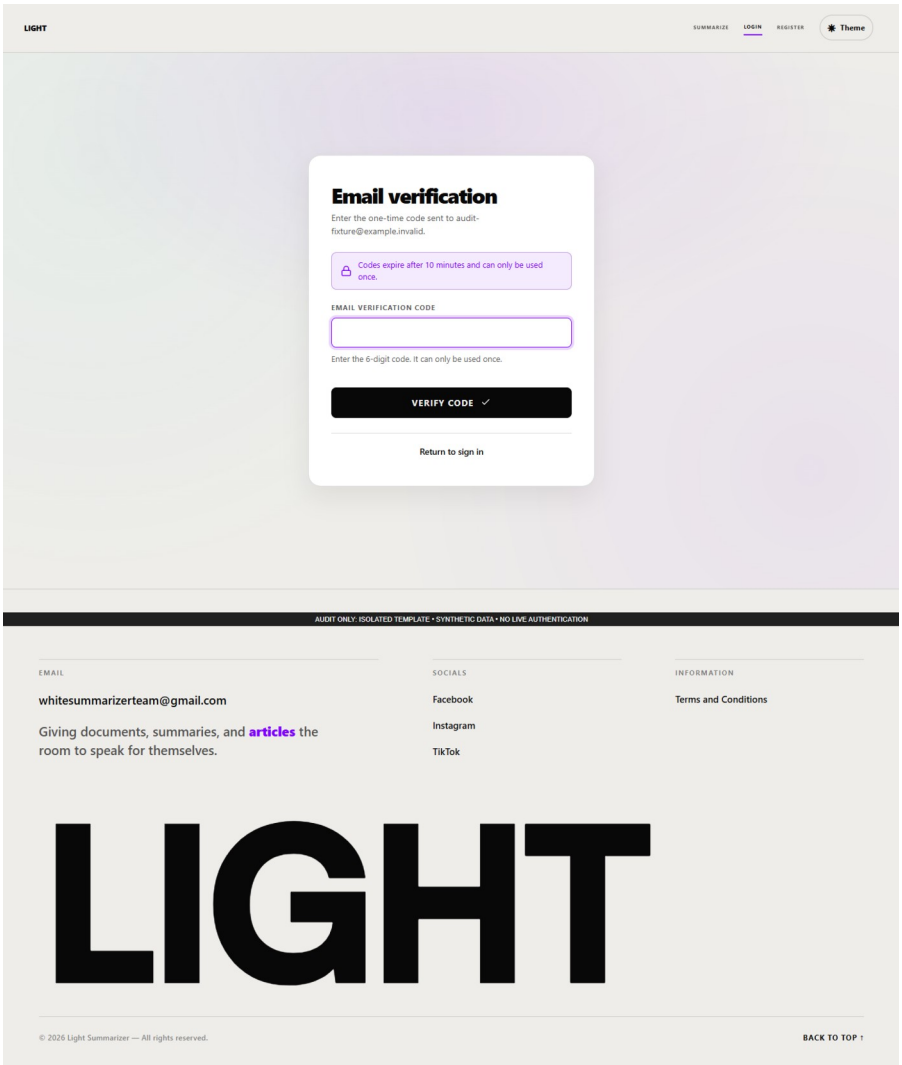



**Figure 55. Conditional administrator email-code screen using a synthetic pending-login fixture.**

For the administrator login flow, the application can replace the credential form with the code screen in Figure 55. Enter the six-digit code sent to the displayed masked/hinted address and select **Verify code**. The interface states that codes expire after

ten minutes and can be used once. **Return to sign in** cancels the pending code step. The old administrator challenge setup/change addresses are compatibility redirects and do not present an active challenge-configuration screen.

Empty fields, invalid credentials, expired codes, inactive accounts, and request failures can prevent access. Follow the displayed message and retry the appropriate step. No actual login or email verification was performed during this documentation audit. Password visibility was tested only with synthetic text. Its current visibility buttons are excluded from ordinary Tab navigation; this is recorded as an accessibility issue.

## C. Create Account

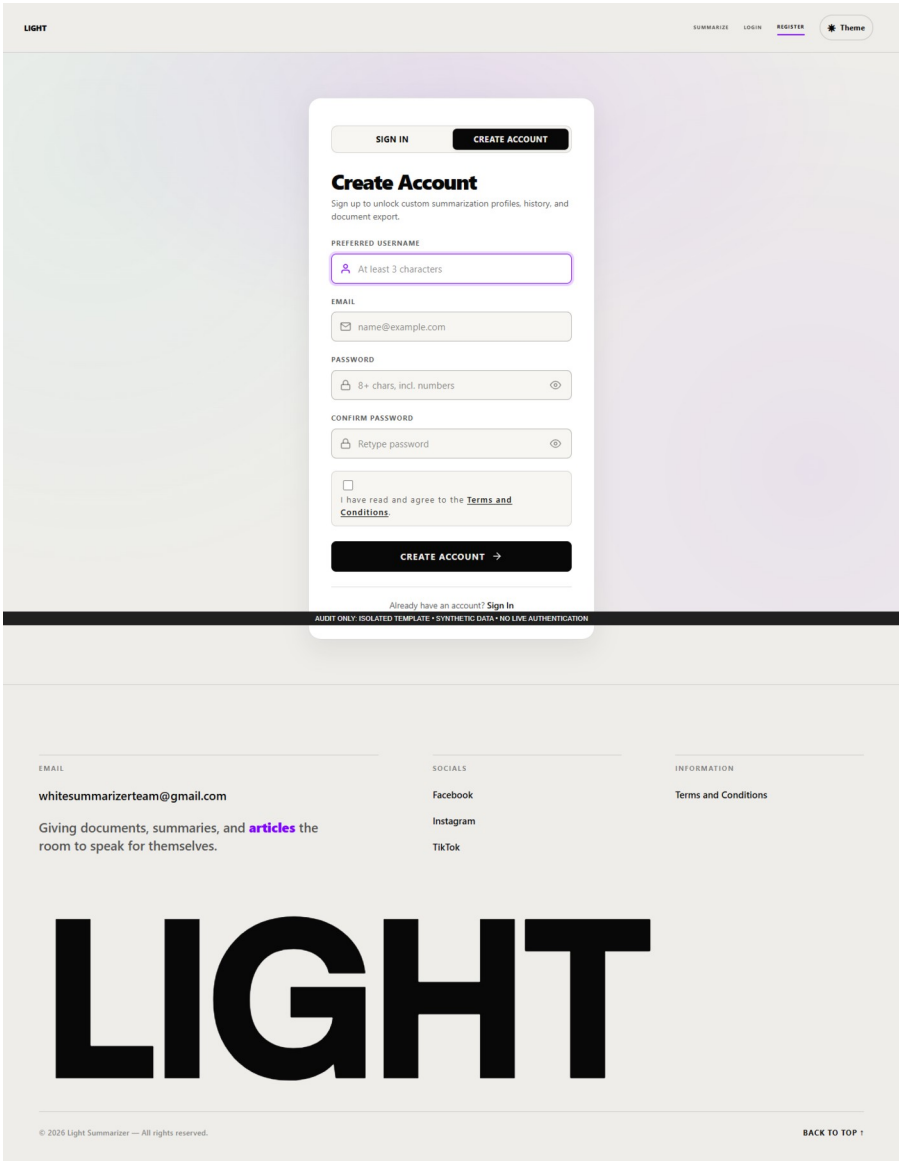



**Figure 56. Registration form rendered without submitting account data.**

The registration form in Figure 56 collects a preferred username, email address, password,

password confirmation, and acceptance of the terms.

1. Select **Create Account** or **Register**.
2. Enter the username and email address. The interface requests a username of at least three characters.
3. Enter a password and repeat it in **Confirm Password**.
4. Read the linked **Terms and Conditions**, then select the agreement checkbox if accepting them.
5. Select **Create Account** and follow the response.

The server password policy checks a length of 8–72 bytes, at least one number, and at least one uppercase letter or special character. For ordinary ASCII passwords this corresponds to 8–72 characters; multibyte characters consume more than one byte. Strength and match hints provide immediate feedback, while the server makes the final validation decision. The eye buttons can show or hide each password field.

No account was created during the audit. The form's mention of “custom summarization profiles” should not be interpreted as an

editable saved-profile screen: the discovered dashboard can infer preferred settings from past summaries, and the Profile page displays account details and deactivation.

### D. Recover Password

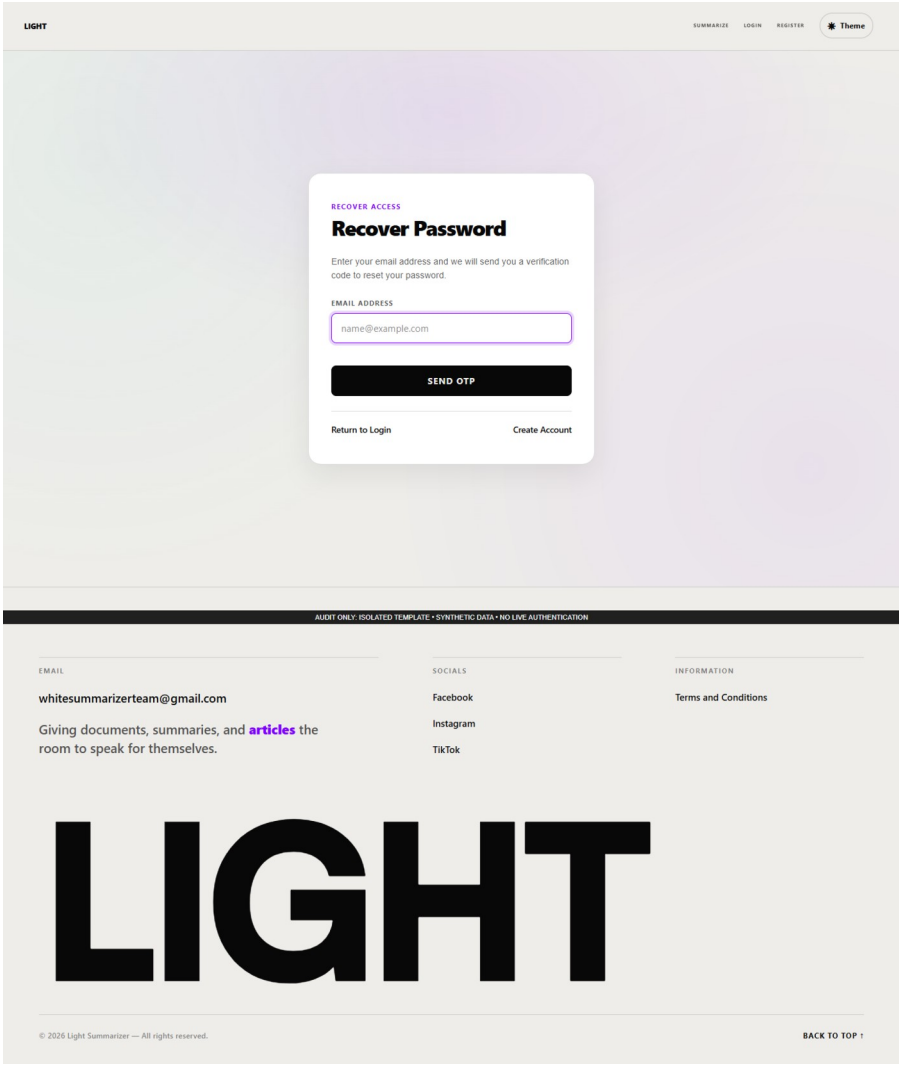



**Figure 57. Password-recovery form rendered without sending an email.**

Use Figure 57's form when the account password is forgotten. Enter the account email and select **Send OTP**. Follow the recovery response and use the received verification code in the reset form. **Return to Login** and **Create Account** provide alternative navigation. A success-style response alone should not be treated as proof that an email has arrived; inspect the actual inbox and the application's message. Delivery and recovery outcomes were not exercised during the audit.

## E. Set a New Password

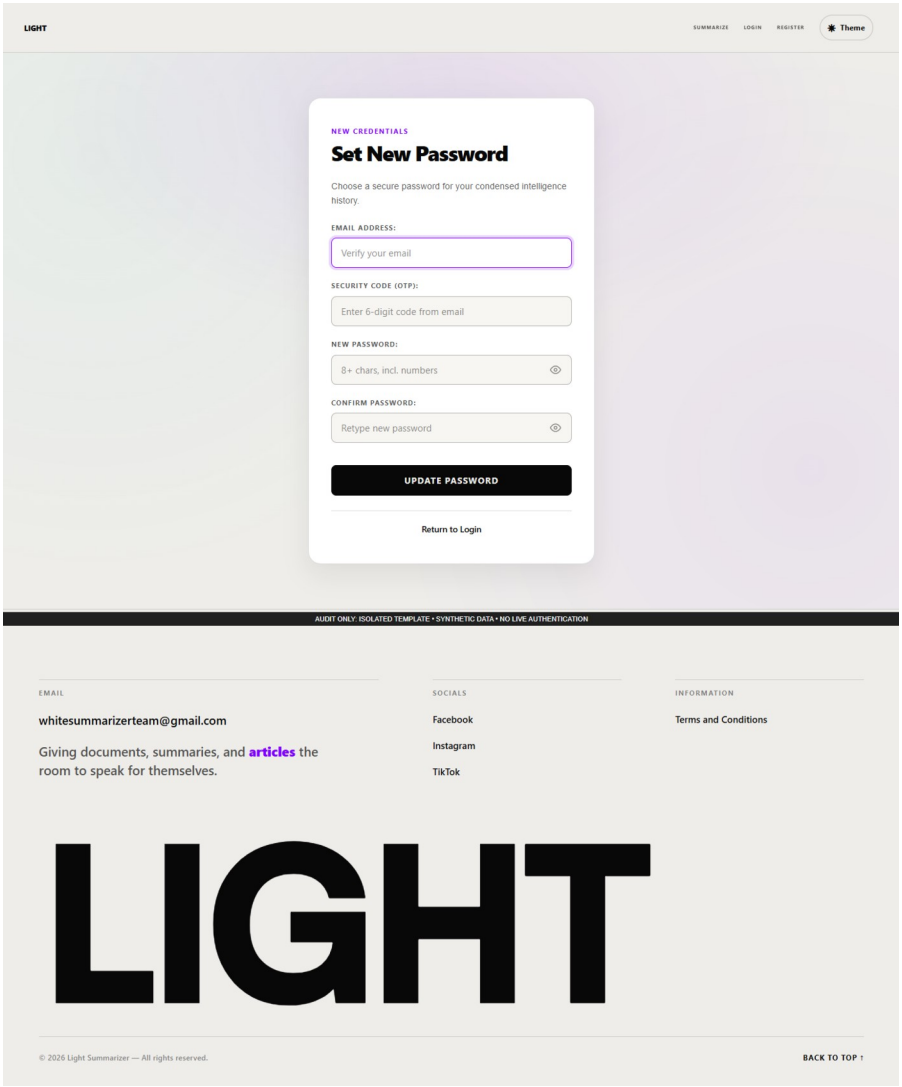



**Figure 58. Password-reset form rendered with no recovery request in progress.**

The reset page in Figure 58 asks for the email address, security code, new password, and confirmation. Email may already be filled when the application has a recovery session.

1. Enter or check the email address used for recovery.
2. Enter the code from the recovery email.
3. Enter a new password that meets the policy and repeat it exactly.
4. Use the visibility and strength hints as needed, then submit the reset form.
5. Follow the success/error message and return to sign in when the change succeeds.

An invalid, expired, or already-used code and mismatched or weak passwords can prevent completion. Request a new code through recovery when the displayed response requires it. No account password was changed during the audit.

## F. Terms and Conditions

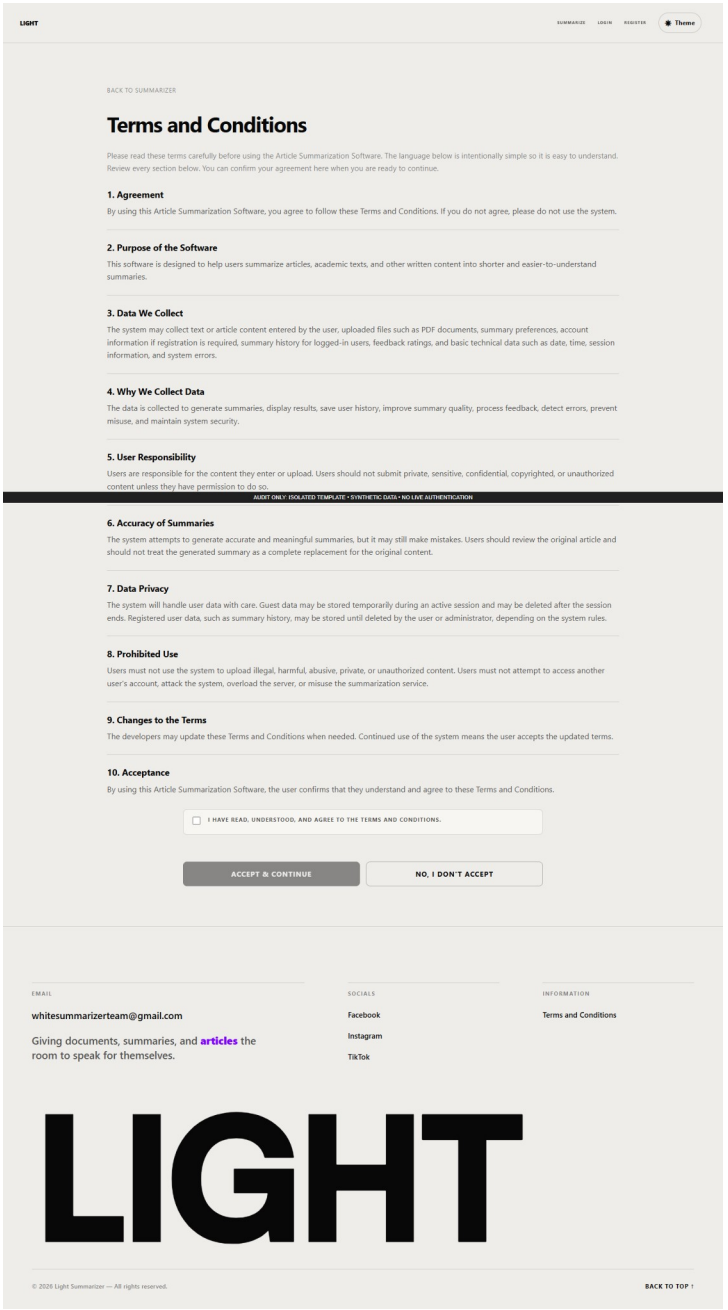



**Figure 59. Public terms page rendered with an anonymous fixture.**

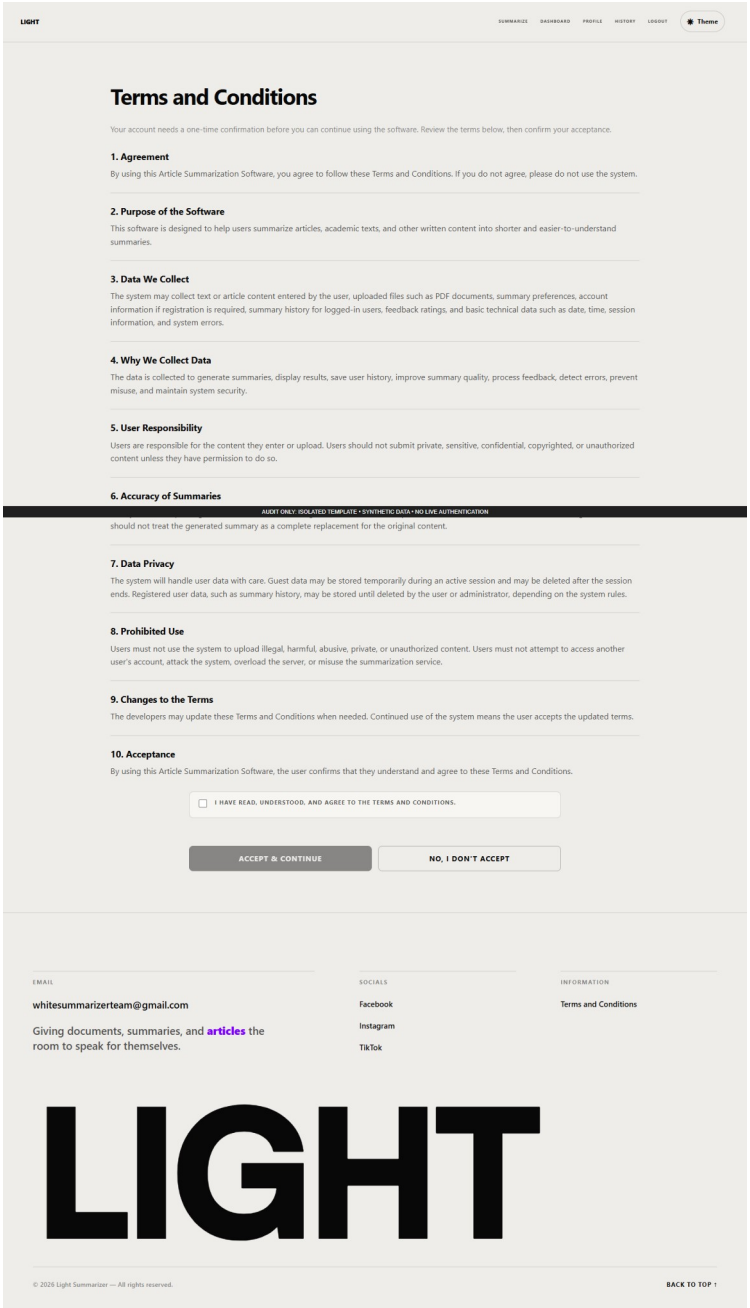



**Figure 60. Pending-acceptance page using a synthetic registered-account fixture.**

The application contains a public terms page and an acceptance page. Figure 59 shows the

public version; Figure 60 shows the acceptance controls that appear for an account awaiting confirmation. The footer's acceptance-page link allows guests to read the content, but that page does not show its account-confirmation form to guests.

1. Read the terms sections before agreeing.
2. Where the agreement form appears, select the acceptance checkbox.
3. Select **Accept & Continue** to submit agreement and follow the resulting destination.
4. If declining, select **No, I Don't Accept**. The current displayed control shows a message and returns to the landing page after a short delay.

The checkbox enabling the acceptance button was verified locally. Agreement was not saved. The visible decline button does not submit the server's separate decline action, so the manual does not claim that clicking it signs the user out or deletes acceptance state. That mismatch is an audit finding.

## G. Summarizer Workspace

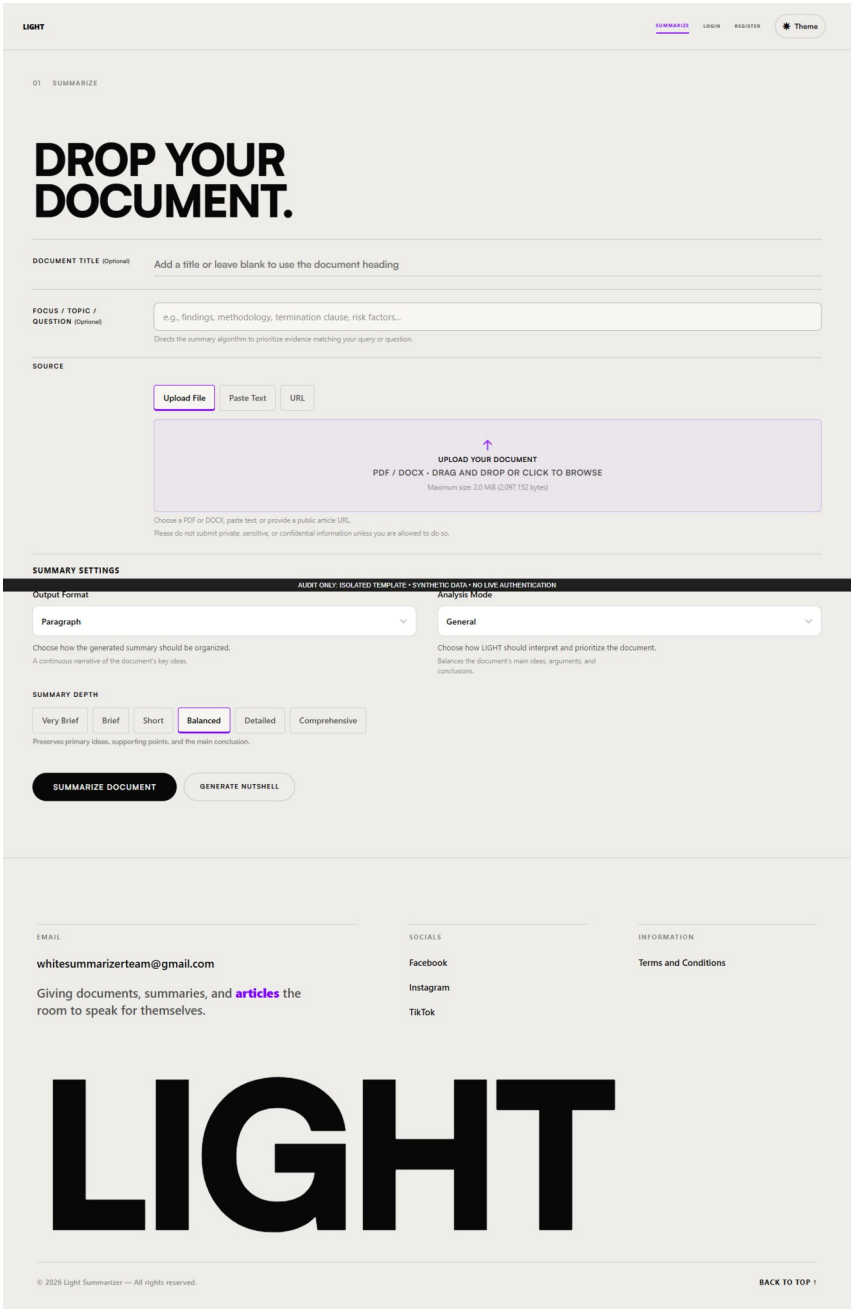



**Figure 61. Summary workspace showing source selection and generation settings.**

Figure 61 contains the source and settings needed for a summary. The workspace is accessible to guests and registered accounts. Provide only documents or text that may be processed under the applicable usage conditions.

### G.1. Select the source

1. Optionally enter **Document Title**. Leaving it blank allows the application to use a document heading or derived title.
2. Optionally enter **Focus / Topic / Question** to request emphasis on a subject. The visible field limits input to 300 characters. This focuses the summary; it is not a promise that a question will be answered correctly.
3. Choose exactly one source method:

**Table 54. Source methods and user actions.**

| Source method | User action | Restriction or recovery |
|---|---|---|
| Upload File | Browse for or drag a PDF/DOCX into the upload area. Use **Replace** or **Remove** to change the selected file. | Read the maximum size printed on the actual running page. Its value depends on configuration and PHP upload limits. The main form selects one file. |
| Paste Text | Paste readable document text into the text area. | Stay within the byte limit shown by the running page. Blank or unreadable text is rejected. |
| URL | Paste a complete public HTTP or HTTPS article address. | Invalid URLs or inaccessible/unreadable sources can fail. URLs with embedded credentials are rejected by the browser validation. |

Table 54 presents source methods and user actions. The stated status and qualifications govern its interpretation.

The isolated screenshot's upload limit belongs to the audit PHP environment and must not be treated as the deployment's configured limit. Choosing text/URL and rejecting an empty URL were tested locally. No live website

document upload, article retrieval, or extraction workflow was performed.

### G.2. Choose settings and submit

**Table 55. Summary settings in the main interface.**

| Setting | Available controls |
|---|---|
| Output Format | Paragraph; Bullet Points; Hybrid (Overview + Bullets); Structured Sections; Key Takeaways. |
| Analysis Mode | General; Academic; Executive; Technical; Study; News; Legal & Policy; Scientific. |
| Summary Depth | Very Brief; Brief; Short; Balanced; Detailed; Comprehensive. |

Table 55 presents summary settings in the main interface. The stated status and qualifications govern its interpretation.

Select the desired format, mode, and depth, reading the explanation shown beside each setting. Then select **Summarize Document**. A successful initial request leads to processing or an existing completed result; an unsuccessful request presents an explanation and recovery instruction. Setting changes were verified in isolation. A completed saved website summary was not generated during the audit; isolated Python generation is reported separately.

If the operator has configured the optional **Generation Method** section, the form may offer evidence-controlled AI synthesis. Read its notice before opting in: selected source passages may be sent to a configured language-model endpoint, including an external endpoint. The interface explains that automated checks are heuristic and that structured sections remain extractive. This conditional control was source-inspected and was not enabled in the isolated renderer.

### G.3. Errors, drafts, and Nutshell

Correct the field named in an error. The empty-URL check, for example, asks the reader to provide the complete HTTP(S) address. Recovery controls may offer retry, a different source method, or sign-in, depending on the error. A file must be selected again after recovery because browsers cannot restore its bytes. The source and settings can be retained within the browser tab for up to one hour; close or clear a shared browser session as appropriate after handling sensitive source material.

**Generate Nutshell** requests a concise rendition in a separate panel. Guests are redirected to registration; that navigation

was verified. Registered users prepare a source, select **Generate Nutshell**, then read the panel's output, not-needed notice, or recovery error. **Copy** appears after a successful output. The current Nutshell endpoint supports six analysis labels and normalizes legal/scientific selections to general; users should not assume that all eight main-summary modes are preserved in Nutshell. No Nutshell was generated in the audit.

### H. Processing and Recovery

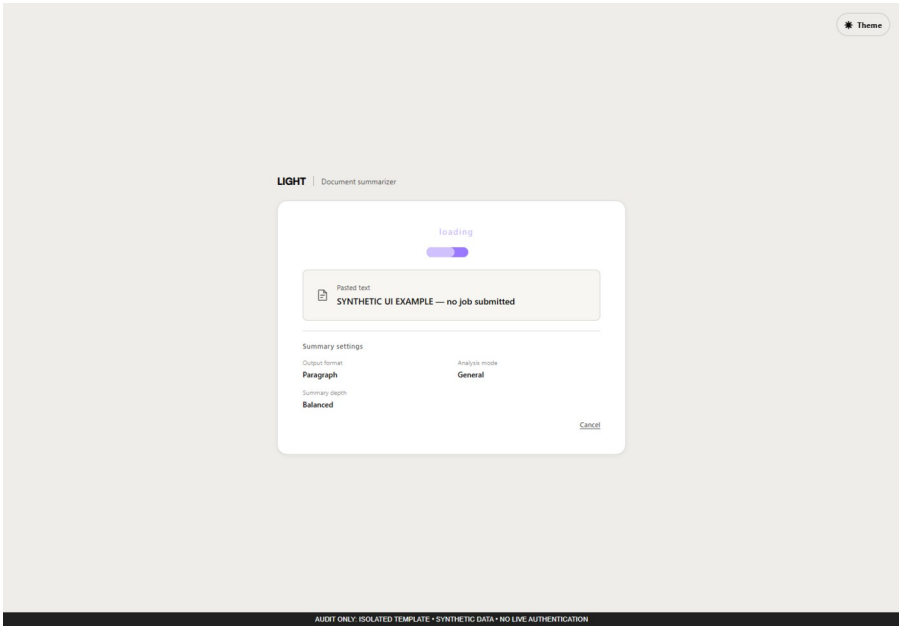



**Figure 62. Processing layout using a synthetic pending request; execution was blocked.**

The processing screen displays the document/source and selected format, mode, and depth. Figure 62 is a layout example only. Normally the page starts execution, checks status, and opens the result when complete. It is not a percentage-complete guarantee.

1. Wait for the displayed status after submitting a source.
2. If the page reports a connection interruption, use **Check connection and status** where offered. This distinguishes checking an existing request from intentionally starting another request.
3. If a recoverable processing failure offers **Retry summary**, review the message and retry as directed.
4. Use the cancel control when available to cancel the current request and return to settings.
5. If returning to a file source, choose the file again.

Opening the processing address without a pending request redirects to the workspace. Cancellation and retry may be unavailable for settled or ineligible jobs. Automatic execution and status requests were blocked in

the screenshot harness, so no successful job, cancellation, or retry is claimed.

## I. Summary Result, Source Review, and Export

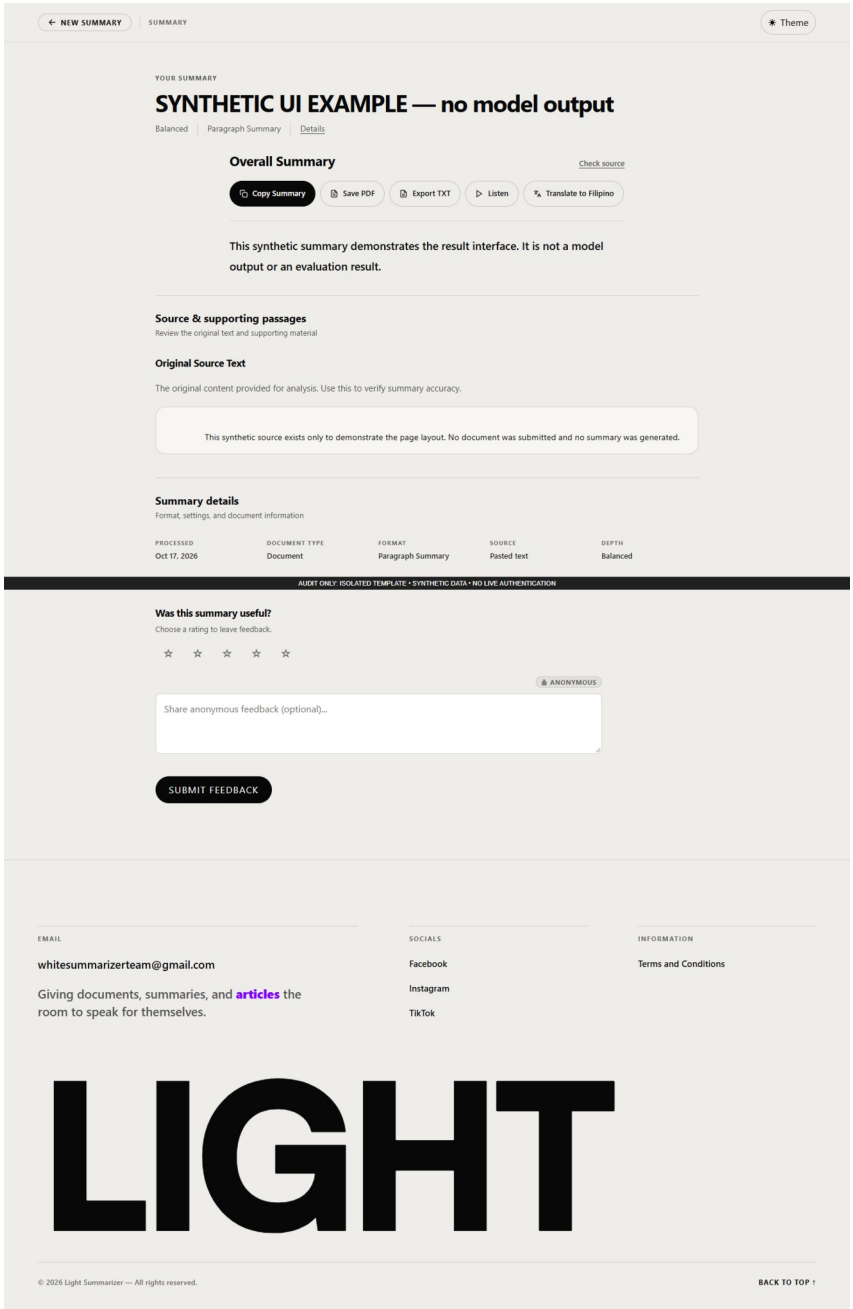



**Figure 63. Result interface with explicitly synthetic summary text, not model output.**

The result page in Figure 63 presents the primary summary and available supporting information. Access depends on the account/guest ownership context or an authorized share token. Saved output may also contain key takeaways, conclusion, section analysis, keywords, supporting passages, coverage, and summary details. Sections appear only when corresponding data exists.

1. Read the main summary and check the document title.
2. Use **Check source**, source disclosures, and numbered evidence links where available to compare claims with the original text.
3. Expand relevant details to understand the source and settings.
4. Use the toolbar actions described below as needed.

**Table 56. Result toolbar actions and verification limits.**

| Action | Procedure and expected behavior | Verification limit |
|---|---|---|
| Copy Summary | Select the copy control, then paste into the intended destination. If browser copying fails, select and copy manually. | Exact synthetic text copied successfully with browser permission. |
| PDF | Select the PDF control, review the browser print preview, then print or choose Save as PDF if available. | This uses the browser print dialog; a server PDF download is not implemented by this button. Printing was not exercised. |
| Text | Select text export and save the .txt download. | Synthetic TXT export was saved and its exact content checked. |
| Listen | Select the audio control and use the player after speech generation succeeds. | Depends on the audio worker; no audio was generated or played in the audit. |
| Translate | Request the displayed Filipino translation and compare it with the original summary. | Depends on the translation worker; no translation was requested. |

| Action | Procedure and expected behavior | Verification limit |
| --- | --- | --- |
| Theme | Select the theme control to switch between light and dark reading styles. | Switching the synthetic result to dark mode was verified. |
| Back | Use the return control to open summary settings. | It does not by itself create or delete a summary. |

Table 56 presents result toolbar actions and verification limits. The stated status and qualifications govern its interpretation.

Model output and automated evidence checks require human judgment. The available summary-quality evidence in this audit does not establish correctness for every source or mode. The code can display multi-document metadata, but the main source form exposes one file and no multi-file selection workflow.

### J. Summary Feedback

The feedback area on the result page asks **Was this summary useful?** Its rating and comment actions are separate from a research survey or assigned blinded review.

1. Select one to five stars.
2. For a low/neutral rating, optionally select the improvement reasons that appear.

3. Optionally enter a comment of up to 500 characters in the visible form.
4. Select **Submit feedback** and wait for confirmation or an error.
5. If feedback already exists, select **Update feedback**, revise it, and submit again.

Selecting a low rating revealed the optional reason choices in the isolated test. No rating or comment was submitted. The interface describes feedback as anonymous, but the implementation associates feedback with account or guest identifiers for access and update behavior; anonymous display does not mean that no internal linkage exists. Expired sessions, access restrictions, rate limits, and validation can prevent saving.

## K. Unavailable Result

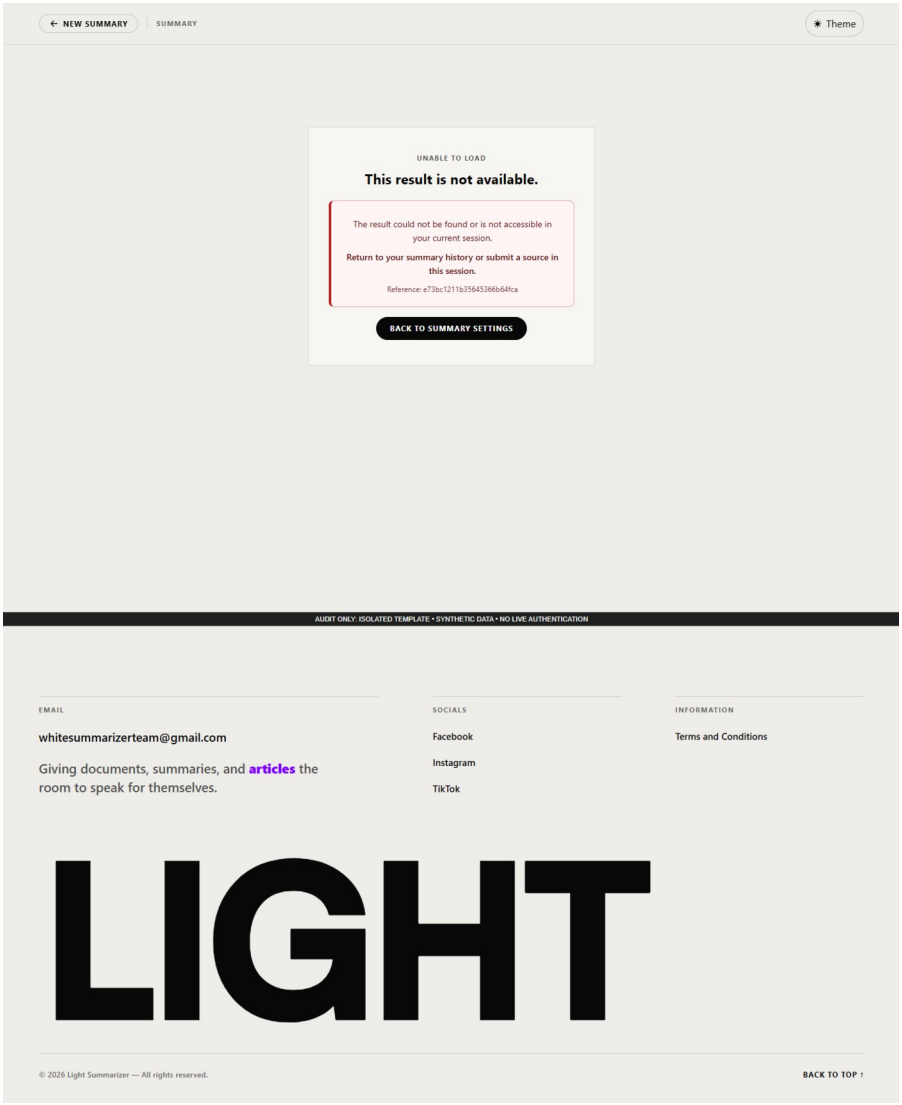



**Figure 64. Unavailable-result state produced with a missing synthetic record.**

Figure 64 shows the recovery view used when a requested result is unavailable or access cannot be granted. Follow **Back to summary settings** or obtain a valid authorized result

link. Avoid assuming that another user's numeric result identifier grants access. Opening the result route without an identifier returns to the landing page. The missing-record view was checked in isolation; no real private result was requested.

## L. Personal Dashboard

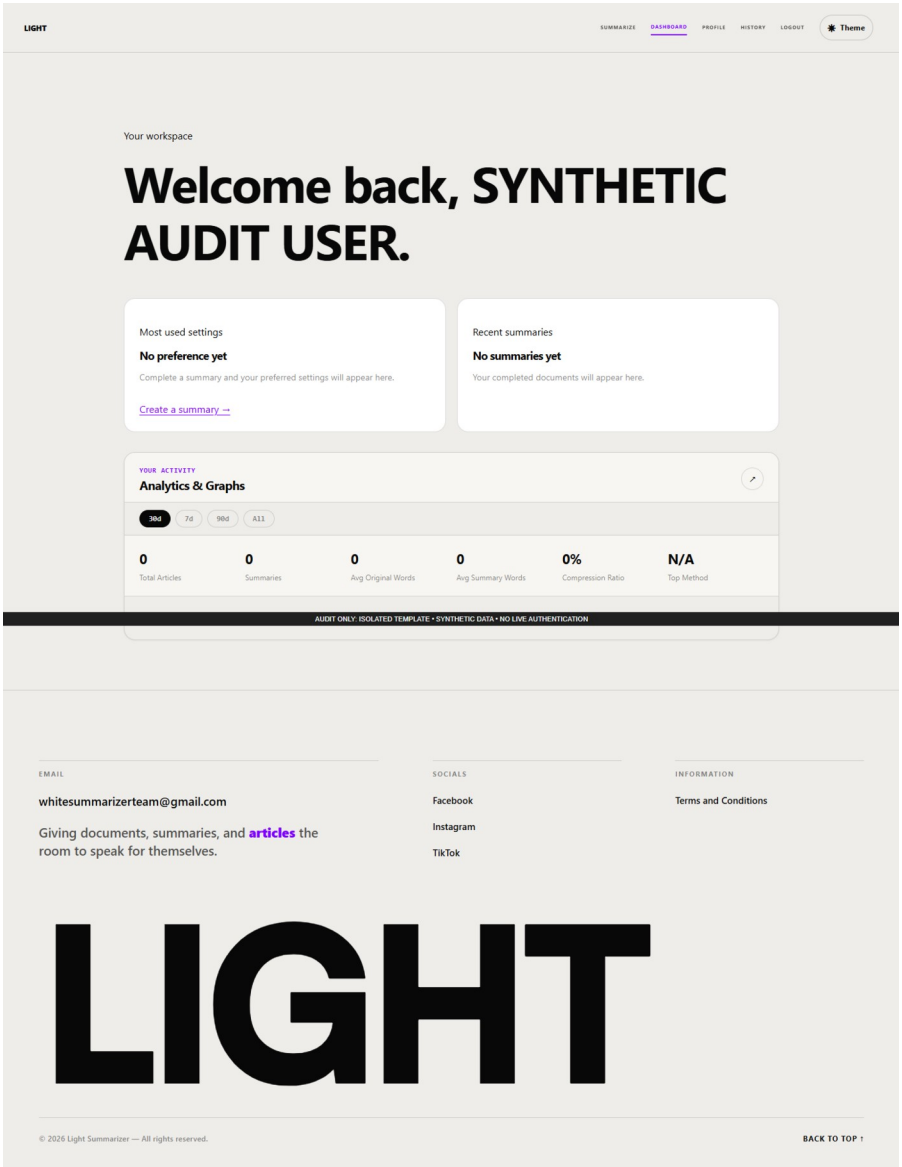



**Figure 65. Personal dashboard with an empty synthetic registered-account fixture.**

The personal dashboard provides recent summary links, activity information, and a starting point for new summaries. Figure 65

uses empty data. Registered nonadministrators use this dashboard; administrators are redirected to the administrative console.

1. Sign in and open **Dashboard.**
2. Select a recent result or begin another summary.
3. If preferred settings are available, use their link to preselect the inferred mode, format, and depth, then review them in the workspace.
4. Use the analytics card's range controls and its **Open full analytics dashboard** link to inspect available activity information.

Preferred settings are inferred from prior saved summaries. This is distinct from editing or naming a saved custom profile. Empty accounts may have no recent summaries or inferred preference. The capture does not establish any real usage counts.

## M. Summary History

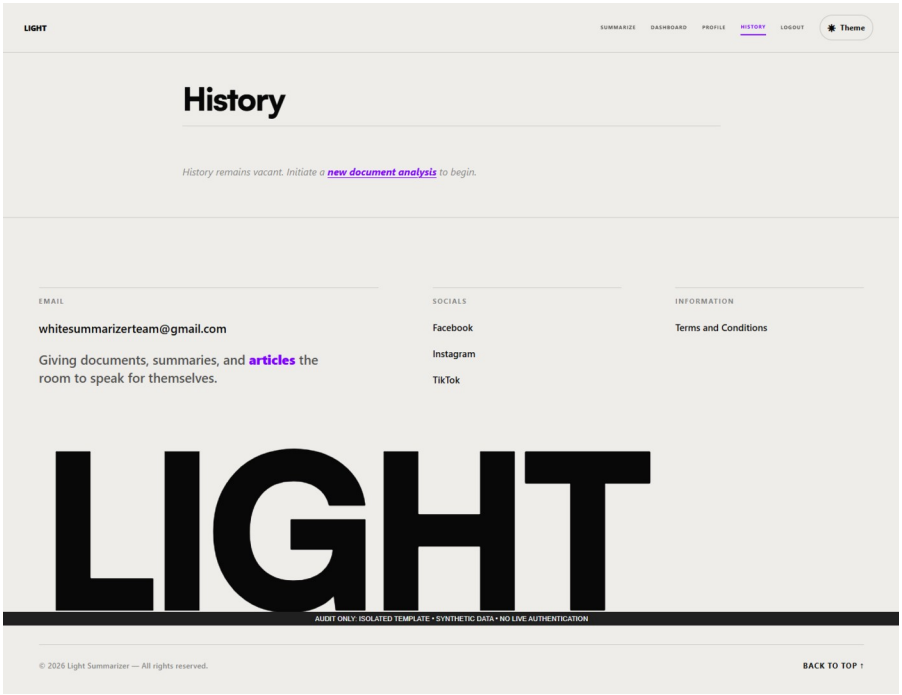



**Figure 66. History page with no synthetic saved summaries.**

Open **History** to view saved summary previews. Select a document title or **Access Full Report** to reopen an accessible result. When completed Nutshell records exist, **Nutshell History** presents their text and any linked main result. Figure 66 illustrates the empty state.

Guests are redirected to registration. The page shows up to 10 main summaries per page and up to 20 completed Nutshell records by default. It does not expose summary editing, renaming, or deletion controls. Although the

backend accepts a page number, the current template has no Previous/Next navigation; access to older records through ordinary UI navigation is a documented limitation. The service's maximum accepted page size of 50 does not change the page's default of 10 main summaries.

## N. Analytics

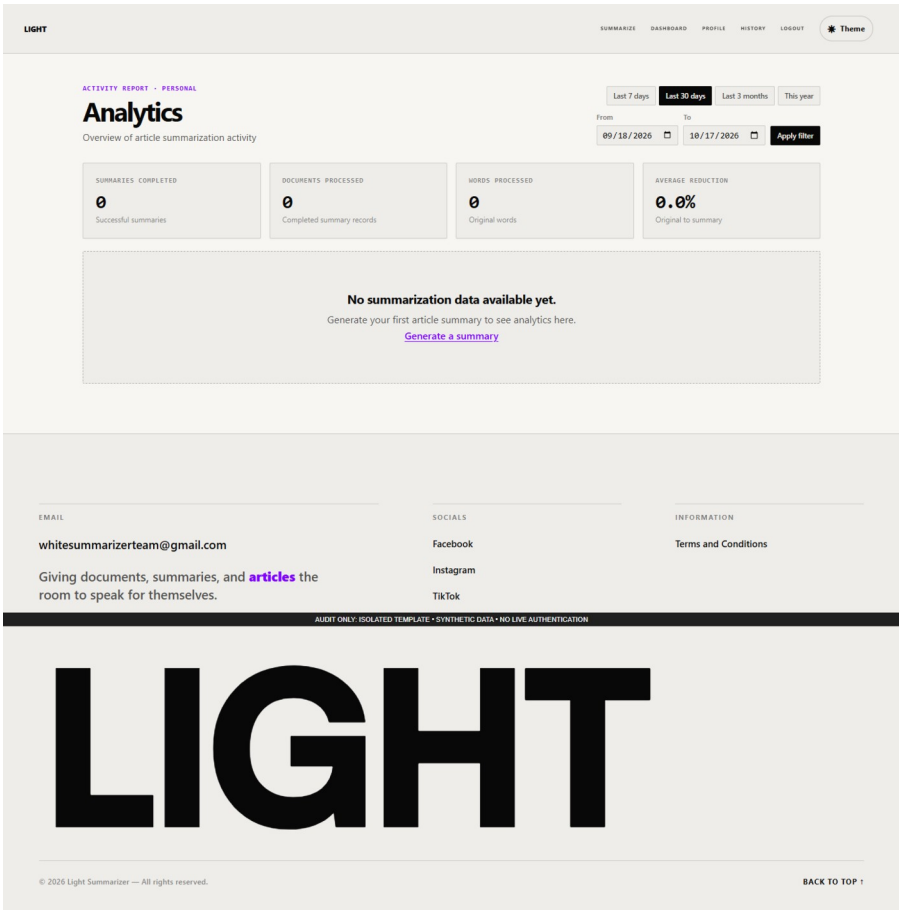



**Figure 67. Analytics page displaying the synthetic empty-data state.**

Analytics describes saved summarization activity. Registered users see personal scope; administrators can see system-wide scope. Figure 67 contains no operational data.

1. Open the full analytics page from the dashboard card, or use the administrator Analytics link.
2. Select **Last 7 days**, **Last 30 days**, **Last 3 months**, or **This year**.
3. For a custom period, set **From** and **To** and choose **Apply filter**.
4. Review available completed summaries, processed documents/words, average reduction, usage distributions, processing durations, and recent activity.

Populated data can enable charts for summaries and Nutshell over time, style/depth use, word comparison, and source types. An empty-data message means that no qualifying records were supplied for the view. An unavailable/error message must not be interpreted as a measured zero. Populated charts and real-date filtering were not tested against existing records.

## O. Profile and Account Deactivation

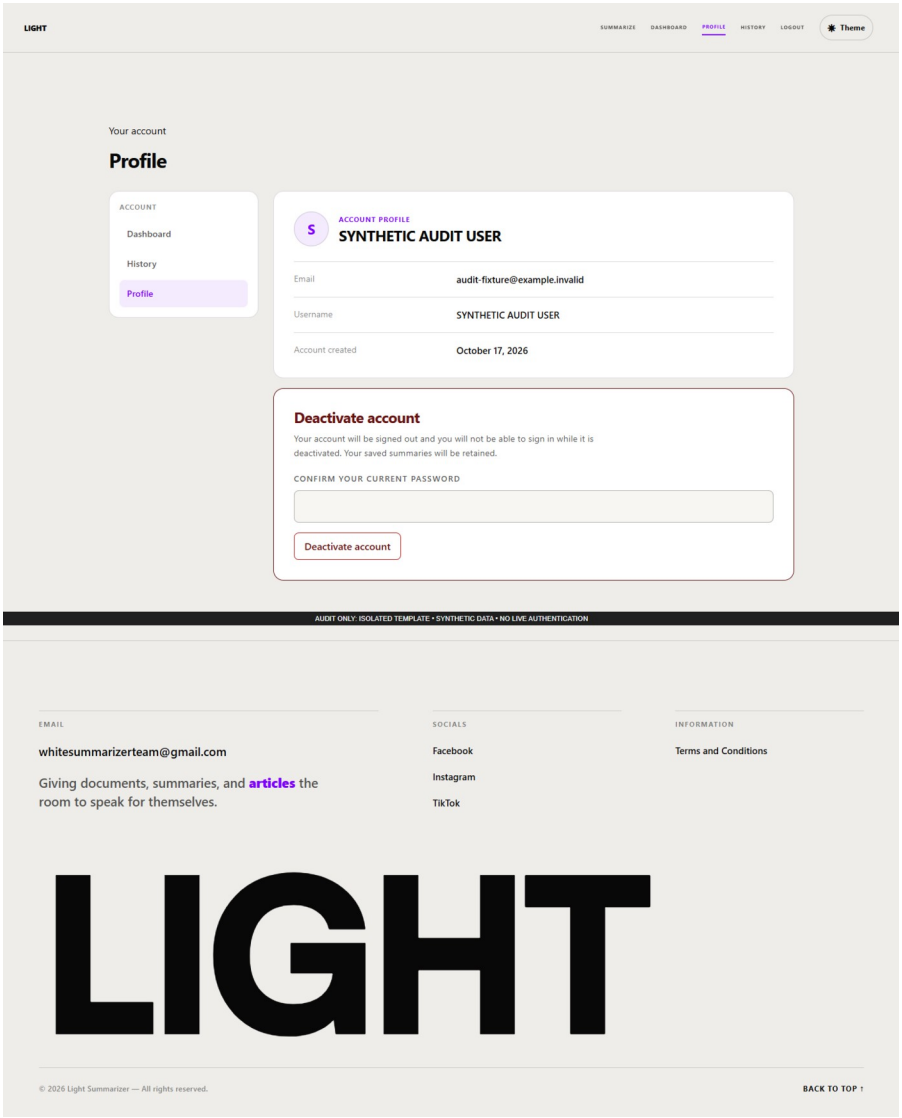



**Figure 68. Profile page displaying a reserved synthetic account identity.**

The profile page in Figure 68 displays username, email, and join date. The sidebar returns to the appropriate dashboard or History. The discovered interface does not

include editing the username/email, uploading a profile picture, or directly changing a password.

For deliberate account deactivation, read the explanation, enter the current password, and select **Deactivate account**. The interface states that the account will be signed out, cannot sign in while inactive, and retains saved summaries. Backend protections apply, including administrator-related restrictions. Deactivation is not permanent data deletion. No account was deactivated during the audit.

## P. Logout Confirmation

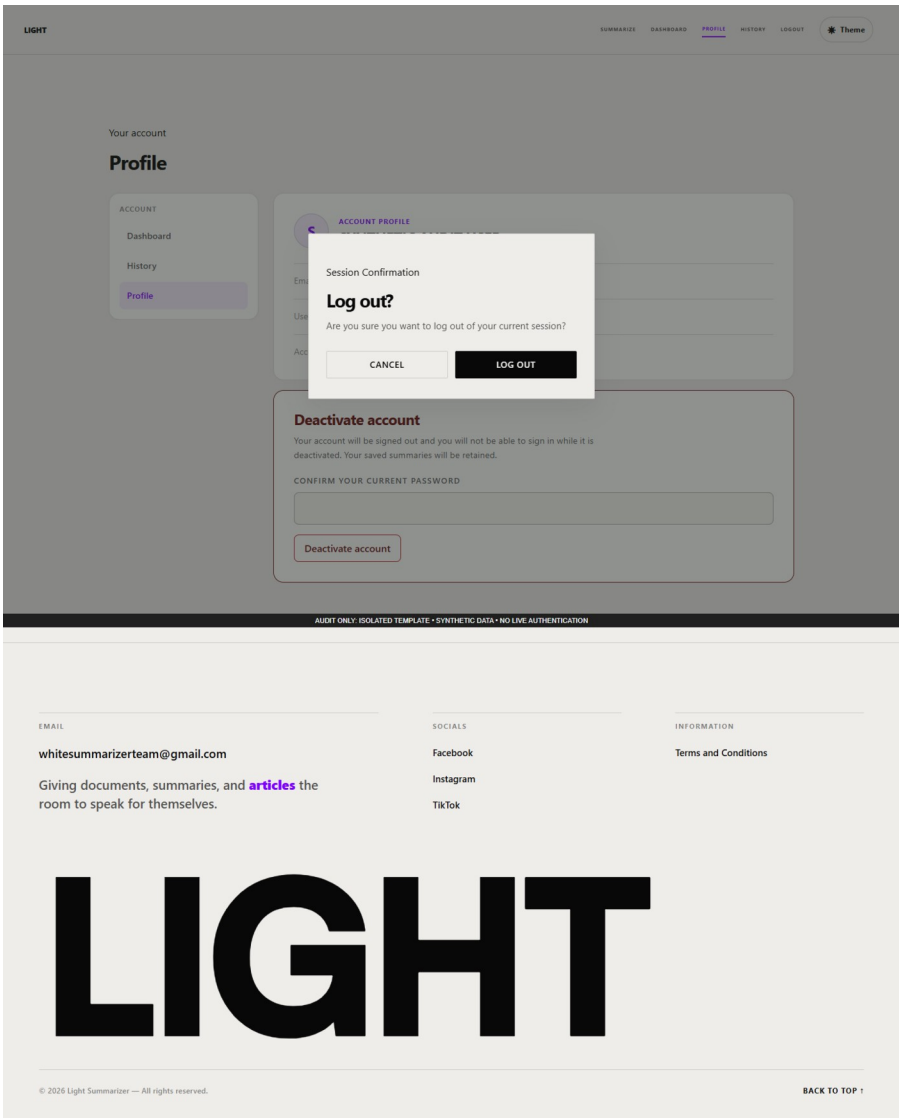



**Figure 69. Logout confirmation opened from a synthetic signed-in profile.**

Select **Logout** to open Figure 69's confirmation. Choose **Cancel**, click the backdrop, or press **Escape** to remain on the page. Choose **Log out** only when ready to leave

the current session. The modal implements keyboard focus movement and returns focus when closed. Escape closure was verified on desktop and mobile; actual logout was not executed.

## Q. Administrative Console

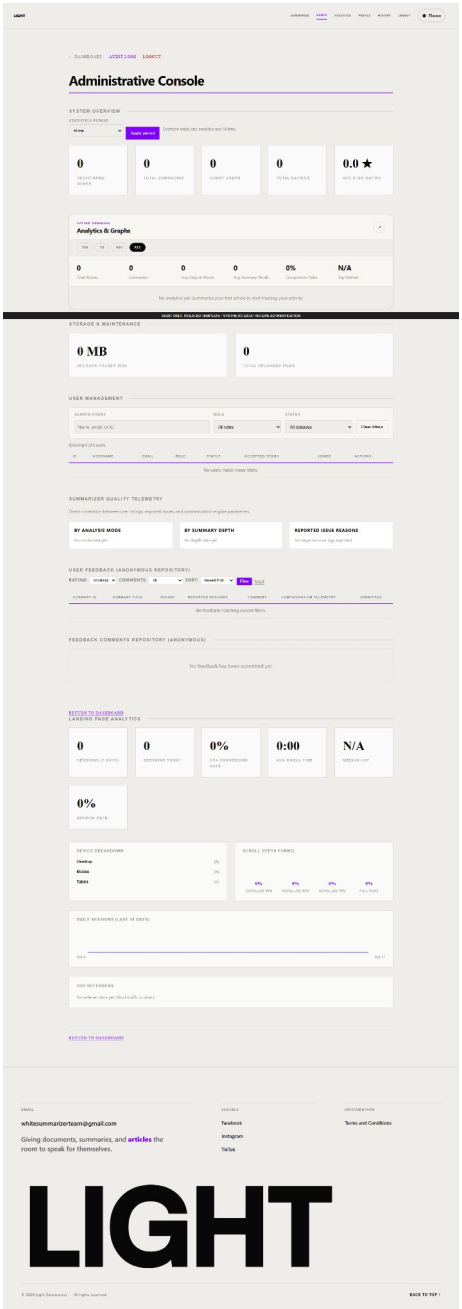



**Figure 70. Administrative console rendered with synthetic administrator access and empty records.**

Figure 70 shows the administrative interface without exposing real accounts, feedback, or analytics. The normal application requires an authenticated administrator. The synthetic role used for this image does not prove that requirement has been tested live.

### Q.1. Review system information

Use **Statistics Period** and **Apply period** to select the overview interval. Review the system cards, analytics graph card, and storage counters. The shared graph card has 30d, 7d, 90d, and All controls and a link to full analytics. Storage cards display upload size and file count. Cleanup and delete-file actions exist in the dispatcher, but no visible maintenance button for those actions was found; this manual does not provide an invented button workflow.

### Q.2. Manage users

1. In **User Management**, search by the supported name/email/id text.
2. Filter by **Role** or **Status**, or choose **Clear filters** to reset the list.
3. Check the intended row and account identity carefully.

4. When an eligible row provides **Deactivate**, read its confirmation before disabling the account.

5. When an eligible row provides deletion, read the permanent-deletion warning before confirming. Deletion is different from deactivation and may remove associated records.

The current visible rows provide deactivation and deletion where permitted; the presence of other server dispatcher actions does not establish additional visible buttons. No real user row was loaded and no administration action was executed. Follow the approved operational authorization and backup procedure before changing accounts.

**Q.3. Review feedback and landing telemetry**

Inspect **Summarizer Quality Telemetry** for available mode/depth/reason aggregates. Filter the feedback repository by rating, whether a comment exists, and sort order, then select **Filter**. **Reset** clears the filter. Result-title links open the referenced result where access allows. A separate comments repository displays submitted feedback comments.

The **Landing Page Analytics** section can show sessions, CTA conversion, dwell time, median LCP, bounce rate, device breakdown, scroll depth, daily trend, and referrer information. Respect the period printed on each panel; not every landing metric shares the overview's selected interval. Missing or unavailable measurements require investigation. These telemetry values are not survey responses or evidence of improved learning. The displayed zero values in Figure 70 are synthetic.

## R. Administrator Audit Logs

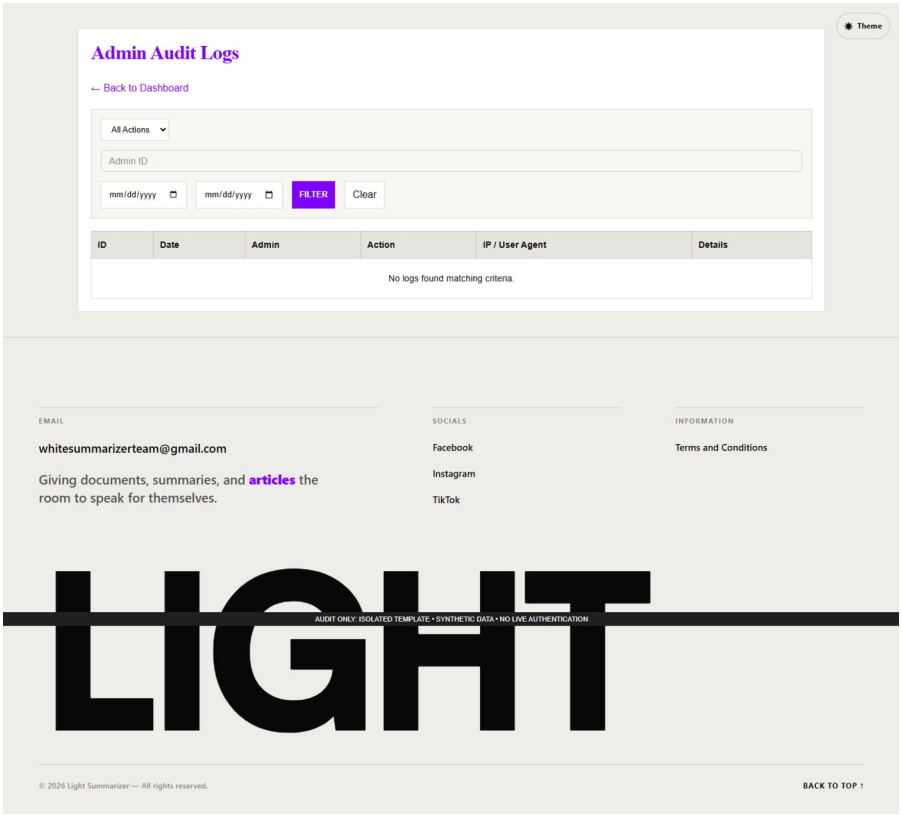



**Figure 71. Audit-log viewer with a synthetic existing table and no log rows.**

Select **Audit Logs** from the administrative console. Figure 71 shows filters for action, administrator identifier, and date range. Select **Filter** to apply them or **Clear** to reset them. When multiple pages exist, use **Previous** and **Next**; the current page indicator identifies the position. **Back to Dashboard** returns to the console.

Available rows show timestamp, administrator, action, request metadata, and details. Treat these records as administrative information. A missing-table message is distinct from a valid empty result; another isolated capture records the missing-table branch. The current filter controls lack explicit accessible labels and require remediation. No actual administrator activity record was read.

## S. Assigned Independent Reviews

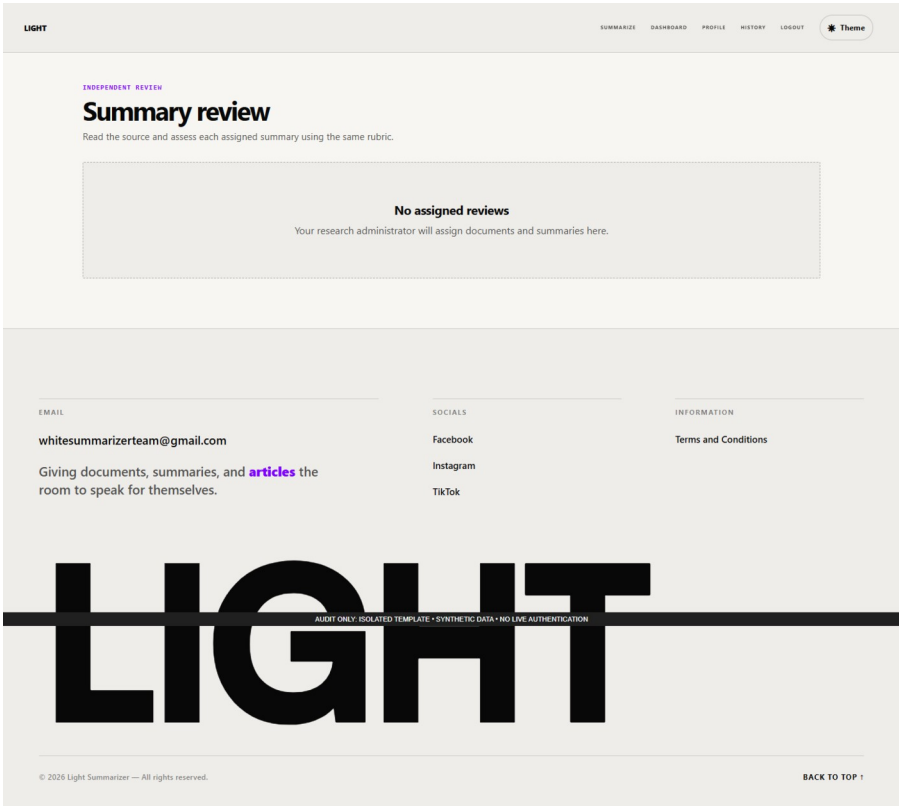



**Figure 72. Assigned-review page with an active synthetic account and no assignments.**

The review page allows an active account to assess assigned summaries independently. Figure 72 shows the empty state, which instructs the reader to await assignments. Anonymous access produces a sign-in-required response. Merely signing in does not supply a review assignment.

When an assignment is available:

1. Open its **Review summary** link while using the assigned account.
2. Read the source document and blinded summary together.
3. Rate relevance, factual consistency, coverage, coherence, readability/fluency, conciseness, and non-redundancy separately.
4. Use the displayed five-level rubric: Very poor, Poor, Acceptable, Good, and Excellent.
5. Optionally add specific comments, then select **Submit assessment**.

The interface explains that submission locks the assessment and that corrections require the research administrator. Submitted items are marked accordingly. No assignment, score, or comment was created in the audit. This content-quality review instrument is separate from the capstone's proposed system-quality questionnaires and cannot substitute for actual IT-consultant or end-user responses.

### T. Evaluation Reports, Compatibility Routes, and Diagnostics

The code contains an administrator-protected evaluation export endpoint for an existing run in JSON, CSV, or HTML format. Its report template presents dataset/run context, metric

availability, system/mode comparisons, human-rating coverage, agreement, paired comparisons, error classifications, challenge measurements, and limitations. Unavailable measurements are explicitly distinct from measured zero values. No existing run was exported or represented as a completed research evaluation in this audit; a live report screenshot is **UNAVAILABLE EVIDENCE.**

The current evaluation.php route redirects administrators to the administrative console. Older tabs and links that target that route's dataset/run/assignment/output views therefore do not provide a working management screen. The associated source partial includes controls for content-unit verification, error classification, and audited rating correction, but its presence is not proof of an accessible current UI. The operator must resolve the route/navigation gap before those procedures can be documented as an operational browser workflow. Obtain run identifiers and assignment links through the project's supported research administration process; no such complete UI process was verified here.

The database-health and feedback-feed addresses are administrator diagnostics returning JSON. The public health address

returns a static service status and cannot establish database or model-worker readiness. Document Q&A and document/multi-document operations exist as API companions; the current normal result and source forms do not expose a chat widget or multi-file selector. Those API interfaces belong in the technical integration documentation rather than being presented as buttons a new user can find.

## U. Common Recovery Guidance and Evidence Trace

**Table 57. Common recovery situations and user guidance.**

| Situation | User guidance |
| --- | --- |
| Source validation message | Correct the named field, check file type/size or use a complete URL, and follow the offered recovery action. |
| Source is unreadable | Try permitted readable text or another supported document; the audit does not claim OCR for image-only files. |
| Connection interrupted during processing | Check the existing request/status using the provided recovery option before intentionally submitting another job. |
| File draft recovered | Select the local file again; the browser does not restore previously selected file bytes. |
| Result unavailable | Return to settings or obtain an authorized result link. |

| Situation | User guidance |
| --- | --- |
| History/analytics asks for registration | Create or sign in to an active account before using registered-only features. |
| Audio or translation unavailable | Read the displayed message and retry only as directed; the service may require operator attention. |
| Review assignment unavailable | Use the correct account and supplied assignment link; contact the research administrator for assignment or correction needs. |
| Terms or sign-in interrupts navigation | Complete the requested prerequisite and continue through the permitted destination. |

Table 57 presents common recovery situations and user guidance. The stated status and qualifications govern its interpretation.

This manual derives its page actions and restrictions from ../system-audit/ui-feature-notes.json (UI-001–UI-051), ../system-audit/ui-route-inventory.json, and the source references they contain. Browser evidence supports only the identified isolated visual and local-interaction checks. The source files for the application were not modified. Desktop and mobile images for each captured state remain available in the screenshot register, including recovery, access-denial, and empty-state variants omitted from the main flow of this appendix.

# APPENDIX G. LIGHT SUMMARIZER MAINTENANCE MANUAL

## 1. Introduction

This manual guides the designated maintainer of LIGHT summarizer in preserving the application, its database, local processing services and stored content. It follows the reference manuscript's ten maintenance sections while describing the inspected LIGHT implementation. The baseline is the 17 October 2026 audit, not a claim of production certification. Procedures and schedules identified as proposed require adoption by the project owner; their inclusion does not mean they have been performed or automated.

The audit made documentation artifacts only. It did not import a schema, migrate the live database, purge records, change accounts, install a scheduled task, send email or restore a backup. Known defects remain in the application and are documented in the security/reliability report.

## 2. System Overview

LIGHT uses PHP pages and services, MariaDB storage and a local Python summarization worker. Users can supply text, an article URL

or a supported PDF/DOCX. The main flow creates a recoverable job, sends the source and preferences to Python, stores the resulting summary/artifact and presents result/history actions. The source also includes a standalone document-combination API and a distinct evaluation subsystem; their storage and authorization behavior differ from the main flow.

The configured database metadata contains 31 tables, 312 columns and 35 foreign keys. The full data dictionary and actual-constraint ERDs accompany Chapter III. The runtime configuration calls the application AI Article Summarizer; LIGHT summarizer is the manuscript title. The current absent engine-version setting resolves to legacy; optional v2/LLM code must not be assumed active.

**Table 58. Maintainer's path and component reference.**

| Component | Verified repository location | Maintenance significance |
|---|---|---|
| Public PHP pages and APIs | app/public | Intended web document root; authenticated and anonymous flows begin here. |

| Component | Verified repository location | Maintenance significance |
| --- | --- | --- |
| Controllers, services, repositories, utilities | app/src | Application logic; whitereaper.php bootstrap has database side effects. |
| Configuration | app/config/ config.php, .en v | Preserve secrets privately; audit documents do not contain credentials. |
| Schema and migrations | database/ schema.sql, database/analyt ics_schema.sql, database/migrat ions | Definitions differ from live password-reset schema; verify before installation/up grade. |
| Local NLP and evaluation engine | python-engine | Requires compatible interpreter, dependencies and separately provisioned optional models. |
| Runtime storage defaults | storage/ uploads, storage/audio, storage/logs, storage/tmp | Protect from direct web access; preserve data according to an approved retention policy. |
| Worker authentication | storage/ worker.token or configured secret | Keep access restricted; never include token in screenshots or reports. |

| Component | Verified repository location | Maintenance significance |
| --- | --- | --- |
| Dependency definitions | composer.json, composer.lock, python-engine/requirements-local.txt, requirements-evaluation.txt | Record installed versions and test upgrades in staging. |
| Operational tools | tools, scripts | Many commands write state even when their names imply checking. |

Table 58 presents maintainer's path and component reference. The stated status and qualifications govern its interpretation.

## 3. Major Features

The inspected backend supports guest and registered-user contexts, account/password flows, administrator email verification, terms acceptance, eight analysis profiles, six main-summary depths, output formatting, text/URL/PDF/DOCX inputs, recoverable processing jobs, history/sharing, ratings/comments, translation and audio integration, usage analytics and administrator maintenance actions. The standalone combination API uses five lengths, omitting the main flow's short option. The evaluation service and CLI support versioned datasets, reference summaries, baseline/ablation

comparisons, blind assignments, seven-criterion ratings, review corrections, metrics and exports.

These are implementation capabilities. Availability of SMTP, external translation, optional models and evaluation management browser controls has not been established by source existence. The current document API lacks owner checks, and the administrator status-toggle path bypasses the guarded deactivate operation. Restrict deployment exposure and resolve the documented defects through a separate authorized implementation task before relying on these paths with real participant data.

**4. System Requirements**

The inspected local PHP executable is 8.1.25; the queried database server is MariaDB 10.4.32. The application setup tool checks PHP 8.1 or later. Required PHP facilities include PDO/MySQL, cURL, DOM/libxml, fileinfo, JSON, mbstring, OpenSSL and enabled proc_open; DOCX package validation also uses ZIP. The audit's extension whitelist confirms these on the inspected executable, which may differ from the PHP instance used by a web server.

Python source uses union-type syntax and strict ZIP iteration, so the old setup statement “Python 3.8 or higher” is insufficient. Use a dependency-compatible Python 3.10-or-newer environment and validate the exact resolved packages; the confirmed isolated engine tests used Python 3.14. A particular version's compatibility with all optional model packages still requires testing. Composer supplies PHPMailer. Python requirement files define NLP, parsing, translation and TTS dependencies; optional embedding/BERTScore/NLI/LLM models require explicit provisioning and configuration. A package name in a requirement file is not proof that its model files exist.

Use a modern browser and an appropriately configured PHP web server. Apache rewrite/header/override configuration controls whether .htaccess protections apply; the PHP development server does not apply them. For deployment, configure the public root, TLS and private runtime paths deliberately. The worker listens on loopback and uses an authentication token. SMTP and the configured translation provider require their own connectivity; operation is not wholly independent of external services.

No measured minimum hardware or supported simultaneous-user count was established. Size CPU, RAM and disk using representative synthetic/approved workloads, chosen models and retention periods. The setup guide's small RAM/disk estimates should be treated as unvalidated legacy guidance. Record actual hardware and versions when reporting timings.

## 5. Preventive Maintenance Procedures

### Database and schema

Keep a versioned schema inventory and compare intended changes against the actual database before each release. Read-only inventory can use direct information_schema queries without application bootstrap. The audit's metadata collector is an example; it excludes application rows and credentials. Do not substitute tools/migrate.php --status for a guaranteed read-only check: it may create/seed the migration ledger.

Resolve the password-reset schema discrepancy in staging before a fresh installation. The service expects password_resets.used_at; the checked-in base schema defines used and created_at. Existing live metadata cannot prove a clean install

works. Verify both an empty-database installation and an upgrade from a representative protected backup. Check constraints, migration ledger and critical flows after each test. Do not claim an automatic repair or run schema import against an existing production database as a troubleshooting shortcut.

**File storage and retention**

Monitor free space, directory permissions and growth in uploads, audio, logs and recovery payloads. Coordinate deletion with active jobs and the approved retention policy. The administrator's “orphan” cleanup actually selects age over 24 hours and does not verify database references; its delete-all action is destructive. Avoid these controls as routine cleanup until their selection has been reviewed. Do not remove arbitrary files while a worker is using them.

Existing CLI cleanup scripts cover landing sessions, feedback comment events, administrator audit records and expired summary recovery data. The first three default to 90 days unless configured otherwise. Their existence does not establish scheduled execution or the appropriateness of that

retention period. Their --dry-run modes load application bootstrap, which may touch guest state and trigger guest cleanup. Validate proposed cleanup on a clone and use direct count queries when a strictly read-only estimate is needed. Guest-token claiming and expiry do not currently cover every feature store; retention documentation must acknowledge that gap.

**Security and dependency maintenance**

Review account privileges and administrator access, audit events and unexpected processing failures. Keep .env, worker tokens, backup files and logs outside public access and grant only the required service accounts access. Use TLS for deployment, inspect effective response/cookie headers, and verify proxy assumptions. Store passwords as hashes through the application; never document plaintext account credentials. Password reset currently does not revoke every persistent token/session, so a reset must not be represented as a complete session-revocation action.

Review dependency updates in staging, retain lock/version records and run relevant tests before deployment. An unverified blanket

package upgrade can change extraction, model output and scoring. Preserve previous version/configuration evidence and a rollback plan. Rotate any exposed secret through a controlled operation, update dependent services and verify it without printing the value.

**Worker and performance monitoring**

Inspect queue age, failure categories, job retries, disk consumption and worker error logs. The Windows scripts can install a limited-privilege current-user logon monitor that checks health every 30 seconds. Installation/running state was not verified. The start script probes protocol 3 and a supervised child self-test; it may terminate a verified stale project worker before restart. Review active work before using start/stop/remove scripts.

The configured 180-second local processing timeout and 600-second TTS timeout are settings, not measured completion times; supervisor limits may further constrain execution. Track processing durations using trustworthy instrumentation. Existing uploaded-source word-count and main-job timing persistence gaps limit current analytics.

Evaluate latency and resource usage with synthetic or approved sources under stated hardware and concurrency.

**Proposed backup and recovery procedure**

The following is a proposed operational procedure, not existing automated backup functionality or a completed recovery test.

1. Identify an accountable operator, backup destination, retention period, encryption/access controls and a recovery window. Record the application revision, PHP/Python/database versions, model inventory, schema and configuration key names. Keep secret values in a separate protected store.
2. Schedule a coordinated maintenance window or use a documented database/file consistency strategy. Pause new processing and wait for active jobs to settle when consistent application/file snapshots cannot otherwise be guaranteed. Capture the database with a compatible MariaDB/MySQL logical-backup utility using an operator-managed login/option file; do not place passwords in command history. A transactional snapshot protects InnoDB

consistency but does not by itself synchronize files or concurrent schema changes.

3. Include the database, required stored user artifacts, deployment source/revision, dependency records and protected configuration recovery material. Record hashes, creation time, size and tool version. Store an encrypted access-controlled copy separately from the running host. Verify that backups can be read, rather than relying only on a command exit code.

4. Restore first into a separately named disposable database and isolated storage tree on a staging host. Set the clone's configuration to those targets before loading any application code. Disable real email, external callbacks and scheduled jobs; use synthetic identities or an approved protected data-handling procedure. Never restore over the live database merely to test a backup.

5. Validate schema/constraints, representative record relationships, file accessibility and critical synthetic flows. Check that job state, audio, history and evaluation

artifacts agree. Exercise login/reset only with staging-controlled delivery. Record elapsed restoration time and data-loss window as measured observations; do not invent recovery-time or recovery-point guarantees.

6. If a real recovery is required, obtain the responsible owner's deployment decision with a reviewed backup identifier, target, expected data-loss window and rollback procedure. Preserve the pre-recovery state where feasible, perform the approved cutover and recheck critical flows. Record who acted, what changed and the outcome.

## 6. Troubleshooting Guide

**Table 59. Evidence-based troubleshooting guide.**

| Symptom | Plausible inspected cause | Maintainer action |
| --- | --- | --- |
| Cannot connect to database | Service stopped, wrong local configuration, missing PDO extension | Check service/configuration privately and compare CLI/web PHP extensions; do not print credentials or run schema import against live data. |

| Symptom | Plausible inspected cause | Maintainer action |
| --- | --- | --- |
| Reset fails after fresh installation | used_at absent from base-schema installation | Compare actual metadata with service SQL; prepare a staging-tested compatibility migration. |
| Administrator cannot complete login | OTP expired, SMTP unavailable, rate limit or inactive account | Review safe server logs and email transport configuration; use the current email-OTP flow rather than obsolete security-question instructions. |
| Upload rejected | PHP/form limit, unsupported format, MIME/package failure, permission issue | Check the displayed effective limit and uploaded file integrity; verify private upload permissions. Do not blindly raise limits. |
| Ordinary text reported unreadable | Cleaning rules can remove an entire paragraph containing navigation-like words such as “register” | Preserve a synthetic reproduction, inspect cleaned-text diagnostics and escalate the documented cleaning defect. Do not claim the original source was inherently unreadable. |

| Symptom | Plausible inspected cause | Maintainer action |
| --- | --- | --- |
| Job remains queued/running | Worker unavailable, stale state, timeout or process permission failure | Inspect job/log evidence; recovery reads may change stale state. Review retry eligibility and active process before restarting. |
| Python command unavailable | Windows app alias or mismatched environment | Use an explicit configured interpreter path and verify packages in that environment. |
| Translation unavailable | External provider/network/configuration failure | Check provider configuration and safe errors; avoid sending confidential text to a provider without an approved policy. |
| Audio missing or seeking inconsistent | Model/binary unavailable, worker timeout, missing cache file or suffix-range defect | Verify model paths and logs in staging; distinguish generation failure from the reproduced HTTP range bug. |
| Guest output disappears after login | Incomplete guest-to-user claiming across stores | Inspect ownership in a controlled test; do not promise every feature's guest data migrates. |

| Symptom | Plausible inspected cause | Maintainer action |
|---|---|---|
| Summary analytics inconsistent | Uploaded-source count/timing omission or failures stored only as jobs | Reconcile the metric's data source; do not infer research effectiveness from dashboard totals. |
| Evaluation metric missing | Optional model/reference unavailable or explicit worker failure | Preserve missing/error status, dependency versions and provenance; do not replace missing values with zero. |
| Private source visible by document ID | Document API lacks owner validation | Restrict exposure and prioritize the ownership fix; validate denial using synthetic accounts in staging. |

Table 59 presents evidence-based troubleshooting guide. The stated status and qualifications govern its interpretation.

## 7. Roles and Responsibilities

The designated administrator manages application accounts, reviews logs and coordinates retention and access. The deployment maintainer manages the web/database/Python environment, secrets, backup validation, worker scheduling and release records. Developers investigate

defects, implement reviewed changes and preserve regression evidence. Researchers maintain corpus provenance, consent/permissions where required, evaluation assignments and interpretation of results. Assigned evaluators independently score their allocated summaries; evaluator assignment is not a separate stored application role. Ordinary users remain responsible for choosing permitted source material and reviewing generated text against the original.

These responsibilities are proposed operating assignments. The audit did not identify or appoint production staff, verify institutional approval, or certify any completed maintenance activity.

**8. Data Security and Privacy**

Treat uploaded text, generated summaries, original artifacts, recovery payloads, audio, account details, telemetry and evaluation comments as potentially sensitive. Selected masking of original-text fields does not guarantee that generated evidence or summary text is anonymous. An “Anonymous” feedback label does not remove every stored relationship. Share links grant access to their holders and should be distributed

accordingly. Audio currently carries a public long-lived cache directive despite access checks; review this before deployment.

Collect only content needed for the approved use, define retention for every store and review third-party transmission paths. Limit backup/export access, redact secrets and direct identifiers from screenshots, and inspect free-text notes before publishing evaluation exports. The applicable institutional and legal requirements require responsible review; implementation controls alone do not establish compliance. No claim of completed ethical approval, participant consent, anonymization or legal certification is made here.

## 9. Maintenance Schedule

**Table 60. Proposed maintenance schedule; adoption and execution are not verified.**

| Frequency / trigger | Proposed activity | Evidence to retain |
|---|---|---|
| Each operating day | Review failed/stale jobs, worker errors and disk space; create a protected database/artifact backup according to the adopted consistency plan | Dated checks, backup ID/hash and exceptions |
| Weekly | Verify backup readability, review access/audit events, inspect storage growth and tested retention candidates | Review record and approved deletion preview |
| Monthly | Review accounts, dependencies, secrets/access controls and data-retention exceptions | Account/ dependency review and change tickets |

| Frequency / trigger | Proposed activity | Evidence to retain |
|---|---|---|
| Each release | Test installation and upgrade in staging; validate critical flows, schema and representative output; preserve rollback materials | Versioned test report, schema comparison and release decision |
| Quarterly | Restore into isolated staging and measure recovery; review privacy/export practices and optional model configuration | Restore log, measured recovery figures and unresolved findings |
| Annually or after a major change | Review architecture, threats, capacity assumptions and maintenance ownership | Updated audit/manual and responsible-owner sign-off |
| Security incident or failed backup | Preserve evidence, restrict affected service as appropriate, follow the approved incident/recovery decision | Incident timeline, actions, recovery validation and follow-up |

Table 60 presents proposed maintenance schedule; adoption and execution are not

verified. The stated status and qualifications govern its interpretation.

An existing worker monitor checks every 30 seconds when installed, but that is a process-health mechanism rather than the proposed database backup or maintenance schedule. Cleanup scripts are available; their scheduling and successful operation were not established.

## 10. Conclusion

Maintenance of LIGHT requires a current schema inventory, controlled source/dependency changes, restricted storage and secret access, observable processing, and verified recovery procedures. The immediate priorities are the documented authorization, account-status, reset-token, schema-installation and text-cleaning defects. Record each maintenance action and its evidence so future maintainers can distinguish implemented behavior, tested behavior and proposed practice.

## APPENDIX H. PROPOSED TECHNICAL EVALUATION INSTRUMENT

### Instrument Status and Purpose

PROPOSED DRAFT — NOT ADMINISTERED; ADVISER AND INSTITUTIONAL REVIEW REQUIRED.

Project: **LIGHT summarizer**.

Researchers: Robert Martin Baltisoto, Shielo Mariell Leodones, Gieselle Sabbaluca and Sofiah Heaven Galman.

This original instrument asks qualified technical reviewers to assess observed application behavior under agreed staged tasks. It contains 28 items across seven selected domains from the historical reference template. Maintainability is not included. It is not a reproduced ISO questionnaire, a complete assessment of ISO/IEC 25010:2011 or 25010:2023, or a security certification. The older edition's withdrawn status is acknowledged in the References. Domain selection and item wording require approval before use.

Study code: [assigned code]. Date: [actual date]. Instrument version: [approved version]. Application revision/engine: [record]. Reviewer expertise category: [approved

nonidentifying category]. Browser/device and environment: [record]. Tasks/evidence provided: [record]. Participant-information and consent reference: [record; no consent inferred by this blank form].

**Directions and Response Scale**

Complete the assigned tasks or inspect the designated evidence before answering. Select one response for each item. Base the response on what was observed, and add a brief example for a disagreement or unavailable feature. An observed failure is evidence for disagreement; it is not "not applicable." Choose N/A only when an item genuinely does not apply or the required evidence was not available, and state the reason. Do not test other people's accounts or private records.

**Table 61. Response scale for the proposed technical and end-user questionnaires.**

| Code | Response | Direction |
|---|---|---|
| 1 | Strongly Disagree | The statement clearly does not describe the observed evidence or experience. |
| 2 | Disagree | The statement generally does not describe the observed evidence or experience. |

| Code | Response | Direction |
|---|---|---|
| 3 | Agree | The statement generally describes the observed evidence or experience. |
| 4 | Strongly Agree | The statement clearly describes the observed evidence or experience. |
| N/A | Not observed / Not applicable | No numeric score; explain the missing exposure or inapplicability. |

Table 61 defines the response convention for both proposed questionnaires. N/A remains outside the numeric scale so missing exposure is not mistaken for a neutral judgment. Proposed exposure includes text and supported-file submission, a configured URL example, preference changes, result/history access, a controlled invalid input, and available supporting actions. Technical reviewers may inspect staging evidence for access denial, job recovery, installation and configuration. Unavailable models or services must be recorded, not presented as successful demonstrations. The known authorization defects require correction or an

institutionally reviewed restricted environment before any real-user trial.

**Functional Suitability**

**Table 62. Proposed technical functional-suitability items.**

| Item | Statement to be assessed | Response |
|---|---|---|
| T-FS1 | The tested text-input workflow produces an output that can be traced to the submitted source. | [1/2/3/4/N/A] |
| T-FS2 | The tested PDF/DOCX or URL pathway performs the documented input-handling task for the supplied example. | [1/2/3/4/N/A] |
| T-FS3 | Changes to supported profile, depth and format settings are reflected appropriately in the tested output. | [1/2/3/4/N/A] |

| Item | Statement to be assessed | Response |
|---|---|---|
| T-FS4 | Tested result, history and feedback actions perform their documented functions without losing the relevant record association. | [1/2/3/4/N/A] |

Table 62 connects functional suitability to actual input, preference and record-handling tasks. It requires a traceable example instead of assuming that the presence of a control establishes successful behavior.

Evidence/example and N/A reason: [reviewer entry]. Assess source fidelity using the supplied original; do not infer accuracy from fluent wording alone.

**Performance Efficiency**

**Table 63. Proposed technical performance-efficiency items.**

| Item | Statement to be assessed | Response |
|---|---|---|
| T-PE1 | Processing time for the stated input size is acceptable against the documented evaluation task target. | [1/2/3/4/N/A] |

| Item | Statement to be assessed | Response |
|---|---|---|
| T-PE2 | The interface provides timely progress or completion information while the tested request is processed. | [1/2/3/4/N/A] |
| T-PE3 | Observed resource use remains within the stated evaluation-environment limits during the tested workload. | [1/2/3/4/N/A] |
| T-PE4 | Timeout or cancellation behavior bounds the tested operation according to the documented task conditions. | [1/2/3/4/N/A] |

Table 63 asks reviewers to judge performance against stated task conditions and available measurements. It distinguishes responsiveness and bounded operation from an unsupported general claim of speed.

Evidence/example and N/A reason: [reviewer entry]. Record hardware, source size, timing target and actual measurement separately. If no target or resource evidence was provided, mark the relevant item N/A rather than inventing a benchmark.

## Compatibility

### Table 64. Proposed technical compatibility items.

| Item | Statement to be assessed | Response |
|---|---|---|
| T-CO1 | The tested browser displays the required controls and outputs without preventing the assigned tasks. | [1/2/3/4/N/A] |
| T-CO2 | The tested file/input format is interpreted consistently with its documented support and limitations. | [1/2/3/4/N/A] |
| T-CO3 | The configured PHP-to-Python integration exchanges the required request and response information in the staged test. | [1/2/3/4/N/A] |
| T-CO4 | The tested external or supporting service handles its documented interface without disrupting unrelated core tasks. | [1/2/3/4/N/A] |

Table 64 covers the particular browser, input format and service interfaces included in the staged evaluation. Conclusions must remain limited to those tested combinations.

Evidence/example and N/A reason: [reviewer entry]. Record tested browsers, formats and service configuration; one environment does not demonstrate universal compatibility.

**Usability**

**Table 65. Proposed technical usability items.**

| Item | Statement to be assessed | Response |
|---|---|---|
| T-US1 | Labels and instructions make the tested input and summary-setting choices understandable. | [1/2/3/4/N/A] |
| T-US2 | Keyboard operation and focus behavior allow completion of the assigned navigation tasks. | [1/2/3/4/N/A] |
| T-US3 | Error messages identify the tested problem and provide an appropriate next action. | [1/2/3/4/N/A] |

| Item | Statement to be assessed | Response |
|---|---|---|
| T-US4 | The result layout distinguishes source information, generated output and supporting actions clearly. | [1/2/3/4/N/A] |

Table 65 focuses on comprehensible controls, keyboard tasks, error guidance and result presentation. These items provide a bounded usability review rather than a complete accessibility audit.

Evidence/example and N/A reason: [reviewer entry]. This short domain does not constitute a comprehensive accessibility-conformance assessment.

**Reliability**

**Table 66. Proposed technical reliability items.**

| Item | Statement to be assessed | Response |
|---|---|---|
| T-RE1 | Repeated execution of the assigned task completes without unexplained loss of state under the recorded conditions. | [1/2/3/4/N/A] |

| Item | Statement to be assessed | Response |
|---|---|---|
| T-RE2 | Invalid input produces a controlled failure without exposing unrelated data or damaging the tested session. | [1/2/3/4/N/A] |
| T-RE3 | The staged retry/cancel scenario maintains a consistent job outcome without unintended duplicate records. | [1/2/3/4/N/A] |
| T-RE4 | The supplied recovery demonstration restores the selected data and critical functions as documented. | [1/2/3/4/N/A] |

Table 66 requires evidence of repeated operation, controlled failure, job consistency and demonstrated recovery. The recovery item cannot be answered favorably from the existence of a maintenance recommendation alone.

Evidence/example and N/A reason: [reviewer entry]. No recovery demonstration has been supplied by this manuscript; T-RE4 remains N/A until one is actually performed.

## Security

**Table 67. Proposed technical security items.**

| Item | Statement to be assessed | Response |
|---|---|---|
| T-SE1 | The staged ownership tests deny a synthetic user access to another user's private records. | [1/2/3/4/N/A] |
| T-SE2 | The tested account-security actions enforce the documented authentication and session controls. | [1/2/3/4/N/A] |
| T-SE3 | The tested state-changing requests reject missing or invalid authorization/CSRF requirements. | [1/2/3/4/N/A] |
| T-SE4 | The reviewed logs, errors and exported material avoid exposing the test secrets or prohibited identifying information. | [1/2/3/4/N/A] |

Table 67 associates security judgments with controlled access, account, mutation and disclosure checks. It does not invite

reviewers to infer security from appearance or to test private production data.

Evidence/example and N/A reason: [reviewer entry]. Use synthetic records and explicit test authorization. A favorable perception cannot override a reproduced defect. The documented document-API ownership failure must remain visible in the assessment evidence.

**Portability**

**Table 68. Proposed technical portability items.**

| Item | Statement to be assessed | Response |
|---|---|---|
| T-PO1 | The documented runtime and dependency requirements identify what the staged installation actually needs. | [1/2/3/4/N/A] |
| T-PO2 | A clean installation in the designated staging environment completes the documented critical checks. | [1/2/3/4/N/A] |

| Item | Statement to be assessed | Response |
|---|---|---|
| T-PO3 | Configuration allows the tested deployment paths and service settings to be changed without editing unrelated logic. | [1/2/3/4/N/A] |
| T-PO4 | Moving the selected test data and configuration to the designated second environment preserves the assigned functions. | [1/2/3/4/N/A] |

Table 68 distinguishes stated requirements from an actual clean installation or transfer. A second-environment claim requires a real staged demonstration and cannot be inferred from configurable paths alone.

Evidence/example and N/A reason: [reviewer entry]. Do not rate installation or transfer as successful from documentation alone. The fresh-install password-reset schema discrepancy is a relevant unresolved finding.

**Reviewer Comments and Researcher Coding**

Most significant observed strength: [optional response].

Most significant observed problem and reproduction: [optional response].

Priority improvement: [optional response].

Unavailable task/evidence and reason: [response where applicable].

Researcher use only: valid item counts [ ]; N/A counts and reasons [ ]; protocol deviations [ ]; approved inclusion/exclusion decision [ ]. Preserve original answers; do not replace disagreement with missing values. Chapter II defines the proposed analysis convention. No responses or computed scores are populated in this document.

## APPENDIX I. PROPOSED END-USER EVALUATION INSTRUMENT

### Instrument Status and Directions

PROPOSED DRAFT — NOT ADMINISTERED; REQUIRES APPROVED PARTICIPANT INFORMATION, CONSENT AND TASKS.

Project: **LIGHT summarizer**. Study code: [assigned code]. Date: [actual date]. Instrument version: [approved version]. Device/browser: [record]. Tasks attempted and help received: [record].

After completing the assigned activities, select 1 (Strongly Disagree), 2 (Disagree), 3 (Agree), 4 (Strongly Agree), or N/A (not observed/not applicable) for each statement. N/A has no numeric score. Give an example when something did not work or was unclear. There are no preferred answers, and declining participation must not affect grades or ordinary services. The final institution-approved participation notice should provide the actual contact, withdrawal conditions, data use and retention details before the instrument is used.

Use only the prepared or otherwise approved reading material. Compare important claims

with the original article. Do not upload confidential personal information. The tasks should record which functions were available; a feature shown in a screenshot is not necessarily a completed task.

**Functional Suitability**

**Table 69. Proposed end-user functional-suitability items.**

| Item | Statement to be assessed | Response |
|---|---|---|
| U-FS1 | I could submit the assigned article using the input method provided in the task. | [1/2/3/4/N/A] |
| U-FS2 | The summary helped me identify the article's main ideas during the assigned review. | [1/2/3/4/N/A] |
| U-FS3 | The available length and presentation choices suited the assigned reading task. | [1/2/3/4/N/A] |
| U-FS4 | I could use the assigned supporting action, such as history, nutshell, translation or audio, for its stated purpose. | [1/2/3/4/N/A] |

Table 69 relates end-user functional judgments to the assigned article and the supporting action actually attempted. The responses describe perceived usefulness in that task, not a measured improvement in academic achievement.

Example, difficulty or N/A reason: [participant response]. Identify the supporting action actually attempted. This domain records experience, not an objective test of learning or factual accuracy.

**Perceived Performance Efficiency**

**Table 70. Proposed end-user performance items.**

| Item | Statement to be assessed | Response |
|---|---|---|
| U-PE1 | The waiting time for the assigned summary was acceptable for that task. | [1/2/3/4/N/A] |
| U-PE2 | The application showed enough progress information while I waited. | [1/2/3/4/N/A] |
| U-PE3 | The tested buttons and page changes responded promptly during my use. | [1/2/3/4/N/A] |

| Item | Statement to be assessed | Response |
|---|---|---|
| U-PE4 | I could continue the assigned reading activity without unnecessary repeated waiting or resubmission. | [1/2/3/4/N/A] |

Table 70 captures the participant's experience of waiting, progress feedback and repeated actions. Interpret it alongside the recorded device and network conditions rather than treating the ratings as elapsed-time measurements.

Example, difficulty or N/A reason: [participant response]. These are perceptions under the recorded device/network conditions; they are not measured processing-speed benchmarks.

**Usability**

**Table 71. Proposed end-user usability items.**

| Item | Statement to be assessed | Response |
|---|---|---|
| U-US1 | I understood where to enter an article and start the assigned task. | [1/2/3/4/N/A] |

| Item | Statement to be assessed | Response |
|---|---|---|
| U-US2 | The labels and summary-setting explanations were understandable to me. | [1/2/3/4/N/A] |
| U-US3 | The result text and controls were easy to read and locate on my device. | [1/2/3/4/N/A] |
| U-US4 | When I needed to change an input or respond to a message, I understood what to do next. | [1/2/3/4/N/A] |

Table 71 covers starting the task, understanding settings, reading results and responding to messages. It permits specific interface difficulties to be recorded without assuming that every participant used every feature.

Example, difficulty or N/A reason: [participant response].

Most useful part of the assigned task: [optional response].

Part that was confusing or did not work: [optional response].

Suggested improvement: [optional response].

Researcher use only: actual task exposure [ ]; response validation [ ]; N/A explanation [ ]; protocol deviations [ ]. All fields remain blank until genuine collection. The 12 items use the same four-point convention as Appendix H; ordinary application star ratings and the separate five-point summary-review rubric must remain separate datasets.

# APPENDIX J. DATA-GATHERING AND SUPPORTING DOCUMENTS

## Status of Research-Event Evidence

Authentic photographs, attendance records, signed permissions, participant information/consent records and completed questionnaires for a LIGHT evaluation were not supplied. This appendix preserves their required places without manufacturing a research event. Screenshots from the software audit show interface or test evidence; they are not photographs of participants or proof that a campus demonstration occurred.

**Table 72. Supporting-document register and current evidence status.**

| Document or evidence | Current status | Required completion |
|---|---|---|
| Researcher draft and embedded diagrams | Supplied; extracted text and image review retained | Confirm version/date and intended use. |
| April–October 2026 Gantt chart | Supplied planning artifact; completion unverified | Identify whether planned, revised or as-completed; supply activity records. |
| Adviser appointment/decision | Not supplied | Insert authentic institutional record. |

| Document or evidence | Current status | Required completion |
|---|---|---|
| Permission for data gathering | Not supplied | Insert signed permission/reference and conditions. |
| Required ethics/institutional determination | Not supplied | Record actual determination applicable to the final protocol. |
| Participant information and consent | Not supplied | Use approved documents and protected records; publish only suitable blank/redacted forms. |
| Instrument validation/pilot records | Not supplied | Record actual reviewers, revisions and findings without invented coefficients. |
| Recruitment/attendance/ task logs | Not supplied | Retain coded records and actual collection dates. |
| Completed responses and codebook | Not supplied | Preserve protected raw data and reproducible analysis copy. |
| Event photographs | Not supplied | Use authentic permitted images with actual date, context and privacy review. |

| Document or evidence | Current status | Required completion |
|---|---|---|
| Technical audit and synthetic test artifacts | Available in the audit package | Retain their isolation/configuration limits and distinguish them from participant evidence. |

Table 72 distinguishes supplied planning and technical material from unavailable human-research records. Its status entries should change only when the corresponding authentic document or dataset is obtained.

**Photograph and Activity-Record Template**

[Insert authentic image only if supplied and approved for publication.]

Actual activity: [record]. Actual date and location: [record]. Participants shown: [describe only as permitted]. Photographer/source and permission reference: [record]. Associated task/consent/attendance reference: [record].

This placeholder receives no figure number because it contains no actual figure. Do not use a generated image, stock photograph or reconstructed scene as documentary evidence of research participation.

## APPENDIX K. RESEARCHER CURRICULUM VITAE

Only the four researcher names below are confirmed by the user's explicit instruction. Academic records, personal details, roles, qualifications and achievements have not been inferred. Each researcher should complete and verify their own entry using the institution's approved format. Include only information necessary for submission; sensitive personal identifiers and contact details should not be added automatically.

**Robert Martin Baltisoto**

Full name: Robert Martin Baltisoto.

Degree program and current academic status: [researcher to confirm].

Institution/campus and academic year: [researcher to confirm].

Educational background: [verified institutions, qualifications and dates].

Relevant technical skills: [researcher-supplied accurate account].

Actual contribution to LIGHT summarizer: [specific responsibilities and supporting work].

Relevant projects, training or certifications: [verified entries only].

Professional/contact information permitted for submission: [optional, approved entry].

Verification and date: [researcher confirmation].

**Shielo Mariell Leodones**

Full name: Shielo Mariell Leodones.

Degree program and current academic status: [researcher to confirm].

Institution/campus and academic year: [researcher to confirm].

Educational background: [verified institutions, qualifications and dates].

Relevant technical skills: [researcher-supplied accurate account].

Actual contribution to LIGHT summarizer: [specific responsibilities and supporting work].

Relevant projects, training or certifications: [verified entries only].

Professional/contact information permitted for submission: [optional, approved entry].

Verification and date: [researcher confirmation].

**Gieselle Sabbaluca**

Full name: Gieselle Sabbaluca.

Degree program and current academic status: [researcher to confirm].

Institution/campus and academic year: [researcher to confirm].

Educational background: [verified institutions, qualifications and dates].

Relevant technical skills: [researcher-supplied accurate account].

Actual contribution to LIGHT summarizer: [specific responsibilities and supporting work].

Relevant projects, training or certifications: [verified entries only].

Professional/contact information permitted for submission: [optional, approved entry].

Verification and date: [researcher confirmation].

**Sofiah Heaven Galman**

Full name: Sofiah Heaven Galman.

Degree program and current academic status: [researcher to confirm].

Institution/campus and academic year: [researcher to confirm].

Educational background: [verified institutions, qualifications and dates].

Relevant technical skills: [researcher-supplied accurate account].

Actual contribution to LIGHT summarizer: [specific responsibilities and supporting work].

Relevant projects, training or certifications: [verified entries only].

Professional/contact information permitted for submission: [optional, approved entry].

Verification and date: [researcher confirmation].

## APPENDIX L. TECHNICAL EVIDENCE AND REPRODUCTION GUIDE

### Purpose and Boundaries

The accompanying technical package preserves the basis of implementation claims and known limitations. It is not a substitute for institutional research records. Evidence files should be reviewed with their stated environment, synthetic-data and source-only qualifications. Reproduction of mutating application commands belongs in an isolated authorized environment, not against the working database by default.

**Table 73. Technical evidence supplied with the manuscript.**

| Evidence area | Artifact location relative to docs | What it supports |
|---|---|---|
| Overall examination | system-audit/COMPLETE_SYSTEM_AUDIT.md | Audit scope, implementation inventory and limits. |
| Backend and database | system-audit/BACKEND_AND_DATABASE_AUDIT.md | Source-supported services and live-schema consistency findings. |

| Evidence area | Artifact location relative to docs | What it supports |
|---|---|---|
| Security and reliability | system-audit/SECURITY_AND_RELIABILITY_AUDIT.md | Prioritized defects, present controls and limited reproductions. |
| Processing engine | system-audit/PYTHON_ENGINE_AUDIT.md | Python architecture, configuration and processing findings. |
| Interface examination | system-audit/UI_UX_AUDIT.md | Page/interaction observations and rendering limitations. |
| Functional evidence | system-audit/FUNCTIONAL_TEST_RESULTS.md | Expected/actual/status evidence with test boundaries. |
| Feature/workflow mapping | system-audit/FEATURE_INVENTORY.md, system-audit/SYSTEM_WORKFLOWS.md | Actual entry points, actors, data movement and limitations. |
| Schema metadata | system-audit/evidence/live-schema.json | Observed tables, columns, constraints and database version, without application row data. |
| Isolated PHP fixtures | system-audit/evidence/backend-isolated-checks.json | Two ownership failures and one suffix-range failure using synthetic data. |

| Evidence area | Artifact location relative to docs | What it supports |
|---|---|---|
| Source inventory | system-audit/evidence/backend-feature-notes.json | Curated capabilities, file hashes and method locations. |
| Dictionary and ERD | capstone-manuscript/tables/DATA_DICTIONARY.md, capstone-manuscript/diagrams/ERD_ACTUAL_SCHEMA.mmd | 312-column dictionary and actual 35-FK relationship source. |
| Researcher-source concordance | capstone-manuscript/reference-analysis/RESEARCHER_DRAFT_IMAGE_REVIEW.md | Supplied image review, schedule transcription and corrections. |

Table 73 identifies the supporting files and the claims each can substantiate. The source maps and metadata should be tied to the reviewed version; later application edits can invalidate line references and findings. The manuscript's build reports and generated contents/lists document presentation checks separately. An editable document's successful export does not establish substantive research completeness.

**Controlled Reproduction Principles**

Preserve the original application and create a designated isolated copy with synthetic fixtures. Prevent the clone from using production database settings, email delivery, external callbacks or scheduled cleanup. Use the explicitly recorded interpreter and dependencies. Run only relevant tests, retain their raw output, and compare observed outcomes with the stated expected behavior. Treat expected-denial failures as findings, not as a reason to discard the case.

Read-only schema verification should query metadata directly without the application bootstrap. The web bootstrap can touch guest state, and certain maintenance status/dry-run commands have side effects. Do not infer a read-only guarantee from their command names. For future benchmark or human-evaluation reproduction, preserve corpus permissions, source/reference versions, run configuration, model availability and analysis rules before generating results.